

电子信息科学与工程系列

数字集成电路 EDA 设计实验教程

庞志勇 陈弟虎 黄以华 编 著

電子工業出版社

Publishing House of Electronics Industry

北京·BEIJING

内 容 简 介

本书针对数字集成电路 EDA 设计的特点,以 Altera 公司 Quartus II 软件为 EDA 设计软件平台,以备受全球青睐的 DE2 系列开发板为 EDA 设计硬件平台,将数字集成电路设计与硬件描述语言 VHDL 相结合,循序渐进地介绍了基于 Quartus II 和 DE2-115 的数字集成电路 EDA 设计的基本过程和方法。

全书共 5 章,第 1 章简要介绍数字集成电路设计;第 2 章和第 3 章分别介绍了目前主要流行的 EDA 设计工具 QuartusII 和 EDA 仿真工具 ModelSim 的内容和具体操作方法;第 4 章介绍了 EDA 设计硬件实验平台 DE2-115,方便读者快速熟悉和了解 EDA 设计硬件实验平台;第 5 章给出了 13 个典型的 EDA 设计实验,实验内容针对数字集成电路设计的知识结构需要,按从简单到复杂,从分离模块实验到系统集成芯片设计实验的顺序安排设计,其中系统集成芯片设计实验内容包括串行通信、红外通信、图像显示、CPU 芯片设计等实验,涵盖了微电子类、电子信息类、计算机类专业的相关实验内容。针对每一个实验,都提供了具体的实验目的、实验原理、实验内容、实验步骤。除实验一、实验二给出了完整的程序外,其余的实验只提供了部分程序,每个实验都强调编写 TestBench 仿真文件,对提高学生数字集成电路设计水平非常有帮助。

本书可作为高等学校微电子类、电子信息类、计算机类专业本、专科的数字集成电路 EDA 设计实验教材,也可作为数字集成电路芯片设计人员、电子工程技术人员的参考用书。

未经许可,不得以任何方式复制或抄袭本书之部分或全部内容。

版权所有,侵权必究。

图书在版编目(CIP)数据

策 划:

责任编辑:

印 刷:

装 订:

出版发行:

北京市海淀区万寿路 173 信箱 邮编 100036

开 本: × 1/16 印张: 字数: 千字

版 次: 2018 年 3 月第 1 版

印 次: 2018 年 3 月第 1 次印刷

印 数: 定价: 元

凡所购买电子工业出版社图书有缺损问题,请向购买书店调换。若书店售缺,请与本社发行部联系,联系及邮购电话: (010)88254888, 88258888。

质量投诉请发邮件至 zlt@phei.com.cn, 盗版侵权举报请发邮件至 dbqq@phei.com.cn。

本书咨询联系方式: classic-series-info@phei.com.cn。

前 言

半个世纪以来，半导体工艺和处理器技术高速发展，集成电路规模越来越大，数字集成电路电子设计自动化（Electronics Design Automation, EDA）正成为全球数字集成电路芯片设计的大趋势。特别是在 20 世纪末，在系统编程技术（In-System Programmability, ISP）的出现，才实现了真正意义上的数字集成电路电子设计自动化。数字集成电路 EDA 技术现在已是、将来更是电子设计的必经之路，也是微电子、电子技术教学必不可少的内容。

现在对 EDA 的概念或范畴用得很宽。本书中，EDA 技术是指以计算机为工具，设计者在 EDA 软件平台上，用硬件描述语言 HDL 完成设计文件，然后由计算机自动地完成逻辑编译、化简、分割、综合、优化、布局、布线和仿真，直至对于特定目标现场可编程门阵列（FPGA）或复杂可编程逻辑器件（CPLD）的适配编译、逻辑映射和编程下载等工作。EDA 仿真工具为用户提供了全功能、全频带的分析软件平台，实现了系统结构或电路特性的模拟以及电路参数的优化。电子系统的验证离不开一个功能齐全的 EDA 开发与验证的硬件平台，一块多功能 EDA 硬件开发平台要求具备 FPGA/CPLD 可编程逻辑器件、按键、LED 灯、数码管、LCD 显示、存储器、USB 接口、鼠标接口、键盘接口、VGA 接口、串行通信接口等主要部分。数字集成电路芯片的主要功能都能在 EDA 硬件开发平台上得到验证。

本书针对本书微电子本科专业在学习了“数字集成电路 EDA 设计技术”课程的基础上，开设的一门 EDA 设计实验课程。本书适合于国内大专院校微电子学或电子类本、专科学生，在熟悉了硬件描述语言 VHDL 或 Verilog HDL 的基础上，配套开设的 EDA 实验课程。通过本书的 EDA 设计实验的训练，让学生由浅入深、循序渐进地掌握硬件描述语言，通过实践掌握数字集成电路 EDA 开发软硬件平台，从而提高学生数字集成电路设计和分析的能力，为学生从事各种数字集成电路的设计打下一定的基础。

本书在编写过程中，得到了焦涵、刘动、林樾、谢德英、陈耀和、余思远和邓少芝等同志的帮助和支持，在此向他们表示衷心的感谢！

本书在编写过程中参考了大量国内外相关教材，主要文献资料已详细列于参考文献，但难免会有疏漏，在此一并表示衷心感谢。

由于数字集成电路 EDA 技术发展迅速，限于作者水平有限，书中错漏和不完善之处在所难免，恳请读者予以指正。

编 者

中山大学专用集成电路设计中心
E-mail: stscdh@mail.sysu.edu.cn
stspzy@mail.sysu.edu.cn

目 录

第 1 章 数字集成电路设计概论	1
1.1 集成电路设计方法与工具的演变	1
1.2 自顶向下的系统设计步骤	3
1.3 FPGA/CPLD 特点及优势	4
1.4 数字集成电路 EDA 设计流程	5
第 2 章 EDA 设计软件平台 Quartus II	7
2.1 设计输入	7
2.2 设计项目的编译	13
2.3 仿真	17
2.4 延时分析	22
2.5 设计下载	23
2.6 MAX+PLUS II 与 Quartus II	24
第 3 章 EDA 设计仿真软件 ModelSim	27
3.1 ModelSim 用户接口	27
3.2 ModelSim 仿真窗口	28
3.3 使用 ModelSim 进行仿真	30
第 4 章 EDA 设计硬件平台	35
4.1 DE2-115 平台简介	35
4.2 DE2-115 平台主要组件	37
4.3 DE2-115 平台自检测试	39
4.4 使用 DE2-115 平台	40
第 5 章 VHDL 基础语法	44
5.1 VHDL 程序基本结构	44
5.2 VHDL 语言要素	50
5.3 VHDL 描述语句	68
5.4 VHDL 高级语句	87
第 6 章 EDA 设计实验	95
实验一 基本数字电路芯片设计实验	95
实验二 彩灯显示芯片设计实验	110
实验三 数码管显示芯片设计实验	116
实验四 存储器芯片设计实验	121
实验五 有限状态机芯片设计实验	125
实验六 VGA 彩条显示芯片设计实验	128
实验七 VGA 图像显示芯片设计实验	133
实验八 PS/2 键盘控制芯片设计实验	145

实验九 串行通信芯片设计实验.....	150
实验十 红外通信芯片设计实验.....	154
实验十一 DDS 频率合成器芯片设计实验	158
实验十二 一个简单的 CPU 芯片设计	165
实验十三 LCD 显示 SOC 芯片设计实验	168
附录 DE2-115 实验板引脚配置信息	179
参考文献.....	194

数字集成电路设计概论

1.1 集成电路设计方法与工具的演变

第一块集成电路 (IC) 于 1959 年被设计出来, 当时的设计只有 4 个晶体管。到 1997 年, 一个集成电路芯片上可集成的晶体管数目已高达 40 多亿个。50 多年来, 集成电路技术已发生了巨大的变化, 它经历了小规模 (SSI)、中规模 (MSI)、大规模 (LSI)、超大规模 (VLSI)、特大规模 (ULSI) 阶段, 目前已进入特大规模 (GSI) 阶段。随着集成技术的不断发展和集成度的迅速提高, 集成电路芯片的设计工作越来越复杂, 其设计方法与设计工具也发生了巨大的变革, 经历了从原始的全手工设计到目前最先进的计算机全自动实现的整个发展过程。根据各个历史进程中科技进步的特点及各主要设计手段、设计工具的不同, 可以把它归为原始的手工设计、计算机辅助设计 (CAD)、电子设计自动化 (EDA)、电子系统设计自动化 (ESDA) 和用户现场可编程器件五个阶段, 各阶段的主要特点如下:

(1) 随着 1959 年世界上第一块集成电路的问世, 手工设计方法也就诞生了。没有任何自动化、半自动化的辅助设计手段, 在 IC 设计的所有步骤上, 完全依靠手工操作。在完成了初步的功能设计、逻辑设计直至电路设计之后, 用分立的元器件搭制起硬件模拟电路, 让信号通过这一模拟电路, 以验证其功能及各项参数是否满足原设计的要求。接下来的版图设计, 也完全采用人工进行布局布线, 凭眼力或感觉, 反复调整与斟酌, 确定最佳排版方案, 使之遵从面积最小、连线最短原则。然后, 用尺和笔在方格纸上绘制版图, 分层剥刻红膜, 逐一检查整套版中每一层红膜的每一个窗孔是否存在漏剥漏刻等错误。然后用这套红膜去拍照、微缩、加工出流片所需的光刻掩模板。在试制流片阶段, 整个流片过程中的工艺设计也是人工的。若需采取新工艺或调整一些工艺参数, 则必须通过工艺线上一丝不苟的实验。最后, 整个 IC 芯片试制的成功与否, 设计的正确与否, 尤其是版图及其之后设计部分的正确性, 都需等到最终测试了流片的结果后才能知道。

(2) 正如计算机辅助设计 (CAD) 给机械、纺织等其他行业的设计领域带来了勃勃生机一样, 计算机对集成电路的辅助设计 (IC-CAD) 也给电子行业带来了新的生命。随着计算机软件技术的渗透, 各种各样的计算机模拟软件, 如线路模拟、逻辑模拟、时序模拟、器件模拟、工艺模拟等软件先后问世, 构成了早期 IC-CAD 技术的主要内容。SPICE 是线路模拟软件最成功的代表之一。对于一个规模不是很大的电路, SPICE 完全取代了以往的硬件实物模拟方法, 而代之以计算机简便、灵活、高效、正确的验证方法。它帮助设计者有效地确

定电功能及参数的正确性，成为中小规模 IC 设计中使用最为广泛的有力工具之一。对于规模较大的数字电路来说，以布尔代数为基础的高级计算机逻辑模拟软件也相继问世。早期的逻辑模拟软件以模拟（仿真）电路的逻辑功能为主，多采用零延迟、单位延迟、最大/最小延迟等简单的延迟模型，而将时序分析交给专用的时序模拟程序来完成。除了对电路进行分析模拟的上述模拟软件，器件模拟软件也被广泛应用，器件模拟软件是在给定半导体器件的物理结构和掺杂分布的条件下，通过软件程序直接求解器件内部的数理方程，从而得到器件有关电学特性和电学参数的计算机辅助设计方法。通过这一方法，人们可以更深入地了解影响器件性能的各种因素，从而控制器件的性能和进行优化设计。无论是参数的调整还是性能的改进，都可以在器件实际制作之前得到全面验证，推动器件的改良和新器件的诞生。工艺模拟软件取代了以往人们只能用在工艺线上做实验的方法来选择和确定工艺条件的传统做法，将这项耗时耗资很大的试验工作交给了计算机去完成。工艺模拟软件能在任意时间、任意条件下，由给出的芯片内部半导体器件体内的杂质浓度分布情况，计算出相应的电学参数，如薄层电阻、阈值电压等，为改善集成电路性能提供强有力的工艺研究工具。

(3) EDA 是 IC-CAD 技术发展的必然产物。随着 IC-CAD 软件的不断丰富、成熟与完善，计算机硬件在速度与容量方面的巨大提高，使得计算机在 IC 各独立设计环节上的辅助设计地位逐渐被工作站统一完备的集成化环境所取代。第一代 EDA 称为计算机辅助设计 CAD (Computer Aided Design) 系统，时间大致为 20 世纪 70 年代至 80 年代初，它以交互式图形编辑和设计规则检查为特点，硬件采用 16 位小型机。那时的逻辑图输入、逻辑模拟、电路模拟与版图设计及版图验证是分别进行的，人们需要对结果进行多次比较和修改才能得到正确的设计。第二代 EDA 系统称为计算机辅助工程 CAE (Computer Aided Engineering) 系统，时间大致为 20 世纪 80 年代至 90 年代初。它以 32 位工作站为硬件平台。它集逻辑图输入 (schematic entry)、逻辑模拟、测试码生成、电路模拟、版图设计、版图验证等工具于一体，构成了一个较完整的设计系统。进入 90 年代，芯片的复杂程度越来越高，数万门以至数十万门的电路设计的需求越来越多。单是依靠原理图输入方式已不堪承受，对于数字集成电路采用硬件描述语言 HDL (Hardware Description Language) 的设计方式就应运而生，开始 HDL 主要用于电路仿真，后来设计工作也逐渐从功能级向行为级发展，EDA 向设计的高层次发展。第三代 EDA 系统的特点是高层次设计的自动化 HLDA (High Level Design Automation)。在第三代 EDA 系统中，引入了硬件描述语言，一般采用两种语言即 VHDL 语言和 Verilog HDL 语言，此外还引入了行为综合和逻辑综合工具，采用较高的抽象层次进行设计，并按层次式方法进行管理，可大大提高处理复杂设计的能力，设计所需的周期也大幅度缩短。综合优化工具的采用使芯片的品质如面积、速度、功耗等获得了优化，因而第三代 EDA 系统迅速得到了推广应用。目前，国际上较著名的四大 EDA 供应商为 Cadence、Synopsys、Mentor Graphics 和 Avent。

(4) 随着半导体工艺技术的不断进步，芯片的设计规模越来越大，进入 $0.18\mu\text{m}$ 后，已经可以在一个芯片上实现 1 亿个门的设计规模。这样的规模完全可以将一个完整的电子系统在一个芯片上得以设计实现，即所谓的单芯片系统 (System On a Chip, SOC)。在 SOC 设计中 IP (Intellectual Property) 核得到了广泛应用，IC 设计成果的可重用性得到很大的发展，设计规模也越来越大。对于 SOC 这样一个规模巨大、功能复杂的电子系统，如果仍用人工手段进行功能分析与设计，直至完成 HDL 描述后再移交给 EDA 工具完成芯片设计，同样是

相当困难的。正是这种需求，推动着设计技术向电子系统设计自动化（Electronic System Design Automation, ESDA）的方向发展。ESDA 的目标就是为设计人员提供进行系统级设计的分析手段，进而完成系统级自动化设计。ESDA 设计工具需要解决的关键问题包括：一是建立系统级仿真库，二是实现不同仿真工具的协同模拟。借助于 ESDA 工具完成功能分析后，再利用行为级综合工具将其自动转化为可综合的 RTL（寄存器传输级）HDL 描述，然后交给 EDA 工具完成最后的芯片设计。

(5) 随着时代的发展，VLSI 产品的开发重点已逐渐由通用的 IC 产品向着专用 IC 产品转化，对批量不大、用途众多、性能各异的专用集成电路（ASIC）的需求，又在两个方面向 VLSI 设计领域提出更进一步的要求。第一，重申版图设计必须要有一种规则化、规范化的形式，以使计算机对各种电路都能方便地实现全自动设计。这种规范化由于门阵列具有的半定制特点而逐渐向门阵列母片形式归一。第二，用户可自己编程的通用半导体器件日益为广大系统设计人员所青睐，从需要半导体厂家协助实现的版图可编程器件 ROM、PLA 等（仅对引线孔一块掩膜版进行编程即可实现不同功能），到用户可自己直接编程但规模较小的 EPROM、PAL、GAL 器件，芯片功能逐渐完善，复杂度日益提高，芯片内容量也几十、上百倍地增加。90 年代初，集前人技术于一身的 ASIC 计算机全自动开发系统——现场可编程门阵列（FPGA）问世了。研制人员无须半导体厂家的生产合作，甚至也无须任何与半导体芯片加工有关的技术与设备，自己在一台计算机开发系统的帮助下，即可实现实地编程（现场编程），在商品化的特制芯片（母片）上，开发出具有各种功能、各种用途的 ASIC 样片，其成本之低、周期之短，已堪称进入非常理想的境界了。目前 FPGA 方法已大量应用于某些样片或样机中专用芯片的研制过程，这些电路规模不是十分巨大（数万门数量级以内），批量小，设计周期要求短，从时间与经费角度都不适于制版流片。当用这种方法最终研制成功，需要进一步大批量生产电路时，FPGA 还有自动转换成门阵列版图数据的能力。如果说从手工设计到 EDA、ESDA 技术的出现，反映了 IC 设计技术的发展过程，FPGA 技术并不构成一个独立的发展过程，事实上，各类 EDA 工具中一般都提供了与 FPGA 开发系统的接口，FPGA 为 ASIC 设计提供了一种可能的实现方式。由于 EDA 中的前端设计与工艺无关，因而依据实际需要，如果调用 FPGA 库进行综合，则可完成 FPGA 设计，而如果调用的是其他工艺库，则可以获得相应的产品实现。

1.2 自顶向下的系统设计步骤

芯片设计通常分为正向设计与反向设计两大类。正向设计一般用来实现一个新的设计，而逆向设计是在解剖别人设计的基础上进行某种修改或改进。在这两大类中又可分为“自顶向下”（TOP-DOWN）和“自底向上”（BOTTOM-UP）的设计步骤。自底向上设计是在系统划分和分解的基础上，先进行基本单元设计，然后逐级向上完成功能模块、子系统设计，直到设计出满足需要的系统为止。而基于 EDA 技术的 IC 设计主要是自顶向下（TOP-DOWN）的设计方法，其设计步骤如图 1.1 所示。首先从系统设计入手，在顶层采用可完全独立于目标器件芯片物理结构的硬件描述语言（HDL），在系统的基本功能或行为级上对设计的产品进行描述和定义，结合多层次的仿真技术，在确保设计的可行性与正确性的前提下，完成功能确认。然后利用 EDA 工具的逻辑综合功能，把功能描述转换成某一具体目标

芯片的网表文件,输出给该器件厂商的布局布线适配器,进行逻辑映射及布局布线,再利用产生的仿真文件进行功能和时序的验证,以确保实际系统的性能。即系统的结构构成方式与系统的行为或算法方式相混合的描述(称为混合层次描述),由于 HDL 具有这种描述能力,设计者就可以在抽象度相当高的层次上描述系统的基本结构。自顶向下设计方法的优越性表现在:(1)由于顶层的功能描述可以完全独立于目标器件的结构,在设计的最初阶段,设计人员可不受芯片结构的约束,集中精力对产品进行最适应市场需求的设计,从而避免了传统设计方法中的再设计风险,缩短了产品的上市周期。(2)设计成果的再利用得到保证。就这方面而言,单片机系统的设计成果难以得到再利用。现代的电子应用系统以及电子产品的开发与生产正向模块化发展,或者说向软硬核组合的方向发展。对于以往成功的设计成果稍作修改、组合就能投入再利用,从而产生全新的或派生的设计模块,同时还可以以一种 IP 核的方式进行存档。(3)由于采用的是结构化开发手段,一旦主系统基本功能结构得到确认,即可实现多人多任务的并行工作方式,使系统的设计规模和效率大幅度提高。(4)在选择实现系统的目标器件的类型、规模、硬件结构等方面具有更大的自由度。

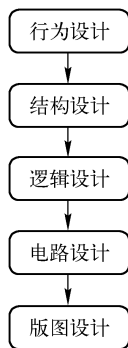


图 1.1 正向设计流程

1.3 FPGA/CPLD 特点及优势

高集成度、高速和高可靠性是 FPGA/CPLD (现场可编程门阵列/复杂可编程逻辑器件) 最明显的特点,其时钟延迟可达纳秒级,结合其并行工作方式,在超高速应用领域和实时测控方面有非常广阔的应用前景。在高可靠应用领域,如果设计得当,将不会存在类似于 MCU 的复位不可靠等问题。CPLD/FPGA 的高可靠性还表现在,几乎可将整个系统下载于同一芯片中,实现所谓片上系统,从而大大缩小了体积,易于管理和屏蔽。

随着集成电路制作工艺的提高,FPGA 在性能、集成规模方面取得了显著提升,可利用先进的 EDA 工具进行更加复杂的数字电子系统设计和产品开发。由于开发工具的通用性、设计语言的标准化以及设计过程几乎与所用器件的硬件结构没有关系,因此设计成功的各类逻辑功能电路有很好的兼容性和可移植性,它几乎可用于任何型号和规模的 FPGA/CPLD 中,从而使得产品设计效率大幅度提高。可以在很短时间内完成十分复杂的系统设计,这正是产品快速进入市场最宝贵的特征。美国 TI 公司认为,一个 ASIC 80% 的功能可用 IP 核等现成逻辑合成。而未来大系统的 CPLD/FPGA 设计仅仅是各类再应用逻辑与 IP 核 (Core) 的拼装,其设计周期将更短。

与 ASIC 设计相比,FPGA/CPLD 显著的优势是开发周期短,投资风险小、产品上市速度快,市场适应能力强,以及硬件升级回旋余地大,而且当产品定型和产量扩大后,可将在生产中达到充分检验的 VHDL 设计迅速实现 ASIC 投产。随着社会经济延伸,各类新型电子产品的开发为我们提出了许多全新的课题和更高的要求。FPGA/CPLD 在 EDA 基础上的广泛应用,从某种意义上说,新的电子系统运转的物理机制又将回到原来的纯数字电路结构,但这是一种更高层次的循环,应是一次否定之否定的运动,它在更高层次上容纳了过去数字技术的优秀部分,对 MCU 系统将是一种扬弃,但在电子设计的技术操作和系统构成的整体上却发生了质的飞跃。如果说 MCU 在逻辑的实现上是无限的话,那么高速发展的

FPGA/CPLD 不仅包括了 MCU 这一特点, 并兼有串、并行工作方式和高速、高可靠性以及宽口径适用性等诸多方面的特点。不仅如此, 随着 EDA 技术的发展和 FPGA/CPLD 在深亚微米领域的进军, 它们与 MCU、MPU、DSP、A/D、D/A、RAM 和 ROM 等独立器件间的物理与功能界限正日趋模糊。特别是软 IP 核产业的迅猛发展, 内嵌处理器 FPGA 的出现, 片上系统已近在咫尺。以大规模集成电路为物质基础的 EDA 技术终将打破软硬件之间最后的屏障, 使软硬件工程师们有了共同的语言。

1.4 数字集成电路 EDA 设计流程

在电子设计技术领域, 可编程逻辑器件的广泛应用为数字系统的设计带来极大的灵活性。由于该器件可以通过软件编程而对其硬件的结构和工作方式进行重构, 使得硬件的设计可以如同软件设计那样方便快捷。这一切极大地改变了传统的数字系统设计方法、设计过程, 乃至设计观念。在传统的数字系统设计中, 用户能够通过编程方式实现逻辑功能只有两种途径, 即微处理器的软件编程 (如单片机) 和特定器件的控制字配置 (如 8255)。在传统的设计概念中, 器件引脚功能的硬件方式的任意确定是不可能的。而对于系统构成的设计过程只能对器件功能和电路板图分别进行设计和确定, 通过设计电路板来规划系统功能。在此期间, 大量的时间和精力花在元件选配和系统结构的可行性定位上。但若采用可编程逻辑器件, 便可利用计算机软件的方式对目标器件进行设计, 而以硬件的形式实现即定的系统功能。在设计过程中, 设计者可根据需要, 随时改变器件的内部逻辑功能和引脚的信号方式。借助于大规模集成的可编程逻辑器件和高效的设计软件, 用户不仅可通过直接对芯片结构的设计实现多种数字逻辑系统功能, 而且由于引脚定义的灵活性, 大大减轻了电路图设计和电路板设计的工作量和难度; 同时, 这种基于可编程逻辑器件芯片的设计大大减少了系统芯片的数量, 缩小了系统的体积, 提高了系统的可靠性。

现代的 EDA 工具软件已突破了早期仅能进行 PCB 版图设计, 或类似某些仅限于电路功能模拟的、纯软件范围的局限, 以最终实现可靠的硬件系统为目标, 配备了系统设计自动化的全部工具。如配置了各种常用的硬件描述语言平台 VHDL、Verilog HDL、ABEL、System C 等; 配置了多种能兼用和混合使用的逻辑描述输入工具, 如硬件描述语言文本输入法 (其中包括布尔方程描述方式、原理图描述方式、状态图描述方式等) 以及原理图输入法、波形输入法等; 同时还配置了高性能的逻辑综合、优化和仿真模拟工具。所有这一切都为今天的电子设计工程技术人员提供了强有力的工具, 在过去令人难以置信的事, 今天已成为平常之事, 利用一台计算机、一套 EDA 软件和一片 CPLD 或 FPGA 芯片, 就能在家中完成大规模数字集成电路设计。基于可编程逻辑器件进行数字集成电路 EDA 设计的典型流程如图 1.2 所示, 主要由设计输入、功能仿真、逻辑综合、布局布线、时序仿真、器件编程下载六步组成。

(1) 设计输入的方式通常有两种: 一种是 HDL (Hardware Description Language) 硬件描述语言输入。现在 EDA 界使用较为广泛的 HDL 是 VHDL 和 Verilog-HDL, 几乎所

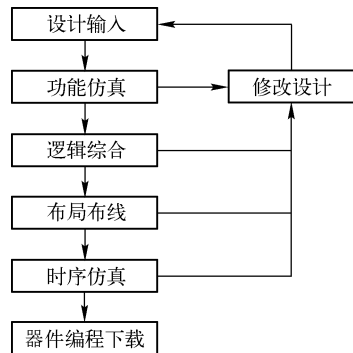


图 1.2 数字集成电路 EDA 设计典型流程

有的设计软件都支持这两种语言的设计输入。其他还有例如 ABEL-HDL、AHDL、System C 语言等。HDL 输入可移植性好，通用性好。但对于初学者来说，很难编写出恰当的语句来准确反映设计意图。另一种常用的输入方式称为原理图输入，它根据设计要求，选用器件、绘制原理图、完成输入过程。原理图输入相对 HDL 输入直观，便于理解，元件库资源丰富。但移植性差，尤其在设计大规模电路时，这种方法的可维护性较差。

以上两种设计输入方法各有利弊，所以通常建议采用原理图和文本输入相结合的方法来进行设计，对于设计不同的部分，灵活选用合适的输入方法。

(2) 功能仿真是在不考虑器件延时的理想情况下仿真设计项目，以验证其逻辑功能的正确性，也称前仿真。通过功能仿真能及时发现设计中的错误，加快设计进度，提高设计的可靠性，但对于简单的设计，可以跳过这一步，只在布线完成以后进行时序仿真就可以了。

(3) 逻辑综合是一个将高层次系统描述向低层次描述转化和优化的过程，即把设计输入的高级语言或原理图综合成底层软件可以识别的网表文件。网表文件中只包含了最简的布尔表达式和信号的连接关系以及其他一些电路设计参数，与具体工艺无关。现在使用较广泛的是 EDIF (Electronic Design Interchange Format, 电子设计交换格式) 网表文件。

通常来说，直接根据高级语言逻辑综合生成的网表文件并不是最简的，还要对其进行优化。优化是根据布尔方程功能等效的原则，用更小更快的综合结果代替一些复杂单元，并与指定的库映射生成新的网表。

(4) 布局布线。在设计通过功能仿真被初步验证之后，根据所选用 PLD 的规格，将综合生成的网表文件转换为器件内部的宏单元、逻辑门之间的连线关系。

(5) 时序仿真。在布局布线中可以获得比前仿真时更精确的 RC 参数，根据这些参数，在考虑具体适配器件的各种延时的情况下，用仿真软件再次验证电路的功能和时序，以确保在各种可能的条件下都能正确响应。这一步骤也称后仿真。

上述任何一步出错，均需回到设计输入阶段，改正错误，重新按设计流程进行设计。

(6) 器件编程下载。通过仿真确认设计无误后，将文件编译成器件可以识别的二进制码并下载到芯片中。

一个集成的 EDA 开发工具，如 Quartus II、MAX+PLUS II、Xilinx ISE 等，应该能完成上述所有的流程，并能使用第三方软件（如 ModelSim）帮助设计。

EDA 设计软件平台 Quartus II

Altera 公司是世界著名的 PLD 生产商之一，是 SOPC（System On a Programmable Chip，可编程单芯片系统）的开创者，它的主要产品是可编程逻辑器件，包括 CPLD、FPGA 和掩模编程器件等。随着产品线的丰富，其开发软件也在不断升级换代，由早期的 MAX+PLUS II 系列逐步发展到目前的 Quartus II。本章主要向读者介绍上述两种设计软件。

MAX+PLUS II 和 Quartus II 是 Altera 公司开发的，集设计输入、处理与校验功能于一体的，完全集成化、易学易用的可编程逻辑设计软件。两者在使用风格上十分接近，不同之处在于两者对器件的支持情况不同，后者比前者增加了对一些新器件的支持，同时取消了对一些老产品的支持。本章着重介绍 Quartus II，对于 MAX+PLUS II，也将做一个简单的说明。

2.1 设计输入

Quartus II 支持多种设计输入方法，主要为原理图式图形设计输入和文本输入两种，并支持由第三方工具编辑的 EDIF、HDL、VQM 等文件。Quartus II 还支持采用其他方法来优化和提高输入的灵活性，如混合设计格式以及利用 LPM 和宏功能模块来加速设计输入。

Quartus II 支持的设计输入文件如图 2.1 所示。

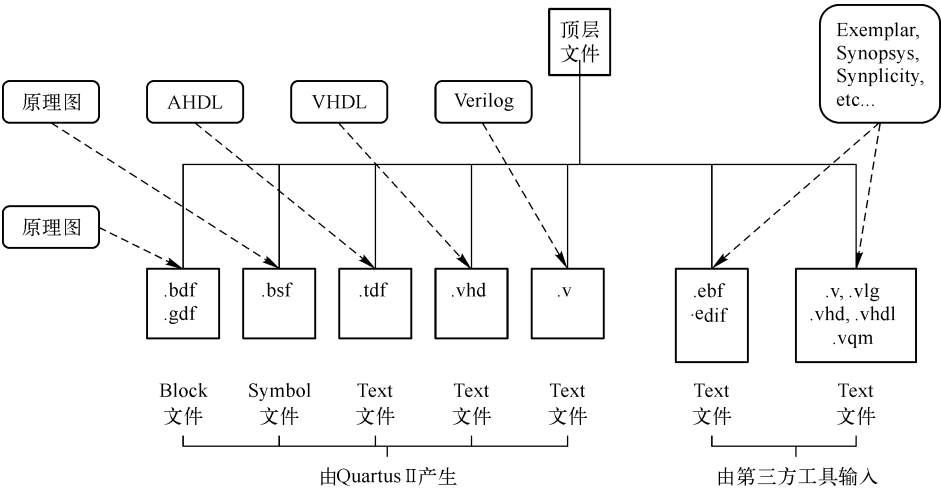


图 2.1 Quartus II 所支持的输入文件

启动 Quartus II 后，其用户界面如图 2.2 所示。

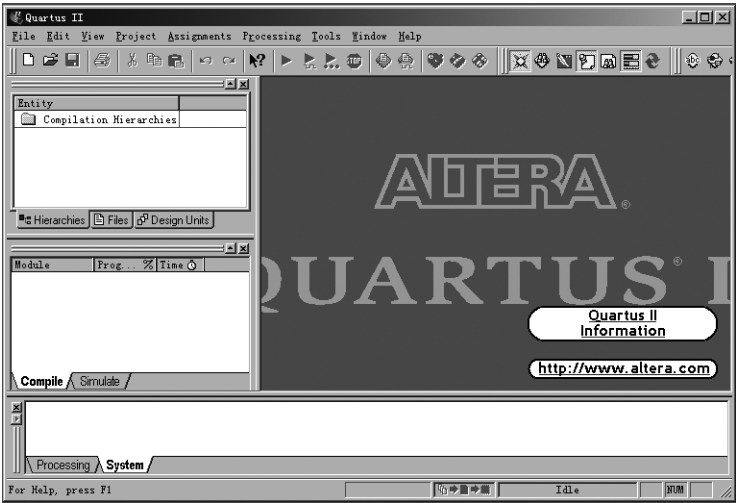


图 2.2 Quartus II 用户界面

1. 图形输入

Quartus II 的图形输入包括图表模块输入和原理图输入。图表模块输入是 Quartus II 中主要的顶层设计的主要方法，它可以“智能”地链接和映射模块；原理图输入是从 MAX+PLUS II 继承的传统的设计输入方法。这两种图形输入方式均在图表和原理图编辑器中完成。

(1) 产生一个新的图表模块/原理图文件，如图 2.3 所示，方法是依次单击 File>New>Block/Schematic document。

(2) 产生一个新模块。

单击工具栏中的“模块”来画一个图表。右键单击模块，选择“Block Properties”属性，在弹出的对话框中输入端口信息，如图 2.4 所示。

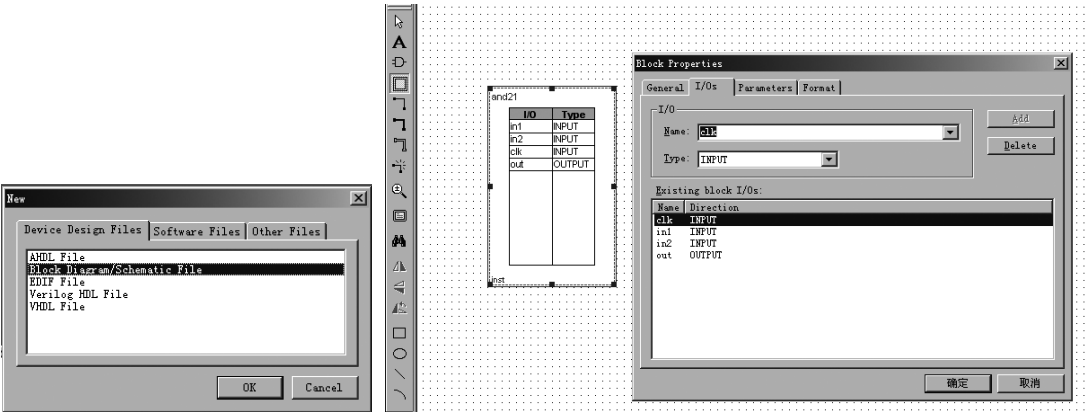


图 2.3 新建文件

图 2.4 新建/定义一个模块

(3) 连接各模块间的端口。

Quartus II 可以“智能”地连接和映射各模块，即连接不同模块时，如果两边端口的名字相

同，就不用标注出来，一个管道可以连接模块之间所有的普通 I/O 端口，如图 2.5 所示。

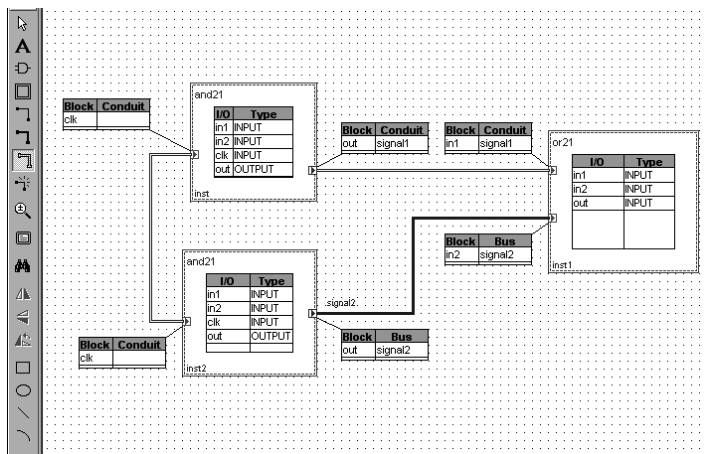


图 2.5 连接模块间的端口

(4) 给模块定义并产生 HDL 或图形设计文件。

右键单击模块，弹出对话框，从菜单中选择“Create Design File from Selected Block...”，如图 2.6 所示。

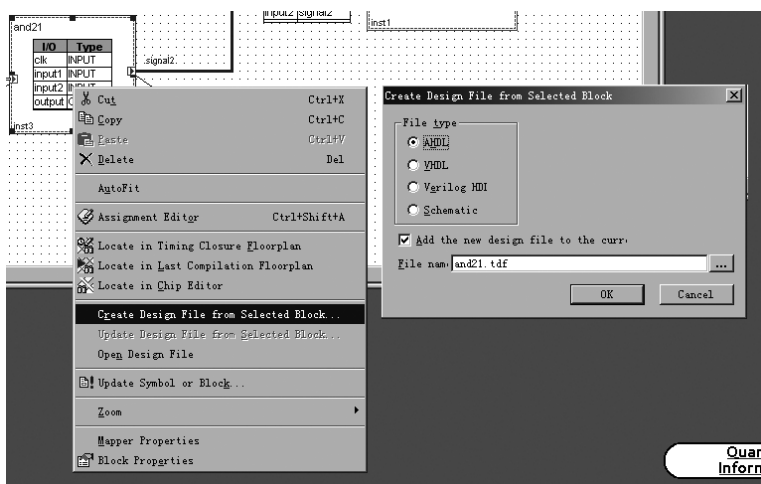


图 2.6 给模块定义并产生 HDL 或图形设计文件

2. 文本输入

Quartus II 的文本输入支持 AHDL、EDIF、Verilog HDL 和 VHDL 文件的输入和编辑。

(1) 产生一个新的文本文件，依次单击 File>New，弹出如图 2.3 所示的新建文件框，在其中选“AHDL File”（或“EDIF File/Verilog HDL File/VHDL File”其中之一，以需要输入的设计文件为准）。

(2) 新建文本后，即打开一个文本编辑框，可在此框中对设计文件进行编辑，如图 2.7 所示。

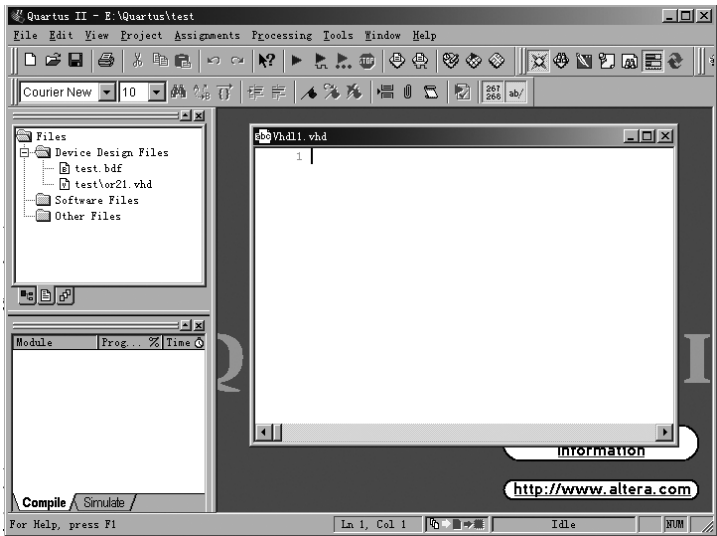


图 2.7 文本编辑框

3. 新建项目

(1) 新建项目指南。

依次单击 File>New Project Wizard..., 进入如图 2.8 所示的新建项目指南, 填写工作目录、项目名称和顶层设计, 其中顶层设计的名字必须和顶层文件名相同, 填写完毕后, 单击“Next”按钮。

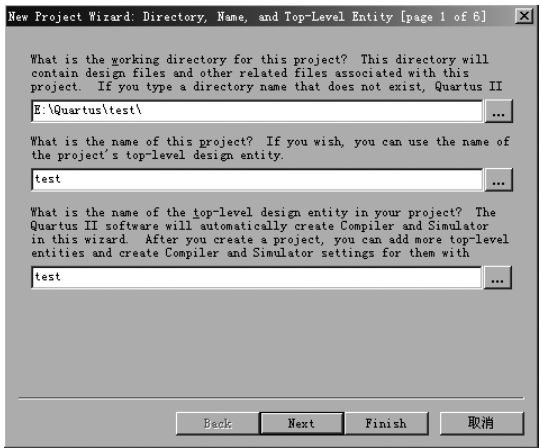


图 2.8 新建项目指南

(2) 加入设计文件和用户自定义库函数, 如图 2.9 所示。

加入设计文件时并不要求加入该项目目录下的所有文件, 但如果顶层设计和顶层文件不同名, 则一定要加入顶层文件名。填写完成后, 单击“Next”按钮。

(3) 设置第三方工具, 如图 2.10 所示。

如果用户需要用第三方工具完成设计, 则可针对 EDA 设计流程的每一步指定相应的第

三方工具。完成后单击“Next”按钮。

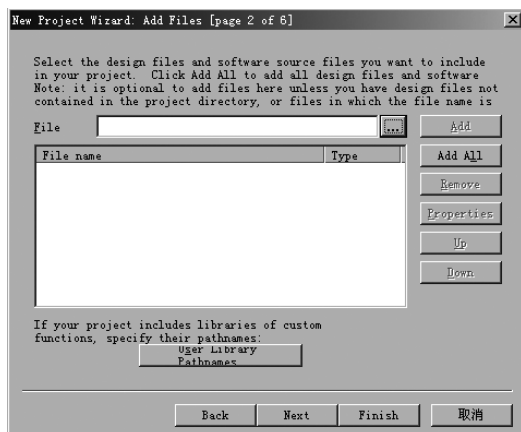


图 2.9 添加设计文件和用户自定义库

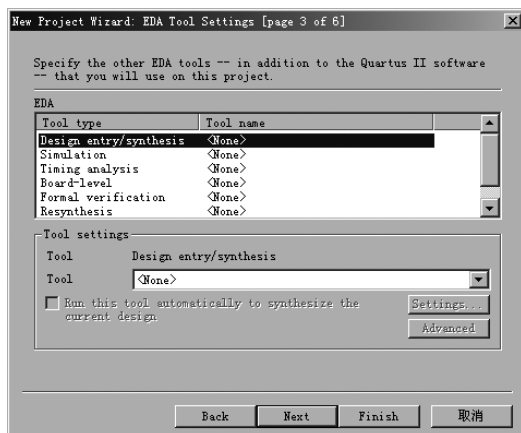


图 2.10 设置第三方工具

(4) 设置目标器件。

首先选择器件系列，如图 2.11 所示，完成后单击“Next”按钮；接着选择目标器件的具体型号，如图 2.12 所示，完成后单击“Next”按钮。Quartus II 将按照用户指定的器件对设计进行配置。若没有选定器件的具体型号，则在图 2.11 中选择“No, I want to allow the Compiler to choose a device”，编译器会根据用户选择的器件系列自动选用合适型号的器件进行配置。



图 2.11 选择器件系列

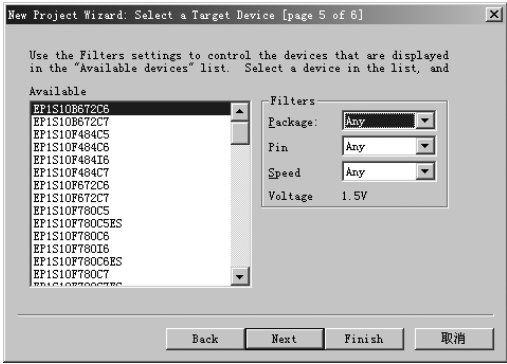


图 2.12 选择目标器件

(6) 在全部选择完成后，会出现如图 2.13 所示的新建项目报告，单击“Finish”按钮，完成新项目的建立。



图 2.13 新建项目报告

若要修改项目的相关设置，则依次单击 Assignments > Settings，进入如图 2.14 所示的项目设置菜单。

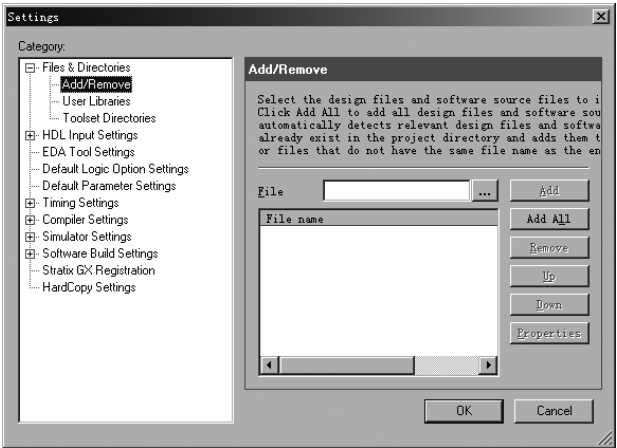


图 2.14 项目设置菜单

2.2 设计项目的编译

Quartus II 的编译包括两种类型：一种只是综合并输出网表，编译器只运行到综合这一步就停止了，综合产生的是门级代码，编译器只产生估算的延时数值；另一种是完全的编译，包括编译、前仿真、综合、后仿真、网表输出，以及将设计配置到 Altera 的器件中去，编译器根据器件特性产生真正的延时时间和给器件的配置文件。

1. 编译设置

(1) 依次单击 Assignments>Compiler Settings Wizard…，进入如图 2.15 所示的编译设置指南，分别指定编译模块和这个编译设置的名字，填写完毕后，单击“Next”按钮。



图 2.15 编译设置指南

(2) 如图 2.16 所示，在此设置框选择是否优化速度或资源利用率以及是否削减节点名字。选择完成后，单击“Next”按钮。

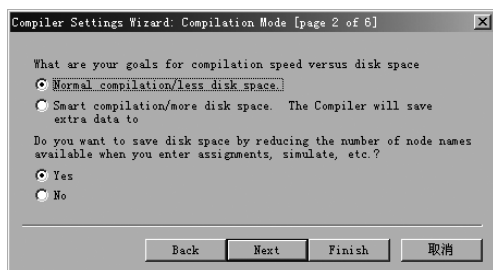


图 2.16 编译设置

(3) 设置目标器件。

首先选择器件系列，如图 2.17 所示，完成后单击“Next”按钮；接着选择目标器件的具体型号，如图 2.18 所示，完成后单击“Next”按钮。若未选定器件的具体型号，则在图 2.17 中选择“No, I want to allow the Compiler to choose a device”，编译器会根据用户选择的器件系列自动选用合适型号的器件进行配置。



图 2.17 选择器件系列

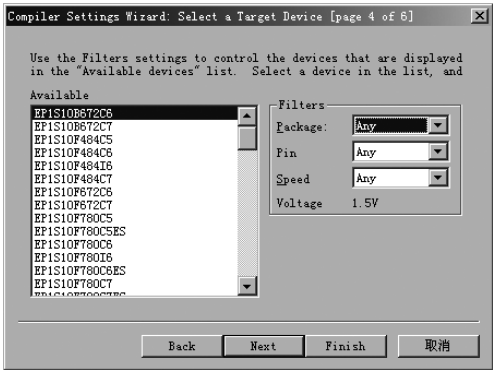


图 2.18 选择目标器件

(4) 是否保存一个节点网表文件，若需要，则输入需要保存的文件名及其路径，完成后单击“Next”按钮，如图 2.19 所示。

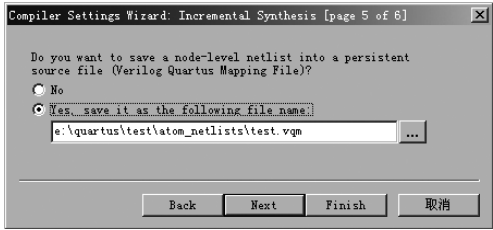


图 2.19 编译设置

(5) 在全部选择完成后，会出现如图 2.20 所示的编译设置报告，单击“Finish”按钮，完成编译设置。

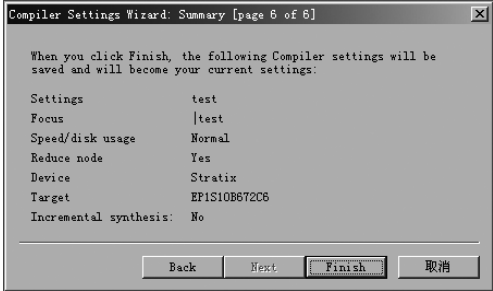


图 2.20 编译设置报告

若要修改编译器的相关设置，依次单击 Assignments>Settings，进入如图 2.21 所示的编译设置菜单。

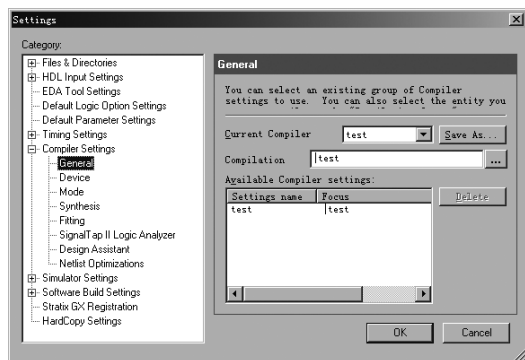


图 2.21 编译设置菜单

2. 设计项目编译

(1) 依次单击 Processing>Start Compilation，Quartus II 即按照用户对编译器的设置对选定的设计文件进行编译，如图 2.22 所示。

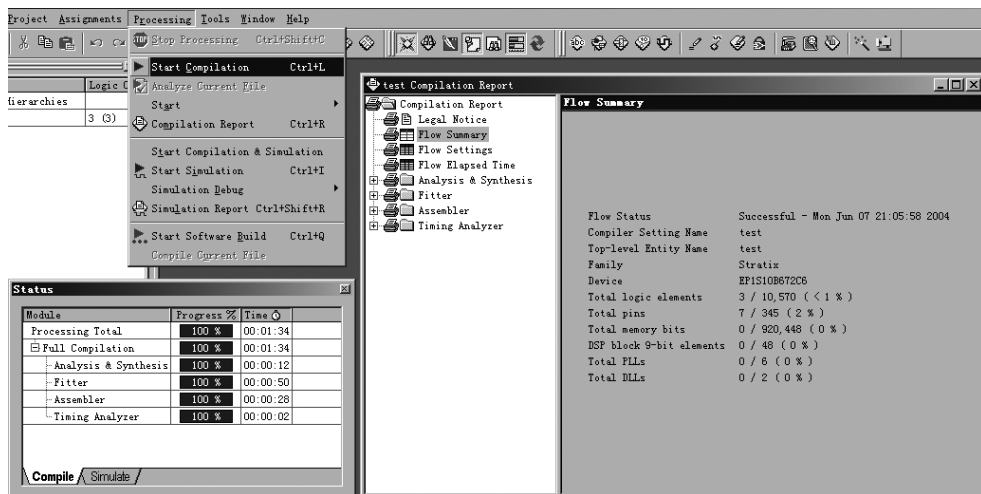


图 2.22 完全编译

(2) (1) 中的编译命令是完全的编译。若只需要对设计进行编译其中的某一步或若干步，可按如图 2.23 所示方法，依次进入 Processing>Start>…，在 Start 菜单中选择相应的编译项目即可。

3. 配置编辑器

配置编辑器可以在编译完成后对器件做进一步的配置，以优化设计。方法是依次单击 Assignments>Assignment Editor，打开如图 2.24 所示的配置编辑器。

4. 编译报告

依次单击 Processing>Compilation Report，可打开编译报告，编译报告中包含了怎样将一个设计放到一个器件中的所有信息，如图 2.25 所示。

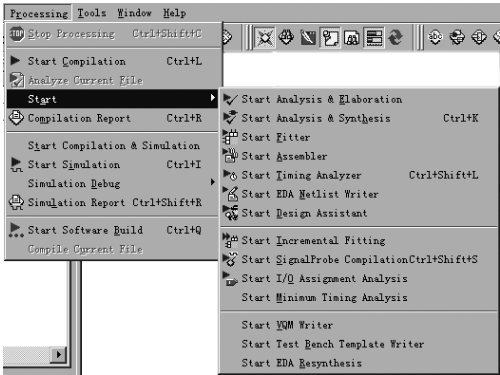


图 2.23 分步编译

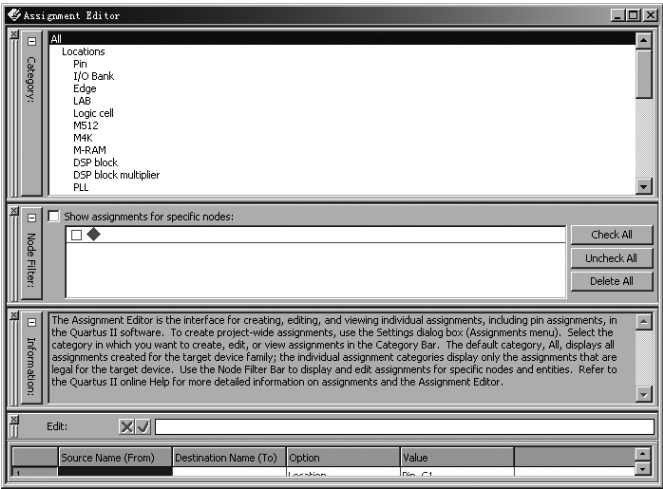


图 2.24 配置编辑器

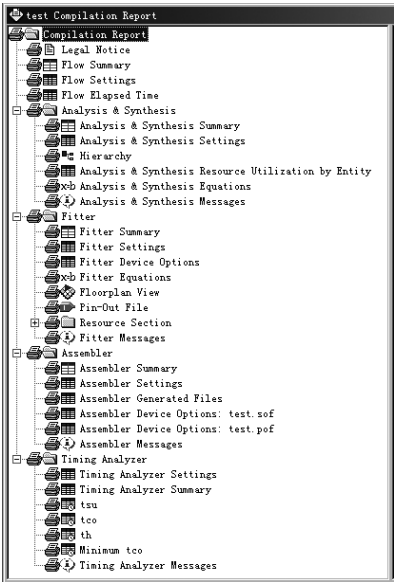


图 2.25 编译报告

2.3 仿 真

Quartus II 支持多种仿真方法。

1. 波形方式输入

这是最常用的仿真输入方法，在图 2.3 所示的新建文件框中，选择 Other Files 标签页中的 Vector Waveform File，即打开如图 2.26 所示的仿真波形编辑框。

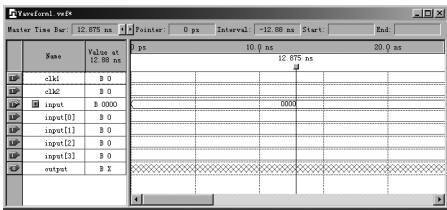


图 2.26 描绘仿真波形

Quartus II 主要支持的图形文件有：

- vwf（向量波形文件）是 Quartus II 中最主要的波形文件；
- vec（向量文件）是 MAX+PLUS II 所支持的文件，Quartus II 同样支持；
- tbl（列表文件）用来将 MAX+PLUS II 中的 .scf 文件输入到 Quartus II 中。

2. testbench

编写 testbench 来仿真与波形仿真相比尤为突出的好处是，当测试数据无比庞大时，可以简易地通过 testbench 中的算法来实现，而另一个更为重要的方面是，可以通过 testbench 对数据文件进行读写操作，从而简化我们的仿真工作。VHDL testbench 常用的一套固定格式总结如下：

```
--测试平台文件(testbench)的基本结构
library ieee;
use ieee.std_logic_1164.all;
entity test_bench is          --测试平台文件的空实体(不需要端口定义)
end test_bench;
architecture tb_behavior of test_bench is
    component entity_under_test    --被测试元件的声明
        port (
            list-of-ports-theri-types-and-modes
        );
    end component;
begin
    instantiation:entity_under_test port map
    (
        port-associations
    );
```

```

process ()          --产生时钟信号
.....
end process;
process ()          --产生激励源
.....
end process;
end tb_behavior;

```

假设我们已经设计好一个六进制计数器芯片的 VHDL 代码如下：

```

Library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity cnt6 is
port
  (clr,en,clk :in std_logic;
   q :out  std_logic_vector(2 downto 0)
  );
end entity;
architecture rtl of cnt6 is
signal tmp :std_logic_vector(2 downto 0);
begin
  process(clk)
  --   variable q6:integer;
  begin
    if (clk' event and clk='1') then
    if (clr='0') then
      tmp<="000";
    elsif (en='1') then
    if (tmp="101") then
      tmp<="000";
    else
      tmp<=unsigned(tmp)+'1';
    end if ;
    end if ;
    end if ;
    q<=tmp;
  --   qa<=q(0);
  --   qb<=q(1);
  --   qc<=q(2);
  end process;
end rtl;

```

按照 Testbench 常用的固定格式，可以设计出相应的 Testbench 仿真程序如下：

```
library ieee;
use ieee.std_logic_1164.all;
    entity cnt6_tb is
end cnt6_tb;
architecture rtl of cnt6_tb is
    component cnt6
        port (
            clr,en,clk :in std_logic;
            q :out  std_logic_vector(2 downto 0)
        );
    end component;
    signal clr :std_logic:='0';
    signal en :std_logic:='0';
    signal clk :std_logic:='0';
    signal q :std_logic_vector(2 downto 0);
    constant clk_period :time :=20 ns;
begin
    instant:cnt6 port map
    (
        clk=>clk,en=>en,clr=>clr,q=>q
    );
    clk_gen:process
begin
    wait for clk_period/2;
    clk<='1';
    wait for clk_period/2;
    clk<='0';
end process;

    clr_gen:process
begin
    clr<='0';
    wait for 30 ns;
    clr<='1';
    wait;
end process;

    en_gen:process
begin
    en<='0';
    wait for 50ns;
    en<='1';
```

```

    wait;
end process;
end rtl;

```

Quartus II 软件还可以自动生成 testbench 文件模板，我们只需要往里面写信号激励即可。主要步骤：依次选择 processing->start->starttest bench template write。这里需要注意的是，要在仿真选项里选择一个仿真工具，比如 ModelSim 或其他仿真工具，然后才会生成 testbench。

自动生成的 testbench 模板格式如下：

```

[c-sharp]
-- Copyright (C) 1991-2008 Altera Corporation
-- Your use of Altera Corporation's design tools, logic functions
-- and other software and tools, and its AMPP partner logic
-- functions, and any output files from any of the foregoing
-- (including device programming or simulation files), and any
-- associated documentation or information are expressly subject
-- to the terms and conditions of the Altera Program License
-- Subscription Agreement, Altera MegaCore Function License
-- Agreement, or other applicable license agreement, including,
-- without limitation, that your use is for the sole purpose of
-- programming logic devices manufactured by Altera and sold by
-- Altera or its authorized distributors. Please refer to the
-- applicable agreement for further details.

-- * * * * *
-- This file contains a Vhdl test bench template that is freely editable to

-- suit user's needs. Comments are provided in each section to help the user
-- fill out necessary details.
-- * * * * *
-- Generated on "03/13/2011 20:05:04"

-- Vhdl Test Bench template for design   : cnt6
--
-- Simulation tool : ModelSim (VHDL)
--

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY cnt6_vhd_tst IS
END cnt6_vhd_tst;
ARCHITECTURE cnt6_arch OF cnt6_vhd_tst IS
-- constants

```

```

-- signals
SIGNAL clk : STD_LOGIC;
SIGNAL clr : STD_LOGIC;
SIGNAL en : STD_LOGIC;
SIGNAL q : STD_LOGIC_VECTOR (2 DOWNTO 0);
COMPONENT cnt6
    PORT (
        clk : IN STD_LOGIC;
        clr : IN STD_LOGIC;
        en : IN STD_LOGIC;
        q : OUT STD_LOGIC_VECTOR (2 DOWNTO 0)
    );
END COMPONENT;
BEGIN
    i1 : cnt6
    PORT MAP (
-- list connections between master ports and signals
        clk => clk,
        clr => clr,
        en => en,
        q => q
    );
    init : PROCESS
-- variable declarations
    BEGIN
        -- code that executes only once
    WAIT;
    END PROCESS init;
    always : PROCESS
-- optional sensitivity list
-- (          )
-- variable declarations
    BEGIN
        -- code executes for every event on sensitivity list
    WAIT;
    END PROCESS always;
    END cnt6_arch;

```

3. 第三方仿真工具

Quartus II 支持的第三方仿真软件非常多，如 ModelSim (Model Technology 公司)、VERILOG-XL (Cadence 公司)、VCS/VSS (Synopsys 公司) 等。其中 ModelSim 的使用将在下面的章节中详细介绍，若使用的是 VHDL 仿真器或 Verilog 仿真器，则

- (1) VHDL 仿真器：利用 Quartus II 产生 .VHO 和 .SDO 文件并利用在 sim_lib 目录中的 *_ATOMs.VHD 和 *_COMPONENTS.VHD 文件（其中 * 视具体器件型号而定）。
- (2) Verilog 仿真器：则利用 Quartus II 产生的 .VO 文件和 .SDO 文件并利用在 sim_lib 目录中的 *_ATOMs.VO 文件（其中 * 视具体器件型号而定）。
- 对于使用何种第三方工具，可在 Assignments > EDA Tool Settings...中设置，如图 2.27 所示。

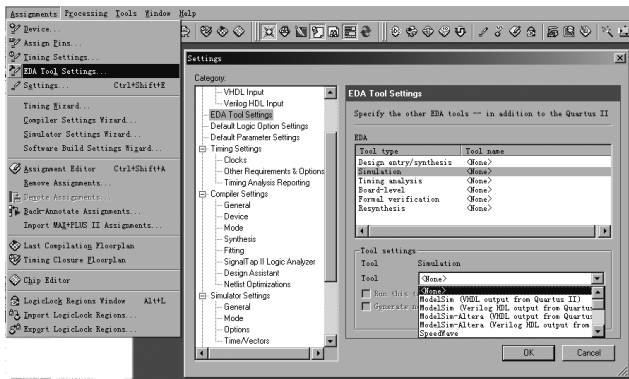


图 2.27 EDA 工具设置

需要指出的是，如图 2.27 所示的 EDA 工具设置菜单，并不仅限于对仿真中需要用到的第三方软件进行配置，其同样可对其他设计过程（如设计输入、时序分析等）中需要用到的第三方软件进行配置。

2.4 延时分析

Quartus II 的延时分析在默认情况下是在编译之后自动调用的。Quartus II 支持对单个时钟或多个时钟的延时分析。单个时钟的延时分析包括最大时钟频率（Fmax）、建立时间（Tsu）、保持时间（Th）、时钟到输出时间（Tco）和整个系统的 Fmax（包括引脚上的输入输出延时）。多个时钟的延时分析可以支持客户分析由不同时钟控制的寄存器之间的延时。另外，Quartus II 会自动检测组合逻辑电路。

延时分析的设置可在 Assignments > Timing Settings...中调整，如图 2.28 所示。

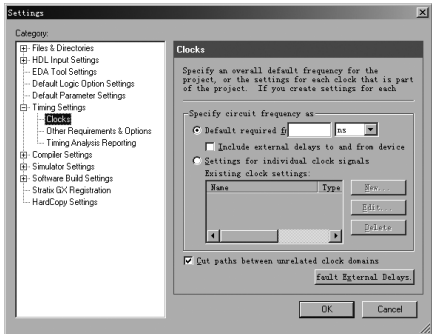


图 2.28 延时分析设置

所有的延时分析信息都包含在编译报告的延时分析目录下，如图 2.29 所示。

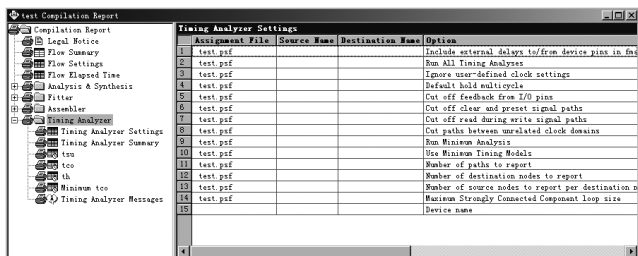


图 2.29 延时分析报告

生成的延时信息也可以 VHDL、Verilog 或标准延时文件（SDF）的格式输出到第三方的 EDA 工具中。

2.5 设计下载

1. 打开下载窗口

依次单击 Tools > Programmer，如图 2.30 所示。

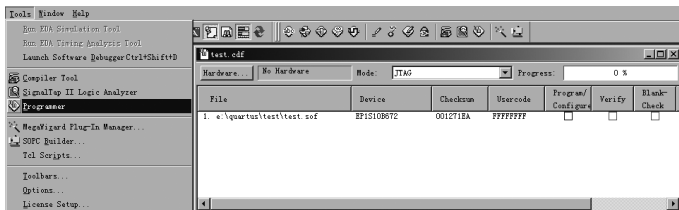


图 2.30 打开下载窗口

2. 设置下载电缆

选择 Hardware... 进行设置，在 Hardware Settings 对话框中选择下载电缆，如图 2.31 所示。



图 2.31 设置下载电缆

3. 开始下载

设置完毕后，依次单击 Processing > Start Programming，即开始下载，如图 2.32 所示。

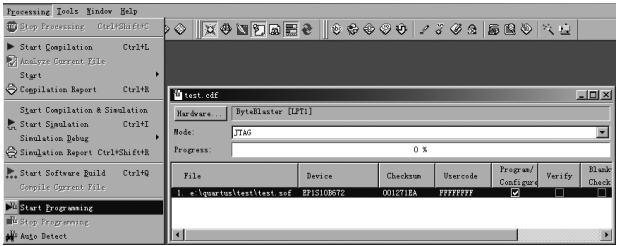


图 2.32 开始下载

下载成功后，使用 Quartus II 进行 PLD 设计的整个流程结束。
之后，还要对配置好的 PLD 进行硬件测试，在测试确定该 PLD 达到设计要求后，该 PLD 就可以投入使用了。

2.6 MAX+PLUS II 与 Quartus II

因为 Quartus II 是由 MAX+PLUS II 发展而来的，相比较而言 Quartus II 的功能比 MAX+PLUS II 多，所以读者如果可以熟练掌握 Quartus II 的使用，那么对 MAX+PLUS II 的使用大同小异。本节主要列出两套软件在支持器件和完成 EDA 设计功能上的异同。

1. 器件支持

Altera 设计软件支持器件如表 2.1 所示。

表 2.1 Altera 设计软件支持器件

器 件	Quartus II	MAX+PLUS II
Stratix (All)	√	
Stratix EP1S10	√	
Stratix GX (All)	√	
Cyclone (All)	√	
HardCopy (All)	√	
APEX II (All)	√	
APEX II EP2A15	√	
APEX 20K (All)	√	
APEX EP20K30E, EP20K60E, EP20K160E	√	
Mercury™ (All)	√	
FLEX 10K (All), FLEX 10KA (All)	√	√
FLEX EPF10K30E, EPF10K50S, EPF10K100E, EPF10K130E, EPF10K200S	√	√
EPF10K10, EPF10K10A, EPF10K20, EPF10K30, EPF10K30A	√	√
Excalibur (All)	√	
Excalibur EPXA1	√	
ACEX (All)	√	√
FLEX 6000 (All)	√	√

续表

器 件	Quartus II	MAX+PLUS II
MAX 9000 (All)		√
MAX 7000 (All)		√
MAX 7000S, MAX 7000B, MAX 7000AE	√	√
MAX 3000 (All)	√	√

2. 软件功能比较

Altera 设计软件功能对比如表 2.2 所示。

表 2.2 Altera 设计软件功能对比

设计流程	功 能	Quartus II	MAX+PLUS II
System-Level Design	SOPC Builder Support	√	
	DSP Builder Support	√	
Embedded Software	Embedded Software Integration	√	
FPGA Design	LogicLock Block-Based Design Methodology	√	
	HardCopy Support	√	
	Advanced Command Line Operation Feature Set	√	
	Tool Command Language (Tcl) Scripting and Synopsys Design Constraints (SDC) Support	√	
	NativeLink Integration	√	
	Text-Based Design Entry	√	√
	Schematic Design Entry	√	√
	Block Design Entry	√	√
	Library of Parameterized Modules (LPM)	√	√
	MegaWizard Plug-In Manager	√	√
	Megacore Functions	√	√
Synthesis	VHDL and Verilog Synthesis	√	√
	Third-Party Synthesis Support	√	√
Device Programming Standard Support	IEEE 1532	√	
	Jam STAPL	√	√
Place-and-Route	PowerFit Place-and-Route	√	
	Fast Fit	√	
	Timing Closure Flow	√	
	Timing Closure Floorplan Editor	√	
	Physical Synthesis	√	
	Design Space Explorer	√	
	Chip Editor	√	
	Incremental Fitting	√	

续表

设计流程	功 能	Quartus II	MAX+PLUS II
Verification	OpenCore Evaluation	✓	✓
	Static Timing Analysis	✓	✓
	Functional Simulation	✓	✓
	Timing Simulation	✓	✓
	ModelSim-Altera	✓	✓
	Testbench Generation	✓	
	SignalTap II Logic Analysis	✓	
	SignalProbe In-System Debugging	✓	
	IBIS Model Generation	✓	
	Hardware/Software Co-Simulation Support	✓	
	Formal Verification	✓	
	PowerGauge Power Analysis	✓	

第 3 章

EDA 设计仿真软件 ModelSim

ModelSim 是由 Mentor Graphics 下属公司 Model Technology 公司提供的数字系统仿真工具。ModelSim 提供了友好的调试环境，支持 VHDL 和 Verilog 混合仿真；采用直接优化的编译技术、Tcl/Tk 技术和单一内核仿真技术，编译仿真速度快；编译的代码与平台无关，便于保护 IP 核；个性化的图形界面和用户接口为用户加快调错提供了强有力的手段；并且全面支持 VHDL 和 Verilog 语言的 IEEE 标准，支持 C/C++ 功能调用和调试。因此，ModelSim 被广泛应用在 FPGA/ASIC 设计的 RTL 级和门级电路仿真中。

ModelSim 得到广泛使用的另一个原因是 Model Technology 公司为各个 PLD 厂家都提供了 OEM 版本的 ModelSim 工具，例如为 Altera 公司提供了 ModelSim Altera。

3.1 ModelSim 用户接口

ModelSim 是一个独立的仿真工具，它在工作时并不需要其他软件协助，在 Altera 公司的集成开发工具 Quartus II 中给 ModelSim 预留了接口，通过这种接口可以从 Quartus II 软件中直接启动 ModelSim 进行仿真。

用 Quartus II 建立新工程时，可以从 New Project Wizard（File 名称）中或者从 Settings 对话框（Assignments 菜单）的 EDA Tool Settings 标签页中选择 EDA 仿真工具。图 3.1 显示了 Settings 对话框的 EDA Tool Settings 标签页。

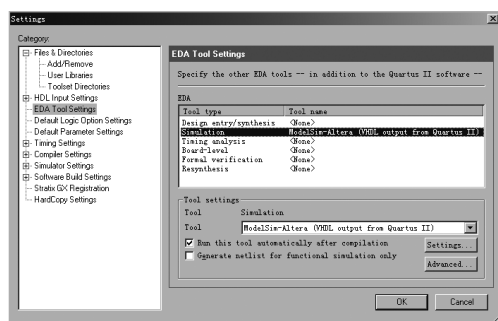


图 3.1 EDA Tool Settings 标签页

如图 3.1 所示，在 Tool 选项框中选择 ModelSim 仿真工具即可，具体的 ModelSim 版本和输出的语言则根据设计者的需要进行选择。

3. List（列表）窗口

列表窗口使用表格的形式显示仿真的结果。窗口被分为两个可调整的部分，右边为信号列表，左边为仿真运行时间以及仿真的 Delta 时间。可以从主窗口中创建列表窗口的第二个副本，两个列表窗口可以进行不同的设置，便于仿真结果的比较。

4. Process（进程）窗口

进程窗口显示了仿真中用到的所有进程的列表，如果在该窗口中选择 View > Active 命令，在窗口中将显示当前工程中的全部进程，若选择 View > In Region，在窗口中只显示当前层次中包含的进程（这里的层次通过结构窗口来指定）。

需要注意的是，如果要查看所有活动的进程，仿真时间必须停止在 0 ps。

5. Signals（信号）窗口

信号窗口主要用来选择需要查看的信号，它分为两部分，左边部分显示了当前层次的信号名称，这里的层次可以使用结构窗口或主窗口工作区的仿真视图（sim）来指定，在结构窗口中选择不同的层次，信号窗口中的信号将根据设计层次的变化而发生变化；信号窗口中的右边部分显示了信号的仿真值，这个值不会跟随波形窗口中指针的变化而变化，它表示当前仿真运行的最后结果。

如果在信号窗口中双击某一信号，那么源程序窗口中将会对相应的信号进行高亮显示。

信号窗口的另一个主要作用是在仿真过程中可以强制使某一个信号的值发生变化，也可以强制任一信号成为时钟信号。

6. Structure（结构）窗口

结构窗口中的信息用来显示设计的结构层次，可以在结构窗口中指定当前需要查看的设计层次。

7. Wave（波形）窗口

波形窗口是除主窗口之外最常用的窗口之一，通过波形窗口可以观察仿真的波形文件。波形窗口主要由三部分组成，如图 3.3 所示。从窗口的左边起第一部分为信号路径和名称；第二部分为指针所在位置信号的当前值，这个值跟随指针的变化而变化；第三部分为波形显示部分。

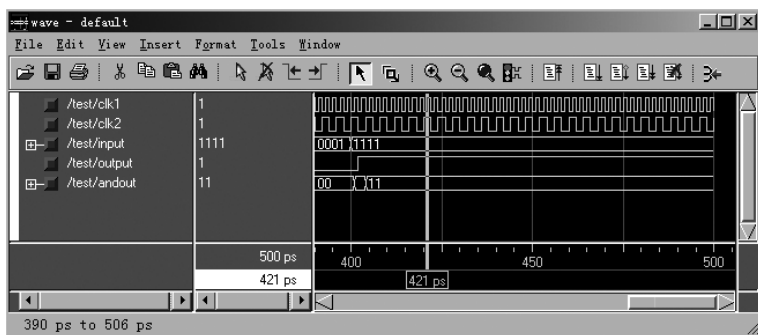


图 3.3 波形窗口

8. Source（源文件）窗口和 Variables（变量）窗口

原文件窗口主要用来显示和编辑 HDL 源文件代码。

变量窗口分为两部分，左边部分为当前进程中用到的所有变量，右边部分为与变量相关的当前值。

3.3 使用 ModelSim 进行仿真

在 3.1 节中提到，ModelSim 是一个独立的仿真工具，它的运行不需要依赖于其他的软件；而 Altera 公司的 EDA 工具 Quartus II 为 ModelSim 提供了接口。通常可以用三种方法启动 ModelSim 进行仿真。

1. 在 ModelSim 独立环境下进行仿真

在 ModelSim 独立环境下仿真，需要设计源文件和 testbench 测试激励文件。

(1) 新建工程

启动 ModelSim 后，依次单击 File > New > Project...，进入如图 3.4 所示的新建工程对话框，输入工程名称、工程保存目录及默认库名（一般选用系统默认库名“work”）。输入完成后单击“OK”按钮，弹出新建工程目录框，单击“确定”按钮。

ModelSim 自动弹出如图 3.5 所示的在工程中添加新项目对话框。选中 Add Existing File，打开添加文件窗口，如图 3.6 所示，先输入源文件的完整路径和文件名（或单击 Browse...，在打开的资源浏览器中查找），输入完毕后，单击“OK”按钮完成源文件添加。添加完源文件后，重复上述步骤添加 testbench 文件。

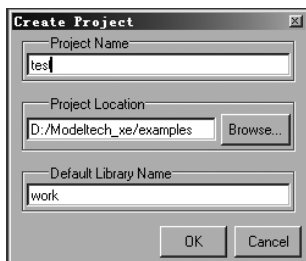


图 3.4 新建工程

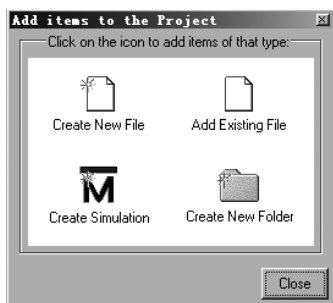


图 3.5 在工程中添加新项目

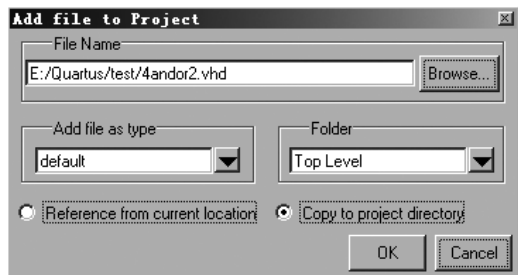


图 3.6 添加源文件

(2) 编译文件

在进行仿真之前，ModelSim 需要先对源文件的语法进行检查，并生成用于仿真使用的伪机器码。

新建工程并将源程序、testbench 文件添加到工程中后，在 ModelSim 主窗口工作区的 Project 标签页会出现当前工程使用的全部文件，如图 3.7 所示，选中全部文件，单击鼠标右键，选择 Compile > Compile All 命令，将文件全部进行编译。

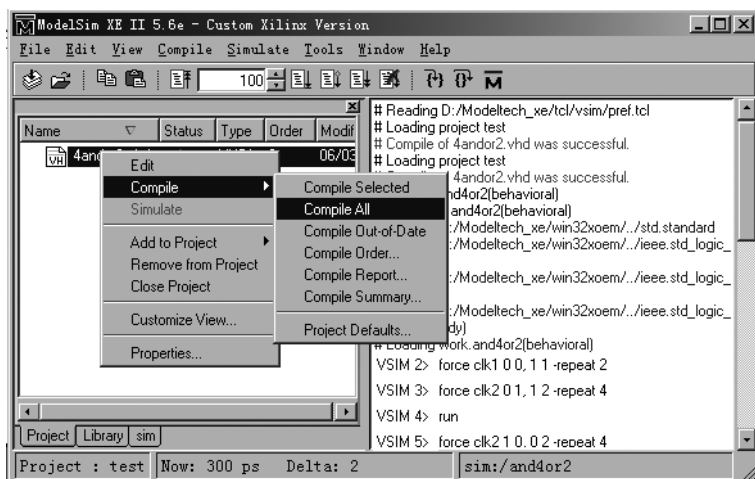


图 3.7 编译源文件

(3) 仿真

编译完成后，打开主窗口 Library 标签页的 Work 库，双击“testbench”，ModelSim 将自动加载仿真文件。

在主窗口中选择 View > All 命令，打开 ModelSim 的全部仿真窗口，然后从 Structure 窗口中选中需要观察的设计层次，按住鼠标左键将其拖动到 Wave 窗口中，然后在 Wave 窗口中单击“run”按钮，完成仿真，可从 Wave 窗口中观察仿真完后的信号波形。

2. 在 Quartus II 中启动 ModelSim 进行仿真

如前所述，Altera 公司的集成 EDA 工具 Quartus II 为 ModelSim 预留了接口，因此可以在 Quartus II 中启动 ModelSim 进行仿真。

以 Quartus II 10.0 为例，在 Quartus II 中启动 ModelSim 进行仿真的步骤如下：

(1) 新建工程，加入源文件，进行逻辑综合，直到综合能够通过。

(2) 选择 Assignments->Settings，在左边目录中选中 Simulation 选项，设置仿真工具为 ModelSim-Altera，选择输出网表格式为 VHDL，如图 3.8 所示。单击“OK”按钮完成设置。

(3) 选择 Processing->start->Start Test Bench Template Writer，等待完成后打开自动生成的 testbench 文件，默认保存在 Simulation \ Modelsim 文件夹下，可以看到 Quartus II 已经为我们完成了一些基本工作，包括端口部分的代码和接口变量的声明，我们要做的就是在这个做好的模具里添加所需要的测试代码。完成测试程序后单击“Save”按钮。

(4) 打开图 3.8 所示的页面，在 NativeLink settings 下选择 Compile test bench，单击右侧的“Test Benches...”按钮加入测试文件。在弹出的对话框中选择 New...，如图 3.9 所示，

在 Test bench and simulation files 栏中将上一步生成的 testbench 文件添加进来；填入 Test bench name 栏（这里是 test_vhd_tst）；酌情设置仿真运行的时间（此处设为 100ns）；单击“OK”按钮完成设置。

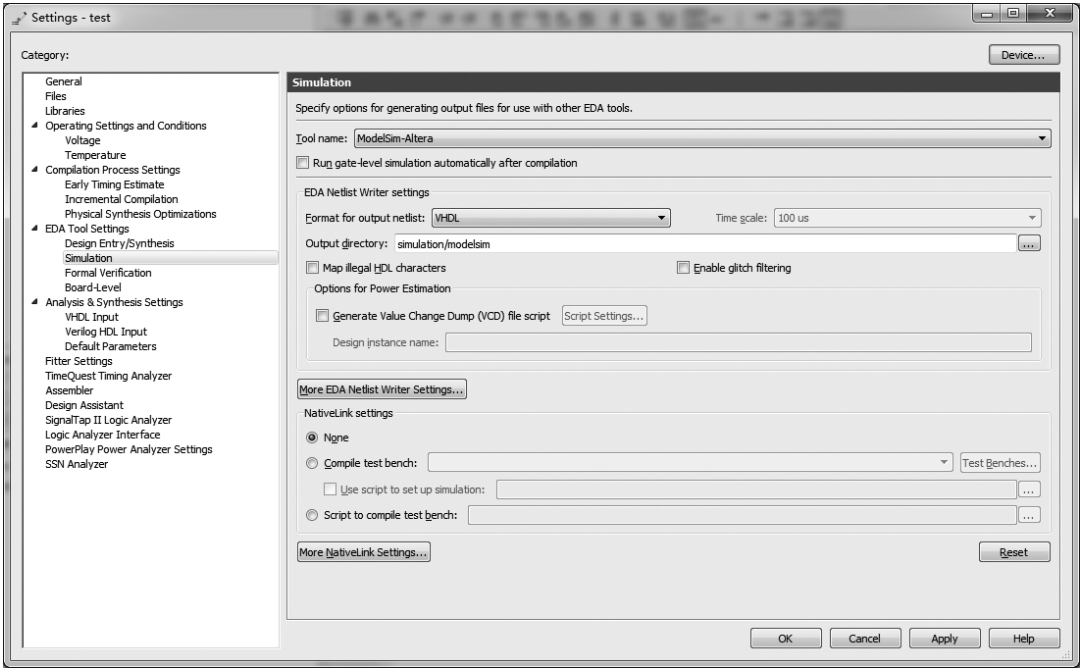


图 3.8 设置仿真工具

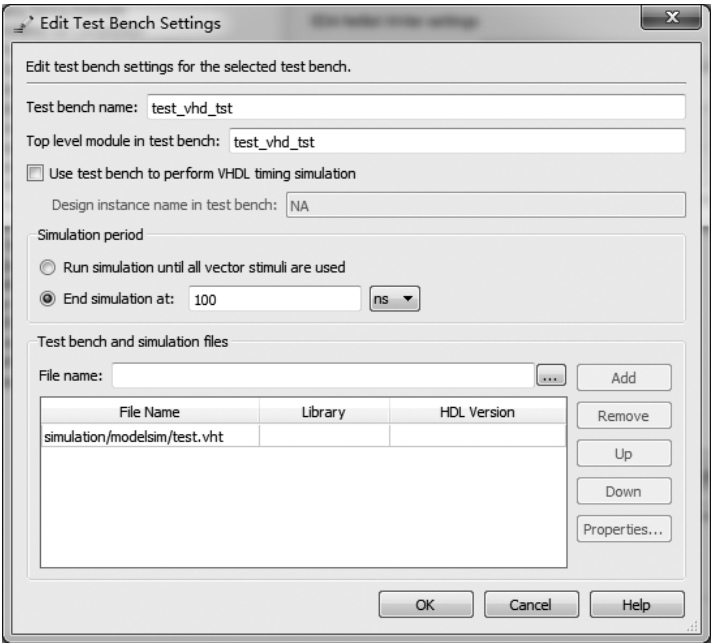


图 3.9 关联仿真文件

(5) 选择 Tools→Run Simulation Tool→RTL Simulation, 自动进入 Modelsim 界面并在波形窗口显示仿真结果, 如图 3.10 所示。

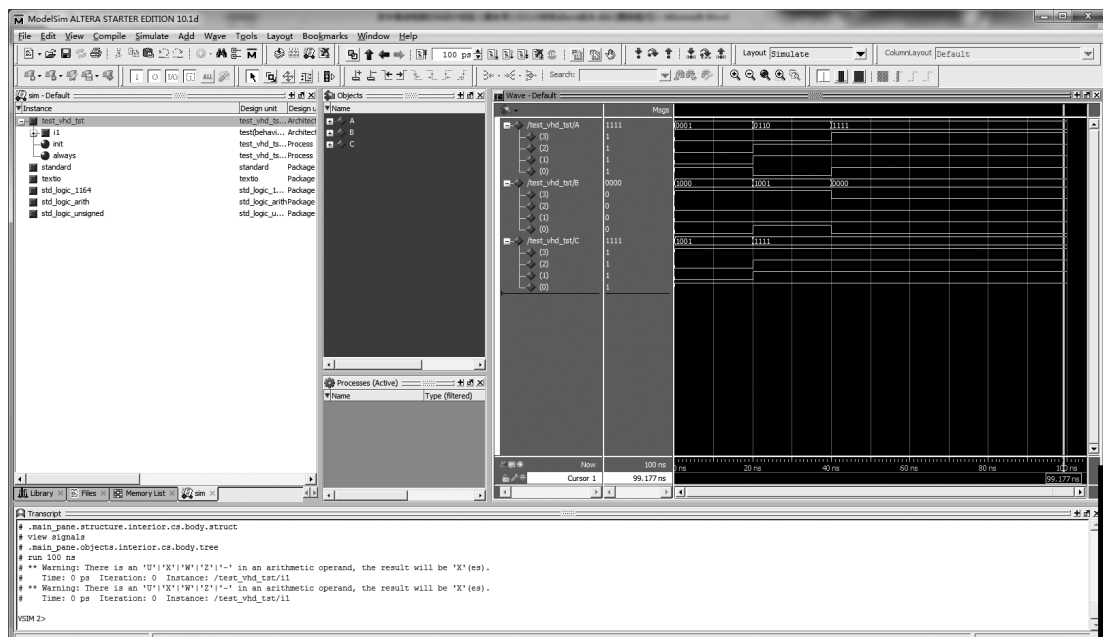


图 3.10 启动 ModelSim 得到 RTL 仿真结果

3. 在 ModelSim 中使用命令

上述两种方法都使用了 testbench 文件来控制仿真时的激励波形, 其实 ModelSim 也支持直接在主窗口中输入激励信号命令, 常用的命令如下:

(1) view <窗口名>

打开某个窗口, 若窗口名为 *, 则打开 ModelSim 的所有窗口。

(2) add <窗口名> <信号名>

将某个信号添加到某个窗口中 (Dataflow、List、Log 或 Wave 中的一个), 并自动打开该窗口。如:

add wave * (将顶层模块的所有信号添加到波形窗口中)

(3) force<信号名> <值> [<时间>] [, <值> <时间> ...] [-repeat <周期>]

强制赋值命令, 方括号中的为可选项, 例如:

force a 0011 (强制信号 a 的值为 0011)

force b 10 0, 5 200 (强制信号 b 在 0 时刻值为 10, 在 200 时刻值为 5)

force clk 0 0, 1 15 - repeat 20 (clk 为周期信号, 周期为 20, 低电平保持 15, 高电平保持 5)

(4) run [值]

运行命令, 例如:

run (运行默认时间, 在主窗口的 Simulate > Simulation Options... 中修改)

run 100ns (运行 100ns)

run 10000 (运行 10000 个单位时间)

(5) restart

使仿真回到起始点。

(6) quit

退出 ModelSim。

第 4 章

EDA 设计硬件平台

本章将简单介绍一款基于 Altera 公司 Cyclone IV FPGA 的 EDA 设计硬件平台 DE2-115, 该实验平台是台湾友晶公司为高等院校学生学习 FPGA 编程和学习基于 NIOS II 软核处理器应用项目编程而开发的硬件实验平台。DE2-115 开发平台所配备的 Cyclone IV EP4CE115 为 Cyclone IV FPGA 系列之最大器件, 此芯片具有 114480 个逻辑单元 (LE)、高达 3.9 Mbits 的随机存储器、内嵌的 266 个乘法器及低功耗等特质。DE2 系列平台位居于国内外 FPGA 教育开发平台的领先地位。因其拥有适应各种应用需求的丰富接口及工业等级的设计资源, 成为全球近千所高校实验室中的首选。第 6 章中的所有 EDA 实验都能在这款 EDA 实验硬件平台上实现。

4.1 DE2-115 平台简介

DE2-115 开发板的正面和背面全貌如图 4.1 和图 4.2 所示, 给出了硬件实验平台的布局, 并标注出了主要连接器和关键部件的位置。

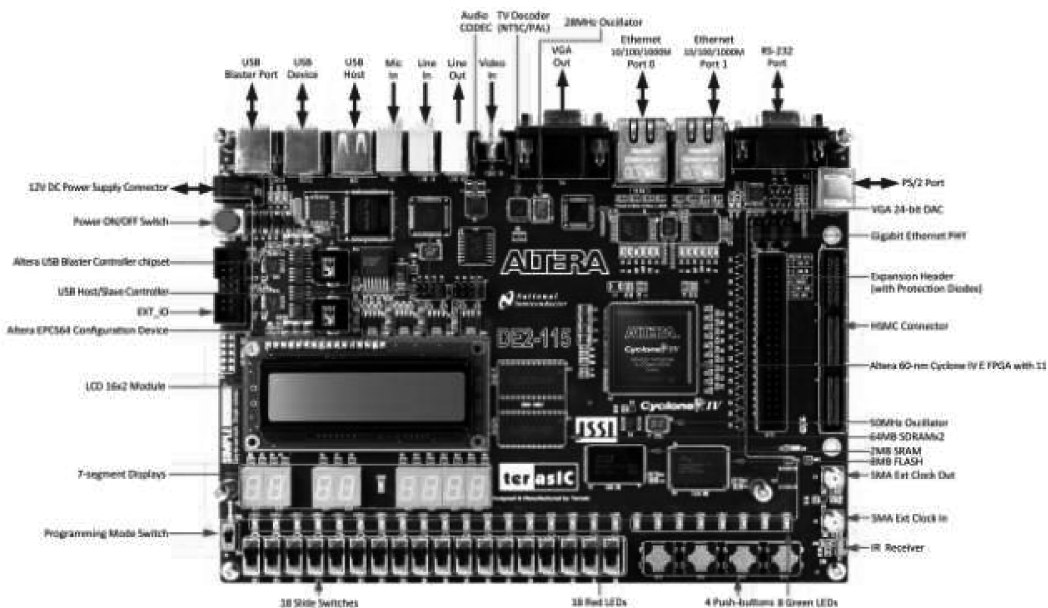


图 4.1 DE2-115 开发板 (正面)

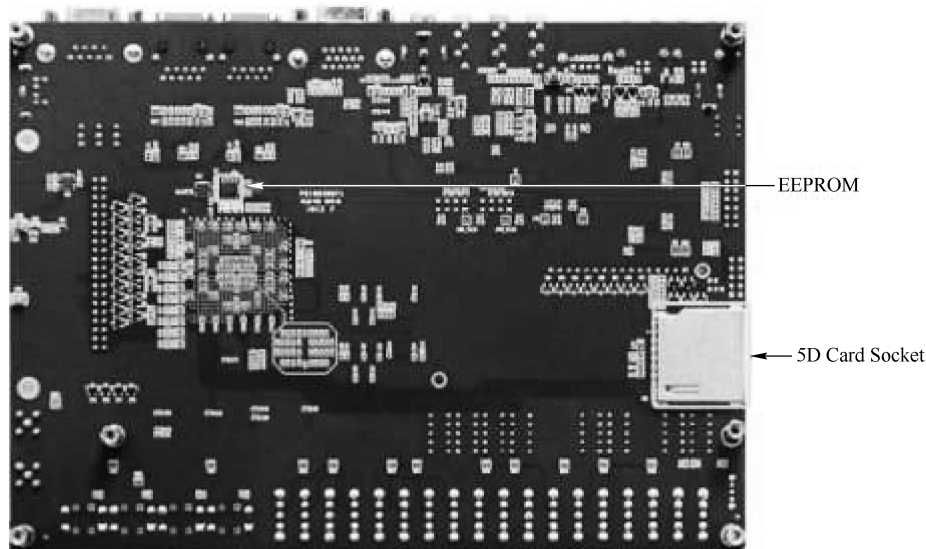


图 4.2 DE2-115 开发板（背面）

DE2-115 实验硬件平台为我们进行 EDA 设计实验提供了广泛的用于实现从简单的电路设计到各种多媒体电路设计的特征，接口部分除了承袭 DE2 系列多媒体开发平台界面，还新增了接口以支持包括千兆以太网（GbE）在内的主流协议。HSMC 插槽提供了通过 HSMC 子卡和电缆的附加功能和连接。亦可通过 HSMC 电缆将多片的 FPGA 板上透过 HSMC 插槽连接起来，以实现大型的 ASIC 原型开发的需求。从图 4.1 和图 4.2 可以看出，DE2-115 实验平台包括以下硬件资源：

- Altera Cyclone® IV 4CE115 FPGA 器件
- Altera 系列配置——EPCS64
- 板上 USB Blaster 用于编程，同时支持 JTAG 模式和 AS 模式
- 2MB SRAM
- 2 片 64MB SDRAM
- 8MB 闪存
- SD 卡插槽
- 4 个按钮开关
- 18 个滑动开关
- 18 个红色 LED
- 9 个绿色 LED
- 50MHz 晶振提供给时钟源
- 24-bit CD-品质声道 CODEC 带有线路输入，线路输出和麦克风输入接口
- VGA DAC（8-bit 高速三通道 DACs）带有 VGA 输出接口
- TV 解码器（NTSC/PAL/SECAM）和 TV 输入接口
- 2 千兆以太网 PHY 带 RJ45 连接器
- 带有 A 类和 B 类 USB 接口的 USB 主从控制器
- RS-232 收发器和 9 针连接器

- PS/2 鼠标/键盘接口
- IR 收发器
- 2 个 SMA 接头, 用于外部时钟输入/输出
- 1 个 40 针扩展口, 带二极管保护
- 1 个 HSMC 连接器
- 16X2 LCD 模组

除了上述硬件结构, DE2-115 开发板还支持标准 I/O 接口, 并提供用于验证 DE2-115 开发板各项组件的软件工具。

4.2 DE2-115 平台主要组件

DE2-115 的系统框图如图 4.3 所示, 整个实验平台以 Cyclone IV E FPGA 器件为核心, 各项主要组件都与它连接, 为我们进行 EDA 设计实验提供极大的便利性, 可以通过 FPGA 来充分实现系统设计。

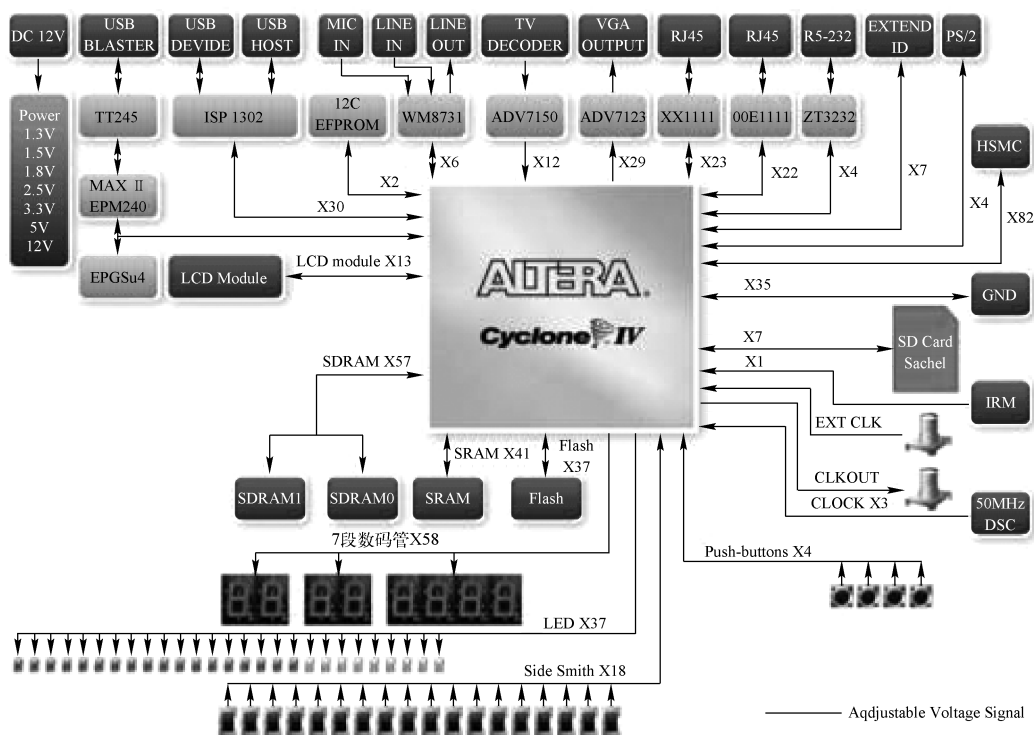


图 4.3 DE2-115 系统框图

DE2-115 实验平台根据功能不同, 可分为 FPGA 器件区、开关与按键区、数码管与彩灯区、PAC 适配板区、单片机扩展区、IC 扩展实验区、引脚接线区、A/D 及 D/A 转换区、电源区、频率选择/输出区和接口区。接下来对图 4.3 中各功能区块进行简单介绍。

(1) FPGA 器件区

该区是整个 EDA 实验平台的核心，主要提供 Altera 公司的 Cyclone IV E FPGA 器件，用以实现基于 FPGA 的芯片设计。该 FPGA 器件具有：

- Cyclone IV EP4CE115F29 器件
- 114480 个逻辑单元
- 432 M9K 内存模块
- 3888 kbits 嵌入式存储器位
- 4 个锁相环

(2) FPGA 器件配置区

该区位于 DE2-115 实验平台背面，主要提供针对 FPGA 器件的编程功能，同时支持 JTAG 模式和 AS 模式，提供系列配置器件——EPCS64，内建 USB Blaster 电路。

(3) 存储器区

该区主要提供一些外部存储器，主要包括：

- 128MB (32Mx32bit) SDRAM
- 2MB (1Mx16) SRAM
- 8 位 8MB (4Mx16) Flash 存储器 8Mx4x8 位闪存
- 32kb EEPROM
- SD 卡接口，提供 SPI 模式 和 4 位 SD 模式用于 SD 卡接入

(4) 连接器（接口）区

该区主要包括：

- 2 个 10/100/1000 以太网接口
- 1 个 HSMC
- 可配置的 I/O 标准（电平：3.3/2.5/1.8/1.5V）
- A 型和 B 型 USB 接口
- 40 针扩展口
- 可配置的 I/O 标准（电平：3.3/2.5/1.8/1.5V）
- VGA 输出接口
- VGA DAC（三通道高速视频 DAC）
- 提供带有流控制的 RS-232 端口的 DB9 连接器
- 提供 PS/2 鼠标/键盘 连接器

(5) 时钟输入区

该区主要包括：

- 3 个 50MHz 晶振
- 1 个 SMA 外部时钟输入

(6) 音频区

- 24 位编码器/解码器 CODEC
- 包括线路输入、线路输出和麦克风输入

(7) 显示及遥控区

该区主要包括：

- 16x2 LCD 模组
- 红外遥控接收模块
- TV 解码器 (NTSC/PAL/SECAM 制式) 和 TV-in 连接器

(8) 开关和数码管区

该区主要包括:

- 18 个滑动开关和 4 个按钮开关
- 18 个红色和 9 个绿色 LED
- 8 个七段显像管

(9) 电源区

该区主要包括:

- 直流电源输入
- 开关和降压调节器 LM3150MH

4.3 DE2-115 平台自检测试

使用实验平台之前, 需要快速验证实验平台是否工作正常, DE2-115 实验平台带有一个出厂默认配置数据平台大部分功能。DE2-115 实验平台开机步骤如下:

1. 用包装盒里的 USB 电缆将 PC 的 USB 端口和 DE2-115 开发板的 USB Blaster 连接器连接起来。为了实现主机和开发板之间的通信, 必须安装 USB Blaster 驱动软件。如该驱动还没有安装在主机上, 可以参考文档 “Getting Started with Altera’s DE2-115 Board” (tut_initialDE2-115. pdf)。该文档可以在 DE2-115 系统 CD 目录 DE2_115_tutorials 中获取。

2. 在将 12V 的适配器连接到 DE2-115 主板前, 按下红色的 ON/OFF 开关以确保电源断开。

3. 将一个 VGA 显示器连接到 DE2-115 开发板的 VGA 接口上。

4. 将耳机插入 DE2-115 主板上的音频输出端口。

5. 将 DE2-115 主板左侧的 RUN/PROG 开关 (SW19) 调到 RUN 状态; PROG 状态仅适用于 AS 模式的编程。

6. 按下 DE2-115 主板上红色的电源开关 OFF /ON 重新上电。

此时, 应该观察到以下现象:

① 所有的 LED 都在闪烁。

② 7 段显像管都是循环显示 0 到 F。

③ LCD 显示 “Welcome to the Altera DE2-115”。

④ VGA 显示器显示图 4.4 中的图像。

⑤ 将滑动开关 SW17 拨到 DOWN 位置; 将会听到一种 1kHz 的声音。请注意, 默认输出的音量较大, 为了避免任何损伤, 请将耳机音量调至最低。

⑥ 将滑动开关 SW17 拨到 UP 位置, 并将音频播放器输出端口插到 DE2-115 主板的线路输入连接器, 通过话筒和耳机, 将会听到从该音频播放器 (MP3、PC、iPod 或类似的设备) 中播放出来的音乐。

⑦ 可以将麦克风接入 DE2-115 开发板的麦克风输入插口, 你的声音就会和音频播放器的音乐混合到一起了。



图 4.4 DE2-115 开机默认的 VGA 输出画面

4.4 使用 DE2-115 平台

4.4.1 配置 Cyclone IV E FPGA 芯片

通常有三种方式将网表文件下载到 FPGA。

(1) JTAG 方式：该方式的名字起源于 IEEE 标准——联合测试行动组。下载 sof 文件直接加载到 Cyclone IV E FPGA 芯片，掉电丢失。也可以将 sof 文件转换为 jic 文件，用 EPCSx 配置，掉电不丢失。

(2) AS 方式：该方式被称为作串行主动编程，它会将配置数据下载到 Altera EPCS64 芯片。它将配置数据保存在非易失性器件中，即使 DE2-115 开发板掉电，数据也不会丢失。每当开发板上电时，EPCS64 芯片里的数据会自动加载到 Cyclone IV E FPGA 芯片，下载的文件类型是 pof 文件。

(3) PS 方式：较陈旧的下载方式，很少使用。

DE2-115 硬件实验平台包含一个存储有 Cyclone IV E FPGA 芯片配置数据的串行配置芯片。每当开发板上电时，芯片中的配置数据会自动从配置芯片加载到 FPGA 芯片。使用 Quartus II 软件，用户可以随时重新配置 FPGA，并可以改变存储在非易失性串行存储器芯片（EPCS）中的数据。

下面介绍 DE2-115 硬件实验平台上的 JTAG 链。

当使用 JTAG 接口配置 FPGA 芯片时，DE2-115 上的 JTAG 链必须形成一个回路，这样 Quartus II 软件才可以正确检测到 JTAG 链上的 FPGA/CPLD 器件。图 4.5 给出了 DE2-115 硬件实验平台上的 JTAG 链。短接 JP3 上的第 1、2 引脚会旁路 HSMC 接头的 JTAG 信号，直接在 DE2-115 上形成 JTAG 回路，这样只有 DE2-115 上的 FPGA 器件（Cyclone IV E）才可以被 Quartus II 软件检测到。如果用户想通过 HSMC 接头包含其他的 FPGA 器件或者包含

FPGA 器件的界面，短路 JP3 的第 2、3 引脚，从而使能 HSMC 接头上的 JTAG 链。JP3 在 DE2-115 硬件实验平台上的位置如图 4.6 所示。

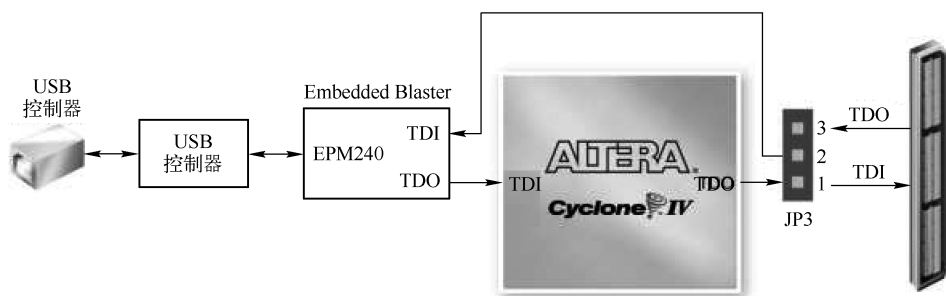


图 4.5 DE2-115 平台 JTAG 链

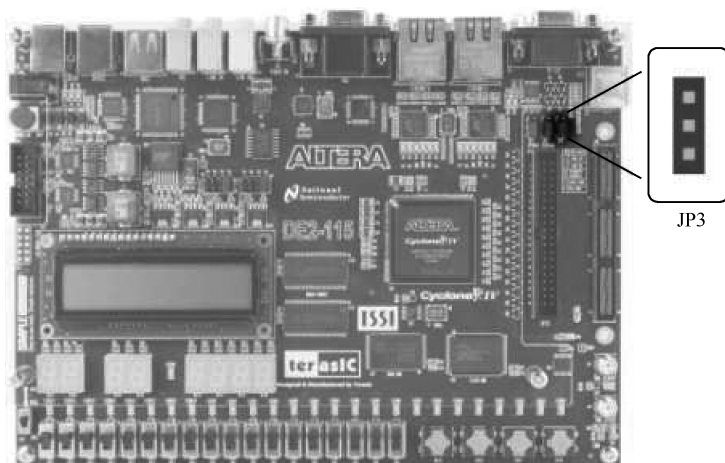


图 4.6 JTAG 链配置跳线 JP3

以下是关于 JTAG 和 AS 编程的操作步骤。对于这两种方式，DE2-115 硬件实验平台均通过 USB 电缆连接到 PC 主机。通过这种连接，主机将开发板当作一个 Altera USB-Blaster 设备。在主机上安装 USB-Blaster 驱动的步骤在文档“Getting Started with Altera’s DE2-115 Board”（[tut_initialDE2-115.pdf](#)）中有详细描述。该文档可以在 DE2-115 的系统光盘中找到。

4.4.2 测试 DE2-115 组件

除了 DE2-115 开机自检程序，DE2-115 开发板附带的控制面板软件允许用户经由 PC 主机访问板上的各种组件。主机通过 USB 与开发板通信。该软件可以用来测试开发板上的各组件功能或作为开发 RTL 代码时的调试工具。

控制面板应用软件位于 DE2-115 系统 CD 中的 DE2_115_tools\DE2_115_control_panel 目录下，无须安装，只要将整个目录复制到用户电脑上并运行 DE2_115_ControlPanel.exe 即可。

注意，对于 Windows 7 64 位的用户，如果在启动控制面板软件时弹出“a missing jtag_

client.dll file (cannot find jtag_client.dll)” 的报错，则需从目录/DE2_115_tools\DE2_115_control_panel\win7_64bits 重新运行 DE2-115_ControlPanel.exe)。

在控制面板软件要求 FPGA 开发板运行指定的任务之前，特定的控制电路将会下载至开发板中。该应用程序将自动启用 Quartus II 工具经由 USB Blaster [USB-0] 连接下载控制电路至 FPGA 开发板。运行如下步骤以激活控制面板应用程序：

1. 确认 Quartus II 10.0 或者更高的版本已成功安装至电脑。
2. 将 RUN/PROG 开关置于 RUN 档。
3. 将提供的 USB 电缆线接到 USB Blaster 端口，连接 12V 电源，然后开启 DE2-115 电源开关。
4. 启动主机上的 DE2_115_ControlPanel.exe 程序，控制面板的用户界面就会出现，如图 4.7 所示。
5. 一旦启动 DE2_115_ControlPanel.exe 程序，DE2_115_ControlPanel.sof 比特流将自动加载至开发板。
6. 如果界面上显示未连接的状态，单击“CONNECT”按钮，.sof 文件将重新加载至开发板。
7. 需要注意的是，控制面板将会一直占用 USB 端口直至关闭这些端口；在未关闭控制面板的 USB 端口之前，将不能使用 Quartus II 下载配置文件至 FPGA。
8. 控制面板程序已安装完成，用户可以设置一些 LED 的 ON/OFF 状态并观察 DE2-115 开发板上的显示结果。



图 4.7 DE2-115 控制面板

DE2-115 控制面板示意图如图 4.8 所示。实现控制功能的“控制电路”将在 FPGA 开发板内运行。它将通过 USB Blaster 连接与主机上的 Control Panel 窗口进行通信。GUI 界面发出命令到控制电路。它将会处理所有的应用需求并实现电脑与 DE2-115 开发板之间的数据传输。

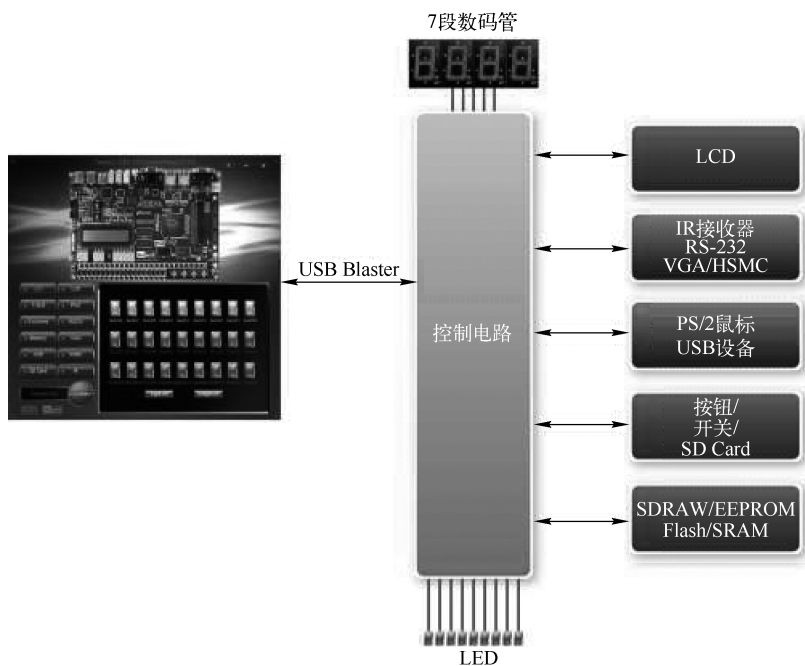


图 4.8 DE2-115 控制面板示意图

DE2-115 控制面板可以用来点亮 LED 灯，改变显示在 7 段数码管和 LCD 显示器上的数值，监测按钮/开关的状态，读/写 SDRAM、SRAM、EEPROM 和 Flash 内存，监测 USB 设备的状态，与 PS/2 鼠标通信，输出 VGA color pattern 至 VGA 显示器，验证 HSMC I/O 的功能，经由 RS-232 接口与 PC 通信和读取 SD 卡规格信息。读取/写入一个字或整个文件的功能允许用户进行多媒体应用的开发（Flash 音频播放器、Flash 图片查看器）而不用担心如何创建一个内存编程。

VHDL 基础语法

美国国防部于 1982 年开发的 VHDL（Very - High - Speed Integrated Circuit Hardware Description Language）语言，是当前广泛使用的 HDL 语言之一，并被 IEEE 和美国国防部采用为标准的 HDL 语言。VHDL 语言的主要特点包括：设计技术齐全，方法灵活，支持广泛；系统硬件描述能力强；编程实现与工艺无关；语言标准、规范，易于共享和复用等。VHDL 语言作为一种标准的硬件描述语言，在语句结构上吸取了 PASCAL、Fortran 和 C 等计算机高级语言的语句，如 IF 语句、FOR 循环语句、函数和子程序等，只要具备高级语言的编程技能和数字逻辑电路的设计基础，就可以在较短的时间内学会 VHDL 语言。

5.1 VHDL 程序基本结构

VHDL 是一种描述数字电路的工业标准语言，该语言在标识符号、数据类型、数据对象以及描述各种电路的语句形式和程序结构等方面具有特殊的规定，为了方便学习，本节先介绍整体的 VHDL 程序结构，再逐步介绍程序中的语法概念。

一个 VHDL 语言的设计程序描述的是一个电路单元，该电路单元可以是一个门电路，或者是一个计数器，也可以是一个 CPU。一个完整的 VHDL 程序由以下五部分组成：（1）库（LIBRARY），储存预先写好的程序和数据的集合；（2）程序包（PACKAGE），声明在设计中将用到的常数、数据类型、元件及子程序；（3）实体（ENTITY），声明电路单元名称及其到其他实体或其他设计的接口，即定义电路单元的输入/输出端口；（4）结构体（ARCHITECTUR），定义实体的实现，电路的具体描述，给出电路单元的内部结构和信号的行为特点；（5）配置（CONFIGURATION），一个实体可以有多个构造体，可以通过配置来为实体选择其中一个构造体。对于初学者，大多数情况下，一个 VHDL 语言程序只包含程序包、实体和结构体三个部分。VHDL 语言程序各个部分的关系如图 5.1 所示。

一个简单非门的 VHDL 程序的文件名称是 notgate. VHD，其中 VHD 是 VHDL 程序的文件扩展名，程序如下：

```
-- *****  
--NOT Gate Simulation  
--Filename:NOTGATE  
-- *****
```

} 注释栏

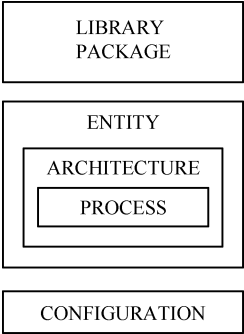


图 5.1 VHDL 语言程序各个部分的关系

```

LIBRARY IEEE;  --IEEE 标准库
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY notgate is
    port (
        A:in STD_LOGIC;
        F:out STD_LOGIC;
    );
End notgate ;
ARCHITECTURE notgate_arch of notgate is
BEGIN
    F<= not  A
ENDnotgate_arch

```

库程序包

实体

结构体

从此例中可以看出，大多数情况下，一段完整的 VHDL 代码至少由以下三个部分组成：

第一部分是程序包（PACKAGE），程序包是用 VHDL 语言编写的共享文件，定义在设计结构体和实体中将用到的常数、数据类型、子程序和设计好的电路单元等，放在文件目录名称为 IEEE 的程序包库中。

第二部分是程序的实体（ENTITY），定义电路单元的输入/输出引脚信号。程序的实体名称为 notgate，是任意取的，但是必须与 VHDL 程序的文件名相同。实体的标识符是 ENTITY，实体以 ENTITY 开头，以 END 结束。其中，定义 A 是输入信号引脚，定义 F 是输出信号引脚。

第三部分是程序的结构体（ARCHITECTURE），具体描述电路的内部结构和逻辑功能。结构体有三种描述方式，分别是行为（BEHAVIOR）描述、数据流（DATAFLOW）描述和结构（STRUCTURE）描述，其中数据流描述方式又称为寄存器（RTL）描述方式，本例中结构体的描述方式属于数据流描述方式。结构体以标识符 ARCHITECTURE 开头，以 END 结尾。结构体的名称 notgate_arch 是任意取的。

小提示：VHDL 每条语句以分号“;”作为结束符，并且 VHDL 对空格是不敏感的，所以符号之间空格的数目是可以自己设定的。可以按自己的习惯任意添加，以增强代码可读性。

5.1.1 VHDL 语言的实体说明语句

实体是 VHDL 程序设计中最基本的组成部分，实体说明主要描述的是一个设计的外貌，即对外的输入/输出端口以及一些用于结构体参数的定义。端口信号名称表示芯片的输入/输出信号的引脚名，这种端口信号通常被称为外部信号，信号的输入/输出状态被称为端口模式，在实体中还定义信号的数据类型。实体类似于电路原理图中的一个部件符号，它并不描述设计的具体功能，只是定义所需的全部输入/输出信号。

实体说明语句的格式为

```

ENTITY 实体名称 IS
GENERIC (
    常数名称 1:类型 [=缺省值];
    常数名称 2:类型 [=缺省值];
    ...
    常数名称 N:类型 [=缺省值];
);
PORT (
    端口信号名称 1:输入/输出状态数据类型;
    端口信号名称 2:输入/输出状态数据类型;
    ...
    端口信号名称 N:输入/输出状态数据类型
);
END 实体名称;

```

小提示: VHDL 语言具有 87 标准和 93 标准两种格式, 以上为 VHDL87 标准, 对于 93 标准, 要使用 END ENTITY 实体名称; 结束实体。注意, 为了保证代码的可综合性与通用性, 最好采用 87 标准的 VHDL 格式, 有些 EDA 工具不一定支持 93 标准的 VHDL 语言格式 (Quartus II 支持 VHDL93、VHDL87 标准)。类属 GENERIC 常用来定义实体端口大小、数据宽度、元件例化数目等。在简单的设计中一般不常用。

例 5.1 一个同步十六进制加法计数器, 带有计数控制、异步清零和进位输出等功能。

电路有 3 个输入端和 5 个输出端, 分别是时钟脉冲输入端 (CLK), 计数器状态控制端 (EN), 异步清零控制端 (Rd), 四位计数输出端 (Q0、Q1、Q2、Q3) 和一个进位输出端 (Co)。当计数器输出 0000~1110 时, Co=0, 只有当计数器输出 1111 时, Co=1。

该设计的实体部分如下:

```

ENTITY cntm16 IS
PORT (
    EN : IN STD_LOGIC;
    Rd : IN STD_LOGIC;
    CLK : IN STD_LOGIC;
    Co : OUT STD_LOGIC;
    Q : BUFFER STD_LOGIC_VECTOR (3 DOWNT0 0)
);
END cntm16;

```

需要说明的几点如下:

(1) 实体名称表示所设计电路的电路名称, 必须与 VHDL 文件名相同, 实体名称是“cntm16”, 所存的 VHDL 文件名必须是“cntm16.VHD”。

(2) 端口信号名称可以表示一个信号, 也可以表示一组信号 (BUS), 由数据类型定义, 如 EN、Rd、CLK、Co 分别表示计数允许信号、异步清零信号、时钟输入信号和进位输出信号, Q 是一组输出信号, 用来表示四位同步二进制计数器的四位计数输出信号。

(3) 端口信号输入/输出状态有以下几种:

- IN——信号进入电路单元。
- OUT——信号从电路单元输出。
- INOUT——信号是双向的,既可以进入电路单元也可以从电路单元输出。
- BUFFER——信号从电路单元输出,同时在电路单元内部可以使用该输出信号。

小提示: OUT 与 BUFFER 信号的区别在于信号是否往内部有反馈,将输出端口定义为 BUFFER 型,可以省去一个用于中间运算的临时信号,但是并不推荐这样做。

(4) 端口数据类型 (TYPE) 定义端口信号的数据类型,在 VHDL 中,常用的端口信号数据类型如下:

- 位 (BIT) 型:表示一位信号的值,可以取值 ‘0’ 和 ‘1’,放在单引号里面表示,如 $X \leq '1'$, $Y \leq '0'$ 。
- 位向量 (BIT_VECTOR) 型:表示一组位型信号值,在使用时必须标明位向量的宽度 (个数) 和位向量的排列顺序,例如: $Q : OUT BIT_VECTOR(3 \text{ downto } 0)$,表示 Q3、Q2、Q1、Q0 四个位型信号。位向量的信号值放在双引号里面表示,例如 $Q \leq "0000"$ 。
- 标准逻辑位 (STD_LOGIC) 型:IEEE 标准的逻辑类型,它是 BIT 型数据类型的扩展,可以取值 ‘U’、‘X’、‘0’、‘1’、‘Z’、‘W’、‘L’、‘H’、‘-’ 等。
- 标准逻辑位向量 (STD_LOGIC_VECTOR) 型:IEEE 标准的逻辑向量,表示一组标准逻辑位型信号值。

VHDL 是与类型高度相关的语言,不允许将一种数据类型的信号赋予另一种数据类型的信号。除了上述介绍的数据类型,还有其他多种数据类型用于定义内部信号和变量,请参见 5.2 节。

小提示: 相同类型 (模型相同,数据类型相同) 的端口可以写在同一行,如:

```
ENTITY cntm16 IS
PORT (
  EN, Rd,CLK : IN STD_LOGIC;
  Co : OUT STD_LOGIC;
  Q : BUFFER STD_LOGIC_VECTOR(3 DOWNT0 0)
);
END cntm16;
```

此外,注意最后一个端口结尾没有分号!

5.1.2 VHDL 语言的结构体

结构体 (ARCHITECTURE) 是 VHDL 程序设计中的核心组成部分,用于描述设计单元的具体结构和功能,在程序中,结构体放在实体的后面。每一个结构体都有名称,结构体的名称是由设计者任意取的,结构体是以标识符 ARCHITECTURE 开头,以 END 结尾。所有能被仿真的实体都由结构体描述,一个实体可以有多个结构体,每个结构体分别代表该实体功能的不同实现方案。结构体可以有三种描述方式:1) 行为描述,反映一个设计的功能和算法,一般使用进程 PROCESS,用顺序语句表达;2) 结构描述,反映一个设计硬件方面的特

征，表达了内部元件间的连接关系，使用元件例化来描述；3) 数据流描述又称为寄存器 (RTL) 描述，反映一个设计中数据从输入到输出的流向，使用并行语句描述。不同的结构体采用不同的描述语句。

结构体的一般格式为

```
ARCHITECTURE 结构体名 OF 实体名称 IS
说明语句
BEGIN
电路描述语句
END 结构体名；
```

结构体说明语句是对结构体中用到的数据对象的数据类型、元件和子程序等加以说明。结构体说明语句定义的信号、常数等只能在该结构体内部使用。若希望用于其他实体或结构体中，则需将其作为程序包处理。信号的定义和实体说明中的端口说明一样，应该有信号名和数据类型说明。不需要方向说明。

电路描述语句是用并行语句来描述电路的各种功能，这些并行语句包括并行信号赋值语句、条件赋值 (WHEN-ELSE) 语句、进程 (PROCESS) 语句、元件例化 (COMPONENT MAP) 语句、块语句和子程序调用语句等。

小提示：结构体中定义的参数 (信号、变量等) 名称不能与其所属实体的端口重名。

结构体的结束语句也可以写成 END ARCHITECTURE 结构体名，或者简写为 END。

例 5.2 设计程序的结构体部分如下：

```
ARCHITECTURE counstr OF cntm16 IS
BEGIN
Co <= .1.WHEN (Q = "1111" AND EN = .1.) ELSE .0.;    --条件赋值语句
PROCESS (CLK, Rd)    --PROCESS 语句
BEGIN
IF (Rd=.0.) THEN    --IF 语句
Q<= "0000";
ELSIF (CLK. EVENTAND CLK=.1.) THEN    --CLK 上升沿计数
IF (EN=.1.) then
Q <= Q+1;
END IF;
END IF;
END PROCESS;
END counstr;
```

结构体的名称是 counstr，该结构体属于行为描述方式，采用多种描述语句，如进程 (PROCESS) 语句、条件赋值语句 (WHEN-ELSE)、顺序语句 (IF-ELSE) 等。

5.1.3 程序包 (PACKAGE)、库 (LIBRARY) 和 USE 语句

程序包定义了一组标准的数据类型说明、常量说明、元件说明、子程序说明和函数说明等，它是一个用 VHDL 语言描写的一段程序，可以供其他设计单元调用。它如同 C 语言中

的 *.H 文件，定义了一些数据类型说明和函数说明。在一个设计单元中，在实体部分所定义的数据类型、常数和子程序在相应的结构体中是可以被使用的（即可见的），但是在一个实体的说明部分和结构体部分中定义的数据类型、常量及子程序却不能被其他设计单元的实体和结构体使用（即不可见）。程序包就是为了使一组类型说明、常量说明和子程序说明对多个设计单元都可以使用而提供的一种结构。程序包分为两大类，即 VHDL 预定义标准程序包和用户定义的程序包。VHDL 设计中常用的标准程序包的名称和内容如表 5.1 所示。用户定义的程序包是设计者把预先设计好的电路单元设计定义在一个程序包中，放在指定的库中，以供其他设计单元调用，如果在设计中要使用某个程序包中的内容，可以用 USE 语句打开该程序包。有关程序包的设计方法参见 5.4.5 节的内容。

表 5.1 IEEE 两个标准库 STD 和 IEEE 中的程序包

库名	程 序 包 名	定义的内容
STD	STANDARD TEXTIO	定义 VHDL 的数据类型，如 BIT、BIT_VECTOR 等 TEXT 读写控制数据类型和子程序等
IEEE	STD_LOGIC_1164	定义 STD_LOG、STD_LOGIC_VECTOR 等
	STD_LOGIC_ARITH	定义有符号与无符号数据类型，基于这些数据类型的算术运算符，如“+”、“-”、“*”、“/” SHL、SHR 等
	STD_LOGIC_SIGNED	定义基于 STD_LOGIC 与 STD_LOGIC_VECTOR 数据类型上的有符号的算术运算
	STD_LOGIC_UNSIGNED	定义基于 STD_LOGIC 与 STD_LOGIC_VECTOR 类型上的无符号的算术运算

库（LIBRARY）是专门用于存放预先编译好的程序包的地方，它实际上对应一个文件目录，程序包的文件就存放在此目录中。库名与目录名的对应关系可以在编译程序中指定，库的说明总是放在设计单元的最前面。例如，对 IEEE 标准库的调用格式为：LIBRARY IEEE。

1. 常用的库和程序包的种类

VHDL 程序中常用的库有 STD 库、IEEE 库和 WORK 库等。其中，STD 库和 IEEE 库中的标准程序包由提供 EDA 工具的厂商提供，用户在设计程序时可以用相应的语句调用。

(1) STD 库

STD 库是 VHDL 语言的标准库，库中定义了 STANDARD 和 TEXTIO 两个标准程序包。STANDARD 程序包中定义了 VHDL 的基本数据类型，如字符（CHARACTER）、整数（INTEGER）、实数（REAL）、位型（BIT）和布尔量（BOOLEAN）等。用户在程序中可以随时调用 STANDARD 包中的内容，不需要任何说明。TEXTIO 程序包中定义了对文本文件的读和写控制的数据类型和子程序。用户在程序中调用 TEXTIO 包中的内容时，需要用 USE 语句加以说明。

(2) IEEE 库

IEEE 标准库是存放用 VHDL 语言编写的多个标准程序包的目录，IEEE 库中的程序包有 STD_LOGIC_1164、STD_LOGIC_ARITH、STD_LOGIC_UNSIGNED 和 STD_LOGIC_SIGNED 等。其中，STD_LOGIC_1164 是 IEEE 库的标准程序包，定义了 STD_LOGIC 和 STD_LOGIC_VECTOR 等多种数据类型，以及多种逻辑运算符子程序和数据类型转换子程序等。STD_LOGIC_ARITH 和 STD_LOGIC_UNSIGED 等程序包是 SYNOPSYS 公司提供的，包中定义了 SIGNED 和 UNSIGNED 数据类型以及基于这些数据类型的运算符子程序。用户使用包中的内

容时，需要用 USE 语句加以说明。

(3) WORK 库

WORK 库是用户进行 VHDL 设计的当前目录，用于存放用户设计好的设计单元和程序包。在使用该库中的内容时不需要进行任何说明。

2. 库、包和 USE 语句的格式

用户在用到标准程序包中的内容时，除了 STANDARD 程序包，其余都要在设计程序中加以说明，首先用 LIBRARY 语句说明程序包所在的库名，再用 USE 语句说明具体使用哪一个程序包和具体的子程序名。各种标准程序包中的内容非常多，初学者一时难以全面了解，可以用下面的格式，以免出现不必要的错误。

库和包的调用格式：

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
```

小提示：写任何模块时最好先加上以下四个语句，以免出现库没包含全的问题：

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
```

5.1.4 配置

一个实体可以用多个结构体描述（反之不成立），多个结构体代表实体实现的多种方式，同一个实体的各结构体之间地位等同，具体综合时，选择哪一个结构体来综合是由配置确定的，仿真时用配置语句进行配置可节省大量时间。但是，在综合时，配置语句是不可综合的，所以尽量每个实体仅一个结构体表述完整，这样比较清晰，整体化。

配置格式：

```
CONFIGURATION 配置名 OF 实体名 IS
    FOR 选配结构体名;
    END FOR;
END CONFIGURATION;
```

5.2 VHDL 语言要素

VHDL 语言和其他高级编程语言一样，除了具有一定的程序结构，还定义了一些语言要素，语言要素是组成编程语句的基本单元。VHDL 语言要素主要有：文字规则、数据对象、数据类型、操作数、运算符。除了掌握 VHDL 程序结构，还有必要详细了解 VHDL 语言的语言要素。

5.2.1 VHDL 文字规则

任何一种程序设计语言都有自己的一套书写符号和语法规则，这些符号和语法规则构成了程序设计语言的文字规则。

1. 标识符

标识符是程序员为了书写程序所规定的一些词，用来表示常数、变量、信号、子程序、结构体和实体等名称。VHDL 基本的标识符的组成规则如下：

- (1) 标识符由英文字母、数字及下划线 “_” 组成；
 - (2) 标识符必须以英文字母开头；
 - (3) 标识符中不能有两个连续的下划线 “_”，标识符的最后一个字符不能是下划线；
 - (4) 标识符中的英文字母不区分大小写；
 - (5) 标识符最长可以是 32 个字符；
 - (6) VHDL 定义的保留字或称关键字，不能用作标识符；
- 例如：CLK, QO, DAT1, SX_1, NOT_Q 是合法的标识符。

3DA, _QD, NA_C, DB-A, DB_ 是非法的标识符。

小提示：93 标准定义了扩展标识符，可以以数字开头，使用 VHDL 保留字等，不过习惯上依然使用 87 标准。

2. 保留字

VHDL 中的保留字是具有特殊含义的标识符号，只能作为固定的用途，用户不能用保留字作为标识符。例如 ENTITY、ARCHITECTURE、PROCESS、BLOCK、BEGIN 和 END 等。VHDL 保留字如表 5.2 所示。

表 5.2 VHDL 保留字

abs	access	after	alias	all
and	architecture	array	assert	attribute
begin	block	body	buffer	bus
case	component	configuration	constant	disconnect
downto	else	elsif	end	entity
exit	file	for	function	generate
generic	group	guarded	if	impure
in	inertial	inout	is	label
library	linkage	literal	loop	map
mod	nand	new	next	nor
not	null	of	on	open
or	others	out	package	port
postponed	procedure	process	pure	range
record	register	reject	rem	report
return	rol	ror	select	severity
signal	shared	sla	sll	sra

续表

srl	subtype	then	to	transport
type	unaffected	units	until	use
variable	wait	when	while	with
xnor	xor			

3. 界符

界符是作为 .VHDL 语言中两个部分的分隔符用的。例如，每个完整的语句均以“;”结尾，用双减号“--”开头的部分是注释内容，不参加程序编译；信号赋值符号是“<=”，变量赋值符号是“:=”等。在 VHDL 中，常用的界符如表 5.3 所示。

表 5.3 VHDL 中的界符

,	:	;	>	<	=
+	-	*	/	&	<>
--	_	()	<=	:=	

4. 注释符

在 VHDL 中，为了便于理解和阅读程序，常常加上注释，注释符用双减号“--”表示。注释语句以注释符开头，到行尾结束。注释可以加在语句结束符“;”之后，也可以加在空行处。

5.2.2 VHDL 数据类型

VHDL 是一种强类型语言，在 VHDL 中，定义了三种数据对象，即信号、变量和常数，编程时非常注重数据类型的匹配问题，要求设计实体中的数据对象必须具有确定的数据类型，只有相同数据类型的量才能进行相互传递或赋值。VHDL 定义了多种标准的数据类型，而且每一种数据类型都具有特定的物理意义。例如，BIT 型、STD_LOGIC 型、INTEGER 型和 REAL 型等数据类型。VHDL 的数据类型较多，根据数据用途分类可分为四大类：标量型、复合型、存取型和文件型。

标量型是单元素的最基本数据类型，通常用于描述一个单值数据对象。标量型包括整数类型、实数类型、枚举类型和时间类型，其中位（BIT）型和标准逻辑位（STD_LOGIC）型属于枚举类型。复合型可以由最基本的数据类型复合而成，如可由标量复合而成，主要包括数组（ARRAY）型和记录（RECORD）型。存取型为给定的数据对象提供存取方式。文件型用于提供多值存取型。这些数据类型又可以分为两大类：在 VHDL 程序包中预定义的数据类型和用户自定义的数据类型。预定义的数据类型是 VHDL 中较常用的、基本的数据类型，这些数据类型都已在 VHDL 的标准库中进行了定义，这些程序包放在 EDA 软件中的 IEEE 和 STD 目录中，供用户随时调用。在预定义的各种数据类型的基础上，用户可以根据实际需要自己定义数据类型和子类型，如标量型和数组型。使用用户定义的数据类型和子类型可以使设计程序的语句简练，易于阅读，简化设计电路硬件结构。

值得注意的是，各种 EDA 工具不能完全支持 VHDL 的所有数据类型，只支持 VHDL 的子集。

1. VHDL 预定义的数据类型

VHDL 预定义的数据类型是一种已在 STANDARD 程序包中预先定义的数据类型，在实际使用中能自动包含在 VHDL 的源文件中。

(1) 整数 (INTEGER) 数据类型

整数数据类型与数学中整数的定义是相同的，整数类型的数据代表正整数、负整数和零。VHDL 整数类型定义格式为：

```
TYPE INTEGER IS RANGE  $-(2^{31}-1) \sim +(2^{31}-1)$  ;
```

实际上，一个整数是由 32 位二进制码表示的带符号数的范围。

正整数 (POSITIVE) 和自然数 (NATURAL) 是整数的子类型，定义格式为：

```
SUBTYPE POSITIVE IS INTEGER RANGE 1 TO INTEGER'HIGH ;
SUBTYPE NATURE IS INTEGER RANGE 0 TO INTEGER'HIGH ;
```

其中，INTEGER'HIGH 是数值类属性，代表整数上限的数值，即 $2^{31}-1$ 。所以正整数表示的数值范围是 $1 \sim 2^{31}-1$ ，自然数表示的数值范围是 $0 \sim 2^{31}-1$ 。实际使用过程中为了节省硬件组件，常用 RANGER...TO...限制整数的范围。例如：

```
SIGNAL A :INTEGER; --信号 A 是整数数据类型
VARIABLE B :INTEGER RANGE 0 TO 15; --变量 B 是整数数据类型,变化范围是 0 到 15.
SIGNAL C :INTEGER RANGE 1 TO 7; --信号 C 是整数数据类型,变化范围是 1 到 7.
```

(2) 实数 (REAL) 数据类型

VHDL 实数数据类型与数学上定义的实数相似，VHDL 的实数就是带小数点的数，也称为浮点数，只能在仿真中使用，由正号、负号、小数点和数字组成。实数有两种书写形式，即小数形式和科学计数形式，不能写成整数形式。例如，1.0、1.0E4、-5.2 等实数是合法的。实数数据类型的定义格式为：

```
TYPE REAL is range -1.0E+38 to 1.0E+38;
```

例如：

```
SIGNAL A, B, C :REAL ;
A<= 5.0;
B <= 3.5E5;
C <= -4.5;
```

小提示：整数与实数均可以由下划线分割，便于阅读，例如：45_133_134；124_452_112.113_429；

此外，不同进制的数可以由如下格式表示：

基数#数字文字#E 指数

如：2#1111_1110# = 254；16#E#E1 = $14 * 161 = 224$ ；

(3) 位 (BIT) 数据类型

位数据类型属于二值枚举型，通常用来表示信号的取值。位数据类型的位值用字符

‘0’和‘1’表示，将值放在单引号中，表示二值逻辑的0和1。这里的0和1与整数型的0和1不同，可以进行算术运算和逻辑运算，而整数类型只能进行算术运算。位数据类型的数据对象为信号和变量，可以进行逻辑运算，其结果仍为位数据类型。

位数据类型的定义格式为：

```
TYPE BIT is ('0', '1');
```

例如：

```
RESULT : OUT BIT;  
RESULT<= .1.;
```

表示将 RESULT 引脚设置为高电平。

(4) 位向量 (BIT_VECTOR) 数据类型

位向量是基于位数据类型的数据，常用来表示总线的状态。位向量是用双引号括起来的数字序列，如“1011”、X“00EA”。VHDL 位向量的定义格式为：

```
TYPE BIT_VECTOR is array (NATURAL range <>) of BIT;
```

使用位向量必须注明位宽，即数组的个数和排列顺序，位向量的数据要用双引号括起来。

例如“1010”、X“A8”。其中1010是4位二进制数，用X表示双引号里的数是十六进制数。

例如：

```
SIGNAL A :BIT_VECTOR (3 DOWNT0 0 );  
A <= "1110" ;
```

表示A是4个BIT型元素组成的一维数组，数组元素的排列顺序是A3=1，A2=1，A1=1，A0=0。

(5) 布尔 (BOOLEAN) 数据类型

布尔数据类型属于二值枚举型数据类型，常用于关系运算和逻辑判断。一个布尔量具有真 (TRUE) 和假 (FALSE) 两种状态。布尔量没有数值的含义，不能用于数值运算，它的数值只能通过关系运算产生。例如，在IF语句中，A>B是关系运算，如果A=3，B=2，则A>B关系成立，结果是布尔量TRUE，否则结果为FALSE。

在VHDL中，布尔数据类型的定义格式为：

```
TYPE BOOLEAN IS (FALSE, TRUE);
```

(6) 字符 (CHARACTER) 数据类型

在STANDARD程序包中预定义了128个ASCII码字符类型，字符类型是用单引号括起来的ASCII字符，可以是数值，也可以是符号或字母，如‘R’、‘A’、‘*’、‘Z’。与VHDL标识符不区分大小写不同，字符类型中的字符大小写是不同的。字符型数据仅包括ASCII码字符表中的全部字符。

(7) 字符串 (STRING)

字符串数据类型是用双引号括起来的字符序列，是由字符型数据组成的数组，也称字符

向量或字符串数组。常用于程序的提示和说明。在 STANDARD 程序包中，字符串的定义是：

```
TYPE STRING is array (POSITIVE range <>) of CHARACTER;
```

字符串数据类型是一维的字符数组，须放在双引号中。VHDL 中有两种类型的字符串：文字字符串和数位字符串。如“ERROR”、“BOTH S AND Q EQUA TO L”、“X”、“BB\$CC”。

(8) 时间 (TIME) 数据类型

时间数据类型是用来表示时间的数据类型，取值范围是整数所定义的范围，仅在系统仿真时，用来表示信号延时的时间，不能用于逻辑综合。一个完整的时间类型包括整数表示的数值和时间单位两部分，数值和单位之间至少留一个空格，如 1 ms、20 ns 等。

STANDARD 程序包中定义时间格式为：

```
TYPE TIME is range -9223372036854775808 to 9223372036854775807
UNITS
fs; --飞秒
ps = 1000 fs; --皮秒
ns = 1000 ps; --纳秒
us = 1000 ns; --微秒
ms = 1000 us; --毫秒
sec = 1000 ms; --秒
min = 60 sec; --分
hr = 60min; --小时
END UNITS;
```

(9) 错误等级 (SEVERITY_LEVEL) 数据类型

错误等级数据用于表征系统的工作状态，包括 NOTE (注意)、WARNING (警告)、ERROR (出错)、FAILURE (失败)。

在仿真过程中，可输出这 4 种值来提示被仿真系统当前的工作状态。

定义格式：

```
TYPE severity_level IS (note,warning,error,failure);
```

2. IEEE 预定义的标准逻辑位和标准逻辑位向量

在 IEEE 库的 STD_LOGIC_1164 程序包中，定义了两个非常重要的数据类型：标准逻辑位 (STD_LOGIC)；标准逻辑矢量 (STD_LOGIC_VECTOR)。使用时可打开 IEEE 库，调用 STD_LOGIC_1164 程序包。如：

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
```

(1) 标准逻辑位 (STD_LOGIC) 数据类型

STD_LOGIC 是位 (BIT) 数据类型的扩展，是 STD_ULOGIC 数据类型的子类型。它是一个逻辑型的数据类型，取代 BIT 数据类型的两种取值数值 (0 和 1)，它扩展定义了 9 种值，在 IEEE STD1164 程序包中，STD_ULOGIC 和 STD_LOGIC 数据类型定义格式为：

```

TYPE std_ulogic IS (
'U', -- Uninitialized
'X', -- Forcing Unknown
'0', -- Forcing 0
'1', -- Forcing 1
'Z', --High Impedance
'W', -- Weak Unknown
'L', --Weak 0
'H', --Weak 1
'-' -- Don't care
);

FUNCTION resolved ( s : std_ulogic_vector ) RETURN std_ulogic;

SUBTYPE std_logic IS resolved std_ulogic;

```

小提示: STD_LOGIC 中的数据类型必须要大写, 不能使用小写字母代替, 在实际的 IC 集成时, 一般只使用 ‘0’、‘1’、‘Z’、‘X’ 四种数据类型, 其余的是不可综合的。

STD_LOGIC 和 STD_ULOGIC 数据类型的区别在于 STD_LOGIC 数据类型是经过重新定义的, 可以用来描述多路驱动的三态总线, 而 STD_ULOGIC 数据类型只能用于描述单路驱动的三态总线。

(2) 标准逻辑位向量 (STD_LOGIC_VECTOR) 数据类型

STD_LOGIC_VECTOR 数据类型是基于 STD_LOGIC 数据类型的标准逻辑一维数组, 和 BIT_VECTOR 数组一样, 使用标准逻辑位向量时必须注明位宽和排列顺序, 数据要用双引号括起来, 常用于描述数字系统中的总线。赋值原则: 同位宽、同类型矢量之间才能进行赋值。

例如:

```

SIGNAL SA1 :STD_LOGIC_VECTOR (3 DOWNT0 0 );
SA1 <= "0110" ;

```

在 IEEE_STD_1164 程序包中, STD_LOGIC_VECTOR 数据类型定义格式为

```

TYPE std_logic_vector IS ARRAY ( NATURAL RANGE <>) OF std_logic;

```

3. 其他预定义的数据类型

在 IEEE 库的 STD_LOGIC_ARITH 程序包中, 包含有两个符合 IEEE 标准的数据类型: 无符号 (UNSIGNED) 和带符号 (SIGNED) 数据类型, 这两种数据类型主要用来进行算术运算。定义格式为

```

TYPE UNSIGNED is array (NATURAL range <>) of STD_LOGIC;
TYPE SIGNED is array (NATURAL range <>) of STD_LOGIC;

```

这两种数据类型主要用于算术运算, 使用时可打开 IEEE 库, 调用 STD_LOGIC_ARITH 程序包。如:

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_ARITH.ALL;

```

(1) 无符号 (UNSIGNED) 数据类型

无符号数据类型是由 STD_LOGIC 数据类型构成的一维数组，用于程序中无符号数的运算，代表一个无符号的数值，即一个二进制数。在一个结构体中，当一个数据除了执行算术运算还要执行逻辑运算时，就必须定义成 UNSIGNED，而不能是 SIGNED 或 INTEGER 类型。例如：

```
SIGNAL DAT1 : UNSIGNED(3 DOWNTO 0);  
DAT1 <= "1001";
```

定义信号 DAT1 是 4 位二进制码表示的无符号数据，数值是 9。

(2) 带符号 (SIGNED) 数据类型

带符号 (SIGNED) 数据类型表示一个带符号的整数，其最高位用来表示符号位，用补码表示数值的大小。当一个数据的最高位是 0 时，这个数表示正整数，当一个数据的最高位是 1 时，这个数表示负整数。例如：

```
VARIABLE DB1, DB2 : SIGNED(3 DOWNTO 0);  
DB1 <= "0110" ;  
DB2 <= " 1001" ;
```

定义变量 DB1 是 6，变量 DB2 是-7。

4. 用户自定义的数据类型

在 VHDL 中，除了一些标准的预定义数据类型，用户还可以根据设计需要，自己定义新的数据类型，称为用户自定义的数据类型。常用用户自定义数据类型有：子类型、枚举型 (ENUMERATION)、数组型 (ARRAY)、记录型 (RECORD) 等。

(1) 子类型

子类型是通过对预定义的数据类型做一些范围限定而形成的一种新的数据类型。这种定义数据类型的方法有如下几种格式：

```
TYPE 数据类型名称 IS 数据类型名 RANGE 数据范围;
```

例如：

```
TYPE DATA IS INTEGER RANGER 0 TO 9 ;
```

定义 DATA 是 INTEGER 数据类型的子集，数据范围是 0~9。

```
SUBTYPE 数据类型名称 IS 数据类型名 RANGE 数据范围;
```

例如：

```
SUBTYPE DB IS STD_LOGIC_VECTOR(7 DOWNTO 0);
```

定义 DB 是 STD_LOGIC_VECTOR 数据类型的子集，位宽 8 位。

(2) 枚举型

在数据类型定义中，直接列出新的数据类型的所有取值的数据类型，称为枚举数据类型。枚举数据类型是用文字、符号表示的一组实际的二进制数。定义该种数据类型的格式为：

```
TYPE 数据类型名称 IS (取值 1, 取值 2, ...);
```

例如：

```
TYPE BIT IS (.0, .1.);
TYPE STATE_M IS (S0, S1, S2, S3);
```

定义 BIT 数据类型，取值 0 和 1。定义 STATE_M 是数据类型，表示状态变量 S0、S1、S2、S3。在 VHDL 中，为了便于阅读程序，可以用符号名来代替具体的数值，前例中 STATE_M 是状态变量，用符号 S0、S1、S2、S3 表示 4 种不同的状态取值 00、01、10、11。

小提示：使用枚举数据类型定义后，综合器会自动将字符类型从 0 开始进行二进制编码，编码的位数由枚举元素个数决定。

(3) 数组 (ARRAY) 型

数组是一组具有相同数据类型的元素组合。数组型属于复合型，是将相同类型的单个数据组合在一起而形成的数据类型。可以是一维数组（一个下标）和多维数组（多个下标），下标的数据类型必须是整数。前面介绍的位向量 (BIT_VECTOR) 和标准逻辑位向量 (STD_LOGIC_VECTOR) 数据类型都属于一维数组类型。数组定义的格式为：

```
TYPE 数据类型名称 IS ARRAY 数组下标的范围 OF 数组元素的数
```

小提示：VHDL 多维数组定义，多维数组声明即将第一维的数组作为第二维数组的元素定义即可。

```
TYPE 1 维数据类型名称 IS ARRAY 数组下标的范围 OF 数组元素的数据类型；
TYPE 2 维数据类型名称 IS ARRAY 数组下标的范围 OF 上面所定义的 1 维数据类型；
```

根据数组元素下标的范围是否指定，把数组分为非限定性数组和限定性数组两种类型。

非限定性数组不具体指定数组元素下标的范围，而是用 NATURAL RANGER <> 表示，当用到该数组时，再定义具体的下标范围。如前面介绍的位向量 (BIT_VECTOR) 和标准逻辑位向量 (STD_LOGIC_VECTOR) 数据类型等在程序包中预定义的数组属于非限定性数组。

例如，在 IEEE 程序包中定义 STD_LOGIC_VECTOR 数据类型的语句是

```
TYPE std_logic_vector IS ARRAY ( NATURAL RANGE <>) OF std_logic;
```

没有具体指出数组元素的下标范围，在程序中用信号说明语句指定。

例如：

```
SIGNAL DAT : STD_LOGIC_VECTOR(3 DOWNT0 0);
```

限定性数组的下标的范围用整数指定，数组元素的下标可以是由低到高，如 0 TO 3，也可以是由高到低，如 7 DOWNT0 0，表示数组元素的个数和在数组中的排列方式。

例如：

```
TYPE D IS ARRAY (0 TO 3) OF STD_LOGIC;
TYPE A IS ARRAY (4 DOWNT0 1) OF BIT;
```

定义数组 D 是一维数组，由 4 个 STD_LOGIC 型元素组成，数组元素的排列顺序是 D (0)，D(1)，D(2)，D(3)。A 数组是由 4 个元素组成的 BIT 数据类型，数组元素的排列顺序是 A(4)，A(3)，A(2)，A(1)。

小提示：对于数组数据类型，可以给一组数据的多个值一起赋值，如上例：

```
SIGNAL ARRAY1: D;  
BEGIN  
  ARRAY1<= ('1','0','0','1');
```

(4) 记录型

记录型是一个数组，由已定义的、不同的数据类型元素构成。记录型也是一种多值的数据类型。但是记录型是由多个不同类型的元素集合而成的。定义格式为：

```
TYPE 记录类型名 IS RECORD  
  元素名: 元素数据类型;  
  元素名: 元素数据类型;  
  ...  
END RECORD [记录类型名]
```

5. 数据类型的转换

在 VHDL 语言中，数据类型的定义是相当严格的，不同类型的数据是不能进行运算和赋值的。为了实现不同类型的数据赋值，就要进行数据类型的转换。转换函数在 VHDL 语言程序包中定义。在 STD_LOGIC_1164、STD_LOGITH_ARITH 和 STD_LOGIC_UNSIGNED 程序包中提供的数据类型转换函数如表 5.4 所示。

表 5.4 数据类型转换函数

程序包名称	函 数 名 称	功 能
STD_LOGIC_1164	TO_BIT TO_BITVECTOR TO_STDULOGIC TO_STDULOGICVECTER	由 STD_LOGIC 转换为 BIT 由 STD_LOGIC_VECTOR 转换为 BIT_VECTOR 由 BIT 转换为 STD_LOGIC 由 BIT_VECTOR 转换为 STD_LOGIC_VECTOR
STD_LOGIC_ARITH	CONV_INTEGER CONV_UNSIGNED CONV_STD_LOGIC_VECTOR	由 UNSIGNED, SIGNED 转换为 INTEGER 由 SIGNED, INTEGER 转换为 UNSIGNED 由 INTEGER, UNSDGNED, SIGNED 转换为 STD_LOGIC_VECTOR
STD_LOGIC_UNSIGNED	CONV_INTEGER	由 STD_LOGIC_VECTOT 转换为 INTEGER

例如：将 INTEGER 类型转换为 STD_LOGIC_VECTOR 类型。

```
SIGNAL A: INTEGER RANGER 0 TO 15;  
SIGNAL B: STD_LOGIC_VECTOR(3 DOWNT0 0);  
B <= CONV_STD_LOGIC_VECTOR ( A );
```

第 1 条语句，将信号 A 定义为 INTEGER 类型。

第 2 条语句，将信号 B 定义为 STD_LOGIC_VECTOR 类型。

第 3 条语句，调用 STD_LOGIC_ARITH 程序包中的函数 CONV_STD_LOGIC_VECTOR ，将信号 A 的数据类型转换为 STD_LOGIC_VECTOR 类型，并赋值给 B。

5.2.3 VHDL 的运算符

与高级语言一样，VHDL 语言的表达式也是由运算符和操作数组成的，操作数是表达式

中各种运算的对象，而运算符（又称操作符）则规定操作数的运算方式。VHDL 标准预定义了 5 种运算符，即逻辑运算符、算术运算符、关系运算符、移位运算符和连接运算符，并且定义了与运算符相应的操作数的数据类型。各种运算符之间是有优先级的，例如在所有运算符中，逻辑运算符 NOT 的优先级别最高。表 5.5 列出了所有运算符的优先级顺序。编写程序时，最好用嵌套括号的方式来表示优先级。

表 5.5 VHDL 运算符列表

运算符类型	运 算 符	功 能	优 先 级
逻辑运算符	AND	逻辑与	最低
	OR	逻辑或	
	NAND	逻辑与非	
	NOR	逻辑或非	
	XOR	逻辑异或	
	NXOR	逻辑异或非	
关系运算符	=	等于	
	/=	不等于	
	<	小于	
	>	大于	
	<=	小于等于	
	>=	大于等于	
移位运算符	SLL	逻辑左移	
	SLA	算术左移	
	SRL	逻辑右移	
	SRA	算术右移	
	ROL	逻辑循环左移	
	ROR	逻辑循环右移	
符号运算符	+	正	
	-	负	
连接运算符	&	位合并	
算术运算符	+	加	
	-	减	
	*	乘	
	/	除	
	MOD	求模	
	REM	求余	
	**	乘方	
	ABS	求绝对值	
逻辑非运算符	NOT	逻辑非	最高

1. 逻辑运算符

在 VHDL 语言中定义了 7 种基本的逻辑运算符，分别是 AND（与）、OR（或）、NOT（非）、NAND（与非）、NOR（或非）、XOR（异或）和 NXOR（异或非）等。

由逻辑运算符和操作数组成了逻辑表达式。在 VHDL 语言中，逻辑表达式中的操作数的数据类型可以是 BIT 和 STD_LOGIC 数据类型，也可以是一维数组类型 BIT_VECTOR 和 STD_LOGIC_VECTOR，要求运算符两边的操作数的数据类型相同、位宽相同。逻辑运算是按位进行的，运算结果的数据类型与操作数的数据类型相同。

在一个逻辑表达式中有两个以上的运算符时，需要用括号对这些运算进行分组。
例如语句

```
X1 <= (A AND B ) OR (C AND B);
X2 <= ( A OR B) AND C;
```

是正确的。

如果一个逻辑表达式中只有 AND、OR 和 XOR 三种运算符中的一种，那么改变运算顺序不会影响电路的逻辑关系，表达式中的括号可以省略。例如，下列语句是正确的：

```
Y1 <= A AND B AND C;
Y2 <= A OR B OR D;
```

逻辑运算符 NOT 处于最高优先级，而其余的逻辑运算符处于最低优先级。

2. 算术运算符

VHDL 语言定义了 5 种常用的算术运算符，如表 5.6 所示。

表 5.6 VHDL 算术运算符

类 型	操 作 符	功 能	操作数数据类型
算术操作符	+	加法运算	整数
	-	减法运算	整数
	&	并置运算	一维数组
	*	乘法运算	整数和实数
	/	除法运算	整数和实数
	MOD	取模运算	整数
	REM	求余运算	整数
	**	乘方运算	整数
	ABS	取绝对值	整数

在算术运算表达式中，两个操作数必须具有相同的数据类型，加法和减法的操作数的数据类型可以是整数、实数或物理量，乘法和除法的操作数可以是整数或实数。为了节约硬件资源，乘法和除法的操作数应该选用 INTEGER、STD_LOGIC_VECTOR 或 BIT_VECTOR 等数据类型。

例如：

```
X<=A+B;
Y<=C*D;
Z<=A-C**2
```

小提示：对于 /、MOD、REM 运算，要求操作符的右操作数必须为 2 的正整数次幂（可以用实际电路移位实现），才可以综合。

3. 关系运算符

关系运算是两个相同类型的操作数进行大小比较或排序判断，比较结果以布尔数据类型表示，可能是 TRUE，也可能是 FALSE。VHDL 语言中定义了 6 种关系运算符，如表 5.7 所示。

表 5.7 VHDL 关系运算符

类 型	操 作 符	功 能	操作数数据类型
关系操作符	=	等于	任何数据类型
	/=	不等于	任何数据类型
	<	小于	枚举与整数，及对应的一维数组
	>	大于	枚举与整数，及对应的一维数组
	<=	小于等于	枚举与整数，及对应的一维数组
	>=	大于等于	枚举与整数，及对应的一维数组

在利用关系运算符对位向量数据进行比较时，比较过程是以从左到右的顺序按位进行比较的，操作数的位宽可以不同，但有时会产生错误的结果。

如果 A、B 是 STD_LOGIC_VECTOR 数据类型，A = “1110”，B = “10110”，关系表达式 A>B 的比较结果是 TRUE，也就是说 A>B。对于以上出现的错误可以利用 STD_LOGIC_ARITH 程序包中定义的 UNSIGNED 数据类型来解决，即把要比较的操作数定义成 UNSIGNED 数据类型。

4. 移位运算符

VHDL93 标准中增加了 6 个移位运算符，如表 5.8 所示。

表 5.8 VHDL 移位运算符

类 型	操 作 符	功 能	操作数数据类型
移位操作符	SLL	逻辑左移	BIT 或布尔型一维数组
	SRL	逻辑右移	BIT 或布尔型一维数组
	SLA	算术左移	BIT 或布尔型一维数组
	SRA	算术右移	BIT 或布尔型一维数组
	ROL	逻辑循环左移	BIT 或布尔型一维数组
	ROR	逻辑循环右移	BIT 或布尔型一维数组

移位运算符的格式是：

```
操作数名称 移位运算符 移位位数；
```

6 个移位运算符所执行的操作如图 5.2 所示。

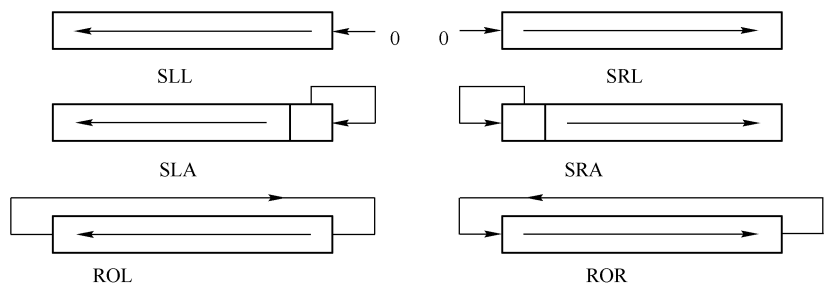


图 5.2 移位运算符

其中，SLL 是将位向量左移，右边移空位补零。SLA 是将位向量左移，右边第一位的数值保持原值不变。SRL 是将位向量右移，左边移空位补零。SRA 是将位向量右移，左边第一位的数值保持原值不变。ROR 和 ROL 是自循环移位方式。

例如：

```
A<="0101";
B <=A SLL 1;
```

仿真的结果是 B = 1010。

5. 连接运算符 (&)

用连接运算符可以将多个数据对象合并成一个新的 一维数组，也可以将两个 一维数组中的元素分解合并成新的一维数组。连接两个操作符产生新的一维数组的位宽等于两个操作数的位宽之和，新的数组元素的顺序是由操作数的位置决定的，连接符 “&” 左边的操作数的元素在左，连接符 “&” 右边的操作数的元素在右。操作数可以是 BIT 或 STD_LOGIC 类型。如果

```
1011,0010 A :BIT_VECTOR(0 TO 7)
0001,0110 B :BIT_VECTOR(0 TO 7)
C <=B(0 TO 3)&A( 5 TO 7)&.1.;
```

则 C=1011, 1101。

5.2.4 VHDL 的数据对象

在算法语言中，定义了多种数据对象，如常数、变量和数组等，用来存放不同类型的数据，如整数、实数、复数、逻辑常数和逻辑变量等。同样，VHDL 语言除了具有一定的语法规则，还定义了可以接受不同数据类型赋值的数据对象。在 VHDL 程序中，常用的数据对象分为三种类型，即常数 (CONSTANT)、变量 (VARIABLE) 和信号 (SIGNAL)，在使用过程中，这三种数据对象除了具有一定的数据功能，还被赋予了不同的物理意义，在应用时要特别注意。

1. 常数 (CONSTANT)

常数被赋值后就保持某一固定的值不变。在 VHDL 中，常数通常用来表示计数器的模的大小、数组数据的位宽和循环计数次数等，也可以表示电源电压值的大小。常数的使用范

围与其在设计程序中的位置有关：定义在程序包中的常量，可在调用此程序包的所有设计实体中使用；定义在设计实体内的常量，仅在该实体内使用；如果常数在结构体中被赋值，则此常数只能在该结构体中使用；如果常数在 PROCESS 语句或子程序中被赋值，则只能供进程或子程序使用，属于局部量。程序设计中使用常数有利于提高程序的可读性以及方便对程序进行修改。通常常数的赋值在程序开始前进行，其数据类型在常数说明语句中指明，赋值符号为“:=”。

常数定义语句的格式为：

```
CONSTANT 常数名称:数据类型 := 表达式
```

例如：

```
CONSTANT width: INTEGER:=16;--width 是整数类型常量,值为 16
CONSTANT delay: TIME:=20ns;--delay 是时间型常量,值为 20ns
CONSTANT fbus: BIT_VECTOR:="11010111";--fbus 是位矢量型常数
其值为 11010111
CONSTANT Vcc: REAL:=5.0;--Vcc 是实数型常量,其值为 5.0
```

注意：常数所赋的值的类型必须与定义的数据类型一致，在程序中常数被赋值后不能再改变。

2. 变量 (VARIABLE)

在 VHDL 程序中，变量常用来存储中间数据，以便实现程序的算法。变量为局部量，它只能在进程语句、函数语句和过程语句的结构中使用。对变量操作有变量定义语句和变量赋值语句，变量在赋值前必须通过定义，可以在变量定义语句中赋初值，变量初值不是必需的，变量初值的赋值符号是“:=”。

小提示：变量与信号赋初值语句仅可用于仿真，在综合时被忽略，不起作用。

变量定义语句的格式为：

```
VARIABLE 变量名称:数据类型 := 初值;
```

例如：

```
VARIABLE S1 :INTEGER := 0 ;--定义 S1 为整数型变量、初值是 0
VARIABLE S2, S3 :INTEGER;--定义 S2,S3 为整数型变量
VARIABLE CON1 :INTEGER RANGER 0 TO 20 ;--定义 CON1 为 0 到 20 整数型变量
VARIABLE D1, D2 :STD_LOGIC ;--定义 D1,D2 为 1 位标准逻辑位型变量。
```

变量赋值语句的格式为：

```
变量名称 := 表达式 ;
```

变量数值的改变是通过变量赋值来实现的。在对变量进行赋值时，要求表达式的数据类型必须与变量定义语句中的数据类型一致，表达式的数据对象可以是常数、变量和信号。变量赋值是立即发生的，没有任何时间延迟，所以变量只有当前值，并且对同一个变量可以多次赋予新值。多个变量的赋值是根据赋值语句在程序中的书写位置，按照自上而下顺序进行的，即变量赋值语句属于顺序执行语句。变量不能放在进程的敏感信号表中，变量不能用于

在进程之间传递数据，变量不能用于硬件连线和存储元件。

例如：

```
PROCESS (D, E)
VARIABLE AV, BV, CV :INTEGER := 0 ;
BEGIN
AV := 1 ;
BV := AV + D ;
AV := E + 2 ;
CV := AV * 2 ;
A <= AV ;
B <= BV ;
C <= CV ;
END PROCESS ;
```

这是一个进程语句，定义 AV、BV、CV 是整数型变量，当敏感信号 D、E 只要有一个发生变化，放在进程中的语句就要全部执行一次。例如 D = 1, E 变化为 2，则这段程序的执行结果是：A = 4, B = 2, C = 8。

3. 信号 (SIGNAL)

在 VHDL 中，信号是描述硬件系统的基本数据对象，它是电子电路内部硬件连接的抽象，可以作为设计实体中并行语句模块间的信息交流通道。信号分为外部端口信号和内部信号。

外部端口信号是设计单元电路的引脚，在程序实体中定义，外部信号对应 4 种 I/O 状态，即 IN, OUT, INOUT, BUFFER，其作用是在设计单元电路之间起互连作用。外部信号可以供整个设计单元，使用属于全局量。例如，在结构体中，外部信号可以直接使用，不需要加以说明，可以通过信号赋值语句给外部输出信号赋值。

内部信号用来描述设计单元内部的传输信号，它除了没有外部信号的流动方向，其他性质与外部信号一致。内部信号的使用范围（可见性）与其在设计程序中的位置有关，内部信号可以在包体、结构体和块语句中定义，如果信号在结构体中定义，则可以供整个结构体使用；如果信号在块语句中定义，只能供块内使用；不能在进程和子程序中定义内部信号。信号在状态机中表示状态变量。

对内部信号操作有信号定义语句和信号赋值语句，内部信号在赋值前必须进行定义，可以在信号定义语句中赋初值，内部信号的初值不是必需的，内部信号的定义格式与变量的定义格式基本相同，只要将变量定义中的保留字 VARIABLE 换成 SIGNAL 即可。

内部信号定义语句的格式为：

```
SIGNAL 信号名称:数据类型 := 初值;
```

例如：

```
SIGNAL S0: STD LOGIC:= '0';-- 定义一个标准逻辑位信号 S0,初值为 0
SIGNAL FA,FB: BIT; -- 定义两个 BIT 位信号 FA 和 FB
SIGNAL D: STD LOGIC VECTOR(0 TO 15);--定义标准逻辑 16 位矢量信号 D
```

信号赋值符号与变量赋值符号不同，信号赋值符号为“<=”。

信号赋值语句的格式为：

信号名称 <= 表达式；

例如： x<=9；

y<=x；

z<=x AFTER 5ns；

对于赋值语句还可以采用如下格式：

```
SIGNAL VEC1,VEC2: BIT_VECTOR( 3 DOWNT0 0);
```

```
VEC1<= ( 0=>.1. OTHERS=>.0.);--表示判别性的整集内涵赋值,VEC 的第三位为 1,其余为 0
```

```
VEC2 (1 TO 2)<="10";--对 VEC2 部分位赋值
```

在对信号进行赋值时，表达式的数据对象可以是常数、变量和信号，但是要求表达式的数据类型必须与信号定义语句中的数据类型一致。在结构体中信号的赋值可以在进程中也可以在进程外，两者的赋值方式是不同的。在进程外，信号的赋值是并行执行的，所以被称为并行信号赋值语句。在进程内，信号的赋值方式具有特殊性。信号不能在进程中定义，但可以在进程中赋值。在进程中，变量赋值是立即起作用的，信号只有在进程被激活（敏感信号发生变化）后，在进程结束时才能赋予新的值。信号具有时间特性，信号赋值不是立即发生的，需要经过固有的时间延迟，所以信号具有过去值和当前发生值，这与实际电路的特性是一致的。

信号的赋值过程分为顺序处理和并行赋值两个阶段。顺序处理是按照自上而下的顺序，用信号原来的值对所有的表达式进行运算，运算结果不影响下一个表达式的运算，直到处理好进程中的最后一个表达式。并行赋值是把表达式的值并行同时赋给信号。整个过程是一个无限循环的过程，循环停止的条件是敏感信号保持不变，所以在进程中的信号赋值语句属于顺序执行语句。在进程之外的信号赋值语句属于并行同时语句。

信号与变量赋值的区别：

(1) 使用场合不同

- 变量在进程、函数和过程中说明；
- 信号在结构体中说明。

(2) 赋值符号不同

- 变量用“:=”赋值，其值被立即使用（无时间延迟）；
- 信号用“<=”赋值，其值可以附加延迟。

例如：

```
SIGNAL D :INTEGER ; --信号要定义在进程外的结构体中
```

```
PROCESS (A,B,C)
```

```
BEGIN
```

```
D <= A ;
```

```
X <= B + D ;
```

```
D <= C ;
```

```
Y <= B + D ;
END PROCESS ;
```

执行的结果是: $D \leq C$;

```
X <= B + C ;
Y <= B + C ; --进程中同一信号多次赋值 只有最后一次生效
PROCESS (A,B,C)
VARIABLE D :INTEGER ; --变量要定义在进程内部
BEGIN
D := A ;
X <= B + D ;
D := C ;
Y <= B + D ;
END PROCESS ;
```

执行的结果是: $X \leq B + A$;

```
Y <= B + C ; --进程中变量多次赋值则立即生效
```

例 5.3 通过一位 BCD 码的加法器的程序比较信号、常量、变量的赋值及使用方法。

```
ENTITY bcdadd IS
PORT (
op1, op2 :IN INTEGER RANGE 0 TO 9 ;
result :OUT INTEGER RANGE 0 TO 31
);
END bcdadder;
ARCHITECTURE a OF bcdadder IS
CONSTANT adj :INTEGER := 6 ; --定义常数 adj=6
SIGNAL binadd :INTEGER RANGE 0 TO 18 ;
--定义信号 binadd 的取值范围是 0~18
BEGIN
binadd <= op1+op2 ; --求 op1+op2 和运算
PROCESS (binadd)
VARIABLE tmp : INTEGER:=0 ; --定义变量 tmp 是整数型,初值是 0
BEGIN
IF binadd > 9 THEN --如果 binadd 大于 9,结果要调整
tmp := adj ; --方法是和加 6,否则,结果加 0.
ELSE
tmp := 0 ;
END IF ;
result <= binadd+tmp ; --给外部信号赋值
END PROCESS;
END a;
```

常量、变量、信号的物理含义如下:

- 常量：电源、地、恒定逻辑值等常数。
- 变量：某些值的载体，或存储单元，常用于描述算法。
- 信号：物理设计中的硬连接线，包括输入/输出端口。

信号与常数相当于全局变量，变量相当于局部变量。变量只能存在于进程 (PROCESS)，函数 (FUNCTION)，过程 (PROCEDURE) 中，不能带出 PROCESS、FUNCTION、PROCEDURE，传送出去，而信号可以。

5.3 VHDL 描述语句

用 VHDL 语言进行设计时，按描述语句的执行顺序进行分类，可将 VHDL 语句分为顺序 (Sequential) 执行语句和并行 (Parallel) 执行语句。在数字系统设计中，这些语句用来描述系统的内部硬件结构和动作行为，以及信号之间的基本逻辑关系。顺序语句必须放在进程或子程序中，用来定义进程或子程序的算法，换句话说，并行语句包含了顺序语句。

在 VHDL 中，并行语句在结构体中的执行是同时并发执行的，其书写次序与其执行顺序并无关联，并行语句的执行顺序是由它们的触发事件来决定的。我们知道，实际的硬件系统中很多操作都是并发的，因此在对系统进行模拟时就要把这些并发性体现出来，并行语句正是用来表示这种并发行为的。

5.3.1 并行语句

相对于传统的软件描述语言，并行语句结构是最具 VHDL 特色的。在 VHDL 中，并行语句具有多种语句格式，各种并行语句在结构体中的执行是同步进行的，或者说是并行运行的，其执行方式与书写的顺序无关。在执行中，并行语句之间可以有信息往来，也可以是互为独立、互不相关、异步运行的（如多时钟情况）。每一并行语句内部的语句运行方式可以有两种不同的方式，即并行执行方式（如块语句）和顺序执行方式（如进程语句）。因此，VHDL 并行语句勾画出了一幅充分表达硬件电路的真实的运行图景。注意，VHDL 中的并行运行有多层含义，即模块间的运行方式可以有同时运行、异步运行、同步运行等方式，从电路的工作方式上讲可以包括组合逻辑运行方式、同步逻辑运行方式和异步逻辑运行方式等。结构体中的并行语句主要有 7 种：(1) 并行信号赋值语句，用“<=”运算符；(2) 条件赋值语句，WHEN-ELSE；(3) 选择信号赋值语句，WITH-SELECT；(4) 块语句，BLOCK；(5) 进程，PROCESS；(6) 元件例化语句，COMPONENT；(7) 生成语句，GENERATE。在结构体语句中，并行语句的位置是：

```
ARCHITECTURE 结构体名 OF 实体名 IS
    说明语句
BEGIN
    并行语句
END 结构体名；
```

1. 并行信号赋值语句

并行信号赋值语句用在结构体中并行执行，其功能是将一个数据或一个表达式的运算结果传送给一个数据对象，这个数据对象可以是内部信号，也可以是预定义的端口信号。并行

信号赋值语句相当于一个进程（用于单个信号赋值）的简化形式，信号赋值语句提供了三种赋值方式，用其代替进程可大大简化程序代码。

小提示：信号赋值语句在顺序语句中也有，顺序语句中可以给信号赋值也可以给变量赋值，但顺序语句里只能对变量说明，不能对信号说明；并行语句刚好相反。

例 5.4 用并行信号赋值语句描述逻辑表达式为 $Y = AB + C \oplus D$ 的电路。

```
ENTITY loga IS
PORT (
A, B, C, D : IN BIT;
Y : OUT BIT
);
END loga; --定义 A,B,C,D 是输入端口信号,Y 是输出端口信号
ARCHITECTURE stra OF loga IS
SIGNAL E : BIT; --定义 E 是内部信号
BEGIN
Y <= (A AND B) OR E; --以下两条并行语句与顺序无关

E <= C XOR D;
END stra;
```

小提示：在进程中的信号赋值语句属于顺序语句，而在结构体中进程外的信号赋值语句则属于并行语句。

2. 条件信号赋值语句 WHEN-ELSE

语法格式为：

```
信号 Y <= 信号 A WHEN 条件表达式 1 ELSE
信号 B WHEN 条件表达式 2 ELSE
...
信号 N;
```

在执行 WHEN-ELSE 语句时，先判断条件表达式 1 是否为 TRUE，若为 TRUE，则 $Y <=$ 信号 A；否则判断条件表达式 2 是否为 TRUE，若为 TRUE，则 $Y <=$ 信号 B；依次类推，只有当所列的条件表达式都为假（FALSE）时， $Y <=$ 信号 N。

例 5.5 用条件信号赋值语句 WHEN-ELSE 实现四选一数据选择器。

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY mux4 IS
PORT (
a0, a1, a2, a3 : IN STD_LOGIC;
s : IN STD_LOGIC_VECTOR (1 DOWNTO 0);
y : OUT STD_LOGIC
);
```

```

END mux4;

ARCHITECTURE archmux OF mux4 IS
BEGIN
y <= a0 WHEN s = "00" else  --当 s=00 时,y=a0
a1 WHEN s = "01" else  --当 s=01 时,y=a1
a2 WHEN s = "10" else  --当 s=10 时,y=a2
a3;  --当 s 取其他值时,y=a3
END archmux;

```

小提示：条件信号赋值语句与 IF 语句的不同之处：

(1) 以上条件信号赋值语句不能进行嵌套，而 IF 语句是可以的。

(2) 由于条件信号赋值语句是并行语句，必须用在结构体中的进程之外（进程是用顺序语句来编写的），而 IF 语句是顺序语句。

(3) 条件信号赋值语句 ELSE 是必须有的，而 IF 可没有。

(4) 条件信号赋值语句与实际的硬件电路十分接近，因此使用该语句要求设计人员具有硬件电路知识，而 IF 语句一般用来进行硬件电路的高级描述，它不要求太多的硬件电路知识。

(5) 一般情况下很少用条件信号赋值语句，只有当用进程语句、IF 语句和 CASE 语句难以对路进行描述时才用。

3. 选择信号赋值语句 WITH-SELECT

语法规式为：

```

WITH 选择信号 X SELECT
信号 Y<= 信号 A WHEN 选择信号值 1,
信号 B WHEN 选择信号值 2,
信号 C WHEN 选择信号值 3,
...
信号 Z WHEN OTHERS;

```

WITH-SELECT 语句不能在进程中应用，它是通过选择信号 X 的值的變化来选择相应的操作。

当选择信号 X 的值与选择信号值 1 相同时，执行 $Y \leq \text{信号 A}$ ；当选择信号 X 的值与选择信号值 2 相同时，执行 $Y \leq \text{信号 B}$ ；只有当选择信号 X 的值与所列的值都不同时，才执行 $Y \leq \text{信号 Z}$ 。

采用选择信号赋值语句 WITH-SELECT 实现的四选一数据选择器结构体如下：

```

ARCHITECTURE archmux OF mux4 IS
BEGIN
WITH s SELECT
y <= a0 WHEN "00",
a1 WHEN "01",
a2 WHEN "10",
a3 WHEN OTHERS;

```

```
END archmux;
```

注意：WITH-SLECT 语句必须指明所有互斥条件，即“s”的所有取值组合，因为“s”的类型为“STD_LOGIC_VECTOR”，其取值组合除了 00、01、10、11，还有 0x、0z、x1……。虽然这些取值组合在实际电路中不出现，但也应列出。为避免麻烦可以用 OTHERS 代替其他各种组合。

4. 块 (BLOCK) 语句

为了实现复杂数字电路的程序设计，常常采用层次化设计和功能模块化设计方法，在 VHDL 语言中，实现这些功能的语句有：块语句、元件定义语句和元件例化语句、子程序（过程和函数），以及包和库等。

块语句可以看作结构体中的子模块，它把实现某一特定功能的一些并发语句组合在一起形成一个语句模块。利用多个块语句可以把一个复杂的结构体划分成多个不同功能的模块，使复杂的结构体结构分明，功能明确，提高结构体的可读性。块与块语句之间的关系是并行执行的，这种结构体的划分方法仅仅是形式上的，处于一个设计层次。块与块之间是不透明的，每个块都可以定义共块内使用的数据对象和数据类型，并且这种说明对其他块是无效的。另外，利用块语句中的保护表达式可以控制块语句的执行或利用块的保护表达式关闭某些信号。块语句的并行工作方式更为明显，块语句本身是并行语句结构，而且它的内部也都是由并行语句构成的（包括进程）。

块语句的格式为：

```
块标号: BLOCK
```

说明语句：

```
BEGIN
```

并行语句区：

```
END BLOCK 块标号;
```

在块语句说明部分中定义块内局部信号、数据类型、元件和子程序，在块内并行语句区可以使用 VHDL 中的所有并行语句。

例 5.6 设计一个电路，包含一个半加器和一个半减器，分别计算出 $A+B$ 和 $A-B$ 的结果。

逻辑表达式：

半加器： $SUM = A \oplus B$

$Co = AB$

半减法器： $SUB = A \oplus B$

$Bo = BA$

把加法和减法分成两个功能模块，分别用两个块语句来表示，设计的程序如下所示：

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
```

```
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY adsu is
PORT (
a, b : IN STD_LOGIC;
co, sum, bo, sub : OUT STD_LOGIC
);
END adsu ;

ARCHITECTURE a OF adsu IS
BEGIN
half_adder : BLOCK -- half_adder
BEGIN
sum <= A XOR B;
co<= A AND B;
END BLOCK half_adder;
half_subtractor: BLOCK -- half_subtractor
BEGIN
sub<= a XOR b;
bo <= NOT a AND b;
END BLOCK half_subtractor;
END a;
```

小提示：块语句只是起一种分隔符的作用，使程序编排更加清晰，有层次。功能上与不用块语句完全相同。块中定义的所有数据类型、数据对象（信号、变量、常量）和子程序等都是局部的；对于多层嵌套的块结构，这些局部定义量只适用于当前块，以及嵌套于本层块的所有层次的内部块，而对此块的外部来说是不可见的。由于 VHDL 综合器不支持保护式块语句（GUARDED BLOCK），因而在此不讨论该语句的应用。基于实用的观点，结构体中功能语句的划分最好使用元件例化（COMPONENT INSTANTIATION）的方式来完成。

5. 进程（PROCESS）语句

进程语句是在结构体中用来描述特定电路功能的程序模块。进程语句是最主要的并行语句，它在 VHDL 程序设计中使用频率极高，也是最能体现硬件描述语言特点的语句之一。进程语句的内部主要是由一组顺序语句组成的。进程中的语句具有顺序处理和并行执行的特点。进程提供了一种用算法（顺序语句）描述硬件行为的方法。在一个结构体中可以包含多个进程语句，多个进程语句之间是并行同时执行的，所以并行语句本身属于并行语句。进程语句可以用来描述组合逻辑电路，也可以描述时序逻辑电路。进程语句的综合是较为复杂的，主要涉及的问题包括：综合后的进程是用组合逻辑电路还是用时序逻辑电路来实现？进程语句中的对象是否有必要用寄存器、触发器、锁存器或 RAM 等存储器件来实现？进程语句结构中至少需要一个敏感信号量，否则除了初始化阶段，进程永远不会被再次激活。这个敏感信号量通常是一个同步控制信号，同步控制信号用在同步语句中，同步语句可以是敏感信号表、WAIT-UNTIL 语句或是 WAIT-ON 语句。一般来说，只有一个同步点的或者具有多个同步点但都使用完全相同的同步控制信号的进程不需要“记忆”在哪个同步点上被挂

起时，不会形成存储器。

进程语句的语法格式为：

```
<进程名称> : PROCESS <敏感信号表>
```

进程说明区：说明用于该进程的常数、变量和子程序。

```
BEGIN
变量和信号赋值语句
顺序语句
END PROCESS <进程名称>;
```

(1) 每个进程语句结构都可以取一个进程名称，但进程语句的名称是可以选用的。进程语句从 PROCESS 开始至 END PROCESS 结束。进程中的敏感信号表 (sensitivity list) 只能是进程中使用的一些信号，而不能是进程中的变量。当敏感信号表中的某个信号的值发生变化时，立即启动进程语句，将进程中的顺序语句按顺序循环执行，直到敏感信号表中的信号值稳定不变为止。如果敏感信号表不存在，那么在 PROCESS 中必须有 WAIT 语句，由 WAIT 语句来产生对信号的敏感；而当敏感信号表存在时，则在 PROCESS 中不能有 WAIT 语句；也可以用 WAIT 语句来启动进程。

(2) 在进程说明部分可定义常数、变量、元件和子程序等，但不能在进程内部定义信号，信号只能在结构体说明部分定义。

(3) 在进程中的语句是顺序语句，包括信号赋值语句、变量赋值语句、IF 语句、CASE 语句和 LOOP 语句等。

(4) 进程语句有两种存在状态，一是等待，当敏感信号没有发生变化时；二是执行，当敏感信号变化时。

用 PROCESS 语句描述计数器的程序如下：

```
PROCESS (CLK, Rd) --进程 (敏感信号表)
BEGIN
IF (Rd=.0.) THEN
Q <= "0000";
ELSIF (CLK. EVENT AND CLK=.1.) THEN
IF (en=.1.) then
Q <= Q+1;
END IF;
END IF;
END PROCESS;
```

在敏感信号表中，信号 Rd、CLK 被列为敏感信号，当这两个信号只要有一个发生变化时，此进程就被执行。注意，EN 并没有被列入敏感表，这是因为 EN 起作用必须发生在时钟的上升沿，这时 CLK 必定发生变化，引起进程的执行。同样，若为同步清零，敏感表中可无 Rd 信号，此时进程如下：

```
PROCESS ( CLK ) --进程 (敏感信号表)
BEGIN
```

```
IF (CLK. EVENT AND CLK=.1.) THEN
IF (Rd=.0.) THEN
Q<="0000";
ELSIF (EN=.1.) then
Q <= Q+1;
ENDIF;
END IF;
END PROCESS;
```

小提示：一个结构体可以包含多个进程，它们之间是并行执行的，但是进程结构中的语句是顺序执行的。进程之间通过信号来实现通信，进程信号的赋值语句是顺序执行的，但是赋值是最后并行执行的。

6. 元件例化语句 (COMPONENT INSTANT)

在 VHDL 程序设计中，一个完整的 VHDL 设计程序包括实体和结构体，实体提供设计单元的端口信息，结构体描述设计单元的结构和功能，设计程序通过综合、仿真等一系列操作后，其最终的目的是得到一个具有特定功能的电路元件，因此，把这种设计好的程序定义为一个元件 (COMPONENT)。元件可以是一个描述简单门电路的程序，也可以是一个描述一位全加器的程序，或者是其他复杂电路的描述程序。元件设计好后保存在当前工作目录中，其他设计体可以通过元件例化的方法调用这些元件。元件例化就是将预先设计好的设计实体定义为一个元件，然后利用映射语句将此元件与当前设计实体中的指定端口相连，从而为当前设计实体引入了一个低一级的设计层次，从而实现层次化设计。在结构体中，描述只表示元件（或模块）和元件（或模块）之间的互连，就像网表一样。当引用库中不存在某一元件时，必须首先进行元件的创建，然后将其放在工作库中，通过调用工作库来引用元件。在引用元件时，要先在结构体的说明部分对元件进行说明，然后在使用元件时进行元件例化。

元件例化语句也是一种并行语句，各个例化语句的执行顺序与例化语句的书写顺序无关，而是按照驱动事件并行执行的。

在进行元件例化时，首先要进行例化元件的说明，元件说明部分使用 COMPONENT 语句，COMPONENT 语句用来说明在结构体中所要调用的模块。如果所调用的模块在元件库中并不存在，那么设计人员必须首先进行元件的创建，然后将其放在工作库中通过调用工作库来引用该元件。

小提示：当前设计实体相当于一个较大（较大时相比于以前设计的实体而言）的电路系统，所声明的例化元件相当于系统板上的芯片，而当前设计实体的“端口”相当于要插入这个电路这块电路板上准备接受此芯片的一个插座。

元件定义语句和元件例化 (PORT MAP) 语句是用于在一个结构体中定义元件和实现元件调用的两条语句，这两条语句分别放在一个结构体中的不同位置，元件定义语句放在结构体的 ARCHITECTURE 和 BEGIN 之间，指出该结构体调用哪一个具体元件，元件调用时必须要进行数据交换，元件例化语句中的 PORT MAP 是端口映射的意思，表示结构体与元件端口之间交换数据的方式。语法格式分别为：

(1) 元件定义 (COMPONENT) 语句的格式:

COMPONENT 元件名称 IS

GENERIC 常量定义信息 (同该元件源程序实体中的 GENERIC 部分)

PORT 元件端口信息 (同该元件源程序实体中的 PORT 部分)

END COMPONENT;

小提示: 元件声明与实体声明基本一致, 元件就是一个实体。

(2) 元件例化 (PORT MAP) 语句的格式:

例化名: 元件名称 PORT MAP 元件端口列表

例 5.7 用元件定义和元件例化语句实现四位全加器的程序设计, 所调用的元件是一位全加器, 元件名称是 fulladder, 用 VHDL 描述的程序文件名是 fulladder. VHD, 四位全加器电路图如图 5.3 所示。

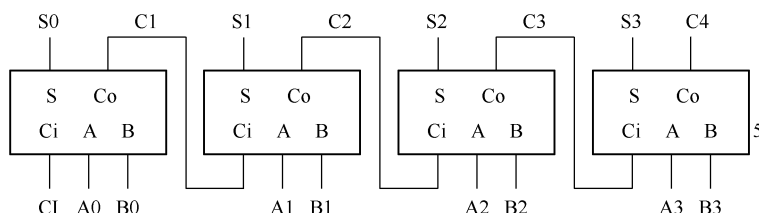


图 5.3 四位全加器电路图

四位全加器的程序文件名为 adder4. VHD, 内容如下:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY adder4 IS
PORT (
A, B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
CI : IN STD_LOGIC;
S : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
C : BUFFER STD_LOGIC_VECTOR(4 DOWNTO 1)
);
END adder4;
ARCHITECTURE a OF adder4 IS
COMPONENT fulladder --元件定义,fulladder 是元件名称
PORT(--端口名表
A, B, Ci : IN STD_LOGIC;
Co, S : OUT STD_LOGIC
);
END COMPONENT;
BEGIN
```

```

U0: fulladder PORT MAP (A(0),B(0),CI,C(1),S(0)); --元件例化
U1: fulladder PORT MAP (A(1),B(1),C(1),C(2),S(1));
U2: fulladder PORT MAP (A(2),B(2),C(2),C(3),S(2));
U3: fulladder PORT MAP (A(3),B(3),C(3),C(4),S(3));
END a;

```

在上述程序的元件定义语句中，COMPONENT 语句后面的元件名称是 fulladder，其在 PORT 中的端口信号信息与描述一位全加器的程序名 fulladder. VHD 在实体中 PORT 部分的端口信号信息必须相同，包括端口信号名称、端口信号的输入/输出状态和端口信号的数据类型等。COMPONENT 语句放在结构体的 ARCHITECTURE 和 BEGIN 之间。

在 PORT MAP 部分，元件名称 fulladder 必须与 COMPONENT 语句中的元件名称一致，U0、U1、U2、U3 是四个元件例化名，表明在这个结构体中对 fulladder 单元的四次不同的调用。

PORT MAP 中所列的端口信号列表表示当前设计单元与元件的端口连接方式，在 VHDL 设计中有两种连接方式，一种是位置关联方式，如

```
U0: Fulladder PORT MAP (A(0),B(0),Ci,C(1),S(0));
```

PORT MAP 列出的端口信号名与 COMPONENT 中的 PORT 端口信号名称在顺序、端口状态和数据类型上必须一致，各个端口信号的意义取决于它的位置而不是它的名称；另一种是端口信号名称关联方式，在这种关联方式下，用符号“=>”连接元件端口和设计电路的端口，如上例 U1 中，PORT MAP 列出的端口信号名称是 A(1)，B(1)，C(0)，C(1) 和 S(1)，PORT MAP 语句可以写成如下形式：

```
U1: Fulladder PORT MAP (A=>A(1), B=>B(1), Ci=>C(1), Co=>C(2), S=>S(1));
```

此时，各个端口的意义取决于端口的名称，与位置无关。

元件例化语句与块语句一样属于并行语句，但是元件和元件例化在设计项目中是分层次的，每个元件就是一个独立的设计实体，这样就可以把一个复杂的设计实体划分成多个简单的元件来设计。

7. 生成 (GENERATE) 语句

生成语句是一种循环语句，具有复制电路的功能。当设计一个由多个相同单元模块组成的电路时，就可以用生成语句来描述。生成语句有 FOR-GENERATE 和 IF-GENERATE 两种形式，分别说明如下：

(1) FOR-GENERATE 语句格式为：

```

标号:FOR 循环变量 IN 取值范围 GENERATE
并行语句
END GENERATE [标号];

```

FOR-GENERATE 语句与 FOR-LOOP 语句不同，FOR-GENERATE 中所列的语句是并行信号赋值语句、元件例化、块语句和子程序等并行语句。循环变量是一个局部变量，其取值范围可以选择递增和递减两种形式，如“0 TO 5”和“3 DOWNT0 1”等。生成语句所复制的单元模块是按照一定顺序排列的，而单元模块之间的关系却是并行的。所以生成语句属于并行语句。

例 5.8 用 FOR-GENERATE 语句实现四位全加器的程序设计如下：

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY adder4 IS
PORT (
A, B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
Ci : IN STD_LOGIC;
S : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
C : BUFFER STD_LOGIC_VECTOR(4 DOWNTO 0)
);
END adder4;
ARCHITECTURE a OF adder4 IS
COMPONENT fulladder
PORT ( A, B, Ci : IN STD_LOGIC;
Co, S : OUT STD_LOGIC);
END COMPONENT;
BEGIN
C(0) <= Ci;
gen1: FOR I IN 0 TO 3 GENERATE
addx: fulladder PORT MAP (A(I),B(I),C(I),C(I+1),S(I)); --产生四位串行全加器
END GENERATE ;
END a;
```

例 5.9 用生成语句描述用 4 个 D 触发器组成一个四位移位寄存器，电路图如图 5.4 所示。D 触发器用元件定义和元件例化语句调用。元件名称是 dff1，用 VHDL 描述的 D 触发器的程序参见例 5.8，程序的文件名是 dff1.VHD。

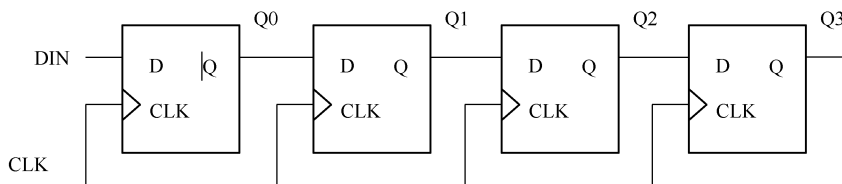


图 5.4 四位移位寄存器

四位移位寄存器的文件名为 shift.VHD，内容如下：

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY shift IS
PORT (
```

```

DIN, CLK : IN STD_LOGIC;
DOUT : OUT STD_LOGIC;
Q : BUFFER STD_LOGIC_VECTOR (3 DOWNTO 0)
);
END shift;

ARCHITECTURE B OF shift IS
COMPONENT dff1
PORT ( D, CLK : IN STD_LOGIC;
Q : OUT STD_LOGIC );
END COMPONENT;
SIGNAL D : STD_LOGIC_VECTOR (0 TO 4);
BEGIN
D(0) <= DIN;
gen2: FOR I IN 0 TO 3 GENERATE
fx: dff1 PORT MAP (D(I), CLK, D(I+1));
END GENERATE ;
Q(0) <= D(1);
Q(1) <= D(2);
Q(2) <= D(3);
Q(3) <= D(4);
DOUT <= D(4)
END B;

```

小提示：从上例可以看出，FOR-GENERATE 用来处理规则的单元模块，对于不规则的单元模块使用 IF-GENERATE 格式。

(2) IF-GENERATE 语句带有条件选择项，其格式为：

```

标号: IF 条件 GENERATE
并行语句
END GENERATE [标号];

```

IF- GENERATE 生成语句主要用来描述一个结构中的例外情况，例如，某些边界条件的特殊性。当执行到该语句时首先进行条件判断，只有条件为“TRUE”才会执行生成语句中的并行处理语句；如果条件为“FALSE”，则不执行该语句。

例 5.10 用 IF-GENERATE 语句描述的四位移位寄存器，其结构体部分程序如下：

```

ARCHITECTURE C OF shift IS
COMPONENT DFF1
PORT (
DIN, CLK : IN STD_LOGIC;
Q : OUT STD_LOGIC
);
END COMPONENT;
SIGNAL D : STD_LOGIC_VECTOR (0 TO 4);

```

```

BEGIN

gen3: FOR I IN 0 TO 3 GENERATE

  IF I = 0 GENERATE

    fx: dff1 PORT MAP (DIN,CLK,D(I+1));

  END GENERATE ;

  IF I /= 0 GENERATE

    fx: dff1 PORT MAP (D(I),CLK, D(I+1));

  END GENERATE ;

END GENERATE ;

Q(0) <= D(1);
Q(1) <= D(2);
Q(2) <= D(3);
Q(3) <= D(4);

END B;
```

5.3.2 顺序语句

顺序语句的执行方式类似于普通计算机语言的程序执行方式，都是按照程序中语句的前后排列方式顺序执行的，一次执行一条语句，在结构层次中前面的语句执行结果会直接影响后面各语句的执行结果，并且从仿真的角度来看是顺序执行的。顺序语句只能出现在进程（PROCESS）、过程（PROCEDURE）、函数（FUNCTION）中。顺序语句可以用来进行算术运算、逻辑运算、信号和变量的赋值、子程序调用等，还可以进行条件控制和迭代。注意，这里的顺序是针对仿真软件的运行和顺应 VHDL 语法的编程思路而言的，其相应的硬件逻辑工作方式未必如此。应该注意区分 VHDL 语言的软件行为与描述综合后的硬件行为的差异。常用的顺序语句有：（1）顺序赋值语句；（2）IF-ELSE 语句；（3）CASE-WHEN 语句；（4）FOR-LOOP 语句；（5）WHILE-LOOP 语句；（6）顺序跳出循环语句；（7）LOOP 语句；（8）WAIT 等待语句。

1. 顺序赋值语句

顺序赋值语句分为变量赋值和信号赋值，它们的赋值是有区别的。首先，在格式上，变量赋值格式为“变量名:=表达式”，而信号的赋值格式为“信号名<=表达式”；其次体现在所用的地方，变量说明和使用都只能在顺序语句中（进程、函数、过程和块模块），而信号的说明只能在同步语句中，但可以在顺序语句和同步语句中使用；再次体现在赋值过程，变量的赋值是立即的，而信号赋值的执行和信号值的更新至少需要 DELTA 延时，只有延时后信号才能得到新值，否则将保持原值，在进程中，信号赋值在结束时起作用。

变量赋值语句语法格式为：

变量赋值目标 := 赋值表达式

例：

```

VARIABLE s:BIT   := '0';

PROCESS (s)
```

```
VARIABLE count:INTEGER := '0'           --变量说明
BEGIN
    count := s+1                           --变量赋值
END PROCESS;
```

信号赋值语句的规范书写格式如下：

目的信号量 <= [TRANSPORT] [INERTIAL]信号变量表达式；

其中 [TRANSPORT] 表示传输延迟，[INERTIAL] 表示惯性延迟。要求“<=”两边的信号变量类型和位长度一致。

例：s <= TRANSPORT t AFTER 10ns； --传输延迟

d <= INERTIAL 2 AFTER 3ns, 1 AFTER 8ns； --惯性延迟

例：s <= a NOR (b AND c)； --信号赋值由敏感信号引起

3 个敏感量 a、b、c 中任何一个发生变化，该语句都将被执行。

2. IF-ELSE 语句

IF-ELSE 语句是最常用的顺序语句之一，主要用于流程控制。其用法和语句格式与普通的计算机高级语言类似，在 VHDL 语言中，它只在进程中使用，根据一个或一组条件来选择某一特定的执行通道。IF 语句不仅可用于选择器设计，还可用于比较器、译码器等进行条件控制的逻辑设计。IF 语句中至少应有一个条件句，条件句必须由布尔表达式构成，条件表达式中可使用关系运算操作及逻辑运算操作的组合表达式。

IF-ELSE 语句常用的格式为：

格式一：

```
IF 条件表达式 1 THEN
    语句方块 A
ELSIF 条件表达式 2 THEN
    语句方块 B
ELSIF 条件表达式 3 THEN
    语句方块 C
:
ELSE
    语句方块 N
END IF
```

格式二：

```
IF 条件表达式 THEN
    语句方块 A
END IF;
```

格式三：

```
IF 条件表达式 THEN
    语句方块 A
```

```
ELSE  
语句方块 B  
END IF
```

格式四:

```
PROCESS (CLK)  
BEGIN  
IF CLK' event AND CLK='1' THEN  
语句方块  
END IF;  
END PROCESS;
```

其中, 格式一是 IF 语句的完整形式, 格式二和格式三是 IF 语句的简化形式, 格式四是 IF 语句的一种特例, 它用于描述带有时钟信号 CLK 上升沿触发的时序逻辑电路。IF 语句可以嵌套使用。

IF 语句中至少应包含一个条件表达式, 先判断条件表达式的结果是否为真, 若为真, 则执行 THEN 后面的语句方块中的语句, 执行完以后就跳转到 END IF 之后的语句。若条件表达式的结果为假, 则执行 ELSE 之后的语句方块。

例如, 采用 IF-ELSE 语句实现的四选一数据选择器程序如下:

```
LIBRARY IEEE;  
USE IEEE.STD_LOGIC_1164.ALL;  
ENTITY mux4 IS  
    PORT (input: IN STD_LOGIC_VECTOR (3 DOWNT0 0);  
          sel: IN STD_LOGIC_VECTOR (1 DOWNT0 0);  
          Y: OUT STD_LOGIC);  
END mux4;  
ARCHITECTURE rtl OF mux4 IS  
BEGIN  
    PROCESS (input, sel)  
    BEGIN  
        IF (sel="00") THEN      Y<=input (0);  
        ELSIF (Sel="01") THEN  Y<=input (1);  
        ELSIF (Sel="10") THEN  Y<=input (2);  
        ELSE                    Y<=input (3);  
        END IF;  
    END PROCESS;  
END rtl;
```

每一个 IF 语句都必须有一个对应的 END IF 语句, IF 语句可以嵌套使用, 即在一个 IF 语句中可以调用另一个 IF 语句。ELSE IF 允许在 IF 语句中出现多次。

例 5.11 用格式四描述一般的 D 触发器的程序如下, D 触发器的电路符号如图 5.4 所示。

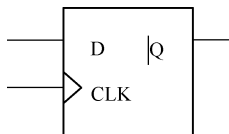


图 5.5 D 触发器

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY dff1 IS
PORT (
CLK, D : IN STD_LOGIC;
Q : OUT STD_LOGIC
);
END dff1;
ARCHITECTURE a OF dff1 IS
BEGIN
PROCESS (CLK)
BEGIN
IF CLK' EVENT AND CLK = '1' THEN
Q <= D;
END IF;
END PROCESS;
END a;

```

程序中，时钟信号（CLK）是敏感信号，用表达式 `CLK.EVENT AND CLK = 1` 判断 CLK 是否产生上升沿（由低电平变成高电平），若 CLK 产生上升沿，则执行 `Q <= D`，否则，Q 保持不变。

小提示：判断上升沿还可以写成 `IF RISING_EDGE(CLK)`；如果要判断时钟信号产生下降沿，可以用表达式 `CLK.EVENT AND CLK = 0`，或者 `FALLING_EDGE(CLK)`。其中 CLK 右上角的 `'` 表示信号的属性，其中 `EVENT` 表示信号内涵变化，是最常用的属性之一。对于信号的其他属性可以查阅相关书籍，由于使用不多，这里不做介绍了。

3. CASE-WHEN 语句

CASE-WHEN 语句属于顺序语句，只能在进程中使用，常用来选择有明确描述的信号。CASE-WHEN 语句根据满足的条件直接选择多项顺序语句中的一项执行，它常用来描述总线行为、编码器、译码器等的结构。

语法格式：

```

CASE 选择信号 X IS
WHEN 信号值 1 =>
语句方块 1

```

```

WHEN 信号值 2 =>
  语句方块 2
WHEN 信号值 3 =>
  :
WHEN OTHERS =>
  语句方块 N

```

CASE-WHEN 语句的功能与 WITH-SELECT 语句的功能相似，都是通过选择信号 X 的值的变化来选择相应的操作。但两者之间不同的是：

(1) CASE-WHEN 语句必须放在进程中，而 WITH-SELECT 语句是并行语句，必须放在进程外；

(2) CASE-WHEN 语句根据选择信号的值，执行不同的语句方块，完成不同的功能。而 WITH-SELECT 语句根据选择信号的值只能执行一个操作。

(3) 使用 CASE-WHEN 语句时，WHEN 语句中的信号值必须在选择信号的取值范围内，如果 WHEN 语句中列举的信号值不能覆盖选择信号的所有取值，就用关键字 OTHERS 表示未能列出的其他可能的取值。

例如，采用 CASE-WHEN 语句实现的四选一数据程序如下：

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY MUX41 IS
  PORT (S1,S2: IN STD_LOGIC;
        A,B,C,D:IN STD_LOGIC;
        Z: OUT STD_LOGIC);
END ENTITY MUX41;
ARCHITECTURE ART OF MUX41 IS
  SIGMA S:STD_LOGIC_VECTOR(1 DOWNT0 0);
BEGIN
  S<=S1 & S2;
  PROCESS (S,A,B,C,D) IS
  BEGIN
    CASE S IS
      WHEN "00" =>Z<=A;
      WHEN "01" =>Z<=B;
      WHEN "10" =>Z<=C;
      WHEN "11" =>Z<=D;
      WHEN OTHERS =>Z<='X' ;
    END CASE;
  END PROCESS;

```

该结构体的功能是：通过 PROCESS 对信号 S 进行感测，当 S = “00” 时，Y=A；当 S = “01” 时，Y=B；当 S = “10” 时，Y=C；当 S = “11” 时，Y=D。程序中用关键字 OTHERS 表示其他情况，Z<='X'。

与 IF 语句相比, CASE 语句组的程序语句是没有先后顺序的, 所有表达式的值都并行处理。IF 语句是有序的, 先处理最起始、最优先的条件, 后处理次优先的条件。

小提示: 使用 CASE 语句的时候要注意:

1. 至少要有一条分支条件;
2. 分支条件之间不能重叠;
3. 分支条件要覆盖完全, 如果不能完全列出, 则要使用 OTHERS 关键字, 以免综合出不必要的锁存器。

4. FOR-LOOP 语句

FOR-LOOP 语句是一种循环执行语句, 它可以使包含的一组顺序语句被循环执行, 其执行的次数可由设定的循环参数决定, 只要设计到重复的动作需求时, 就可以考虑使用循环语句。FOR-LOOP 语句分为递减方式和递增方式, 两种语法格式分别为:

(1) 递减方式

```
FOR I IN 起始值 DOWNTO 结束值 LOOP
```

顺序语句

```
END LOOP;
```

(2) 递增方式

```
FOR I IN 起始值 TO 结束值 LOOP
```

顺序语句

```
END LOOP;
```

在循环语句中, I 是决定循环次数的循环变量, 循环变量的变化范围由起始值和结束值的大小确定, 起始值和结束值都应该取整数。当采用 DOWNTO 递减方式时, 取起始值大于结束值, I 从起始值开始, 每执行一次循环后 I 递减 1, 直到结束值为止; 当采用 TO 递增方式时, 取结束值大于起始值, I 也是从起始值开始执行, 每次循环增加 1。

例 5.12 用 FOR-LOOP 语句描述奇偶校验器中的奇校验。输入四位二进制数, 当检测到数据中 1 的位数为奇数时, 输出 Y=1, 否则, Y=0。

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY loop1 IS
PORT (
D : IN STD_LOGIC_VECTOR(0 TO 7);    --输入 D 是八位二进制数
Y : OUT STD_LOGIC
);
END loop1;
tmp := .0.;
```

```

BEGIN
FOR I IN 0 TO 7 LOOP
tmp := tmp XOR D(I);           --变量赋值语句是立即赋值, tmp=tmp⊕D(I)
END LOOP;
Y<= tmp ;
END PROCESS;
END a;

```

5. WHILE-LOOP 语句

WHILE-LOOP 语句是一种循环语句，WHILE-LOOP 循环是 LOOP 循环的另一种形式，其语法格式为：

```

[标号:] WHILE 条件表达式 LOOP
    顺序语句;
END LOOP [标号];

```

循环标号用作该 WHILE-LOOP 循环语句的标识符，可有可无。WHILE 后面的条件表达式是布尔表达式。WHILE-LOOP 循环语句在每次执行前要先检查条件表达式的值，当条件表达式的值为 TRUE 时，就执行循环体中的顺序处理语句，执行完毕后返回到该循环的开始，然后再次检查条件表达式的值；若表达式的值为 FALSE，那么结束循环并转而执行 WHILE-LOOP 后的语句。

小提示：一般综合器不支持 WHILE-LOOP 语句，所以对于循环，推荐使用 FOR-LOOP 语句。

6. 顺序跳出循环语句

(1) NEXT 跳出循环语句

在 LOOP 中，NEXT 语句用于跳出本次循环，转入下次循环，并重新开始。

NEXT 语句有三种用法，其语法格式如下：

```

NEXT;                               -- 第一种语句格式
NEXT LOOP 循环标号;                 -- 第二种语句格式
NEXT LOOP 循环标号 WHEN 条件表达式; -- 第三种语句格式

```

当 LOOP 标号空缺时，则执行 NEXT 语句时，即刻无条件终止当前的循环，跳回到本次循环 LOOP 语句开始处，开始下一次循环，否则跳转到指定标号的 LOOP 语句开始处，重新开始执行循环操作。若 WHEN 子句出现并且条件表达式的值为 TRUE，则执行 NEXT 语句，进入跳转操作，否则继续向下执行。其中循环标号用来表明结束本次循环后下一次循环的起始位置；条件表达式的值为布尔量，是跳出本次循环的条件。

用法示例：

```

...
L1: FOR CNT IN 1 TO 8 LOOP
    S1:A(CNT):='0';
    NEXT WHEN (B=C);

```

```
S2:A (CNT+8):='0';  
END LOOP L1;
```

当程序执行到 NEXT 语句时, 如果条件判断式 ($B=C$) 的结果为 TRUE, 则执行 NEXT 语句, 并返回到 L1, 使 CNT 加 1 后执行 S1 开始的赋值语句, 否则执行 S2 开始的赋值语句。

(2) EXIT 退出循环语句

在 LOOP 中, EXIT 语句在循环体内用来描述退出循环并结束循环这一功能。EXIT 语句有三种用法, 其语法格式如下:

```
EXIT;                                -- 第一种语句格式  
EXIT LOOP 循环标号;                 -- 第二种语句格式  
EXIT LOOP 循环标号 WHEN 条件表达式; -- 第三种语句格式
```

这种语句格式与前述的 NEXT 语句的格式和操作功能非常相似, 唯一的区别是 NEXT 语句是跳向 LOOP 语句的起始点, 而 EXIT 语句则是跳向 LOOP 语句的终点。

WHEN 后的条件表达式是布尔表达式, EXIT 退出循环语句在条件表达式为真时, 退出循环标号指定的循环体。

EXIT 退出循环语句 WHEN 后面的条件表达式可以空缺, 成为无条件退出循环语句。

当遇到无条件退出循环语句时, 立即从循环标号指明的循环体中退出, EXIT 退出循环语句后的循环标号也可空缺, 成为默认无条件退出循环语句时, 立即从 EXIT 所在的循环体中退出。

小提示: NEXT 与 EXIT 语句类似于 C 语言中的 CONTINUE 与 BREAK 语句。

VHDL 还有 WAIT 语句、ASSERT 语句、NULL 语句, 这些语句并不常用, 如果要了解这些语句的用法, 请查阅相关资料。

7. LOOP 语句

LOOP 语句的语法格式如下:

```
[标号:] LOOP  
顺序语句;  
END LOOP [标号];  
用法示例:  
L2 :LOOP  
    a := a+1;  
    EXIT L2 WHEN a >10 ; --当 a 大于 10 时跳出循环  
END LOOP L2;  
...
```

8. WAIT 语句

WAIT 语句在进程中起到与敏感信号一样重要的作用, 敏感信号触发进程的执行, WAIT 语句同步进程的执行, 同步条件由 WAIT 语句指明。进程在仿真运行中处于执行或挂起两种状态之一。当进程执行到 WAIT 语句时, 将被挂起并设置好再次执行的条件。WAIT 语句有四种用法, 其语法格式如下:

```
WAIT;                                -- 第一种语句格式
WAIT ON 信号表;                      -- 第二种语句格式
WAIT UNTIL 条件表达式;              -- 第三种语句格式
WAIT FOR 时间表达式;                -- 第四种语句格式, 超时等待语句
```

一般只有 WAIT-UNTIL 格式的等待语句可以被综合器接受 (其余语句格式只能在 VHDL 仿真器中使用), WAIT-UNTIL 语句有以下三种表达方式:

```
WAIT UNTIL 信号=Value;              -- (1)
WAIT UNTIL 信号'EVENT AND 信号=Value; -- (2)
WAIT UNTIL NOT 信号'STABLE AND 信号=Value; -- (3)
```

9. 返回语句 (RETURN)

RETURN 语句是一段子程序结束后, 返回主程序的控制语句。它有两种格式:

```
RETURN;                             -- 第一种语句格式
RETURN 表达式;                      -- 第二种语句格式
```

第一种格式只能用于过程, 它后面一定不能有表达式; 第二种格式只能用于函数, 它后面必须有条件表达式, 它是函数结束的必要条件, 函数结束必须用 RETURN 语句。

5.4 VHDL 高级语句

前面详细介绍了 VHDL 常用的并行语句和顺序语句, 在此基础上进一步介绍 VHDL 中用于结构化、模块化和层次化的设计语句, 包括: 子程序、程序包、配置语句、断言语句等。

5.4.1 子程序 (SUBPROGRAM)

子程序由一组顺序语句组成, 是为了在程序中重复使用而设立的。子程序不是一个独立的编译单位, 子程序可以在程序包、结构体和进程中定义, 子程序必须在被定义后才能被调用, 在结构体中定义的子程序对于该结构体来说是局部的, 即不能被其他设计层次的结构体调用。如果要在其他结构体中调用同一个子程序, 就需要把子程序定义到程序包中。主程序调用子程序, 子程序将处理结果返回给主程序, 其含义与其他高级计算机语言中的子程序相同。

主程序和子程序之间通过端口参数列表位置关联方式进行数据传送, 子程序可以被多次调用完成重复性的任务。VHDL 中的子程序有两种类型: 过程 (PROCEDURE) 和函数 (FUNCTION), 它们在被调用后返回数据的方式不同。函数只有输入参数和单一的返回值, 而过程有任意多个输入、输出和双向参数, 可以具有多个返回值。过程被看作一种语句, 而函数通常是表达式的一部分。过程能单独存在, 而函数通常作为语句的一部分被请求。过程和函数都有两种形式: 即并行过程和并行函数, 顺序过程和顺序函数。并行的过程与函数可在进程语句和另一个子程序的外部, 而顺序函数和过程仅存在于进程语句和另一个子程序语句中。

1. 过程语句

过程语句的作用是传递信息，即通过参数进行内外的信息传递。其中，参数需说明（信号、变量及常量）类别、类型及传递方向。

过程语句的格式为：

```
PROCEDURE 过程名称参数列表      --过程首
PROCEDURE 过程名称参数列表 IS    --过程体
```

说明部分

```
BEGIN
```

顺序语句

```
END 过程名称;
```

调用过程语句的格式为：

```
过程名称 参数列表;
```

在 VHDL 中，过程（PROCEDURE）的定义由两部分组成，即过程首和过程体，在进程或结构体中过程首可以省略，过程体放在结构体的说明部分。而在程序包中必须定义过程首，把过程首放在程序包的包首部分，而过程体放在包体部分。

在 PROCEDURE 结构中，参数列表中的参数可以是输入也可以是输出，在参数表中可以对常数、变量和信号三类数据对象做出说明。用 IN、OUT 和 INOUT 定义这些参数的端口模式，在没有特别指定时，端口模式 IN 的参数默认为常数，端口模式 OUT 和 INOUT 看作变量。在子程序调用时，IN 和 INOUT 的参数传送数据至子程序，子程序调用结束返回时，OUT 和 INOUT 的参数返回数据。在过程语句执行结束后，如没有特别说明，OUT 和 INOUT 参数将按变量对待，将值传递给调用者的变量。如果调用者需要 OUT 和 INOUT 作为信号使用，则在过程参数定义时要指明是信号。过程的调用是一条语句，调用时通过其接口返回零个或多个值。根据环境的不同，过程调用有两种方式，即顺序语句方式和并行语句方式。在一般的顺序语句自然执行过程中，若一个过程被执行，则属于顺序语句方式；当某个过程处于并行语句环境中时，其过程体中定义的任一 IN 或 INOUT 的目标参量发生改变时，将启动过程的调用，这时的调用属于并行语句方式。

例 5.13 用一个过程语句来实现数据求和运算程序。

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY fun IS
PORT (A:IN std_logic_Vector(0 TO 2);
      M:OUT std_logic_Vector(0 TO 2));
```

```

END fun;

ARCHITECTURE art1 OF fun IS
    PROCEDURE sam1 (X,Y,Z: IN BIT;           --过程定义
                    N:OUT BIT) IS

    BEGIN
        N:= (X AND Y) OR  Z;

    END sam1;
BEGIN
    sam1 (A(0), A(1), A(2), N(0));
    sam1 (A(2), A(0), A(1), N(1));
    sam1 (A(1), A(2), A(0), N(2));           --过程调用是一条语句
    M(0)<= N(0);                             --等效于:M<= N(2)& N(1)& N(0);
    M(1)<= N(1);
    M(2)<= N(2);
END art1;

```

2. 函数语句

函数语句的作用是输入若干参数，通过函数运算求值，最后返回直接一个值。

函数语句分为两部分，即函数首和函数体。

(1) 函数首的格式为：

```

FUNCTION 函数名称 (参数列表)
RETURN 数据类型名;

```

(2) 函数体的格式为：

```

FUNCTION 函数名称 (参数列表)
RETURN 数据类型名 IS

```

说明部分

```

BEGIN

```

顺序语句

```

RETURN 返回变量
END 函数名称;

```

函数首是程序包与函数的接口界面。如果要将一个函数组织成程序包入库，则必须定义函数首，且函数首应放在程序包的说明部分，而函数体应放在程序包的包体内。如果只在一个结构体中定义并调用函数，则只需定义函数体即可。其中，参数表中包括参数名、参数类别及数据类型，函数的参数为信号或常数，默认情况为常数；在 RETURN 后面的数据类型为函数返回值的类型；说明部分用来说明函数体内引用的对象和过程；顺序语句就是函数体，用来定义函数的功能。在 PROCEDURE 结构中，参数可以是输入也可以是输出。而 FUNCTION 语句中括号内的所有参数都是输入参数或输入信号，因此在括号内指明端口方向的“IN”可以省略。

调用函数语句的格式为：

$Y \leftarrow$ 函数名称 (参数列表);

例 5.14 函数应用实例：

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY fun IS
PORT (A:IN std_logic_Vector(0 TO 2);
      M :OUT std_logic_Vector(0 TO 2));
END fun;
ARCHITECTURE art OF fun IS
    FUNDTION sam (X,Y,Z: BIT) RETURN BIT IS --函数定义
    BEGIN
        RETURN (X AND Y) OR Z;
    END sam;
PROCESS (A)
BEGIN
    M(0) <= sam(A(0), A(1), A(2));
    M(1) <= sam(A(2), A(0), A(1));
    M(2) <= sam(A(1), A(2), A(0)); --函数调用是一个表达式.

```

当 A 的 3 个位中任何一位有变化时，将启动对 sam 函数的调用，并将返回值赋给 M 输出。

```

END PROCESS;
END art;

```

小提示：子程序相当于 C 语言中的自定义函数（函数相当于有返回值的函数，过程相当于有输出参数的函数），可以在结构体的进程中进行直接调用，使语句更加简洁明了。但是也有区别：VHDL 中每调用一次子程序就相当于生成了一个相应的电路模块，所以不能无限调用子程序，而是要严格控制调用次数。

5.4.2 程序包的设计

在 VHDL 中，数据类型、常量及子程序在实体说明和结构体内定义，而这些数据类型、常量及子程序对其他设计实体是不可见的。为使它们对其他设计实体可见，VHDL 提供了程序包 (Package)。用程序包可定义一些公用的子程序、常量及自定义数据类型等。各种 VHDL 编译系统都含有多个标准程序包，如 Std-Logic-1164 和 Standard 程序包。用户也可自行设计程序包（保存到 WORK 库中）。程序包由两个独立的单元组成：即包首和包体，结构为：

(1) 包首部分

PACKAGE 程序包名称 IS

包首说明

END 程序包名称;

(2) 包体部分

PACKAGE BODY 程序包名称 IS

包体说明语句

END 程序包名称;

包首说明部分定义数据类型、元件和子程序等。包体说明语句部分具体描述元件和子程序的内容。在程序包结构中，程序包体不是必需的，因为在程序包首也可以具体定义元件和子程序的内容。

例 5.15 用 FUNCTION 语句描述在两个数中找出最大值，并用函数调用方式求出最大值。

这段程序放在一个程序包 (PACKAGE) 中。

(1) 在名称为 BF1 的程序包中定义名称为 MAX1 的函数，程序包文件名为 bf1. VHD，放在当前的 WORK 库中。

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
PACKAGE packexp IS                                --定义程序包
    FUNCTION max( a,b : IN STD_LOGIC_VECTOR)
        RETURN STD_LOGIC_VECTOR ;                --定义函数首
END ;
PACKAGE BODY packexp IS
    FUNCTION max( a,b : IN STD_LOGIC_VECTOR)
        RETURN STD_LOGIC_VECTOR IS              --定义函数体
        BEGIN
            IF a > b THEN RETURN a;
            ELSE RETURN b;
            END IF;
        END FUNCTION max;                        --结束 FUNCTION 语句
END;                                              --结束 PACKAGE BODY 语句
```

(2) 调用函数 MAX 的程序，文件名是 SMAX. VHD

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE WORK.packexp.ALL ;
ENTITY axamp IS
    PORT (dat1,dat2 : IN STD_LOGIC_VECTOR (3 DOWNTO 0));
    dat3,dat4 : IN STD_LOGIC_VECTOR (3 DOWNTO 0);
    out1,out2 : OUT STD_LOGIC_VECTOR (3 DOWNTO 0) );
END;
ARCHITECTURE bhv OF axamp IS
BEGIN
```

```

out1 <= max(dat1,dat2);      --用在赋值语句中的并行函数调用语句
PROCESS (dat3,dat4)
BEGIN
out2 <= max(dat3,dat4);      --顺序函数调用语句
END PROCESS;
END;
```

小提示：程序包是由 VHDL 语言编写的，所以其源程序要以 XXX.VHD 文件类型保存。若要使用程序包中声明的内容，在设计实体的开始，要使用 USE WORK.XXX.ALL 打开程序包，再调用程序包的内容。

5.4.3 配置语句

配置语句描述了层与层之间的连接关系，以及实体与构造体之间的连接关系。设计者可以利用配置语句选择不同的构造体，使其与要设计的实体相对应；在仿真某一个实体时，可以利用配置选择不同的构造体进行性能对比实验，以得到性能最佳的构造体。

配置语句的基本格式为：

```

CONFIGURATION 配置名 OF 实体名 IS
    [配置说明];
END 配置名;
```

配置语句根据不同情况，其说明语句有简有繁。最简单的默认配置格式为：

```

CONFIGURATION 配置名 OF 实体名 IS
    FOR 选配结构体名
        END FOR;
END 配置名;
```

这种配置用于选择不包含块（BLOCK）和元件（COMPONENTS）的构造体。在配置语句中只包含实体所选配的构造体名，其他什么都没有。

例如，加入了配置的 4 位等值比较器设计程序如下：

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY comp4 IS
    PORT(a, b: IN STD_LOGIC_VECTOR(3 downto 0);
         y: OUT STD_LOGIC);
END comp4;
--结构体 1:
ARCHITECTURE behavior OF comp4 IS
BEGIN
Comp: PROCESS(a, b)
    BEGIN
        IF a=b THEN
            y<= '1';
```

```

        ELSE
            y<='0';
        END IF;
    END PROCESS comp;
END behavior;
--结构体 2:
ARCHITECTURE dataflow OF comp4 IS
BEGIN
    y<='1' WHEN(a=b) ELSE '0';
END dataflow;
--结构体 3:
ARCHITECTURE structural OF comp4 IS
    COMPONENT xnor2
        PORT(in1, in2: IN STD_LOGIC;
              Out:      OUT STD_LOGIC);
    END COMPONENT;
    COMPONENT and4
        PORT(in1, in2, in3, in4: IN STD_LOGIC;
              Out:      OUT STD-LOGIC);
    END COMPONENT;
    SIGNAL s: STD_LOGIC_VECTOR(0 to 3);
BEGIN
    u0: xnor2 PORT MAP(a(0),b(0),s(0));
    u1: xnor2 PORT MAP(a(1),b(1),s(1));
    u2: xnor2 PORT MAP(a(2),b(2),s(2));
    u3: xnor2 PORT MAP(a(3),b(3),s(3));
    u4: and4  PORT MAP(s(0),s(1),s(2),s(3),y);
END structural;
--配置:
CONFIGURATION comp4_con OF comp4 IS
    FOR behavior
        END FOR;
END comp4_con;
```

5.4.4 ASSERT 断言语句

主要用于程序仿真、调试中的人机对话，它可以给出一个文字串作为警告和错误信息，基本书写格式如下：

```

ASSERT<条件>
REPORT<输出信号>    ——字符串
SEVERITY<错误级别>; ——有四种 NOTE、WARNING、ERROR 和 FAILURE
```

如果程序在仿真或调试过程中出现问题，断言语句就会给出一个文字串作为提示信息，

当程序执行到断言语句时，就会对 ASSERT 条件表达式进行判断，如果返回值为 TRUE，则断言语句不做任何操作，程序向下执行；如果返回值为 FALSE，则输出指定的提示信息和出错级别。

断言语句可以分为顺序断言语句和并行断言语句。并行断言语句可以放在实体说明、结构体和块语句中使用，可以放在任何要观察和调试的点上，而顺序断言语句只能在进程、函数和过程中使用。其实断言语句的顺序使用格式和并行使用格式是一样的，因此断言语句可以应用在任何场所，格式请看顺序的说明。

第 6 章

EDA 设计实验

第 1 章介绍了 EDA 设计流程，第 2 章和第 3 章介绍了 EDA 设计软件平台，第 4 章介绍了 EDA 设计硬件平台，第 5 章简单介绍了 VHDL 语言，至此，读者对整个 EDA 设计过程和设计工具都应该有了比较清楚的认识和理解。在此基础上，本章将给出 13 个 EDA 设计实验。这些实验都可在第 4 章所介绍的 EDA 设计硬件平台 DE2-115 上实现。对每一个实验，都给出了实验目的、实验原理、实验内容、实验步骤。在实验顺序和内容的安排上，按从简单到复杂，从分离模块到系统集成芯片设计的顺序安排。实验一和实验二均给出了详细的实验程序，主要目的是通过现成的设计程序，使读者能够迅速地熟悉 EDA 设计软件使用，同时对 EDA 硬件设计平台的功能、连接、按键和输出显示等输入/输出有所了解。从实验三开始，没有给出具体的实验程序，但对实验原理、端口设置都有较详细的描述，或给出其中较难的程序段，目的是为了提高读者对实验原理和实验内容的理解，提高用 VHDL 语言进行 EDA 设计的能力。因此，要求在做实验前做好充分准备，最好编写好实验程序。实验一到实验五属于基础实验，建议全部完成。后面的实验，可以根据专业不同进行选择。

实验一 基本数字电路芯片设计实验

一、实验目的

1. 掌握用 VHDL 语言实现组合逻辑电路和时序逻辑电路芯片的设计。
2. 掌握 DE2-115 硬件实验平台的使用。
3. 掌握数字集成电路 EDA 芯片设计流程和 Quartus II 软件的使用。

二、实验原理

1. 组合逻辑电路 (简称组合电路)

组合电路是由各种集成逻辑门按一定要求连接并实现某种逻辑功能的电路。这类电路的特点是在任意指定时刻的稳态输出仅取决于该时刻的输入变量的状态，而与之前各时刻的输入状态无关。

(1) 基本逻辑门

基本逻辑门电路是数字逻辑电路的基本单元电路，常用的有与非门、或非门和异或门等。

① 二输入与非门

二输入与非门的逻辑方程为 $y = a * b$ ，其真值表如表 6.1 所示。

表 6.1 二输入与非门真值表

a	b	y
0	0	1
0	1	1
1	0	1
1	1	0

根据逻辑方程和真值表，可写出二输入与非门的 VHDL 程序如下：
行为描述：

```
ENTITY nand2 IS
PORT ( a, b : IN STD_LOGIC;
      y : OUT STD_LOGIC );
END nand2;
ARCHITECTURE Behaviroal OF nand2 IS
BEGIN
    y <= a NAND b;
END behavioral;
```

数据流描述：

```
ENTITY nad2 IS
PORT ( a, b : IN STD_LOGIC;
      y : OUT STD_LOGIC );
END nand2;
ARCHITECTURE dataflow OF nand2 IS
BEGIN
    PROCESS ( a, b )
        VARIABLE comb: STD_LOGIC_VECTOR ( 1 DOWNT0 0 );
        Comb := a & b;
        CASE comb IS
            WHEN "00" => y <= '1';
            WHEN "01" => y <= '1';
            WHEN "10" => y <= '1';
            WHEN "11" => y <= '0';
            WHEN OTHERS => y <= 'X';
        END CASE;
    END PROCESS;
```

```
END dataflow;
```

② 二输入异或非门

二输入异或非门也称同或门，其逻辑方程为 $y = ab + \overline{ab}$ ，简记为： $y = a \odot b$ ，真值表如表 6.2 所示。

表 6.2 二输入异或非门真值表

a	b	y
0	0	1
0	1	0
1	0	0
1	1	1

根据逻辑方程和真值表，可写出二输入异或非门的行为描述 VHDL 程序如下：

```
ENTITY xnor2 IS
    Port (a,b: IN  std_logic;
          y:  OUT std_logic
          );
END xnor2;

ARCHITECTURE Behavioal OF xnor2 IS
    BEGIN
        y<=a XNOR b;
    END behavioral;
```

(2) 编/译码器

编码器、译码器和选择器是常用的组合电路。编码就是用二进制数码来表示一个特定的信息（十进制数、文字、符号和指令等），实现这一功能的逻辑电路称为编码器。把代码的特定含义翻译出来的过程称为译码，实现译码功能的逻辑电路称为译码器。显然译码是编码的逆过程，译码可以将二进制代码翻译成十进制数、字符或其他进制的代码，也可将十进制数或其他进制代码译成所需进制的信号码。

译码器在数字技术中有着广泛的应用：例如各种数字显示译码器，在数字通信设备中广泛使用的规则码发生器和分配器都是在译码器的基础上构成的。在计算技术中也广泛使用译码器，例如，存储器的地址译码器，控制器中的指令译码器等。译码器依据用途可分为通用译码器、码制转换译码器、数字显示译码器等类型。

数据选择器（多路选择器、多路器、多路开关）的功能是从多路输入数据中选择一路数据输出。它可以用来完成并行码输入/串行码输出的转换；也可作 n 线至 1 线的转换器；还可用它设计各种逻辑电路以及实现逻辑函数。它是应用非常广泛的一种中规模集成组件。

每种编码器、译码器和选择器也都有相应的逻辑方程和真值表，可以根据它们编写相应的 VHDL 代码来实现。

138 译码器通常用于计算机的地址译码，A、B、C 三条译码线确定 8 组译码输出地址的低位，其高位地址由三条译码允许线 $\overline{G2A}$ 、 $\overline{G2B}$ （低电平允许译码）和 G1（高电平允许译码）的组合逻辑确定。若 A、B、C 三条译码线与具有输出锁存功能的 I/O 口或数据锁存器输出口连接，则可组成 8 选 1 唯一选通电路，即每次只有一条线选通（低电平选通），在硬件电路设计中也比较常用。其真值表如表 6.3 所示。

表 6.3 138 译码器真值表

输 入					输 出							
G1	G2 *	C	B	A	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
X	1	X	X	X	1	1	1	1	1	1	1	1
0	X	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

* G2=G2A+G2B

138 译码器的行为描述 VHDL 程序如下：

```
ENTITY deco138 IS
    Port (A,B,C,G1,G2A,G2B: IN  std_logic;
          y:                      OUT std_logic_vector(7 downto 0)
          );
END deco138;
ARCHITECTURE behaviroal OF deco138 IS

    SIGNAL indata: std_logic_vector(2 downto 0);
BEGIN
    indata<=C & B & A;
    PROCESS(indata,G1,G2A,G2B)
    BEGIN
        IF (G1=' 1' and G2A=' 0' and G2B=' 0' ) THEN
            CASE indata IS
                WHEN  "000"=>y<= "11111110";
                WHEN  "001"=>y<= "11111101";
                WHEN  "010"=>y<= "11111011";
```

```
        WHEN "011" => y <= "11110111";
        WHEN "100" => y <= "11101111";
        WHEN "101" => y <= "11011111";
        WHEN "110" => y <= "10111111";
        WHEN "111" => y <= "01111111";
        WHEN others => y <= "XXXXXXXX";

    END CASE;

ELSE
    y <= "11111111";

END IF;

END PROCESS;

END behaviroal;
```

组合电路中还包括各种运算器，如半加器、全加器、乘法器等。它们也都有相应的逻辑方程和真值表，可以根据它们编写相应的 VHDL 代码来实现。

2. 时序逻辑电路 (简称时序电路)

时序电路是指电路的稳定输出不仅与该时刻的输入状态有关，而且与过去的输入状态有关的逻辑电路。因此在时序电路中，除了有反映现在输入状态的组合电路，还应包含能记忆过去状态的存储电路。

时序电路分为同步时序电路和异步时序电路两大类。在同步时序电路中，有一个统一的时钟同步脉冲，只有在时钟脉冲的特定时刻存储电路状态才变化，并且时钟脉冲间隔不能太短；只有在前一时钟脉冲所引起的电路响应完全结束之后，也就是电路进入新的稳定状态之后，下一个时钟脉冲才能到来，否则，电路状态就会发生混乱。在异步时序电路中，或者没有时钟脉冲（例如，由两个与非门构成的基本 R-S 触发器），或者虽有时钟脉冲，但是存储电路状态的变化与时钟脉冲是异步的。在实际应用的时序电路中，大多数属于同步时序电路，常用的时序电路有触发器、锁存器、寄存器、计数器等。

(1) 触发器

常用的触发器有 D 触发器、带异步复位/置位 D 触发器、J-K 触发器等。触发器一般用于数据锁存或产生跳变信号。触发方式有正跳变触发、负跳变触发和电平触发，根据触发器的真值表，可以很容易地写出各种触发器的 VHDL 程序。

表 6.4 所示是通用 J-K 触发器的真值表。

表 6.4 通用 J-K 触发器真值表

输 入 端					输 出 端	
PSET	CLR	CLK	J	K	Q	/Q
0	1	X	X	X	1	0
1	0	X	X	X	0	1
0	0	X	X	X	X	X
1	1	↑	0	1	0	1
1	1	↑	1	1	翻转	

续表

输 入 端					输 出 端	
PSET	CLR	CLK	J	K	Q	/Q
1	1	↑	0	0	Q ₀	/Q ₀
1	1	↑	1	0	1	0
1	1	0	X	X	Q ₀	/Q ₀

从真值表可以看出，这是一个带异步置位、清零的正跳变 J-K 触发器。PSET 和 CLR 端分别是置位和清零端，它们的组合逻辑决定触发器的状态。

由真值表可以写出此 J-K 触发器的 VHDL 行为描述为：

```
entity jkdff is
  Port (pset,clr,clk,j,k: in std_logic;
        qa,qb: buffer std_logic
        );
end jkdff;

architecture Behaviroal of jkdff is
begin
  process (pset,clr,clk,j,k)
  begin
    if pset='0' and clr='1' then
      qa<='1' ;
      qb<='0' ;
    elsif pset='1' and clr='0' then
      qa<='0' ;
      qb<='1' ;
    elsif pset='1' and clr='1' then
      if clk' event and clk='1' then
        if j='0' and k='1' then
          qa<='0' ;
          qb<='1' ;
        elsif j='1' and k='0' then
          qa<='1' ;
          qb<='0' ;
        elsif j='1' and k='1' then
          qa<=not qa;
          qb<=not qb;
        else
          null;
        end if;
      elsif clk='0' then
        null;
      end if;
    end if;
```

```
end if;  
end process;  
end Behavioral;
```

(2) 寄存器

寄存器是数字系统中的重要基本模块，它的种类较多，有通用寄存器、移位寄存器、循环移位寄存器等。这里给出一个移位寄存器的例子。

移位寄存器主要用于各种数据的变换，变换方式非常灵活：串行输入/串行输出、串行输入/并行输出、并行输入/串行输出，还有循环左移、循环右移等。

8 位移位寄存器采用 8 位串/并行输入，串行输出。当 8 位并行数据输入时，在同步时钟脉冲的控制下，高位首先移出，然后逐位从串行输出口输出，低位由通过串行输入口输入的数据补充。其移位时序图如图 6.1 所示。

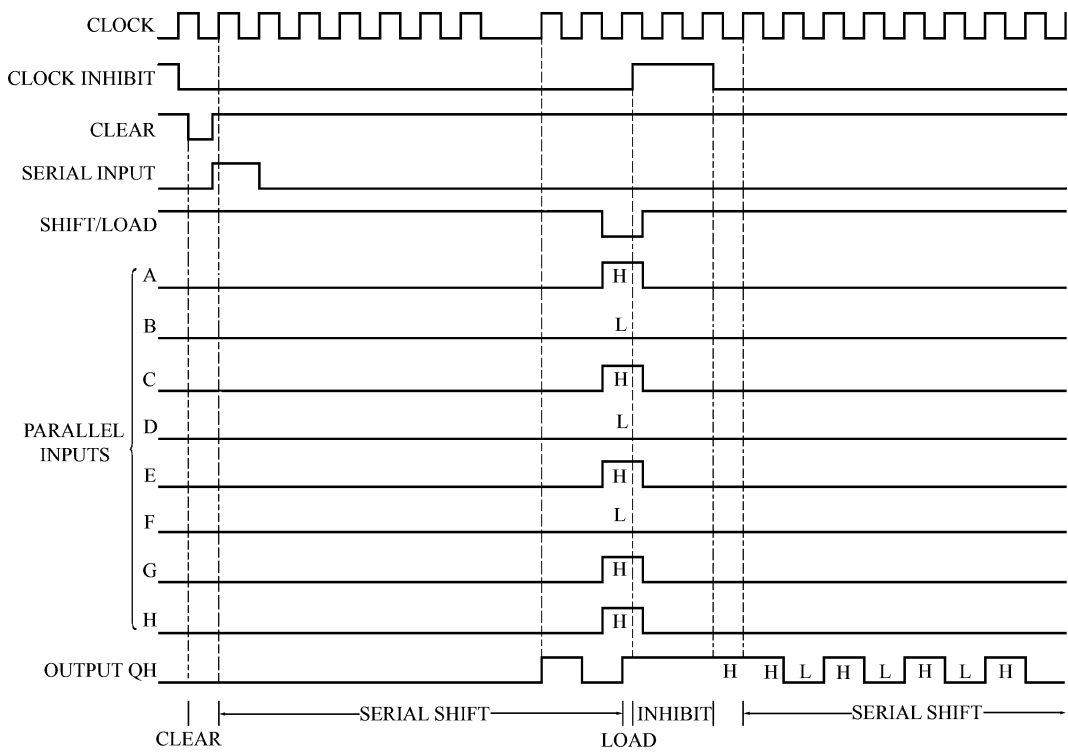


图 6.1 8 位移位寄存器移位时序图

移位寄存器的引脚如图 6.2 所示，各引脚功能如下：

- A-H：8 位并行数据输入端；
- SI：串行数据输入端；
- CLK：同步脉冲输入端；
- CKIH：时钟信号禁止端；
- S/L：串行、并行输入控制端；
- CLR：异步清零输入端；
- Q_H：串行数据输出端。

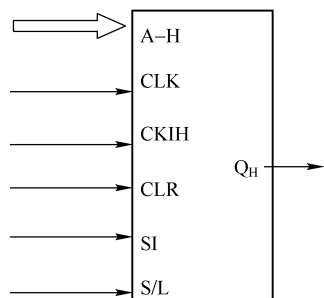


图 6.2 移位寄存器引脚

移位寄存器的引脚如从移位时序图可知：

- 当 CLR=0 时，内部寄存器及串行输出端清零。
- 当 CKIH=1 时，禁止移位，不管同步时钟信号如何变化，输出 Q_H 端状态保持不变。
- 当 S/L=0 时，允许并行数据输入，8 位数据储存在 $Q_{A0} \sim Q_{H0}$ 的 8 位内部寄存器中。
- 当 S/L=1 时，允许串行数据输入，1 位串行数据首先存入内部寄存器的最低位

Q_{A0} 中。

根据以上移位操作过程，可描述其行为如下：

```
entity shift01 IS
    port (clk,clr,ckih,s_l: IN std_logic;
          SI,A,B,C,D,E,F,G,H: IN std_logic;
          QH: OUT std_logic);
END shift01;

ARCHITECTURE Behaviroal OF shift01 IS
    SIGNAL Q:std_logic_vector(7 downto 0);
    BEGIN
    P1:PROCESS (CLR,CLK,CKIH,S_L,q(7))
    BEGIN
        QH<=Q(7);
        if clr='0' then
            Q<="00000000";
        elsif clk' event and clk='1' then
            if ckih='1' then
                Q<=Q;
            elsif s_l='0' then
                Q(0)<=A;
                Q(1)<=B;
                Q(2)<=C;
                Q(3)<=D;
                Q(4)<=E;
                Q(5)<=F;
                Q(6)<=G;
                Q(7)<=H;
            else
                Q(0)<=SI;
                G:FOR i in 6 downto 0 LOOP
                    Q(i+1)<=Q(i);
                END LOOP;
            end if;
        elsif clk='0' and ckih='0' then
            null;
        end if;
    END PROCESS;
```

END Behavioral;

三、实验内容

1. 根据 3-8 译码器真值表写出逻辑表达式,并画出逻辑实现框图,用逻辑门实现对 3-8 译码器的 VHDL 程序设计;用 FOR 循环语句实现 3-8 译码器,对各种 VHDL 程序实现进行逻辑综合和 ModelSim 仿真,比较综合和仿真结果。

2. 选择 3 位拨码开关作为输入,8 个 LED 作为输出,对 3-8 译码器电路芯片在 DE2-115 硬件实验平台上进行板级验证。

四、实验步骤

1. 创建新工程

打开 QuartusII 10.0 软件,如图 6.3 所示。



图 6.3 Quartus II 10.0 工作环境界面

选取创建新工程 Create a New Project (New Project Wizard), 出现新建工程向导 New Project Wizard: Introduction 对话框,单击“Next”按钮,出现 New Project Wizard: Directory, Name, Top-Level Entity [page 1 of 5] 对话框。输入工程工作路径、工程文件名以及顶层实体名。注意,这里输入的顶层实体名必须与之后设计文件的顶层实体名相同,默认的顶层实体名与工程文件名相同,本次实验采用这种命名方法。用户也可以根据需求输入不同的顶层实体名。输入结束后,如图 6.4 所示。单击“Next”按钮。

进入 Add File 对话框,在此如果没有已经编辑好的 VHDL 源文件就单击“Next”按钮,进入 New Project Wizard: Family & Device Settings [page 3 of 5] 对话框。根据 DE2-115 硬件实验平台上 FPGA 的规格,在 Family 下拉对话框中选择“CycloneIV E”,在 Target device 栏选择“Specific device selected in Available devices list”,在 Available devices 选择框中选择“EP4CE115F29C7”,设定好如图 6.5 所示,单击“Next”按钮。

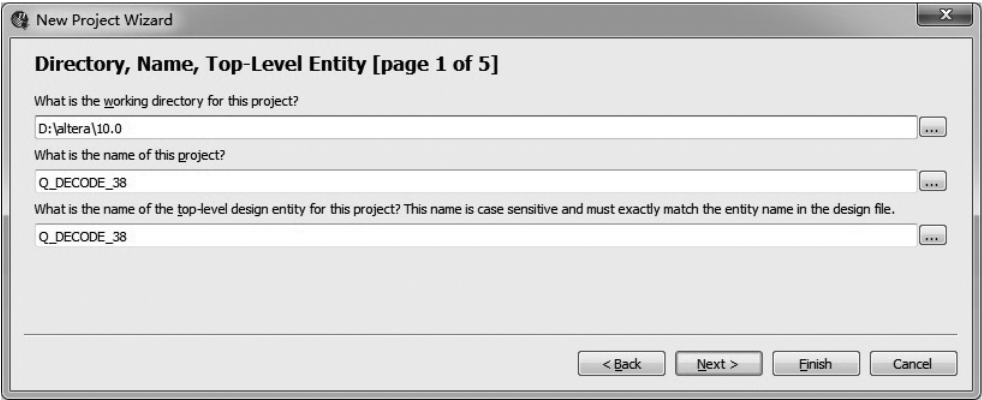


图 6.4 输入新建工程信息

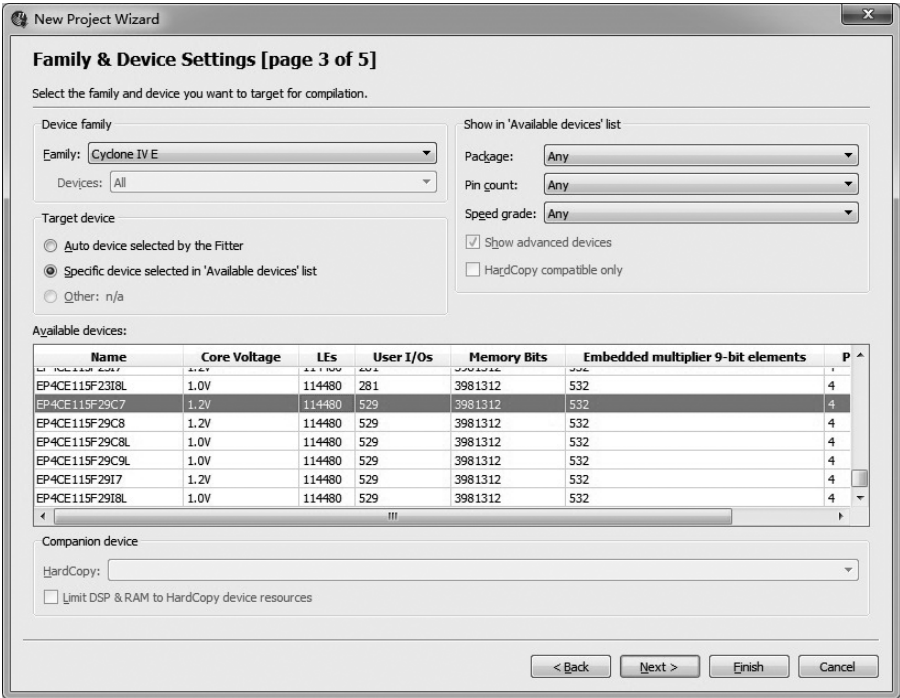


图 6.5 选择相应 FPGA 器件

进入 New Project Wizard: EDA Tool Settings 对话框，这里主要设置仿真工具，在 Simulation 的 Tool Name 栏设定仿真工具为“ModelSim-Altera”，在 Format(s) 栏选择“VHDL”，设定好如图 6.6 所示，单击“Next”按钮。如果实验中不使用这些工具，可以选择“None”。

进入 New Project Wizard: Summary 对话框，注意检查所有设定是否正确，检查无误后单击“Finish”按钮建立新建工程。

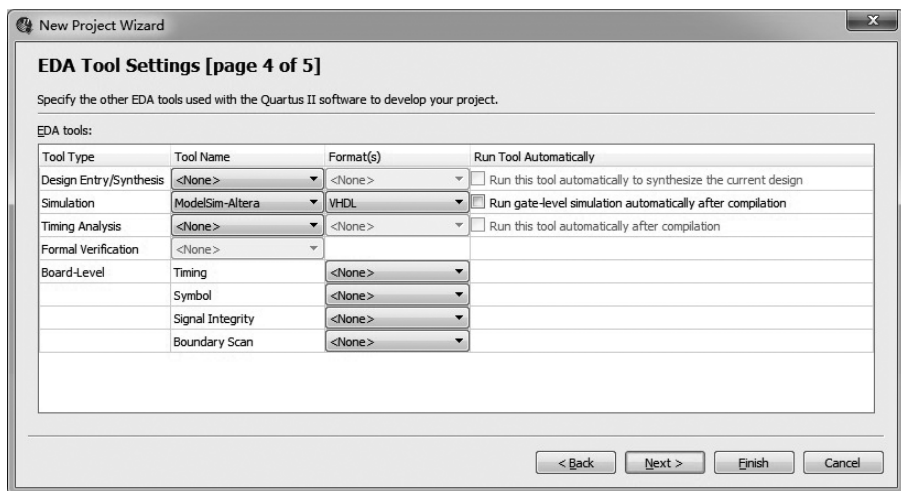


图 6.6 设置 EDA 仿真工具

2. 设计输入

选取菜单选项 File > New, 在 New 对话框里选择“VHDL File”, 单击“OK”按钮进入 VHDL 编辑器, 输入 3-8 译码器的 VHDL 源代码如图 6.7 所示。

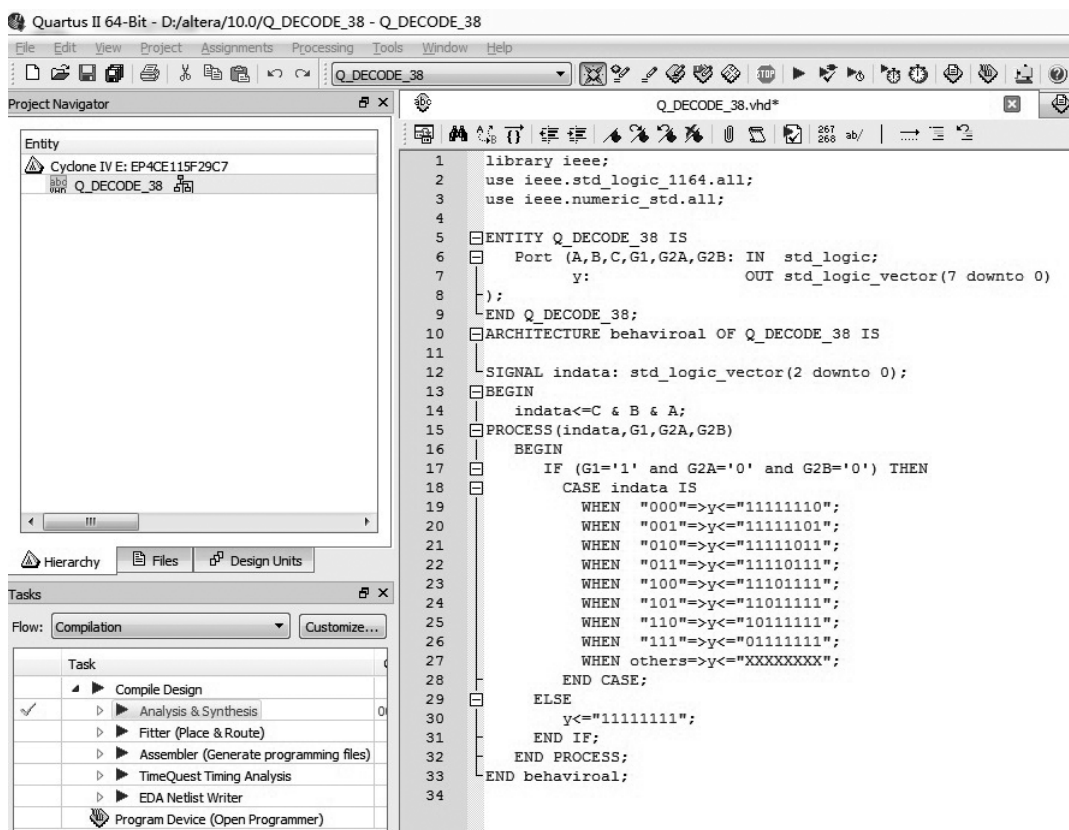


图 6.7 VHDL 代码编辑

保存设计。可单击菜单项 File->Save，单击图标，或者使用快捷键“Ctrl+S”保存设计。

3. 功能仿真

功能仿真可以使用 Quartus II 自带的波形仿真工具，推荐使用第三方软件 ModelSim 完成功能仿真，ModelSim 支持两种方式的仿真，一种是直接在 ModelSim 主窗口中输入命令，另一种是生成测试激励源，包括文本文件或波形文件。读者可自行选择一种，结果应是一致的。相对来说，第一种方法比较灵活，可以实时控制功能模拟，但如果一旦修改了源程序，所有命令都需要重新输入，比较烦琐；第二种方法不存在输入命令的问题，在使用时灵活运用。具体仿真过程请参考本书第 4 章内容进行。

4. 分析与综合

Quartus II 软件的编译器（Compiler）可以完成的功能包括：VHDL 程序语法检查、综合逻辑电路、布局布线、时序分析等。

对于简单设计，基本不用进行参数设置，选取菜单项 Processing>Star Compilation 或按钮工具栏的图标，就可以自动完成分析与综合的过程。编译完成时，会出现编译结果总结框，可以得到 FPGA 资源利用率等一些芯片设计关键信息。双击“RTL Viewer”，可观察生成的 RTL 级视图。

5. 分配引脚

在完成上述步骤之后，需要添加约束文件，主要是 FPGA 引脚的指定。具体引脚指定必须依据 DE2-115 硬件实验平台的具体硬件连线进行。在 DE2-115 实验平台使用手册中给出了所有 FPGA I/O 引脚与外围设备器件的引脚连接对应关系。本实验主要用到拨码开关作为输入，LED 作为显示输出。具体对应引脚分配如下：

（1）拨动开关引脚配置

Signal Name	FPGA Pin No.	Description	I/O Standard
SW[0]	PIN_AB28	Slide Switch[0]	Depending on JP7
SW[1]	PIN_AC28	Slide Switch[1]	Depending on JP7
SW[2]	PIN_AC27	Slide Switch[2]	Depending on JP7
SW[3]	PIN_AD27	Slide Switch[3]	Depending on JP7
SW[4]	PIN_AB27	Slide Switch[4]	Depending on JP7
SW[5]	PIN_AC26	Slide Switch[5]	Depending on JP7
SW[6]	PIN_AD26	Slide Switch[6]	Depending on JP7
SW[7]	PIN_AB26	Slide Switch[7]	Depending on JP7
SW[8]	PIN_AC25	Slide Switch[8]	Depending on JP7
SW[9]	PIN_AB25	Slide Switch[9]	Depending on JP7
SW[10]	PIN_AC24	Slide Switch[10]	Depending on JP7
SW[11]	PIN_AB24	Slide Switch[11]	Depending on JP7
SW[12]	PIN_AB23	Slide Switch[12]	Depending on JP7
SW[13]	PIN_AA24	Slide Switch[13]	Depending on JP7

续表

SW[14]	PIN_AA23	Slide Switch[14]	Depending on JP7
SW[15]	PIN_AA22	Slide Switch[15]	Depending on JP7
SW[16]	PIN_Y24	Slide Switch[16]	Depending on JP7
SW[17]	PIN_Y23	Slide Switch[17]	Depending on JP7

(2) LED 引脚配置

Signal Name	FPGA Pin No.	Description	I/OStandard
LEDR[0]	PIN_G19	LED Red[0]	2. 5V
LEDR[1]	PIN_F19	LED Red[1]	2. 5V
LEDR[2]	PIN_E19	LED Red[2]	2. 5V
LEDR[3]	PIN_F21	LED Red[3]	2. 5V
LEDR[4]	PIN_F18	LED Red[4]	2. 5V
LEDR[5]	PIN_E18	LED Red[5]	2. 5V
LEDR[6]	PIN_J19	LED Red[6]	2. 5V
LEDR[7]	PIN_H19	LED Red[7]	2. 5V
LEDR[8]	PIN_J17	LED Red[8]	2. 5V
LEDR[9]	PIN_G17	LED Red[9]	2. 5V
LEDR[10]	PIN_J15	LED Red[10]	2. 5V
LEDR[11]	PIN_H16	LED Red[11]	2. 5V
LEDR[12]	PIN_J16	LED Red[12]	2. 5V
LEDR[13]	PIN_H17	LED Red[13]	2. 5V
LEDR[14]	PIN_F15	LED Red[14]	2. 5V
LEDR[15]	PIN_G15	LED Red[15]	2. 5V
LEDR[16]	PIN_G16	LED Red[16]	2. 5V
LEDR[17]	PIN_H15	LED Red[17]	2. 5V
LEDG[0]	PIN_E21	LED Green[0]	2. 5V
LEDG[1]	PIN_E22	LED Green[1]	2. 5V
LEDG[2]	PIN_E25	LED Green[2]	2. 5V
LEDG[3]	PIN_E24	LED Green[3]	2. 5V
LEDG[4]	PIN_H21	LED Green[4]	2. 5V
LEDG[5]	PIN_G20	LED Green[5]	2. 5V
LEDG[6]	PIN_G22	LED Green[6]	2. 5V
LEDG[7]	PIN_G21	LED Green[7]	2. 5V
LEDG[8]	PIN_F17	LED Green[8]	2. 5V

进行 FPGA I/O 引脚指定常用的方式有两种，一种是直接写约束文件，另一种是采用可视化图形操作方式。这里给出后一种方式的具体步骤如下：

选取菜单选项 Assignments > Pin Planner，将光标移至 Pin Planner 对话框下侧的

“Location” 栏位下双击左键，即可依据上表进行 I/O 引脚指定，完成结果如图 6.8 所示。

选取菜单选项 Assignments>Device，在 Device 对话框中选择 “Device and Pin Options”，选取右侧分类中的 “Unused Pins”，确认 “Reserve all unused pins” 选项设为 “As input tri-stated with pull-up”。在此处若 “Reserve all unused pins” 选项设为 “As input tri-stated”，则在 FPGA 开发板上就要将这些未使用的 I/O 引脚接地或给定电压，避免空接状态。空接状态会增加 FPGA 功耗，还容易产生干扰噪声信号。相对地将 “Reserve all unused pins” 选项设为 “As input tri-stated with pull-up”，内部会直接拉高点位，就不需要在 FPGA 开发板上做实际的连线。

关闭 Pin Planner 对话框，重新进行编译。

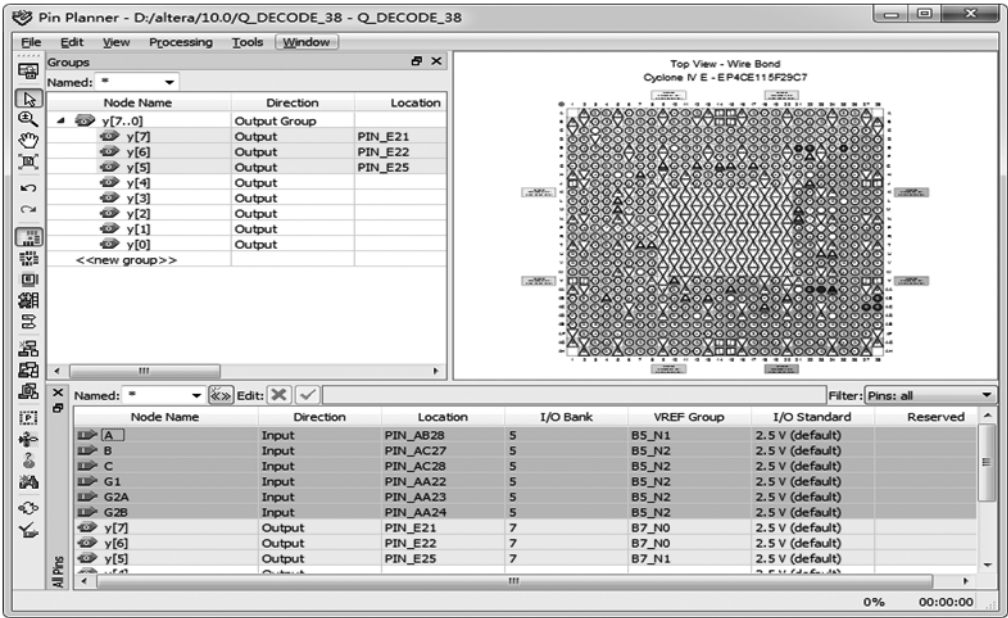


图 6.8 FPGA 引脚指定

6. FPGA 烧录（下载配置文件）

上述过程完成无误之后，就可以进行 FPGA 烧录，具体在 DE2-115 平台进行硬件测试。在 FPGA 烧录之前，一定要用 USB 连接线将 DE2-115 硬件实验平台的 USB Blaster 接口和计算机 USB 接口连接好，并将 DE2-115 平台接上电源。在第一次打开电源时会出现 “寻找新增硬件向导” 对话框，选择 “从清单或特定位置安装”，再按 “下一步” 按钮，将 “请选择您的搜寻和安装选项” 设定为 “搜寻时包含这个位置：Altera 软件安装硬盘盘符如 D:\Altera\10.0\quartus\drivers\usb-blaster”，设定好按 “下一步” 按钮。若出现警告窗口，选择 “继续安装”，安装完成后，按 “完成” 按钮。这样 USB 驱动就安装成功了，如图 6.9 所示。也可以在计算机硬件设备配置里进行上述 Altera Usb-Blaster 驱动安装。

接下来就可以执行下载实现文件步骤。选取菜单选项 Tools>Programmer，打开烧录窗口 “Programmer”，选取 “Hardware Setup”，打开 “Hardware Setup” 对话框，选择 “Hardware Setting” 页面，在 “Available hardware items:” 下拉对话框中双击 “USB-Blaster”。确认开

发板左下方 RUN/PROG 的滑动开关 (SW19) 拨到 RUN 位置。从烧录窗口中 “Add File” 加入烧录文件 “QDECODER_38.sof”, 并在要烧录文件项目的 Program/Configure 处勾选, 再按 “Start” 键进行 FPGA 烧录。烧录成功窗口如图 6.10 所示。至此, 整个基于 FPGA 的芯片 EDA 设计流程结束。控制 DE2-115 开发板上实验中用到的滑动开关, 观察 LED 彩灯显示是否和想要的逻辑功能一致。

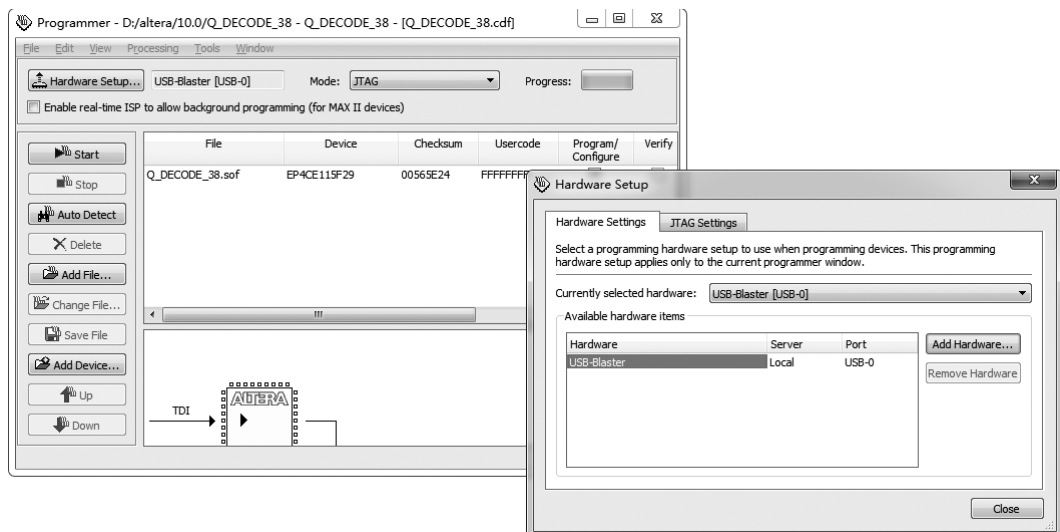


图 6.9 USB-BLASTER 硬件设定

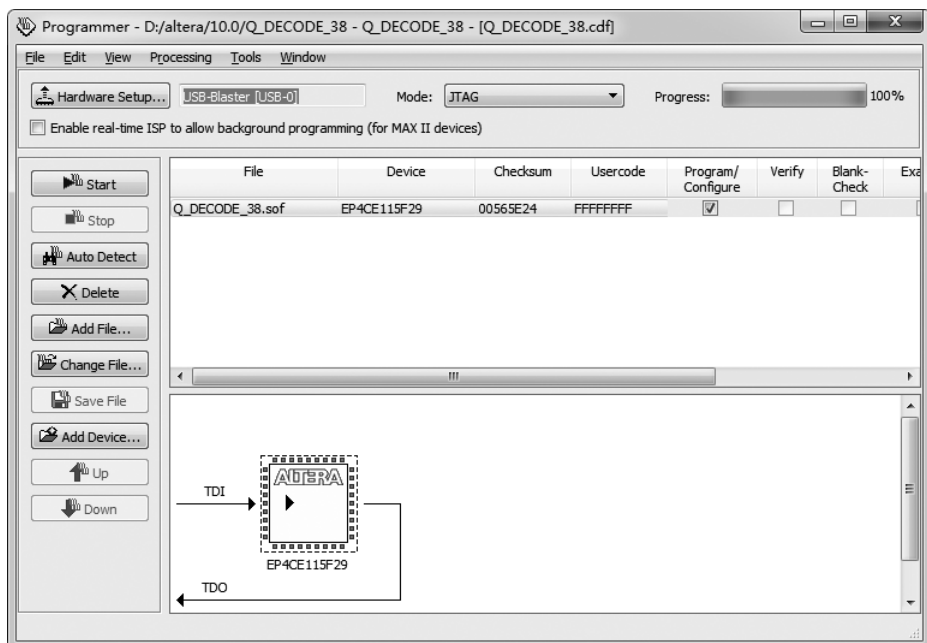


图 6.10 烧录成功窗口

实验二 彩灯显示芯片设计实验

一、实验目的

- 1. 掌握 VHDL 语言实现基本输入/输出芯片设计。
- 2. 掌握基于 FPGA 的数字集成电路 EDA 芯片设计流程。
- 3. 掌握 Quartus II 软件平台以及 DE2-115 硬件平台的使用。

二、实验原理

1. 开关和彩灯 LED 与 FPGA 接线

DE2-115 提供了 4 个按钮开关，每个按钮开关都通过一个施密特触发器进行了去抖动处理。74HC245 芯片有 4 个施密特触发器，它的输出信号分别为 KEY0、KEY1、KEY2、KEY3，直接连接到 Cyclone IV E FPGA，如图 6.11 所示。当按钮没有被按下时，它的输出是高电平，按下去则给出一个低电平。得益于去抖动电路，这些按钮开关适合用来给内部电路提供（模拟的）时钟或复位信号。

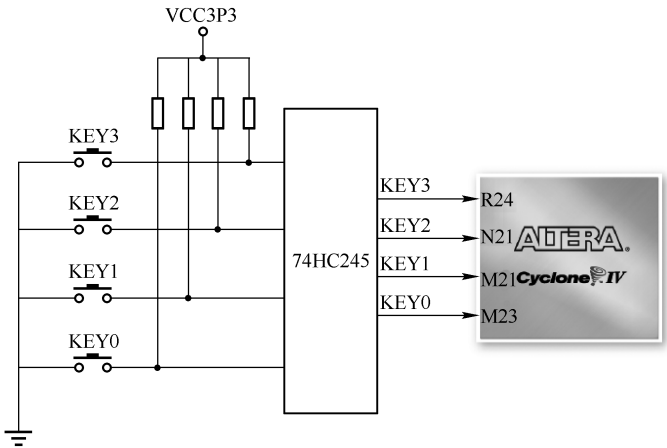


图 6.11 按钮开关和 FPGA 的连接示意图

DE2-115 开发板上还有 18 个拨动开关。这些开关没有去抖动电路，它们可以作为对电平敏感的电路的输入数据。每个开关都直接连接到 Cyclone IV E FPGA，如图 6.12 所示。当拨动开关在 DOWN 位置（靠近开发板边缘）时，输出为低电平，在 UP 位置时，输出为高电平。

DE2-115 实验平台共有 27 个直接由 FPGA 控制的 LED。在实验平台底部 18 个拨动开关的正上方有 18 个红色的 LED，8 个绿色 LED 可以在按钮开关的上方找到（第九个 LED 位于 7 段数码管的中间）。每一个 LED 都由 Cyclone IV E FPGA 的一个引脚直接驱动，其输出高电平则点亮 LED，输出低电平 LED 熄灭。图 6.13 给出了 LED 和 Cyclone IV E FPGA 之间的连接示意图。实验中所用到的彩灯为 18 个红色 LED，在 DE2-115 实验平台上的器件编号

为 LEDR17 ~ LEDR0, 在本实验中作为电路的输入器件使用。

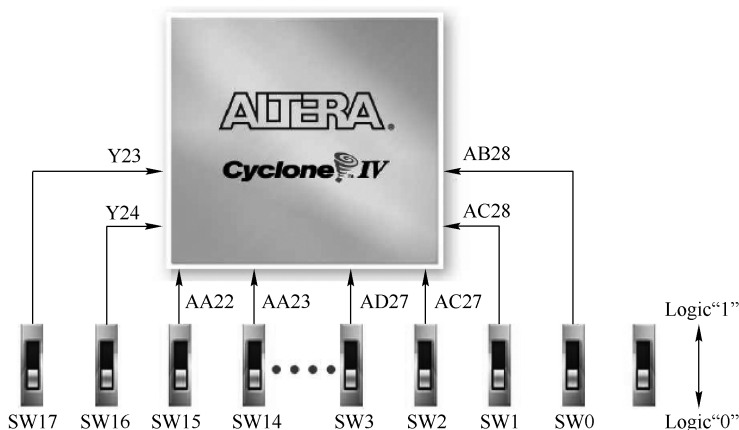


图 6.12 拨动开关和 FPGA 连接示意图

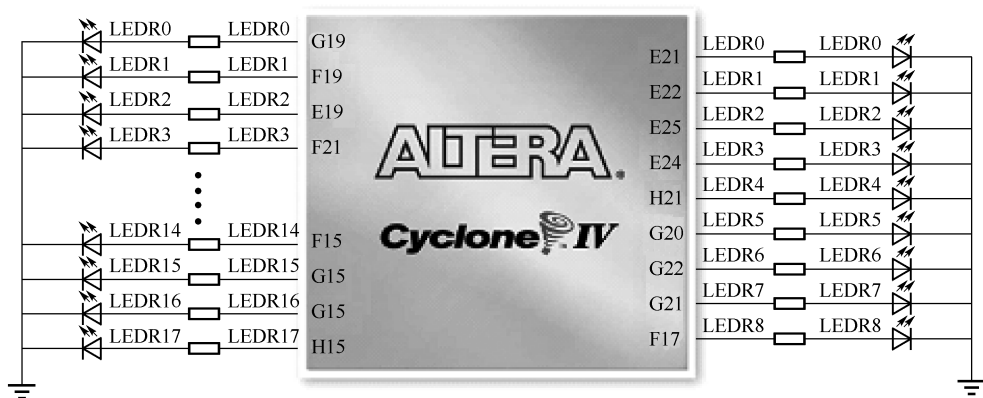


图 6.13 LED 和 FPGA 连接示意图

2. 彩灯移位的实现

实验中可观察到彩灯采用不同组合顺序点亮及熄灭, 下面以模式 1 来讲解彩灯移位点亮的原理。模式 1 的实验现象是: 彩灯从左至右, 即从 D111 至 D101 逐个点亮并熄灭, 当 D101 点亮并熄灭后, 从 D111 重新开始点亮、熄灭, 以此种模式一直循环, 描述模式 1 的 VHDL 源代码段如下:

```
if cled="000000000000" then
    cled(10)<='1';
else
    cled<='0' &cled(10 downto 1);
end if;
```

cled="000000000000" 描述的是彩灯全灭的情况, 即 D101 熄灭后彩灯的状态, 下一个状态应是 D111 点亮, 由语句 cled(10)<='1' 产生。

在 D111 点亮后, 应进入彩灯移位的状态, 在这里使用了语句 cled<='0' &cled(10

downto 1), 此语句使处于高电平的信号不断后移, 所对应的点亮的彩灯也不断后移, 当把高电平信号移出 cled(10 downto 0)时, 出现 cled="00000000000" 的状态, 即彩灯全灭。

综合上述两种状态, 实现了彩灯逐个移位的功能。

三、实验内容

SW1 在 0 位置时, 彩灯关闭, 数码管显示“OFF”; SW1 在 1 位置时, 彩灯处于复位状态, 数码管显示“CLR”。

AN1 键为模式切换按键, 按下 AN1 键可使彩灯的点亮模式在模式 0 至模式 8 之间循环切换, 同时数码管的显示也在“CLR”、“LF1”至“LF8”之间循环切换。

各点亮模式如下:

模式 0: 数码管显示“CLR”, 复位, 彩灯全灭;

模式 1: 数码管显示“LF1”, 彩灯从左至右移位逐个点亮;

模式 2: 数码管显示“LF2”, 彩灯从右至左移位逐个点亮;

模式 3: 数码管显示“LF3”, 彩灯从中间向两边移位逐个点亮;

模式 4: 数码管显示“LF4”, 彩灯从两边向中间移位逐个点亮;

模式 5: 数码管显示“LF5”, 彩灯从左至右依次点亮, 从右至左依次熄灭;

模式 6: 数码管显示“LF6”, 彩灯从中间向两边依次点亮, 反向依次熄灭;

模式 7: 数码管显示“LF7”, 彩灯从两边向中间依次点亮, 反向依次熄灭;

模式 8: 数码管显示“LF8”, 全部彩灯闪烁。

四、实验步骤

1. 编写 VHDL 程序实现实验内容要求的彩灯显示芯片设计, 进行语法检查, 直到语法检查正确。

2. 用 VHDL 语言编写 TestBench 文件, 对实现的彩灯显示 VHDL 程序进行 ModelSim 功能仿真, 如仿真波形有误, 需重新修改 VHDL 程序, 重复以上各个步骤, 直到得到正确的波形仿真结果。

3. 对 VHDL 语言实现的彩灯显示芯片设计进行逻辑综合, 并进行综合后仿真。

4. 对 VHDL 语言实现的彩灯显示芯片设计进行实现过程 (包括翻译、映射、布局布线 3 个小步骤)。

5. 布局布线后进行时序仿真。

6. 参考附录进行 FPGA 引脚分配。

7. PC 机通过写入电缆与硬件实验平台连接, 硬件实验平台与电源连接, 检查无误后, 开启电源, 把仿真正确的 VHDL 编译代码写入 DE2-115 硬件实验平台。

8. 进行硬件在线调试, 检查结果是否符合设计要求。若不符合, 需要修改 VHDL 程序, 重复以上各个步骤, 直到硬件在线调试正确。

五、实验源代码

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```

use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity exp1 is
    Port (clk1:    in  std_logic;
          clk2:    in  std_logic;
          sw1:     in  std_logic;    --总开关
          an1:     in  std_logic;    --模式切换按钮
          segcode2: out std_logic_vector(6 downto 0);  --段码中的数字信号
          segcode1: out std_logic_vector(6 downto 0);  --段码中的数字信号
          segcode0: out std_logic_vector(6 downto 0);  --段码中的数字信号
          cled:     buffer std_logic_vector(10 downto 0) --彩灯信号
    );
end exp1;

architecture Behavioral of exp1 is
    signal mode:std_logic_vector(3 downto 0):="0000";
    signal flag:std_logic;
    signal ancouter:std_logic_vector(9 downto 0);
    signal mode_disp:std_logic_vector(6 downto 0);

begin
    process (clk1,sw1,an1,cled)                                --彩灯控制进程
    begin
        if sw1='1' then                                         --开关
            if clk1' event and clk1='1' then
                if mode="0000" then                             --模式0:复位
                    cled<= (others=>'0');
                elsif mode="0001" then                           --模式1:左至右移位亮
                    if cled="000000000000" then
                        cled(10)<='1';
                    else
                        cled<='0' & cled(10 downto 1);
                    end if;
                elsif mode="0010" then                           --模式2:右至左移位亮
                    if cled="000000000000" then
                        cled(0)<='1';
                    else
                        cled<=cled(9 downto 0)&'0';
                    end if;
                elsif mode="0011" then                           --模式3:从中间向两边移位亮
                    if cled="000000000000" then

```

```

        cled(5)<='1';
    else
        cled(10 downto 5)<=cled(9 downto 5)&'0';
        cled(5 downto 0)<='0' &cled(5 downto 1);
    end if;
elseif mode="0100" then                                --模式 4:从两边向中间移位亮
    if cled="000000000000"then
        cled(10)<='1';
        cled(0)<='1';
    else
        cled(10 downto 5)<='0' &cled(10 downto 6);
        cled(4 downto 0)<=cled(3 downto 0)&'0';
    end if;
elseif mode="0101" then                                --模式 5:从左至右依次亮,从右至左依次灭
    if flag='0' then
        cled<='1' &cled(10 downto 1);
    else
        cled<=cled(9 downto 0)&'0';
    end if;
elseif mode="0110" then                                --模式 6:从中间向两边依次亮,反向依次灭
    if flag='0' then
        cled(10 downto 5)<=cled(9 downto 5)&'1';
        cled(5 downto 0)<='1' &cled(5 downto 1);
    else
        cled(10 downto 5)<='0' &cled(10 downto 6);
        cled(5 downto 0)<=cled(4 downto 0)&'0';
    end if;
elseif mode="0111" then                                --模式 7:从两边向中间依次亮,反向依次灭
    if flag='0' then
        cled(10 downto 5)<='1' &cled(10 downto 6);
        cled(5 downto 0)<=cled(4 downto 0)&'1';
    else
        cled(10 downto 5)<=cled(9 downto 5)&'0';
        cled(5 downto 0)<='0' &cled(5 downto 1);
    end if;
elseif mode="1000" then                                --模式 8:全部彩灯闪烁
    if flag='0' then
        cled<=(others=>'1');
    else
        cled<=(others=>'0');
    end if;
else
    cled<=(others=>'0');

```

```

        end if;
    end if;

    if cled="000000000000" then                --彩灯标志位
        flag<='0';
    elsif cled="1111111111" then
        flag<='1';
    else
        null;
    end if;
else
    null;
end if;
end process;

process (sw1,mode,mode_disp)                  --显示控制进程
begin
    if sw1='1' then
        if mode="0000" then                    --显示“clr”
            segcode0<="1010000";
            segcode1<="0111000";
            segcode2<="0111001";
        else                                    --显示“LF*”,*为模式编号
            segcode0<=mode_disp;
            segcode1<="1110001";
            segcode2<="0111000";
        end if;
        case mode is
            when "0001"=>mode_disp<="0000110";    --显示“1-8”段码
            when "0010"=>mode_disp<="1011011";
            when "0011"=>mode_disp<="1001111";
            when "0100"=>mode_disp<="1100110";
            when "0101"=>mode_disp<="1101101";
            when "0110"=>mode_disp<="1111101";
            when "0111"=>mode_disp<="0000111";
            when "1000"=>mode_disp<="1111111";
            when others=>mode_disp<="0000000";
        end case;
        segcode0<="1110001";                    --显示“OFF”
        segcode1<="1110001";
        segcode2<="0111111";
    end if;
end process;

```

```
process (clk2,an1)                                --键盘防抖
begin
    if an1='1' then
        if clk2' event and clk2='1' then
            if ancounter(9)='0' then
                ancounter<=ancounter+1;
            elsif ancounter(9)='1' then
                null;
            end if;
        end if;
    else
        ancounter<=(others=>'0');
    end if;
end process;

process (ancounter(9),sw1)                        --模式切换
begin
    if sw1='1' then
        if ancounter(9)' event and ancounter(9)='1' then
            if mode="1000" then
                mode<="0000";
            else
                mode<=mode+1;
            end if;
        else
            null;
        end if;
    else
        mode<="0000";
    end if;
end process;
end Behavioral;
```

实验三 数码管显示芯片设计实验

一、实验目的

1. 掌握 8 段数码管的显示原理。
2. 掌握用 VHDL 语言实现对数码管的各种显示控制。
3. 掌握计数器、分频器等基本电路的设计。

二、实验原理

LED 数码管显示是一种常用的显示方式，由于它具有控制简单、亮度高、功耗低、容易做到大型化等优点，因此被广泛应用于时钟、计算器、消息显示等仅需要显示简单数字或符号的设备中。常用的数码管有 7 段式、8 段式和米字型等，其中 8 段式的数码管最为常用，它由 7 段发光线段和一个发光点组成，并按“日”字形排列，每一段都是一个发光二极管，如图 6.14 所示。

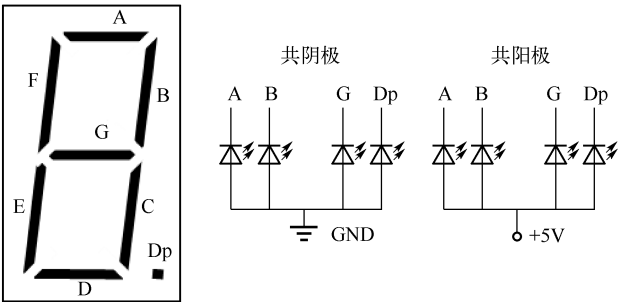


图 6.14 8 段数码管显示模块结构图

LED 数码管有共阴极和共阳极两种。共阴极数码管的 LED 的阴极连接在一起，并将此公共端接地，当某个 LED 的阳极为高电平时，LED 被点亮，相应的段显示；同样，共阳极的数码管则将 LED 的阳极连接在一起，并将此公共端接正电压，当某个 LED 的阴极为低电平时，LED 被点亮，相应的段显示。数字量与段的对应关系如下：

D7	D6	D5	D4	D3	D2	D1	D0
Dp	G	F	E	D	C	B	A

8 段数码管的常用字型码如表 6.5 所示。

表 6.5 8 段数码管常用字型码

8 段 LED 字型码			
显示字符	共阴极	共阳极	
0	00111111	11000000	
1	00000110	11111001	
2	01011011	10100100	
3	01001111	10110000	
4	01100110	10011001	
5	01101101	10010010	
6	01111101	10000010	
7	00000111	11111000	
8	01111111	10000000	
9	01101111	10010000	

续表

8 段 LED 字型码		
显示字符	共阴极	共阳极
A	01110110	10001001
B	01111100	10000011
C	00111001	11000110
D	01011110	10100001
E	01111001	10001110
F	0111001	1000110

DE2-115 实验平台配有八个 7 段数码管（不考虑 Dp 这个管脚）。它们被分成两组，每组 4 个，用来作为数字显示用。正如图 6.15 所示，7 段数码管的每个引脚（共阳模式）均连接到 Cyclone IV E FPGA。FPGA 输出低电压时，对应的字码段点亮，反之则熄灭。每个数码管的字段都从 0 到 6 依次编号，左图给出了它们的编号次序。

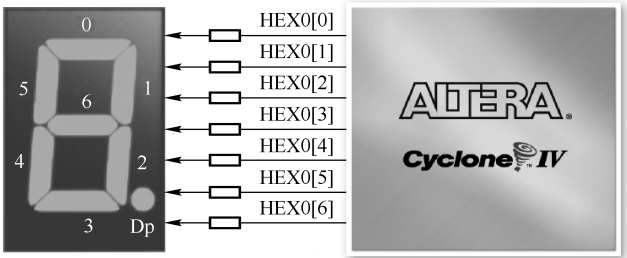


图 6.15 DE2-115 Cyclone IV E FPGA 和 7 段数码管间连接示意图

在 VHDL 软件设计方面，因为硬件设计中选用了译码器作为公共端的选通，因此控制时只需在向译码器发送译码信号的同时把需要显示的数据也从数据线上送出，则可以在被选中的相应序号数码管中显示指定的信息。在频率选择方面，由于每一个 LED 数码管是按照顺序被循环点亮的，如果主频率太低，则会产生一种流动或闪烁的现象，因此为了避免闪烁，产生一个连续稳定的显示效果，必须使用高于 100Hz 的频率作为主频率，使数码管的显示看上去是连续的。部分 LED 显示的 VHDL 程序如下：

```
entity LED is
  Port (
    clk:    in std_logic;
    clk1: in std_logic;
    segcode:out std_logic_vector(7 downto 0);
    row:    out std_logic_vector(2 downto 0));
end LED;

architecture Behaviroal of LED is
```

```

signal q_disp:    std_logic_vector(2 downto 0):="000";
signal data_disp: std_logic_vector(3 downto 0):="0000";

.....
end Behavioral;

```

本硬件实验平台上的 LED 主要是用于为其他如单片机、A/D、D/A 等系统模块提供数据显示的支持。

三、实验内容

1. 实现一个简单的 7 段解码器芯片，输入是 A_0 、 A_1 、 A_2 ，当 $A_0A_1A_2$ 取值 000 ~ 010 时，7 段数码管显示 HELO，输入为其他值时 7 段数码管显示空格。

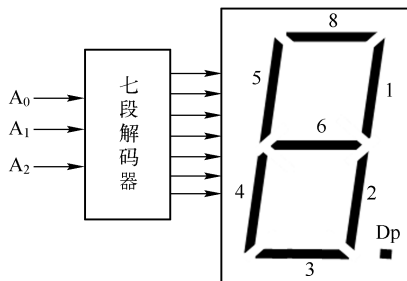


图 6.16

2. 编写 VHDL 程序实现对 8 个 7 段数码管显示的控制。在 LED 上循环显示 0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F 16 个的数字。
3. 编写 VHDL 程序实现数字电子钟，要求实现时分秒、校时、闹钟等功能。

四、实验步骤

4. 编写 VHDL 程序实现实验内容所要求的芯片设计，进行语法检查，直到语法检查正确。
5. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直到得到正确的波形仿真结果。
6. 对 VHDL 语言实现的芯片设计进行逻辑综合，并进行综合后仿真。
7. 对 VHDL 语言实现的芯片设计进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
8. 布局布线后进行时序仿真。
9. 参考附录进行 FPGA 引脚分配。
10. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，开启电源，把仿真正确的 VHDL 编译代码写入 DE2-115 硬件实验平台。
11. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

本实验的所有数码管和 FPGA 芯片的引脚分配连接信息如下：

Signal Name	FPGA Pin No.	Description	I/O Standard
HEX0[0]	PIN_G18	Seven Segment Digit 0[0]	2. 5V
HEX0[1]	PIN_F22	Seven Segment Digit 0[1]	2. 5V
HEX0[2]	PIN_E17	Seven Segment Digit 0[2]	2. 5V
HEX0[3]	PIN_L26	Seven Segment Digit 0[3]	Depending on JP7
HEX0[4]	PIN_L25	Seven Segment Digit 0[4]	Depending on JP7
HEX0[5]	PIN_J22	Seven Segment Digit 0[5]	Depending on JP7
HEX0[6]	PIN_H22	Seven Segment Digit 0[6]	Depending on JP7
HEX1[0]	PIN_M24	Seven Segment Digit 1[0]	Depending on JP7
HEX1[1]	PIN_Y22	Seven Segment Digit 1[1]	Depending on JP7
HEX1[2]	PIN_W21	Seven Segment Digit 1[2]	Depending on JP7
HEX1[3]	PIN_W22	Seven Segment Digit 1[3]	Depending on JP7
HEX1[4]	PIN_W25	Seven Segment Digit 1[4]	Depending on JP7
HEX1[5]	PIN_U23	Seven Segment Digit 1[5]	Depending on JP7
HEX1[6]	PIN_U24	Seven Segment Digit 1[6]	Depending on JP7
HEX2[0]	PIN_AA25	Seven Segment Digit 2[0]	Depending on JP7
HEX2[1]	PIN_AA26	Seven Segment Digit 2[1]	Depending on JP7
HEX2[2]	PIN_Y25	Seven Segment Digit 2[2]	Depending on JP7
HEX2[3]	PIN_W26	Seven Segment Digit 2[3]	Depending on JP7
HEX2[4]	PIN_Y26	Seven Segment Digit 2[4]	Depending on JP7
HEX2[5]	PIN_W27	Seven Segment Digit 2[5]	Depending on JP7
HEX2[6]	PIN_W28	Seven Segment Digit 2[6]	Depending on JP7
HEX3[0]	PIN_V21	Seven Segment Digit 3[0]	Depending on JP7
HEX3[1]	PIN_U21	Seven Segment Digit 3[1]	Depending on JP7
HEX3[2]	PIN_AB20	Seven Segment Digit 3[2]	Depending on JP6
HEX3[3]	PIN_AA21	Seven Segment Digit 3[3]	Depending on JP6
HEX3[4]	PIN_AD24	Seven Segment Digit 3[4]	Depending on JP6
HEX3[5]	PIN_AF23	Seven Segment Digit 3[5]	Depending on JP6
HEX3[6]	PIN_Y19	Seven Segment Digit 3[6]	Depending on JP6
HEX4[0]	PIN_AB19	Seven Segment Digit 4[0]	Depending on JP6
HEX4[1]	PIN_AA19	Seven Segment Digit 4[1]	Depending on JP6
HEX4[2]	PIN_AG21	Seven Segment Digit 4[2]	Depending on JP6
HEX4[3]	PIN_AH21	Seven Segment Digit 4[3]	Depending on JP6
HEX4[4]	PIN_AE19	Seven Segment Digit 4[4]	Depending on JP6
HEX4[5]	PIN_AF19	Seven Segment Digit 4[5]	Depending on JP6
HEX4[6]	PIN_AE18	Seven Segment Digit 4[6]	Depending on JP6
HEX5[0]	PIN_AD18	Seven Segment Digit 5[0]	Depending on JP6
HEX5[1]	PIN_AC18	Seven Segment Digit 5[1]	Depending on JP6

续表

HEX5[2]	PIN_AB18	Seven Segment Digit 5[2]	Depending on JP6
HEX5[3]	PIN_AH19	Seven Segment Digit 5[3]	Depending on JP6
HEX5[4]	PIN_AG19	Seven Segment Digit 5[4]	Depending on JP6
HEX5[5]	PIN_AF18	Seven Segment Digit 5[5]	Depending on JP6
HEX5[6]	PIN_AH18	Seven Segment Digit 5[6]	Depending on JP6
HEX6[0]	PIN_AA17	Seven Segment Digit 6[0]	Depending on JP6
HEX6[1]	PIN_AB16	Seven Segment Digit 6[1]	Depending on JP6
HEX6[2]	PIN_AA16	Seven Segment Digit 6[2]	Depending on JP6
HEX6[3]	PIN_AB17	Seven Segment Digit 6[3]	Depending on JP6
HEX6[4]	PIN_AB15	Seven Segment Digit 6[4]	Depending on JP6
HEX6[5]	PIN_AA15	Seven Segment Digit 6[5]	Depending on JP6
HEX6[6]	PIN_AC17	Seven Segment Digit 6[6]	Depending on JP6
HEX7[0]	PIN_AD17	Seven Segment Digit 7[0]	Depending on JP6
HEX7[1]	PIN_AE17	Seven Segment Digit 7[1]	Depending on JP6
HEX7[2]	PIN_AG17	Seven Segment Digit 7[2]	Depending on JP6
HEX7[3]	PIN_AH17	Seven Segment Digit 7[3]	Depending on JP6
HEX7[4]	PIN_AF17	Seven Segment Digit 7[4]	Depending on JP6
HEX7[5]	PIN_AG18	Seven Segment Digit 7[5]	Depending on JP6
HEX7[6]	PIN_AA14	Seven Segment Digit 7[6]	3. 3V

实验四 存储器芯片设计实验

一、实验目的

1. 掌握用 VHDL 语言实现存储器。
2. 掌握用 LPM 定制方法实现存储器。

二、实验原理

在数字系统中，一般都需要包含一定大小的存储器。存储器的种类很多，从功能上可以分为只读存储器（READ-ONLY MEMORY，ROM）和随机存储器（RANDOM ACCESS MEMORY，RAM）两大类。

ROM 和 RAM 属于通用大规模器件，一般不需要自行设计，特别是采用 PLD 器件进行设计时。但是在数字系统中，有时也需要设计一些小型的存储器件，用于特定的用途，例如临时存放数据、构成查表运算等。此类器件的特点为地址与存储内容直接对应，设计时将输入地址作为给出输出内容的条件；当需要较大的存储器时，一般使用 FPGA 的外部扩充存储器。

ROM（只读存储器）：ROM 的内容是在初始设计电路时就写入到内部的，通常采用电路的固定结构来实现存储；ROM 只需设置数据输出端口和地址输入端口；

下面是一个容量为 256×4 的 ROM 存储的例子，该 ROM 有 8 位地址线 AD_R(0)～AD_R(7)，4 位数据输出线 DOUT(0)～DOUT(3) 及使能端 EN，如图 6.17 所示。

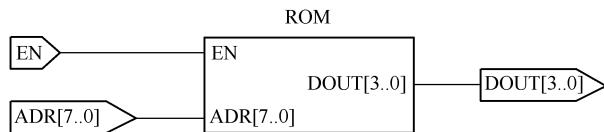


图 6.17 ROM

VHDL 描述代码如下：

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
USE STD.TEXTIO.ALL;
ENTITY ROM IS
    PORT (EN:IN STD_LOGIC;
          ADR:IN STD_LOGIC_VECTOR(7 DOWNTO 0);
          DOUT:OUT STD_LOGIC_VECTOR(3 DOWNTO 0));
END ENTITY ROM;
ARCHITECTURE ART OF ROM IS
    SUBTYPE WORD IS STD_LOGIC_VECTOR(3 DOWNTO 0);
    TYPE MEMORY IS ARRAY(0 TO 255) OF WORD;
    SIGNAL ADR_IN:INTEGER RANGE 0 TO 255;
    VARIABLE ROM:MEMORY;
    VARIABLE START_UP:BOOLEAN:=TRUE;
    VARIABLE L:LINE;
    VARIABLE J:INTEGER;
    FILE ROMIN:TEXT IS IN "ROMIN";
    BEGIN
        PROCESS (EN,ADR) IS
            BEGIN
                IF START_UP THEN                                --初始化开始
                    FOR J IN ROM' RANGE LOOP
                        READLINE(ROMIN,1);
                        READ(1,ROM(J));
                    END LOOP;
                    START_UP:=FALSE;                            --初始化结束
                END IF;
                ADR_IN<=CONV_INTEGER(ADR);                      --将向量转化成整数
                IF (EN='1') THEN
                    DOUT<=ROM(ADR_IN);
                ELSE
                    DOUT<="ZZZZ";
                END IF;
            END PROCESS;
        END ARCHITECTURE ART;
```

RAM (随机存储器): RAM 和 ROM 的主要区别在于 RAM 的描述上有读和写两种操作, 而且在读写上对时间有较严格的要求。下面给出一个 8×8 位的双口 RAM 的 VHDL 描述实例, 如图 6.18 所示。

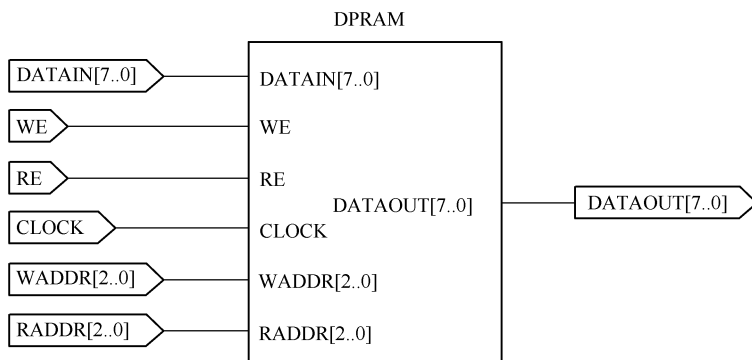


图 6.18 双口 RAM

VHDL 描述代码如下:

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY DPRAM IS
    GENERIC (WIDTH:INTEGER := 8;
             DEPTH:INTEGER := 8;
             ADDER:INTEGER := 3);
    PORT (DATAIN:IN STD_LOGIC_VECTOR (WIDTH-1 DOWNT0 0);
          DATAOUT:OUT STD_LOGIC_VECTOR (WIDTH-1 DOWNT0 0);
          CLOCK:IN STD_LOGIC;
          WE,RE:IN STD_LOGIC;
          WADD:IN STD_LOGIC_VECTOR (ADDER-1 DOWNT0 0);
          RADD:IN STD_LOGIC_VECTOR (ADDER-1 DOWNT0 0));
END ENTITY DPRAM;
ARCHITECTURE ART OF DPRAM IS
    TYPE MEM IS ARRAY (DEPTH-1 TO 0) OF
        STD_LOGIC_VECTOR (WIDTH-1 DOWNT0 0);
    SIGNAL RAMTMP:MEM;
BEGIN
    --写进程
    PROCESS (CLOCK) IS
        BEGIN
            IF (CLOCK' EVENT AND CLOCK =' 1' ) THEN
                IF (WE =' 1' ) THEN
                    RAMTMP (CONV_INTEGER (WADD)) <= DATAIN;
                
```

```
END IF;  
END IF;  
END PROCESS;  
--读进程  
PROCESS (CLOCK) IS  
BEGIN  
IF (CLOCK' EVENT AND CLOCK='1' ) THEN  
IF (RE='1' ) THEN  
DATAOUT<=RAMTMP (CONV_INTEGER (RADD) );  
END IF;  
END IF;  
END PROCESS;  
END ARCHITECTURE ART;
```

Quartus II 提供的参数化功能模块 LPM (Library Parameterized Modules) 可以用来实现如 ROM、RAM、FIFO 等存储器结构。用图形编辑, 进入 mega_lpm 元件库, 调用 lpm_rom 元件, 设置地址总线宽度 address 和数据总线宽度, 并添加输入、输出引脚。在设置 lpm_rom 数据参数选择项 lpm_file 的对应窗口中, 用键盘输入 lpm_ROM 配置文件的路径, 然后设置在系统 ROM/RAM 读写允许, 以便能对 FPGA 中的 ROM 在系统读写。用初始化存储器编辑窗口中编辑 lpm_ROM 配置文件 (文件名 .mif)。全程编译即可实现 ROM 存储器设计。

三、实验内容

1. 调用 LPM 实现 RAM 存储器, 用 DE2-115 硬件实验平台作为其输入、输出对其进行测试。SW15 ~ SW11 用于 5 位地址输入, SW7 ~ SW0 用于 8 位数据, SW17 用于写信号 Write, 用 KEY0 作为时钟信号。将地址值、输入数据、读出数据都在 7 段数码管上显示出来。
2. 用 VHDL 语言实现实验内容 1。

四、实验步骤

1. 编写 VHDL 程序实现实验内容所要求的芯片设计, 进行语法检查, 直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件, 对上述 VHDL 程序进行 ModelSim 功能仿真, 如仿真波形有误, 需重新修改 VHDL 程序, 重复以上各个步骤, 直到得到正确的波形仿真结果。
3. 对 VHDL 语言实现的芯片设计进行逻辑综合, 并进行综合后仿真。
4. 对 VHDL 语言实现的芯片设计进行实现过程 (包括翻译、映射、布局布线 3 个小步骤)。
5. 布局布线后进行时序仿真。
6. 参考附录进行 FPGA 引脚分配。
7. 计算机通过写入电缆与硬件实验平台连接, 硬件实验平台与电源连接, 检查无误后, 开启电源, 把仿真正确的 VHDL 编译代码写入 DE2-115 硬件实验平台。

8. 进行硬件在线调试, 检查结果是否符合设计要求。若不符合, 需要修改 VHDL 程序, 重复以上各个步骤, 直到硬件在线调试正确。

实验五 有限状态机芯片设计实验

一、实验目的

1. 理解和掌握有限状态机的概念。
2. 掌握用 VHDL 语言实现有限状态机芯片设计的方法。

二、实验原理

有限状态机 (Finite-State Machine, FSM) 简称状态机, 是表示有限个状态以及在这些状态之间的转移和动作等行为的数学模型。有限状态机由状态寄存器和组合逻辑电路构成, 状态寄存器逻辑的功能是存储有限状态机的内部状态; 而组合逻辑可以分为次态逻辑和输出逻辑两部分, 次态逻辑的功能是确定有限状态机的下一个状态, 输出逻辑的功能是确定有限状态机的输出。在数字电路系统中, 有限状态机是一种十分重要的时序逻辑电路模块, 能够根据控制信号按照预先设定的状态进行状态转移, 是协调相关信号动作、完成特定操作的控制中心, 是实现高效率高可靠性逻辑控制的重要途径, 它对数字系统的设计具有十分重要的作用。

在实际的应用中, 根据有限状态机是否使用输入信号, 通常将其分为 Moore 型有限状态机和 Mealy 型有限状态机两种类型。Moore 型有限状态机的输出信号仅与当前状态有关, 即可以把 Moore 型有限状态的输出看作当前状态的函数。Mealy 型有限状态机的输出信号不仅与当前状态有关, 而且与所有的输入信号有关, 即可以把 Mealy 型有限状态机的输出看作当前状态和所有输入信号的函数。

状态机的分析可以从状态图入手, 同样, 状态机的设计也可以从状态图入手。

设计状态机时一般先构造出状态图。构造状态图的一般方法是从一个比较容易描述的状态开始, 通常初始态是一个很好的开始状态, 也就是状态机复位以后开始的状态。在建立每个状态时最好都清楚地写出关于这个状态的文字描述, 为硬件设计过程提供清晰的参考资料, 也为最后完成的设计提供完整的设计文档。

状态机设计中常见的编码方式有: 二进制码 (Binary 码)、格雷码 (Gray 码)、独热码 (One-hot 码) 以及二一十进制码 (BCD 码)。

用 VHDL 设计的状态机的一般结构由以下几部分组成:

(1) 说明部分

说明部分中有新数据类型 (TYPE) 的定义及其状态类型 (状态名), 以及在此新数据类型下定义的状态变量。状态类型一般用枚举类型, 其中每一个状态名可任意选取。但为了和使含义明确便于辨认, 状态名最好有明显的解释性意义。状态变量应定义为信号, 便于信息传递, 说明部分一般放在 ARCHITECTURE 和 BEGIN 之间, 例如:

```
ARCHITECTURE ... IS
```

```
    TYPE states IS (st0, st1, st2, st3);
```

```
--定义新的数据类型和状态名
```

```
SIGNAL current_state, next_state: states;    --定义状态变量
...
BEGIN
...
```

(2) 主控时序进程

状态机是随外部时钟信号，以同步时序方式工作的，因此，状态机中必须包含一个对工作时钟信号敏感的进程，作为状态机的“驱动泵”。当时钟发生有效跳变时，状态机的状态才发生变化。状态机的下一状态（包括再次进入本状态）仅仅取决于时钟信号的到来。一般地，主控时序进程不负责所进入的下一状态的具体状态取值。当时钟的有效跳变到来时，时序进程只是机械地将代表下一状态的信号（next_state）中的内容送入代表本状态的信号（current_state）中，而下一状态信号（next_state）中的内容完全由其他的进程根据实际情况来决定，当然此进程中也可以放置一些同步、异步清零或置位等控制信号。总体来说，主控时序进程的设计比较固定、单一和简单。

(3) 主控组合进程

主控组合进程的任务是根据外部输入的控制信号（包括来自状态机外部的信号和来自状态机内部其他非主控的组合或时序进程的信号），或（和）当前状态的状态值确定下一状态（next_state）的取向，即 next_state 的取值内容，以及确定对外输出或对内部其他组合或时序进程输出控制信号的内容。

(4) 普通组合进程

普通组合进程用于配合状态机工作的其他组合进程，如为了完成某种算法的进程。

(5) 普通时序进程

普通时序进程是用于配合状态机工作的其他时序进程，如为了稳定输出设置的数据锁存器等。

状态机通常有三种描述方式：一段式状态机、两段式状态机、三段式状态机。

一段式状态机：当把整个状态机写在一个进程语句中，并且这个进程语句既包含状态转移，又含有组合逻辑输入/输出时，称为一段式状态机。不推荐采用这种状态机，因为从代码风格来讲，一般都会要求把组合逻辑和时序逻辑分开；从代码维护和升级来说，组合逻辑和时序逻辑混合在一起不利于代码维护和修改，也不利于约束。

两段式状态机：所谓的两段式状态机就是采用一个进程语句来实现时序逻辑，采用另一个进程语句来实现组合逻辑，提高了代码的可读性，易于维护。不同于一段式状态机的是，它需要定义两个状态——现态和次态，然后通过现态和次态的转换来实现时序逻辑。

三段式状态机：三段式状态机与两段式状态机的区别在于，两段式直接采用组合逻辑输出，而三段式则通过在组合逻辑后再增加一级寄存器来实现时序逻辑输出。这样做的好处是可以有效地滤去整个逻辑输出的毛刺，同时可以有效地进行时序计算与约束，另外对于总线形式的输出信号来说，容易使总线数据对齐，从而减小总线数据间的偏移，减小接收端数据采样出错的频率。三段式状态机的基本格式是：第一个进程语句实现同步状态跳转；第二个进程语句实现组合逻辑；第三个进程语句实现同步输出。当然，必要时还可以引入第 3 个和第 4 个进程，以完成其他的逻辑功能。

下面示例程序描述的状态机是由两个主控进程构成的，其中进程“REG”是主控时序

进程,“COM”是主控组合进程,此程序可作为一般状态机设计的模板来加以套用。

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY s_machine IS
    PORT ( clk,reset      : IN STD_LOGIC;
          state_inputs : IN STD_LOGIC_VECTOR (0 TO 1);
          comb_outputs : OUT STD_LOGIC_VECTOR (0 TO 1) );
END s_machine;
ARCHITECTURE behv OF s_machine IS
    TYPE states IS (st0, st1, st2, st3);      --定义 states 为枚举型数据类型
    SIGNAL current_state, next_state: states;
BEGIN
    REG:PROCESS (reset,clk)                  -- 时序逻辑进程
    BEGIN
        IF reset = '1' THEN
            current_state <= st0;              --异步复位
        ELSIF clk='1' AND clk' EVENT THEN
            current_state <= next_state;        --当测到时钟上升沿时转换至下一状态
        END IF;
    END PROCESS;
    --由信号 current_state 将当前状态值带出此进程,进入进程 COM
    COM:PROCESS(current_state, state_inputs) -- 组合逻辑进程
    BEGIN
        CASE current_state IS                --确定当前状态的状态值
            WHEN st0 => comb_outputs <= "00"; --初始态译码输出"00"
            IF state_inputs = "00" THEN        --根据外部的状态控制输入"00"
                next_state <= st0;              --在下一时钟后,进程 REG 的状态将维持为 st0
            ELSE
                next_state <= st1;              --否则,在下一时钟后,进程 REG 的状态将为 st1
            END IF;
            WHEN st1 => comb_outputs <= "01"; --对应状态 st1 的译码输出"01"
            IF state_inputs = "00" THEN        --根据外部的状态控制输入"00"
                next_state <= st1;              --在下一时钟后,进程 REG 的状态将维持为 st1
            ELSE
                next_state <= st2;              --否则,在下一时钟后,进程 REG 的状态将为 st2
            END IF;
        END CASE;
    END PROCESS;
END behv;

```

三、实验内容

用 VHDL 语言编程实现一个有限状态机,该有限状态机有一个输入 w 和一个输出 z 。当输入 w 是 4 个连续 0 或 1 时,输出 $z=1$, 否则 $z=0$ 。要求先画出这个有限状态机的状态图,用 VHDL 语言实现三种描述方式(即一段式状态机、两段式状态机、三段式状态机),并分

别仿真出时序图，最后利用 DE2-115 实验平台进行硬件验证。

四、实验步骤

1. 编写 VHDL 程序实现实验内容所要求的芯片设计，进行语法检查，直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直到得到正确的波形仿真结果。
3. 对 VHDL 语言实现的芯片设计进行逻辑综合，并进行综合后仿真。
4. 对 VHDL 语言实现的芯片设计进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
5. 布局布线后进行时序仿真。
6. 参考附录进行 FPGA 引脚分配。
7. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，开启电源，把仿真正确的 VHDL 编译代码写入 DE2-115 硬件实验平台。
8. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

实验六 VGA 彩条显示芯片设计实验

一、实验目的

1. 掌握 VGA 显示原理、VGA 工作时序以及 FPGA 的 VGA 接口电路。
2. 学会 VHDL 语言实现 VGA 接口电路芯片设计。

二、实验原理

计算机显示器的显示有许多标准，常见的有 VGA、SVGA 等，一般这些显示控制都使用专用的显示控制器（如 6845）。本实验尝试用 FPGA 来实现 VGA 图像显示控制器。

常见的彩色显示器一般由 CRT（阴极射线管）构成，彩色是由红（R）、绿（G）、蓝（B）三种基色组成的。显示是通过逐行扫描的方式实现的，阴极射线枪发出的电子束打在涂有荧光粉的荧光屏上，产生 RGB 三基色，合成一个彩色像素，如图 6.19 所示。扫描从屏幕的左上方开始，从左到右、从上到下进行扫描，每扫完一行，电子束回到屏幕的左边下一行的起始位置，在这期间，CRT 对电子束进行消隐，每行结束时，用行同步信号进行行同步；扫描完所有行，用场同步信号进行场同步，并使扫描回到屏幕的左上方，同时进行场消隐，预备下一场的扫描。

VGA 接口是与显示器进行通信的标准接口，在硬件设计中只需将 VGA 口中定义的相应引脚与 CPLD 的 I/O 接口相连接即可。根据 VGA 工业标准，VGA 的输出控制中除了红（R）、绿（G）、蓝（B）三种基色信号，同时还有水平同步（HS）和垂直同步（VS）等信号的控制输出，只有这些信号协同工作时才能实现对 VGA 显示器的控制。由此可见，要通

过 CPLD 器件实现对 VGA 显示的控制，了解接口标准、控制时序和设定恰当的参数是系统设计的关键。VGA 接口各引脚及信号对应关系如图 6.20 和表 6.6 所示。

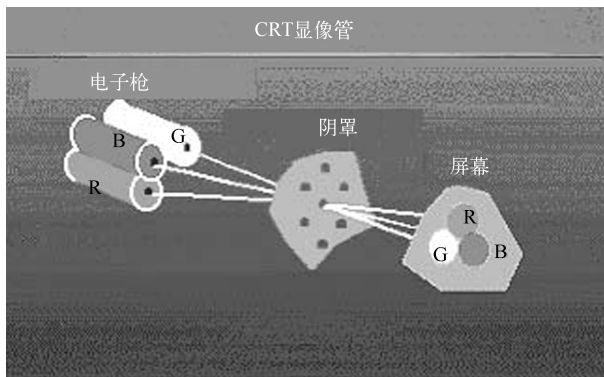


图 6.19 CRT 显示器工作原理

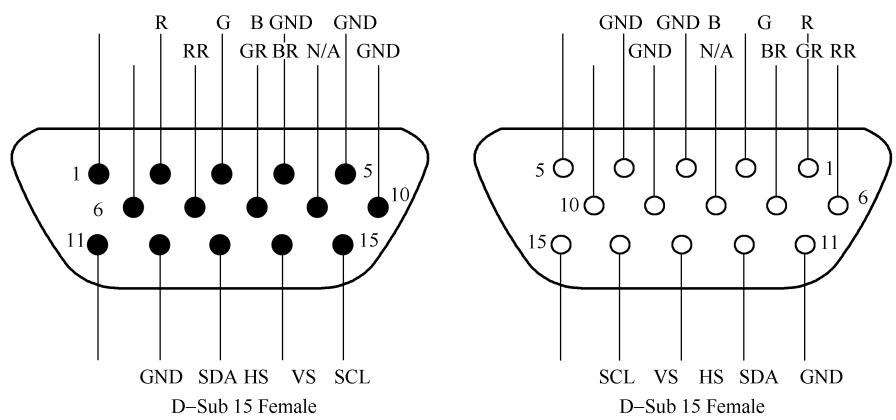


图 6.20 VGA 接口引脚定义图

表 6.6 VGA 引脚定义对照表

引脚序号	引脚名称	引脚序号	引脚名称	引脚序号	引脚名称
1	红色	6	红色返回	11	数字地
2	绿色	7	绿色返回	12	不用
3	蓝色	8	蓝色返回	13	水平同步
4	不用	9	不用	14	垂直同步
5	地	10	数字地	15	不用

在 DE2-115 硬件实验平台上，FPGA 与 VGA 接口电路的连接如图 6.21 所示。Cyclone IV E FPGA 通过 ADV7123 与 15 引脚 D-SUB VGA 接口连接，模拟器件公司的 ADV7123 三通道 10 位（仅高八位连接到 FPGA）高速视频 DAC 芯片用来将 FPGA 输出的数字信号（R、G、B）转换为模拟信号（R、G、B），ADV7123 芯片可支持的分辨率为 SVGA 标准（1280×1024），带宽达 100MHz。VGA 同步信号直接由 Cyclone IV E FPGA 所驱动。

符合 VGA 标准的 CRT 显示器工作过程如下：当 VS=0、HS=0 时，CRT 显示的内容为

亮的过程，即正向扫描过程约为 $26\mu\text{s}$ 。当一行扫描完毕，行同步 $\text{HS}=1$ ，约需 $6\mu\text{s}$ ；其间，CRT 扫描产生消隐，电子束回到 CRT 左边下一行的起始位置 ($X=0, Y=1$)；当扫描完 480 行后，CRT 的场同步 $\text{VS}=1$ ，产生场同步使扫描线回到 CRT 的第一行第一列 ($X=0, Y=0$) 处 (约为两个行周期)。 HS 和 VS 的时序图如图 6.22 所示。

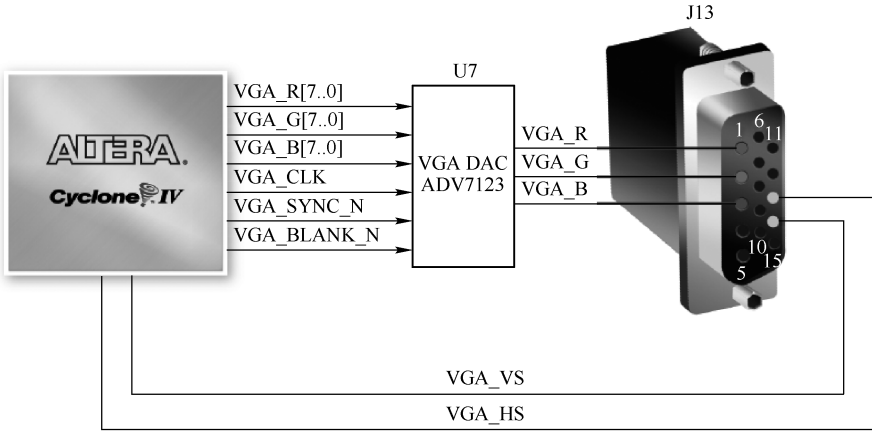


图 6.21 FPGA 与 VGA 接口电路连接示意图

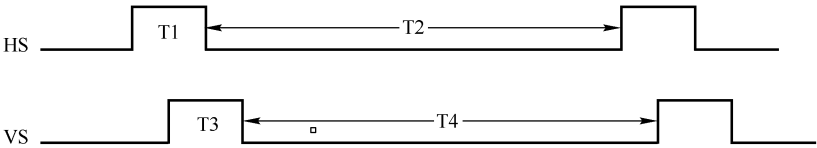


图 6.22 VGA 行频信号和场频信号时序图

图 6.22 中 $T1$ 为行同步消隐时间 (约为 $6\mu\text{s}$)； $T2$ 为行显示时间 (约为 $26\mu\text{s}$)； $T3$ 为场同步消隐时间 (两个行周期)； $T4$ 为场显示时间 (480 个行周期)。

对于 VGA 显示器要严格遵循“VGA 工业标准”，即 $640\times 480\times 60\text{HZ}$ 模式，否则会损害 VGA 显示器。图 6.23 所示的是 VGA 行扫描、场扫描的时序图。

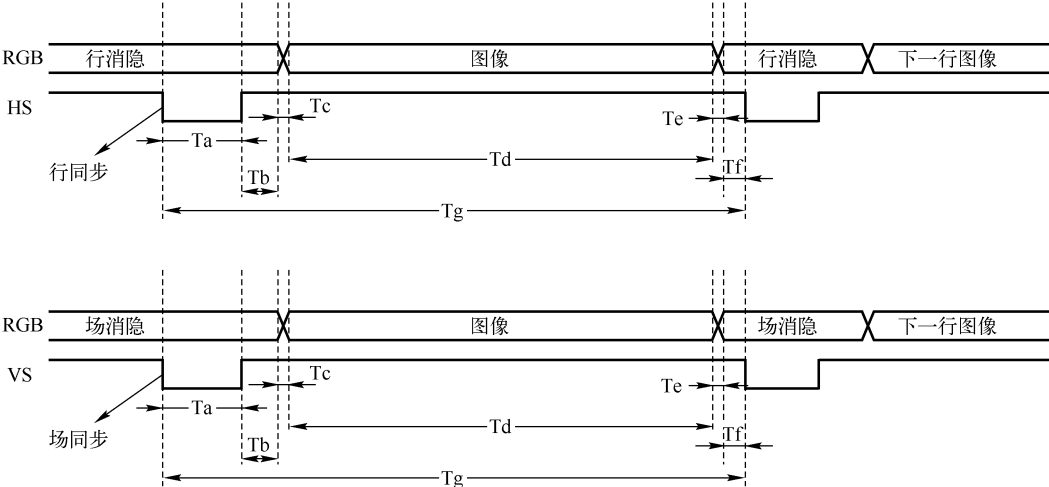


图 6.23 VGA 行扫描、场扫描的时序图

VGA 工业标准要求的频率如下：

- 时钟频率 (Clock frequency)：25.175MHz (像素输出频率)；
- 行频 (Line frequency)：31469Hz；
- 场频 (Field frequency)：59.94Hz (每秒图像刷新频率)。

表 6.7 行扫描时序要求 [单位：像素，即输出一个像素 (Pixel) 的时间间隔]

		行同步头			行图象		行周期
对应位置	Tf	Ta	Tb	Tc	Td	Te	Tg
时间 (Pixels)	8	96	40	g	460	8	800

表 6.8 场扫描时序要求 [单位：行，即输出一行 (Line) 的时间间隔]

		行同步头			行图象		行周期
对应位置	Tf	Ta	Tb	Tc	Td	Te	Tg
时间 (Lines)	2	2	25	8	480	8	525

VGA 工业标准显示模式要求：行同步、场同步都为负极性，即同步头脉冲要求是负脉冲。

设计 VGA 图像显示控制要注意两个问题：一个是时序驱动，这是完成设计的关键，时序稍微有偏差，显示必然不正常，甚至会损坏彩色显示器；另一个是 VGA 信号的电平驱动。

因此，在 VHDL 软件的设计方面也要着重于 VGA 接口控制时序的产生。时序配合的控制在整个显示控制器中起着至关重要的作用，控制器中的各个进程输出的显示数据时序配合都必须满足 VGA 工业标准才能保证送到显示器的数据能正确显示。根据 VGA 工业标准，红、绿、蓝三基色与水平同步和垂直同步信号时序配合的 VHDL 可以设计如下：

```
GRB(2)<=(GRBP(2) xor md) AND HS AND VS;
GRB(3)<=(GRBP(3) xor md) AND HS AND VS;
GRB(1)<=(GRBP(1) xor md) AND HS AND VS;
R<=GRB(2);
G<=GRB(3);
B<=GRB(1);
```

其中，GRBP 为三基色数据信号，由模式控制进程模块的产生；HS 和 VS 分别为行同步信号和场同步信号，由计数器产生，输出的 R、G、B 信号分别为红色、绿色、蓝色信号。此外，还引入了 md 信号作为图像模式的变换控制，以控制图形生成模块产生出横、竖彩条和网格等多种不同的图形。

在频率方面由于 VGA 显示要求的像素填充率较高，虽然在软件设计中已做一定简化，但根据软件设计的要求仍必须使用平台中最高的 12MHz 时钟源作为驱动频率。

三、实验内容

编写 VHDL 程序，实现用 CPLD/FPGA 对 VGA 显示器的控制，使之产生横、竖彩条和网格等多种不同的图形，最好能够利用实验五设计的有限状态机进行控制。部分 VHDL 程序如下：

```

--clk:    扫描时钟
--md:    显示模式选择信号
--hs:    行同步信号
--vs:    场同步信号
--r:     红基色信号
--g:     绿基色信号
--b:     蓝基色信号

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity color is
port (clk,md:    in    std_logic;
      hs,vs,r,g,b: out  std_logic);
end color;
architecture behv of color is
    signal hsl,vs1,fclk,cclk:std_logic;
    signal mmd:          std_logic_vector(1 downto 0);    --方式选择
    signal fs:           std_logic_vector(3 downto 0);
    signal cc:           std_logic_vector(4 downto 0);      --行同步/横彩条生成
    signal ll:           std_logic_vector(8 downto 0);      --场同步/竖彩条生成
    signal grbx:         std_logic_vector(3 downto 1);      --X 横彩条
    signal grby:         std_logic_vector(3 downto 1);      --Y 竖彩条
    signal grbp:         std_logic_vector(3 downto 1);
    signal grb:          std_logic_vector(3 downto 1);
begin
    .....
    .....
end behv;

```

四、实验步骤

1. 编写 VHDL 程序实现实验内容所要求的芯片设计，进行语法检查，直到语法检查正确。

2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直到得到正确的波形仿真结果。

3. 对 VHDL 语言实现的芯片设计进行逻辑综合，并进行综合后仿真。

4. 对 VHDL 语言实现的芯片设计进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。

5. 布局布线后进行时序仿真。

6. 参考附录进行 FPGA 引脚分配。

7. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，

开启电源，把仿真正确的 VHDL 编译代码写入 DE2-115 硬件实验平台。

8. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

实验七 VGA 图像显示芯片设计实验

一、实验目的

1. 学习数字图像原理。
2. 进一步巩固存储器的使用。
3. 掌握用 VHDL 语言实现图像显示芯片的设计。

二、实验原理

数字图像是指以二维数组或矩阵表示图像，如图 6.24 所示。像素（或像元，Pixel）是数字图像的基本元素，像素是在模拟图像数字化时对连续空间进行离散化得到的。每个像素具有整数行（高）和列（宽）位置坐标，同时每个像素都具有整数灰度值或颜色值。

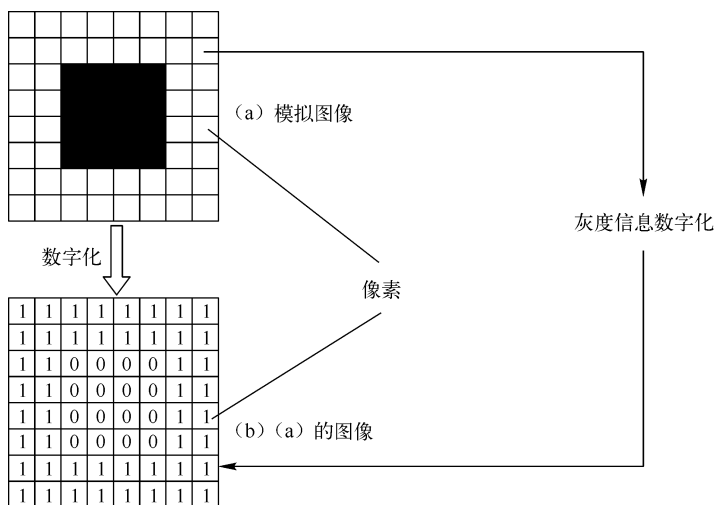


图 6.24 数字图像原理图

本实验要求实现某一分辨率下（如 $640 \times 480 @ 60\text{Hz}$ ）的 VGA 接口图像显示芯片的设计，要显示的图像存储于 FPGA 内部存储器。VGA 接口图像显示芯片是一个较大的数字系统，可采用模块化设计方法、借鉴自顶向下的程序设计思想，进行功能划分并按层次化设计思想进行设计。整个系统设计主要包括：数字锁相环模块（二分频）、VGA 显示时序控制模块及 FPGA 片内 ROM 存储模块（图像数据存储），如图 6.25 所示。

数字锁相环模块把 50MHz 时钟频率二分频成 25MHz 并提供给其他模块作为时钟；VGA 显示时序控制模块用于产生 640×480 显示范围，并控制显示范围和消隐范围，以及产生水平同步时序信号（HS）和垂直同步时序信号（VS）的值；FPGA 片内 ROM 存储模块用于存储要显示的图像数据。下面将对每一个模块的实现原理进行详细介绍。

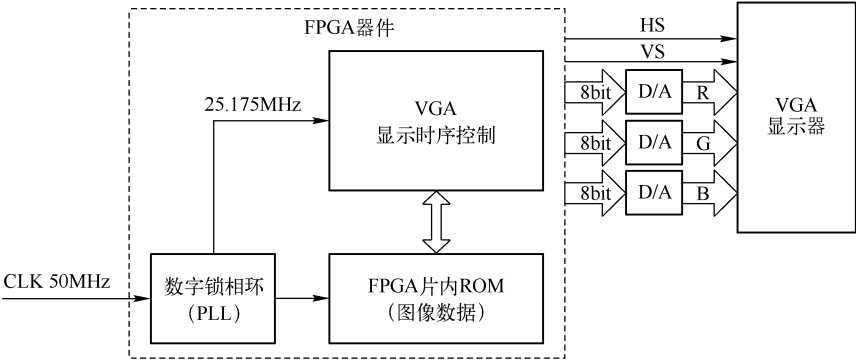


图 6.25 VGA 显示控制系统框图

1. 数字锁相环（二分频）模块设计

VGA 图像显示分辨率是 640×480 和场频是 60Hz，因此需要的像素时钟是 25MHz。由于 DE2-115 开发板提供 50MHz 的时钟信号，不提供 25MHz 的时钟，本实验介绍一种通过 Quartus II 的 MegaWizard Plug-In Manager 工具中的 ATLPLL 锁相环模块对 50MHz 时钟信号进行二分频的设计方法。

设计步骤如下：

(1) 打开 Quartus II 软件，选择 tools 菜单栏下的 MegaWizard Plug-In Manager 工具，弹出下图 6.26 所示对话框，选中 Create a new custom... 单选按钮，单击 next 产生图 6.27 所示对话框，在左栏选择 I/O 项下的 ATLPLL，再选择 Cyclone IV E 器件和 VHDL 语言方式；最后输入 PLL 文件存放的路径和文件名：D:\de2_115\PLL\pll.vhd，单击 Next 按钮。

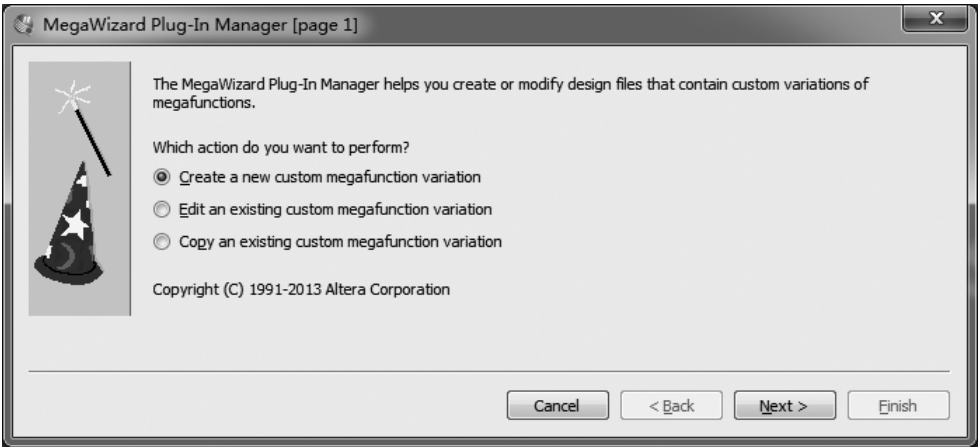


图 6.26 定制新的宏功能块

(2) 在图 6.28 所示窗口中设置参考时钟频率为 50MHz（就是要分频的时钟），单击“Next”按钮显示图 6.29 所示窗口，在 Lock output 设置框里去掉“locked”；单击“Next”按钮，弹出图 6.30 对话框，不需要进行设置；直接再单击“Next”按钮，弹出图 6.31 所示的对话框，设置 C0 的倍频为 1/2；接着一直单击“Next”按钮，最后单击“Finish”按钮完成二分频模块设置。

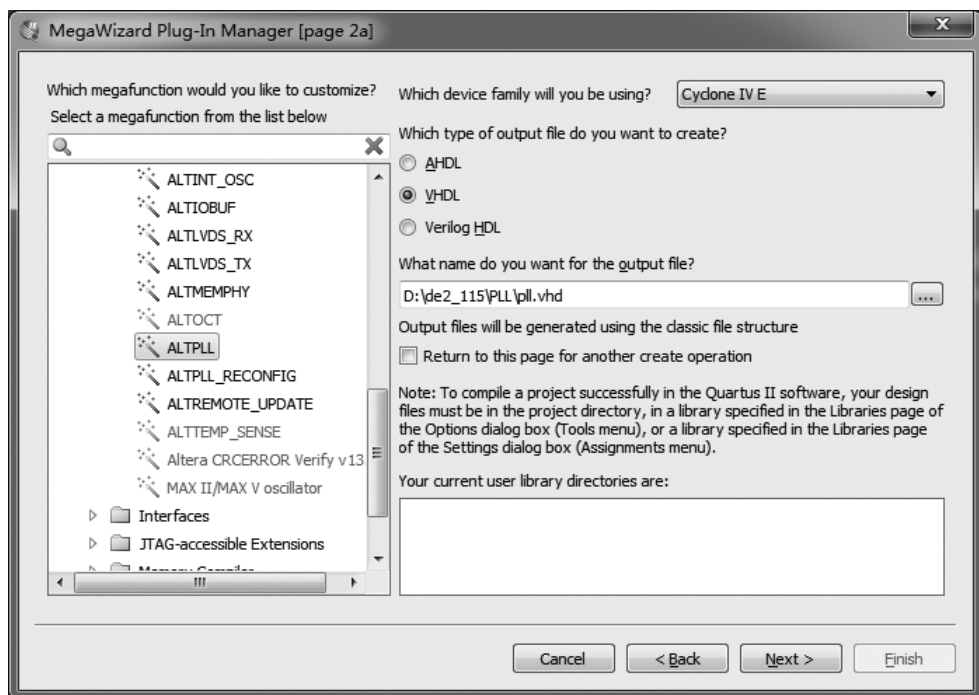


图 6.27 ATLPLL 宏功能块设置

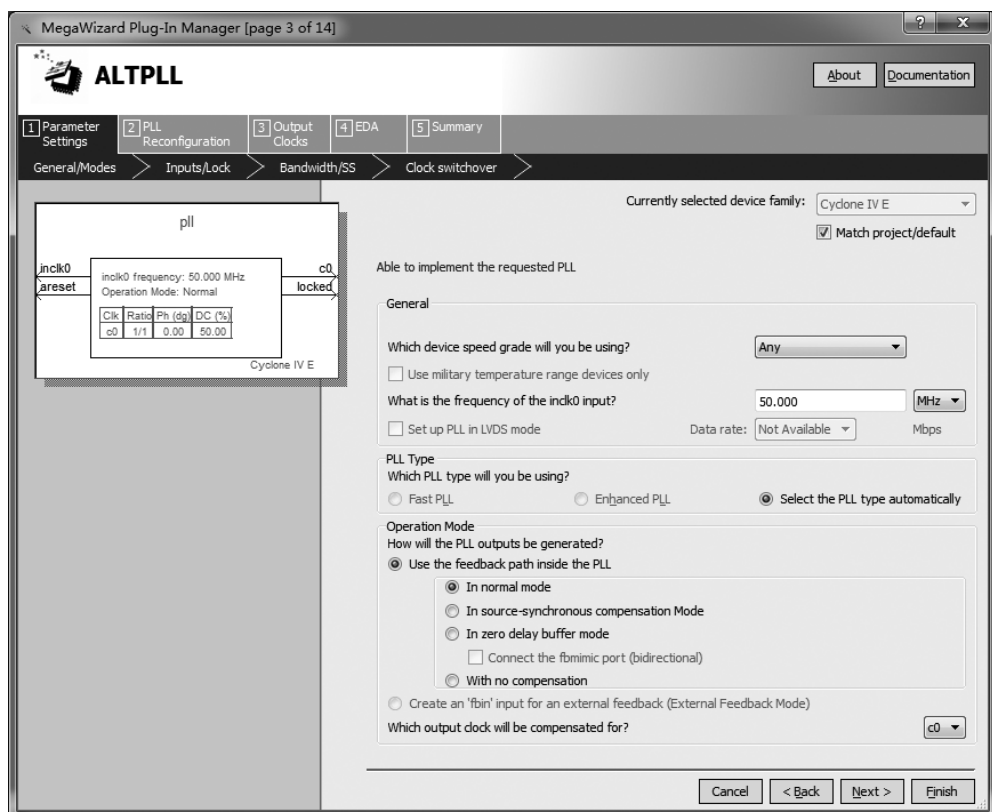


图 6.28 选择参考时钟为 50MHz

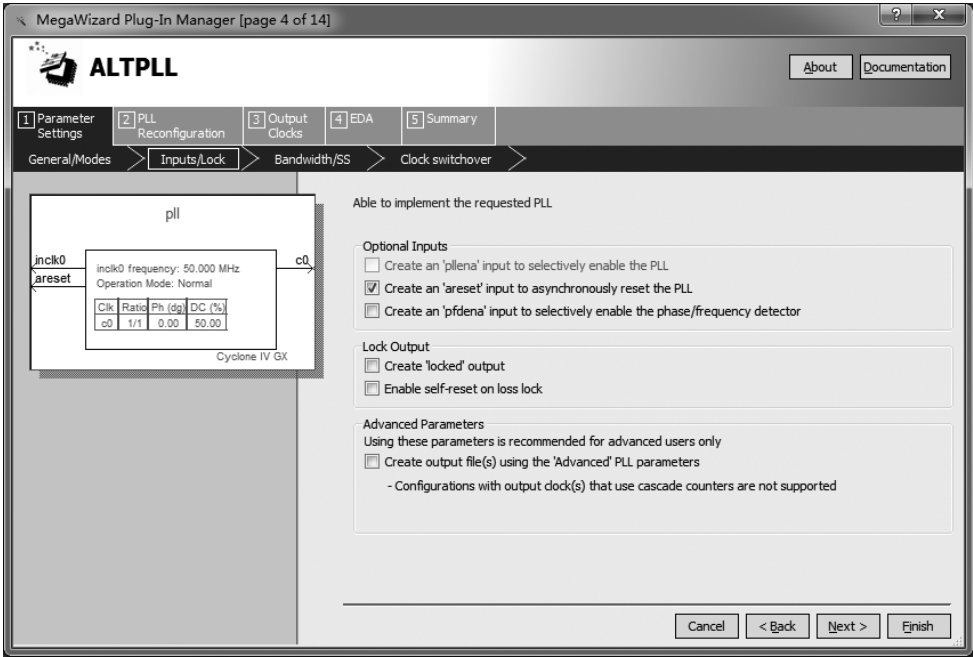


图 6.29 删除 locked 输出线

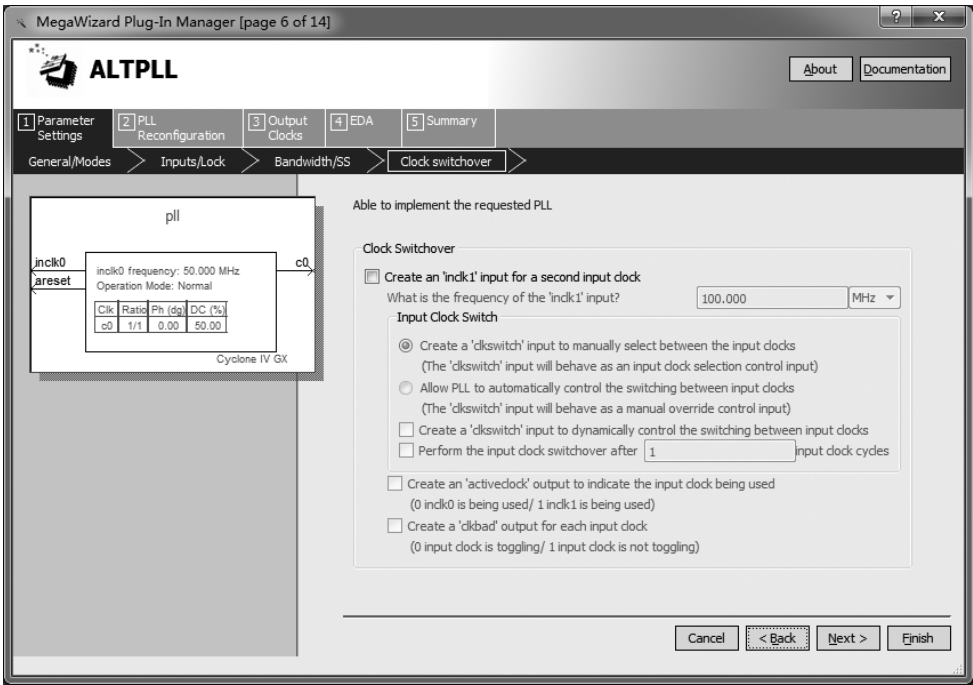


图 6.30 无须增加内部时钟

2. ROM 存储器模块设计

半导体存储器是一种能存储大量二值信息的半导体器件。按存取功能分为只读存储器

(掩模 ROM、PROM、EPROM 等)、随机存储器 (SRAM、DROM 等)。在一些需要特殊运算的应用电路中, 只读存储器 ROM 是关键元件, 设计人员通常利用 ROM 创建各种查找表, 从而简化电路设计, 提高电路的处理速度和稳定性。FPGA 是基于 SRAM 的可编程器件。掉电后 FPGA 上的配置信息将全部丢失, 所以由 FPGA 构造的数字系统在每次上电后要依赖外部存储器来主动配置或在线被动配置。真正意义上的 ROM 应具有掉电后信息不丢失的特性, 因此利用 FPGA 实现的 ROM 只能被认为器件处于用户状态时具备 ROM 功能。

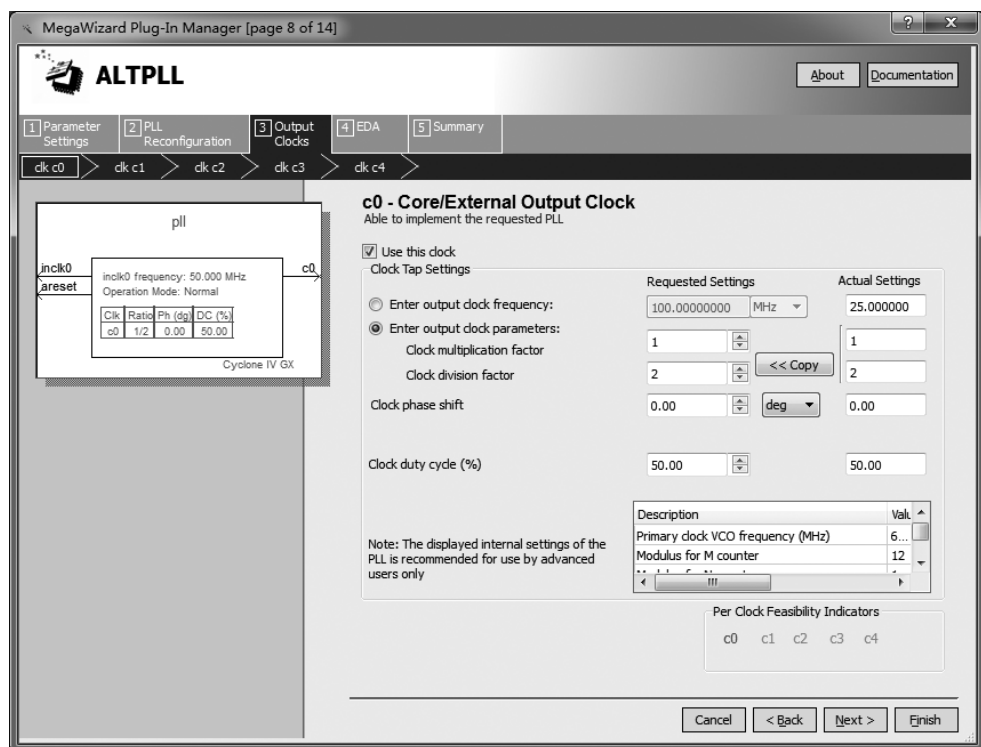


图 6.31 设置分频系数为 1/2

如何在 ROM 单元中对数字图像进行初始化是本实验必须面对的问题。mif (memory initialization file) 文件是存储器初始化文件, 用来配置 RAM 或 ROM 中的数据。通常 mif 文件的格式如下:

```
WIDTH = 8;           //数据位宽为 8 位
DEPTH = 256;         //存储深度为 256
ADDRESS_RADIX = UNS; //地址类型为无符号整型
DATA_RADIX = UNS;    //数据类型为无符号整型
CONTENT
BEGIN
    0:1;              //地址 0 的存储器内容为 1
    1:3;              //地址为 1 的存储器内容的 3
    [2:255]:0;        //地址为 2~255 的存储器内容为 0
END;
```

mif 文件的格式是固定的, 对于前 4 行 (DEPTH, WIDTH, ADDRESS_RADIX, DATA_RADIX), 前 2 个参数应与 lpm_ROM 宏模块 LPM_WIDTHAD 和 LPM_WIDTH 相关联, 后 2 个参数分别是地址和数据, 这里为了方便, 设置为 DEC (十进制)。关键是文件内容 (CONTENT), 在 BEGIN 和 EDN 之间, 冒号左边是地址, 右边是数据; 以分号结尾; mif 文件的使用就是修改 BEGIN 和 EDN 之间的内容。

生成 QuartusII 可用的 mif 文件, 通常有如下几种方式:

- 方法 1: 利用 Quartus 自带的 mif 编辑器。优点是, 对于小容量 RAM 可以快速方便地完成 mif 文件的编辑工作, 不需要第三方软件的编辑; 缺点是, 一旦数据量过大, 一个一个地输入会令人崩溃。
- 方法 2: 利用 mif 软件来生成, 例如 Mif Maker2010 软件。
- 方法 3: 使用高级语言, 例如 C 语言或 Matlab 语言等来生成。

这里介绍一种采用方法 3 将一幅数字图像转换成 mif 文件的方法。

如何将数字图像文件转换为 mif 文件? 对数字图像进行存储、处理、传播, 必须采用一定的图像格式, 也就是把图像的像素按照一定的方式进行组织和存储, 把图像数据存储成文件就得到图像文件。图像文件格式是记录和存储影像信息的格式, 它决定了应该在文件中存放何种类型的信息, 文件如何与各种应用软件兼容, 文件如何与其他文件交换数据。

目前比较流行的图像格式包括光栅图像格式 (BMP、GIF、JPEG、PNG 等) 和矢量图像格式 (WMF、SVG 等)。BMP 格式图像作为一种很常见的图像格式, 无压缩处理, 且具备相对简单的文件结构, 易于处理, 在本实验中采用了 BMP 图像作为研究对象。BMP 格式的位图文件相对于 mif 文件较为复杂, BMP 文件包括位图信息头结构、位图文件头结构、位图颜色表和位图像素数据四部分。位图文件的保存顺序: 文件头→信息头→颜色表→像素数据。

当下应用较广泛的 24 位真彩色图像的存储文件中没有图像颜色表, 图像中每一像素由 R、G、B 三个分量构成, 每个分量分为 8 位, 取值范围都是 0 ~ 255, 每个像素需要 24 位。本实验采用 FPGA 内部存储器, 为了节省存储空间, 采用高彩色图像, R、G、B 信号分别由 5 位、6 位、5 位来表示, 即一个像素值由 16 位来表示。这样做, 可使得存储数据量比真彩色图像减少一半, 同时又能满足显示效果。

如何将 BMP 图像文件转换为 mif 文件? 因为没有现成的转换程序, 所以需要自行编写转换程序。Matlab 软件具有强大的数字图像数据处理能力, 这里介绍自行编写 Matlab 转换程序的方法。

代码如下:

```
clear;
clc;
close all;
fid=fopen('data_8bits.txt');           % 打开待转换数据的 .txt 文件
temp=fscanf(fid,'%x');                 % 扫描文件
n=1024;
fid=fopen('convert_data.mif','w');      % 打开待写入的 .mif 文件
fprintf(fid,'WIDTH=8;\n');              % 写入存储位宽 8 位
fprintf(fid,'DEPTH=1024;\n');           % 写入存储深度 1024
```

```

fprintf(fid,' ADDRESS_RADIX=UNS;\n' );           % 写入地址类型为无符号整型
fprintf(fid,' DATA_RADIX=HEX;\n' );             % 写入数据类型为 16 进制
fprintf(fid,' CONTENT BEGIN\n' );               % 起始内容
for i=0:n-1
    fprintf(fid,' \t% d:% x;\n' ,i,temp(i+1));
end
fprintf(fid,' END;\n' );
fclose(fid);                                     % 关闭文件
clear all;
clc;
p=imread(' lena.bmp' );                         % 打开待转换数字图像的 .bmp 文件
pr=uint16(p(:,:,1));
pg=uint16(p(:,:,2));
pb=uint16(p(:,:,3));
[pm,pn]=size(pr);
pixs=pm*pn;
c=1023;
d=1047552;
f=240;
h=248;
pr_new=bitshift(pr,-3);                         % 取红色分量 R 是 5 位
pg_new=bitshift(pg,-2);                         % 取绿色分量 G 是 6 位
pb_new=bitshift(pb,-3);                         % 取蓝色分量 B 是 5 位
fid=fopen(' D:\vga\lena16.mif' ,'w' );          % 打开待写入的 .mif 文件
fprintf(fid,' --Generated by MifMaker,Used by QuartusII\n\n' );
fprintf(fid,' WIDTH=16;\n' );                    % 写入存储位宽 16 位
fprintf(fid,' DEPTH=% d;\n\n' ,pixs);            % 写入存储深度 pixs
fprintf(fid,' ADDRESS_RADIX=UNS;\n' );           % 写入地址类型为无符号整型
fprintf(fid,' DATA_RADIX=UNS;\n\n' );          % 写入数据类型为无符号整型
fprintf(fid,' CONTENT BEGIN\n' );               % 起始内容
for i=1:pm
    for j=1:pn
        fprintf(fid,' \t% ld:% ld;\n' ,(i-1)*pn+j-1,pr_new(i,j)*2048+pg_new
            (i,j)*32+pb_new(i,j));
    end
end
end
fprintf(fid,' END;\n' );
fclose(fid);

```

使用这段程序转化之前，除了阅读板子手册查看 RAM 的容量能装下多大的图像数据，还需要找一张合适的图片，这里提示一下，可以用 Windows 自带的画图软件将图片打开，右键重新调整大小去掉保持纵横比，选择像素，这样就可以任意修改图像大小了！

先将 BMP 格式文件转换成 mif 格式文件，再经过 Quartus II 直接将 mif 文件转换为 Hex

文件，最终将图像数据烧写入 ROM 中。

设计基于 FPGA 的 ROM 存储器通常可以采用两种方法：一是用 Quartus II 软件自带的参数化模块库定制，只需要对端口和参数做必要的设置即可，这是首选的设计方法；二是自己采用 VHDL 等硬件描述语言进行设计。下面简单介绍使用参数化模块库定制 ROM 的过程。

(1) 用 MegaFunction 创建 ROM 模块，首先单击菜单项 Tools→MegaWizard Plug-In Manager...

(2) 打开后默认选择第一项，单击“Next”按钮。

(3) 在接下来的界面中，在左侧选择 Installed Plug-Ins→Memory Compiler→ROM:1-PORT，然后在右侧输出文件类型选择“VHDL”，并在右侧给出的输出文件名栏目中设定输出文件的目录和名字为：D:\de2_115\VGA\VGA_ROM，即生成的 VHDL 文件名为 vga_rom.vhd。

(4) 单击“Next”按钮进入下一步，在此界面中参数设置如图 6.32 所示。之后单击“Next”按钮。

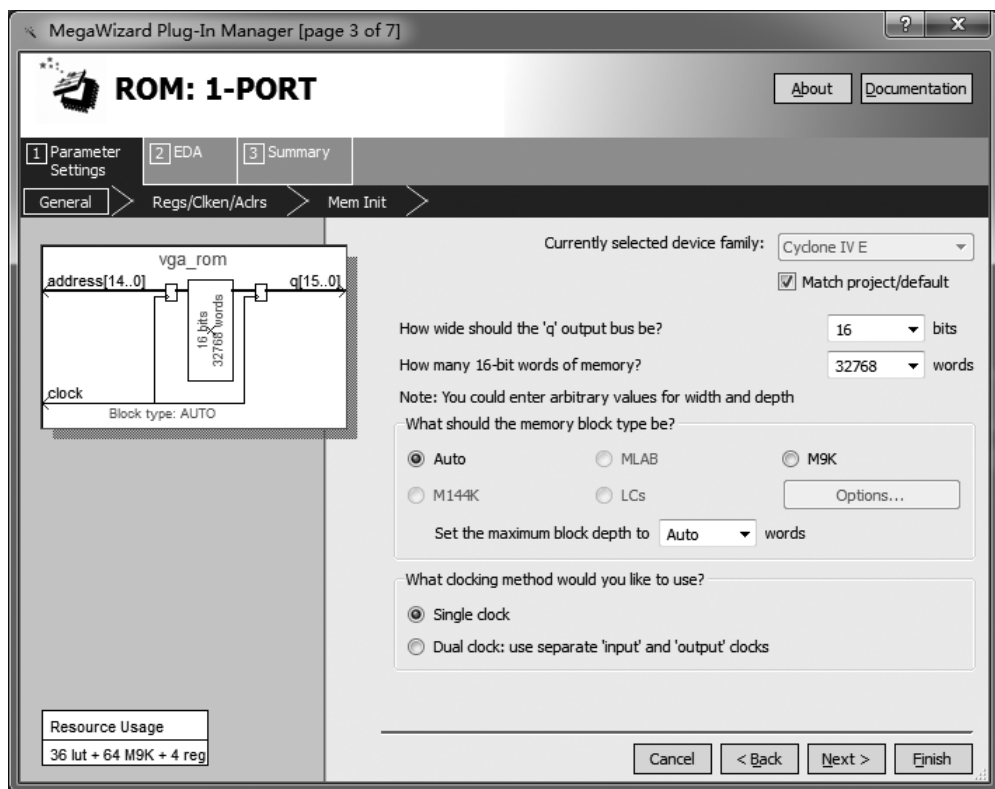


图 6.32 设置 ROM 模块的参数

(5) 在下一个界面中无须改变设置，继续单击“Next”按钮。

(6) 之后要指定 ROM 模块的初始化数据文件（储存 LENA 图像的 lena16.mif），如图 6.33 所示，最后单击“Finish”按钮，完成定制过程。完成后如果有对话框问是否添加此 ip，选择“否”。

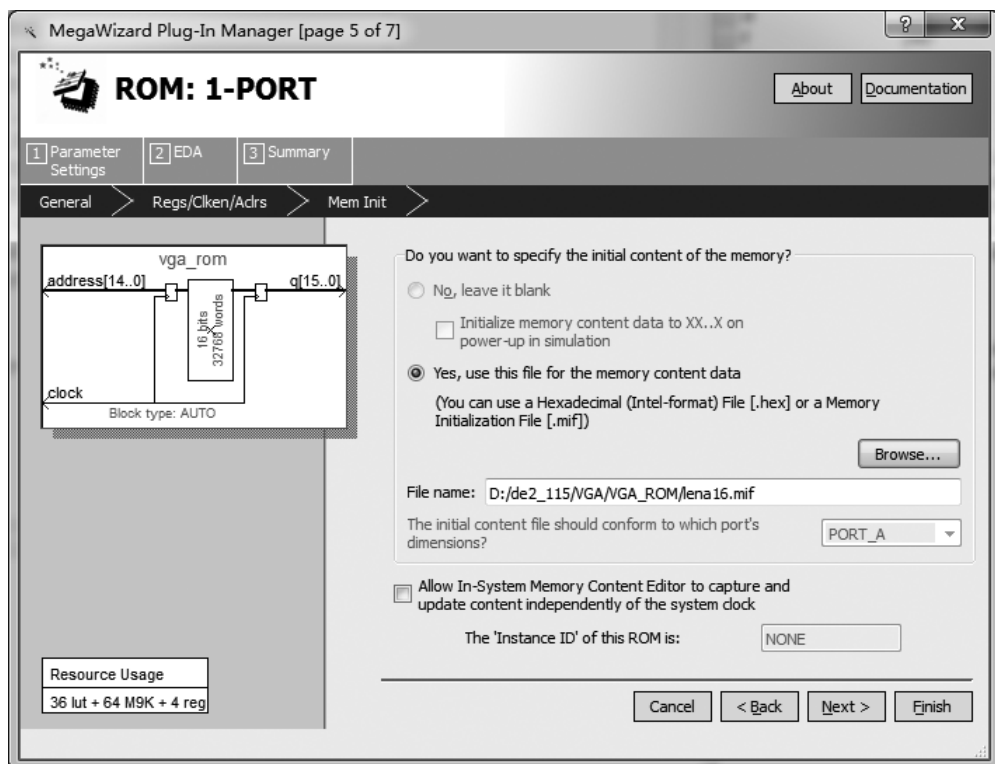


图 6.33 设置 ROM 模块的初始数据

要把一幅图像的数据写入 ROM，如何写入文件的格式这一问题是要解决的。

memory initialization file (.mif) 格式作为 Quartus II 中 ROM 的初始化数据文件格式，其具备较简单的记录方式，其中地址和数据都可以为十进制或十六进制，地址值位于冒号左边，对应的数据则在右边，并以分号结尾。

相比 BMP 格式的位图文件，mif 格式的位图文件略为简单，BMP 文件它包括位图信息头结构、位图文件头结构、位图颜色表和位图像素数据四部分。以下是位图文件的保存顺序：文件头→信息头→颜色表→像素数据。位图文件的大小、宽、高、实现调色板、图像像素值得到等可以通过处理位图时依据文件的这些结构得到^[11]。用当下应用较广泛的 24 位真彩色图像举例分析，其特点是：24 位真彩色图像的存储文件中没有图像颜色表；图像中每一像素由 R、G、B 三个分量构成，每个分量分为 8 位，每个像素需要 24 位；f red、f green、f blue 颜色分量值的取值范围都是 0 ~ 255。

对于要用到颜色表的位图来说，图像数即为该像素颜色存在于调色板中的索引值。对于真彩色的图像，图像数据则是实际的 R、G、B 值，即一个像素由 3 个字节 24 位组成，前 8 位用于表示 B，中间 8 位用于表示 G，后 8 位用于表示 R。因为硬件存储资源的限制，取图像的像素位深为 3，总共可显示为 8 种颜色。首先需要把 BMP 图像数据的 RGB 分量读出，并且各取 1 位，颜色分量值也取值 '1' 或 '0'。针对 24 位的真彩色图像特点，可以先对颜色分量值进行取样，但较为粗略，当颜色的分量值为 f red > 127 时，取 f red = 1；反之取 f red = 0。同理，颜色的分量值 f green、f blue 可取得相应的 0、1 二值^[12]。

最后需特别注意是，如果利用 Visual C++ 编写转换程序，BMP 文件需按照从左至右，

从下至上的顺序排列，即读取文件时，图像底行的左边第一个像素最先被读到，顶行最右边一个像素则最后被读，这一过程与 VGA 显示器的扫描顺序恰好是相反的。

3. 显示控制模块设计

显示器扫描方式分为逐行扫描和隔行扫描：逐行扫描是扫描从屏幕左上角一点开始，从左向右逐点扫描，每扫描完一行，电子束回到屏幕的左边下一行的起始位置，在这期间，CRT 对电子束进行消隐，每行结束时，用行同步信号进行同步；当扫描完所有的行，形成一帧，用场同步信号进行场同步，并使扫描回到屏幕左上方，同时进行场消隐，开始下一帧。隔行扫描是指电子束扫描时每隔一行扫一线，完成一屏后再返回来扫描剩下的线，如图 6.34 所示。

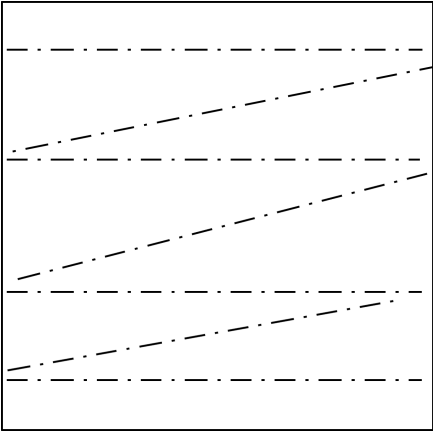


图 6.34 VGA 屏幕扫描原理

VGA 显示控制模块按照 640×480@ 60Hz 的标准产生行扫描和场扫描信号，从 ROM 模块读取图像数据进行处理，并将处理好的 RGB 信号通过端口输出到 ADV7123。

VGA 图像动态显示是基于动画帧的思想，当场计数器每计到 524 时，屏计数器加 1，用屏计数器控制画面显示的先后顺序，在快速扫描时，要实现流畅的动画显示，每秒至少应该显示 24 幅图片。在实际应用过程中，屏计数器的值不能是无限大的，应该根据设计中定义的寄存器的大小来锁定某一个值，否则之前出现过的图片又会重现。

4. 原理图设计过程

(1) 建立工程，将工程命名为 vga_top，并将之前设置好的 PLL 相关文件、ROM 相关文件和控制模块文件 vga640480.vhd 复制到工程目录，同时在软件中将这这些文件添加进这个工程。

(2) 选择 Project Navigator 下方的 Files 选项卡，分别选中这三个模块的 VHDL 文件，右键单击执行 Create Symbol Files for Current File，以便在原理图中调用这些模块。

(3) 如图 6.35 所示，选择 File->New...，选中 Block Diagram/Schematic File，在新建的原理图编辑窗口双击，产生元件查找对话框，找出已生成的 vga640480、pll、vga_rom 模块，如图 6.36 所示。

(4) 在原理图中将模块和引脚进行连线组成系统原理图，如图 6.37 所示。

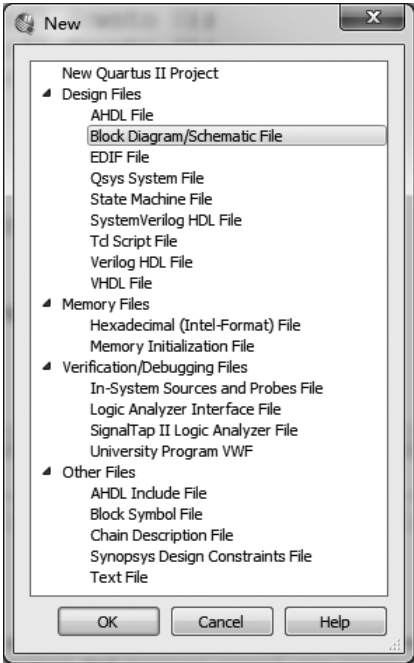


图 6.35 新建设计文件选择窗口

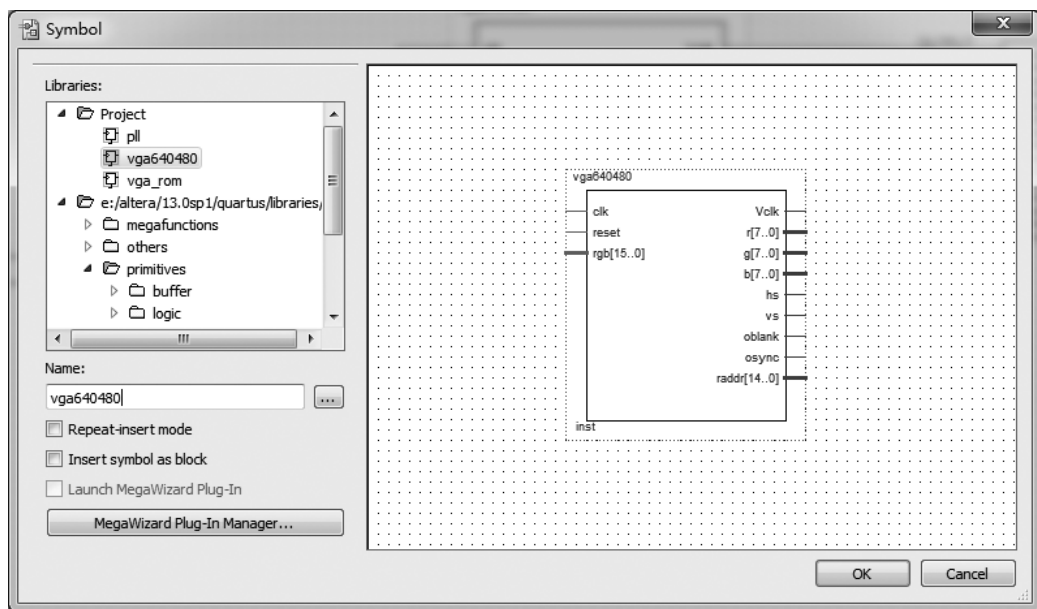


图 6.36 查找模块

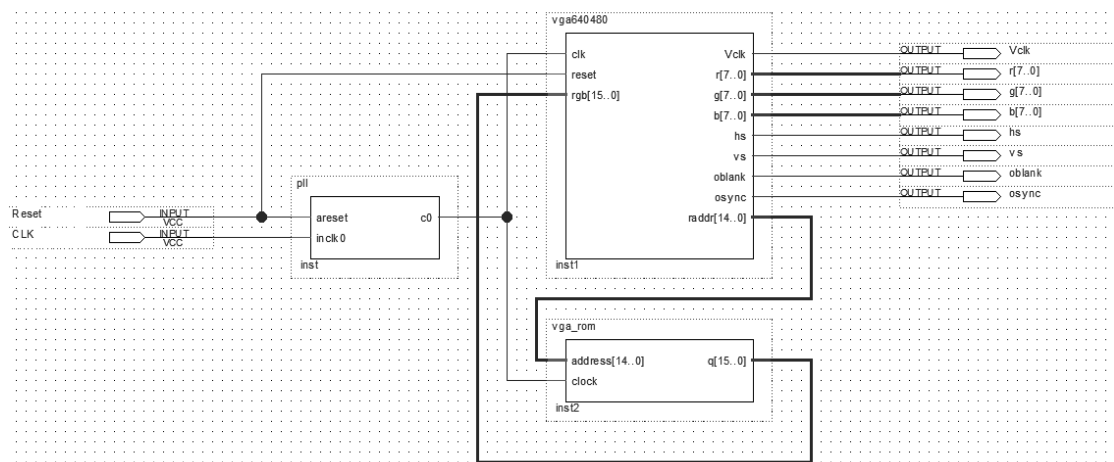


图 6.37 系统原理图

二、实验内容

1. 静态显示芯片的设计与实现

用 VHDL 语言编写 VGA 控制模块，并在顶层工程中调用锁相环芯片和存储芯片，实现用 FPGA 对 VGA 显示器的控制，使之静态显示图像。VGA 控制模块的部分代码如下：

```
--clk:          输入时钟
--v_clk: 扫描时钟
--reset: 复位信号
--rgb: 读取 ROM 数据
```

```

--raddr: ROM 地址信号
--hs:    行扫描电平
--vs:    场扫描电平
--r:     红基色信号
--g:     绿基色信号
--b:     蓝基色信号
--oblank: RGB 信号输出无效
--osync: 同步信号
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity vga640480 is
port (clk,reset:instd_logic;
      rgb : in std_logic_vector(15 downto 0)
      r,g,b:outstd_logic_vector(7 downto 0)
      v_clk,hs,vs,oblank,osync: out std_logic;
      raddr : out std_logic_vector(14 downto 0)
    );
end vga640480;
.....
.....

```

2. 动态显示芯片的设计与实现

用 VHDL 语言编写 VGA 控制模块，并在顶层工程中调用锁相环芯片和存储芯片，实现用 FPGA 对 VGA 显示器的控制，使之动态显示图像，用有限状态机控制图像按照一定路线移动，例如，按菱形轨迹运动或沿着显示器的四周运动。

模块中实现屏计数器的代码为：

```

--vcount:场计数器、
--k:     屏计数器
process (clk, vcount)
begin
    if (clk' event and clk=1) then
        if (vcount=524 and k/=1000) then
            k<=k+1;
        elsif (vcount=524 and k>=1000) then
            k<=1000; --当屏计数到 1000 时锁定其值,防止 k 的寄存器数值溢出
        end if;
    end if;
end process;

```

本实验的 FPGA 引脚分配如表 6.9 所示。

表 6.9 FPGA 引脚分配

信 号 名	引 脚	信 号 名	引 脚
clk	PIN_Y2	g[2]	PIN_F8
v_clk	PIN_A12	g[1]	PIN_G11
r[7]	PIN_H10	g[0]	PIN_G8
r[6]	PIN_H8	b[7]	PIN_D12
r[5]	PIN_J12	b[6]	PIN_D11
r[4]	PIN_G10	b[5]	PIN_C12
r[3]	PIN_F12	b[4]	PIN_A11
r[2]	PIN_D10	b[3]	PIN_B11
r[1]	PIN_E11	b[2]	PIN_C11
r[0]	PIN_E12	b[1]	PIN_A10
g[7]	PIN_C9	b[0]	PIN_B10
g[6]	PIN_F10	hs	PIN_G13
g[5]	PIN_B8	vs	PIN_C13
g[4]	PIN_C8	obblank	PIN_F11
g[3]	PIN_H12	osync	PIN_C10

四、实验步骤

1. 编写对 VGA 显示器控制的 VHDL 程序，进行语法检查，直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直至得到正确的波形仿真结果。
3. 进行逻辑综合，并进行综合后仿真。
4. 进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
5. 布局布线后进行时序仿真。
6. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，开启电源，把仿真正确的 VHDL 编译代码写入硬件实验平台的 CPLD 或 FPGA 中。
7. 把一台显示器的 VGA 端口与实验板上的 VGA 接口相连。
8. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

实验八 PS/2 键盘控制芯片设计实验

一、实验目的

1. 掌握 PS/2 键盘接口原理、接口协议、工作模式和协议数据包格式。
2. 掌握用 VHDL 语言实现 PS/2 键盘控制芯片的设计。

二、实验原理

硬件实验平台通过 PS/2 连接器支持键盘和鼠标的连接。但这只是提供了一个通过 PS/2

电缆到 FLEX 芯片的电器连接。因此，要想使键盘和鼠标能够与 CPLD/FPGA 芯片通信，必须采用逻辑编程在 CPLD/FPGA 芯片上设计一个硬件接口。而且，还需要采用移位寄存器来实现串/并转换。

PS/2 端口有 6 个引脚，分别是地线、电源（VDD）线、键盘数据线和 一个键盘时钟线。UP 1 开发板为鼠标和键盘提供电源，键盘数据线是 FLEX 芯片的第 31 脚，键盘时钟线是 FLEX 芯片的第 30 脚。时钟线要停止来自键盘的数据传输时，通常是用键盘控制的，但是它也可以由计算机系统或 FLEX 芯片控制。键盘与开发板系统能够交换的命令和消息分别如表 6.10 和表 6.11 所示。

表 6.10 发送给 PS/2 键盘的命令和消息

Commands Sent to Keyboard	Hex Value
Reset Keyboard Keyboard returns AA, 00 after self-test	FF
Resend Message	FE
Set key typematic (autorepeat) XX is scan code for key	FB, XX
Set key make and break	FC, XX
Set key make	FD, XX
Set all key typematic, make and break	FA
Set all keys make	F9
Set all keys make and break	F8
Make all keys typematic (autorepeat)	F7
Set to Default Values	F6
Clear Buffers and start scanning keys	F4
Set typematic (autorepeat) rate and delay Bits 6 and 5 are delay (250ms to sec) Bits 4 to 0 are rate (all 0's=30x/sec to all 1's 2x/sec)	F3, XX
Read keyboard ID Keyboard sends FA, 83, AB	F2
Set scan code set XX IS 01, 02, or 03	F0, XX
Echo	EE
Set Keyboard LEDs XX is 00000 Scroll, Num, and Caps Lock bits 1 is LED on and 0 is LED off	ED, XX

表 6.11 键盘发送的 PS/2 命令和消息

Messages Sent by Keyboard	Hex Value
Resend Message	FE
Two bad messages in a row	FC
Keyboard Acknowledge Command Sent by Keyboard after each command byte	FA
Response to Echo command	EE
Keyboard passed self-test	AA
Keyboard buffer overflow	00

键盘通常是由行列开关矩阵进行编码的。每一个键盘的按键都有唯一的扫描码，这个扫描码是由按键所在的行列开关矩阵的位置决定的。为了判断所有的按键是否改变，所有的行列都通过键盘码进行周期检测或扫描。键盘数据由已知的扫描码通过串行方式传送给计算机。键盘扫描码由 make 码和 break 码组成。每当有一个按键被按下时，就会有一个 make 码被发送。当有一个按键被释放，就会有一个 break 码被发送。大多数按键的 break 码是由 F0 后面加上 make 码组成的。需要注意的是，当我们打字的时候，通常是释放一个按键，然后才击打下一个按键。采用这样的配置，系统就可以判断是否有按键被按下，即使有多个按键同时被按下，系统也可以区分开哪一个按键已经被释放了。

PS/2 串行数据传输是这样进行的：扫描码以 11 比特在双向数据线上被串行地发送。当没有按键被按下，计算机也不需要发送数据时，数据线和时钟线处于高电平。单独一个按键或命令的传输格式如图 6.38 所示，它由 1bit 的起始位、8bits 数据位、1bit 奇偶校验位、1bit 停止位组成。

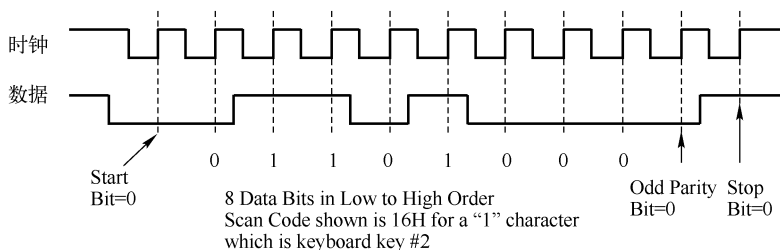


图 6.38 键盘一个扫描码的传输

当通过键盘发送一个命令时，有下列事件依次发生：

- (1) 首先，键盘检测时钟线和数据线是否被占用。如果是高电平，则表示没有被占用。如果两者都没有被占用，键盘则把数据线拉为低电平，准备发送起始比特。
- (2) 键盘的时钟线处于低电平大概持续 $35\mu\text{s}$ 。
- (3) 然后，键盘将以大约 $70\mu\text{s}$ 的时钟周期发送其余的 10bits。键盘负责驱动时钟线和数据线。
- (4) 负责辨识起始比特和接收的串行数据。串行数据是由 8bits 数据，接着 1bit 奇偶校验位和 1bit 停止位组成的。如果想接着发送更多的数据，则下一个起始位就是第 12bit。

当计算机或 FLEX 芯片发送一个命令给 PS/2 键盘时，如图 6.39 所示，过程如下：

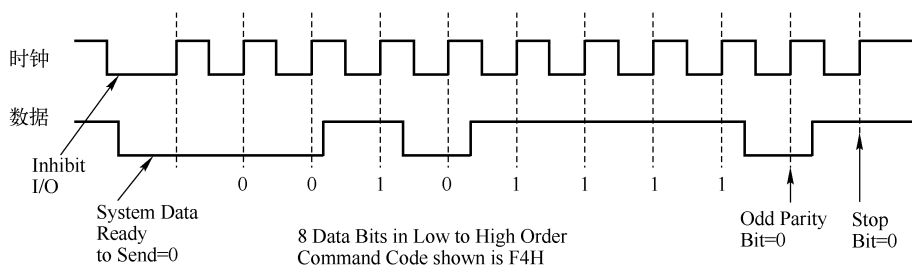


图 6.39 系统传输一个命令给 PS/2 设备

- (1) 系统驱动时钟线为低电平，持续大约 $60\mu\text{s}$ ，以抑止任何新的键盘数据传送。这个时钟线是双向的。

- (2) 系统驱动数据线为低电平，然后释放时钟线发出信号给键盘，有数据到来。
- (3) 接收命令中的串行比特数据，键盘将产生时钟信号。
- (4) 系统将发送 8bits 数据，后面接着一个奇偶校验位和一个停止位。
- (5) 停止位变为高电平后，数据线被释放。

一旦执行完一个命令字节，键盘将发送一个应答信号（ACK），如果键盘接收的信号正确，则发应答信号 FA。如果系统没有释放数据线，键盘将持续地产生时钟信号，直到完成，它将发送一个重发命令信号，FE 或 FC 给系统。奇偶校验错误，或者丢失停止比特也将产生一个重发命令信号。

PS/2 键盘处理一个扫描码需要两个步骤，一个键盘码是用来查找扫描码的，键盘码需要的扫描码表如表 3 所示，这个表是针对英语的键盘布局的。每当有一个按键被按下，就会有一个 make 码被发送。当有一个按键被释放，就会有一个 break 码被发送。当同时有多个按键被按下时，就会有多个 make 码被发送，然后才有一个 break 码被发送。采用预定义的扫描码是不需要发送一个命令给键盘的。预定义的键盘扫描码如表 6. 12 所示。

表 6. 12 键盘扫描码

Key#	Make Code	Break Code	Key#	Make Code	Break Code	Key#	Make Code	Break Code
1	OE	F0 OE	31	1C	F0 1C	90	77	F0 77
2	16	F0 16	32	1B	F0 1B	91	6C	F0 6C
3	1E	F0 1E	33	23	F0 23	92	6B	F0 6B
4	26	F0 26	34	2B	F0 2B	93	69	F0 69
5	25	F0 25	35	34	F0 34	96	75	F0 75
6	2E	F0 2E	36	33	F0 33	97	73	F0 73
7	36	F0 36	37	3B	F0 3B	98	72	F0 72
8	3D	F0 3D	38	42	F0 42	99	70	F0 70
9	3E	F0 3E	39	4B	F0 4B	100	7C	F0 7C
10	46	F0 46	40	4C	F0 4C	101	7D	F0 7D
11	45	F0 45	41	52	F0 52	102	74	F0 74
12	4E	F0 4E	43	5A	F0 5A	103	7A	F0 7A
13	55	F0 55	44	12	F0 12	104	71	F0 71
15	66	F0 66	46	1A	F0 1A	105	7B	F0 7B
16	0D	F0 0D	47	22	F0 22	106	79	F0 79
17	15	F0 15	48	21	F0 21	110	76	F0 76
18	1D	F0 1D	49	2A	F0 2A	112	05	F0 05
19	24	F0 24	50	32	F0 32	113	06	F0 06
20	2D	F0 2P	51	31	F0 31	114	04	F0 04
21	2C	F0 2C	52	3A	F0 3A	115	0c	F0 0C
22	35	F0 35	53	41	F0 41	116	03	F0 03
23	3C	F0 3C	54	49	F0 49	117	0B	F0 0B
24	43	F0 43	55	4A	F0 4A	118	83	F0 83
25	44	F0 44	57	59	F0 59	119	0A	F0 0A
26	4D	F0 4D	58	14	F0 14	120	01	F0 01
27	54	F0 54	60	11	F0 11	121	09	F0 09
28	5B	F0 5B	61	29	F0 29	122	78	F0 78
29	5D	F0 5D	62	E0 11	E0 F0 11	123	07	F0 07

The remaining key codes are a function of the shift, control, alt, or num-lock keys.

Key	No Shift or Num Lock		Shift *		Num Lock On	
	#	Make	Break	Make	Break	Make
76		E0 70	E0 F0 70	E0 F0 12 E0 70	E0 F0 70 ED 12	E0 12 E0 70
76		E0 71	E0 F0 71	E0 F0 12 E0 71	E0 F0 71 E0 12	E0 12 E0 71
79		E0 6B	E0 F0 6B	E0 F0 12 E0 6B	E0 F0 6B E0 12	E0 12 E0 6B
80		E0 6C	E0 F0 6C	E0 F0 12 E0 6C	E0 F0 6C E0 12	E0 12 E0 6C
81		E0 69	E0 F0 69	E0 F0 12 E0 69	E0 F0 69 E0 12	E0 12 E0 69
83		E0 75	E0 F0 75	E0 F0 12 E0 75	E0 F0 75 E0 12	E0 12 E0 75
84		E0 72	E0 F0 72	E0 F0 12 E0 72	E0 F0 72 E0 12	E0 12 E0 72
85		E0 7D	E0 F0 7D	E0 F0 12 E0 7D	E0 F0 7D E0 12	E0 12 E0 7D
86		E0 7A	E0 F0 7A	E0 F0 12 E0 7A	E0 F0 7A E0 12	E0 12 E0 7A
89		E0 74	E0 F0 74	E0 F0 12 E0 74	E0 F0 74 E0 12	E0 12 E0 74
* When the left Shift Key is held down, the 12-F0 12 shift make and break is sent with the other scan codes. When the right Shift Key is held down, 59-F0 59 is sent.						

Key	Scan Code		Shift Case *	
	#	Make	Break	Make
95		E0 4A	E0 F0 4A	E0 F0 12 E0 4A
* When the left Shift Key is held down, the 12-F0 12 shift make and break is sent with the other scan codes. When the right Shift Key is held down, 59-F0 59 is sent. When both Shift Keys are down, both sets of codes are sent with the other scan codes.				

Key	Scan Code		Control Case, Shift Case		Alt Case	
	#	Make	Break	Make	Break	Make
124		E0 12 E0 7C	E0 F0 7C E0 F0 12	E0 7C	E0 F0 7C	84

Key #	Make Code	Control Key Pressed
126 *	E1 14 77 E1 F0 14 F0 77	E0 7E E0 F0 7E
* This key does not repeat		

三、实验内容

设计如图 6.40 所示的芯片结构，实现从键盘读取扫描码。

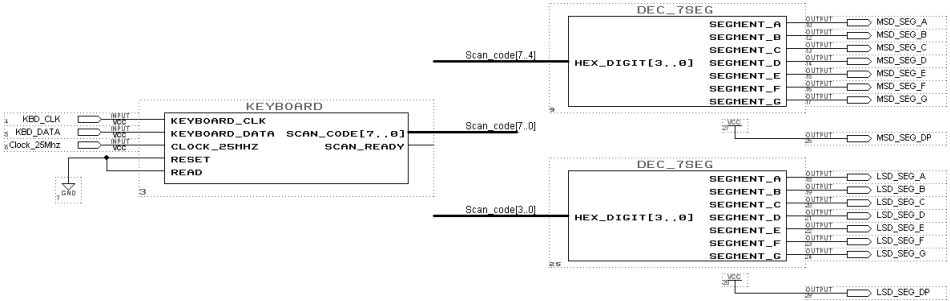


图 6.40 PS/2 接口键盘控制系统

四、实验步骤

1. 按照实验内容的要求编写 VHDL 程序，进行语法检查，直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直至得到正确的波形仿真结果。
3. 对 VHDL 语言实现的系统进行逻辑综合，并进行综合后仿真。
4. 对 VHDL 语言实现的系统进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
5. 布局布线后进行时序仿真。
6. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，开启电源，把仿真正确的 VHDL 编译代码写入硬件实验平台的 CPLD 或 FPGA 中。
7. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

实验九 串行通信芯片设计实验

一、实验目的

1. 理解串行通信的基本原理。
2. 掌握用 VHDL 语言实现串行通信芯片的设计。

二、实验原理

EIA-232 标准是一种应用十分广泛的串行通信标准。它由美国电子工业协会（Electronic Industries Association, EIA）制订，至今已有数十年历史，其前称为 RS-232 标准（RS 是 Recommended Standard 的缩写，即“推荐标准”）。目前比较普及的是 1969 年公布的第三个修订版本 RS-232C（EIA-232C）。现在仍常用“RS-232C”或“RS-232”来称呼这一版本。

RS-232 标准定义了 DTE（Data Terminal Equipment，数据终端设备，例如计算机）和 DCE（Data Circuit-Terminating Equipment，数据电路终接设备或数据通信设备，例如调制解调器）之间的接口标准。因此从通信的角度来说它是属于物理层的标准，适用于短距和低速的通信场合，其最大传输距离为 15m，最大位速率为 20kbps。如果需要远距离通信，一般需要调制解调器做中间设备。

RS-232 标准定义了接口的电气、机械、功能等方面的特性。对逻辑电平的定义如表 6.13 所示。

表 6.13 RS-232 标准定义

RS-232 信号定义		
电平	-15V ~ -5V	+5V ~ +15V
逻辑	1	0
信号	传号（MASK）	空号（SPACE）
功能	OFF	ON

EIA-232-32 标准虽然定义了信号线的功能,但并未定义接口与电缆的物理特性,因此存在多种类型的 RS-232 接口,如 DB25、DB15、DB9、甚至 RJ-45 等。如计算机的 COM 口使用的就是 DB9 接口。DE2-115 实验平台采用的也是 DB9 接口。DB9 接口分为 DB9 Male (9 针) 和 DB9 Female (9 孔) 两种,前者用于 DTE (如 PC 的 COM 口),后者用于 DCE (如 Modem 的接口)。DE2-115 实验平台采用的是后者,如图 6.41 所示。

DTE 上的 DB9 为公 (Male) 接口,如图 6.42 所示。

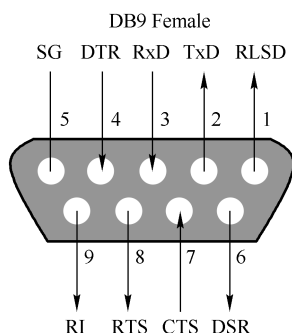


图 6.41 9 孔 D 型串行通信接口引脚定义

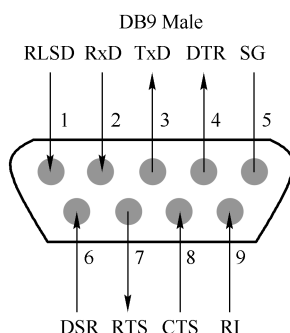


图 6.42 9 针 D 型串行通信接口引脚定义

由引脚定义可见,对于分别使用这两种接口的设备,使用平行线将两者对应引脚连通即可进行通信。DE2-115 实验平台采用的是三线连接方式,如图 6.43 所示。

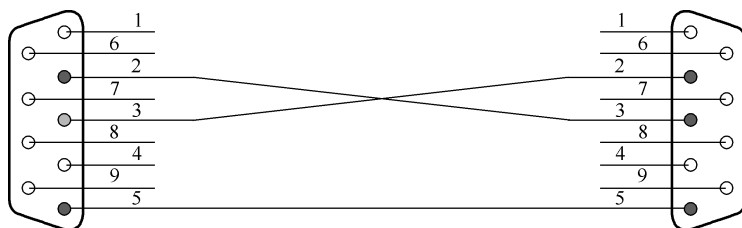


图 6.43 计算机与实验平台的串行接口连接

RS-232 使用的电平与电路中的 TTL 电平并不兼容,因此实际应用中需要电平转换电路进行电平转换。DE2-115 实验平台采用 MAXIM 公司的 MAX232 芯片进行电平转换。

MAX232 是一款双通道 EIA-232 驱动器/接收器芯片。它采用 +5V 单电压供电,内部含有一个电压倍增器和电压反相器,只需加上若干电容即可实现 TTL/CMOS 电平与 RS-232 电平之间的转换,并可输出 $\pm 10V$ 的电压。MAX232 的引脚分配如图 6.44 所示。

根据 MAX232 的 Datasheet, MAX232 外接 4 个 $1\mu F$ 的电容即可正常工作。它与系统的连接如图 6.45 所示。

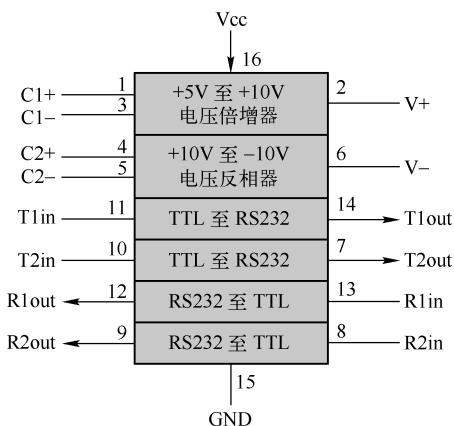


图 6.44 MAX232 的引脚图

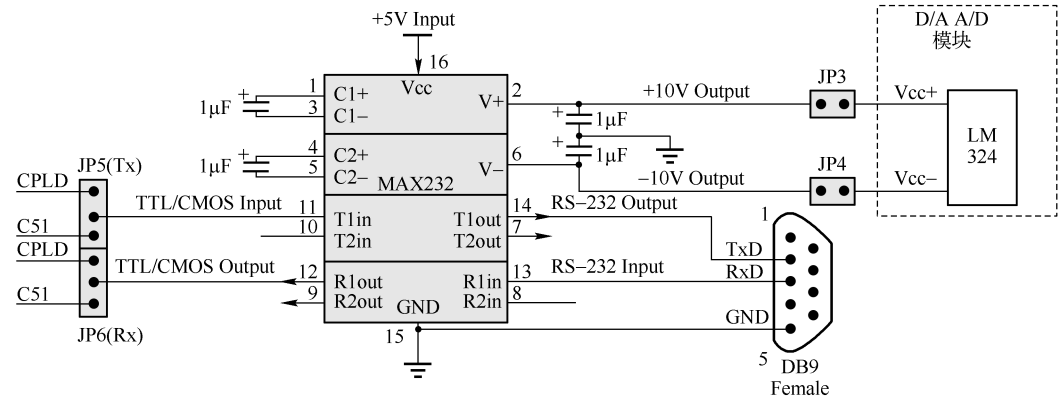


图 6.45 MAX232 电路原理图

图 6.45 中 MAX232 的 2、6 引脚输出±10V 的电压与 D/A 电路中的运算放大器 LM324 相连，提供 LM324 所需的负电压。

MAX232 的 11、12 引脚分别连接到两个跳线 JP5、JP6 上，可由这两个跳线决定 MAX232 是与 CPLD 相连还是与单片机相连。与 CPLD 之间的连接通过两个反相器缓冲隔离。具体连接如图 6.46 所示。

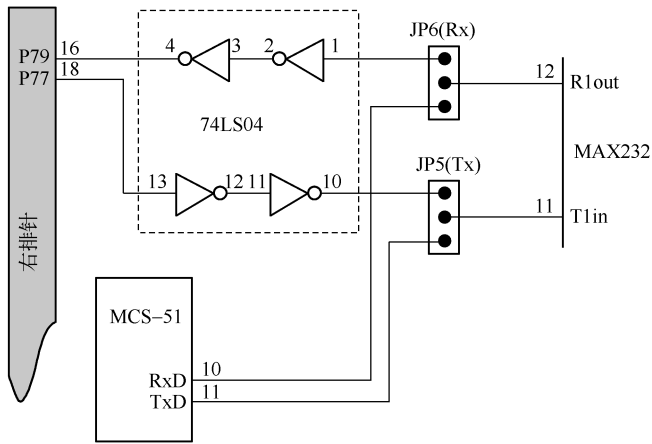


图 6.46 MAX232 电路原理图

串行异步传输时的数据格式如图 6.47 所示。

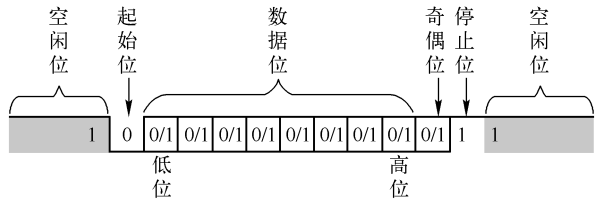


图 6.47 异步串行通信数据格式（一帧数据）

因此，通信双方需要就两个方面进行协定：帧格式和各个位的宽度（或者位的传输速率）。图 6.47 中的数据位并非只能是 8 位，数据位还可以是 5 位、6 位或 7 位，奇偶校验位用

于对数据进行奇偶校验，这个位是可以省略的。而停止位的长度可以是 1 位、1.5 位或 2 位。常用的帧格式为“N. 8. 1”格式，即 1 帧含 1 个起始位、8 个数据位、1 个停止位，无奇偶校验位。

位的传输速率用波特率（Baud Rate）表示。对于低速传输的情况，一般可认为波特率即每秒比特数（bits per second, bps）。对于 N. 8. 1 格式的帧，一帧有 10 位，则波特率为 2400bps 时每秒可以传输 2400bits，即可传输 $2400/10=240$ 帧=240 字符。一般通信双方都采用相同的波特率才能正常通信。

由于接收器、发送器异步工作，无须连接接收和发送时钟。对于发送方，只需要按照协定的帧格式和波特率将需要发送的字符逐位发送出去即可。对于接收方，可一直监测接收线电平，出现低电平时则表示出现起始位。然后根据协定的波特率逐位接收，转换成字符，如果接下来的位是停止位，则表示接收正确，否则表示接收错误，放弃所接收的字符。

但有时接收线出现的负跳变只是噪声信号而不是真正的起始位，用上述方法判断就会造成错误，所以需要采用更可靠的方法来检测起始位。一般的方法是采用频率是波特率若干倍的采样时钟进行采样，例如常用 16 倍频率的采样时钟。这样，对每一位可进行 16 次采样。当在接收线上检测到低电平时，开始连续采样 8 次，若每次采样结果都是低电平，则可认为被采样的信号是真正的起始位。此后每 16 个采样时钟采样一次，即可确保每次采样都是每位的中间位置。

三、实验内容

根据上述串行通信的基本原理，用 VHDL 语言描述 RS-232 串行异步接收器，实现硬件实验平台与计算机的串行通信，并把接收的数据在数码管上显示出来。本实验的部分 VHDL 代码如下：

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity simprx232 is
    Port ( sclk : in std_logic;           --时钟
          rxd : in std_logic;           --接 MAX232 的 12 脚
          dout : out std_logic_vector(7 downto 0); --用于输出接收到的数据
          sel : in std_logic_vector(2 downto 0));
end simprx232;
```

本实验的 CPLD 引脚分配如表 6. 14 所示。

表 6. 14 CPLD 引脚分配

信 号 名	引 脚	信 号 名	引 脚
sclk	P9	dout5	P6

续表

信 号 名	引 脚	信 号 名	引 脚
rxld	P79	dout4	P5
sel0	P80	dout3	P4
sel1	P81	dout2	P3
sel2	P82	dout1	P2
dout7	P83	dout0	P1
dout6	P7		

P9 对应时钟电路的 CLK OUT1，为一个 12MHz 的时钟脉冲。接收器可以接收 9600bps、8. N. 1 格式的串行数据，并将最后接收到的数据显示在数码管上。

四、实验步骤

1. 编写 RS-232 串行异步接收器的 VHDL 程序，进行语法检查，直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直至得到正确的波形仿真结果。
3. 对 VHDL 语言实现的异步接收器分别进行逻辑综合，并进行综合后仿真。
4. 对 VHDL 语言实现的异步接收器分别进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
5. 布局布线后进行时序仿真。
6. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，开启电源，把仿真正确的 VHDL 编译代码写入硬件实验平台的 CPLD 或 FPGA 中。
7. 将实验板右端串口控制 JPRS2321、JPRS2322 全部右插。
8. 将频率选择区 CLK1（即 JP157）用短路帽接 12MHz。
9. 把串口线的一端接在计算机的 COM1 端口上，另一端接在实验板串行通信口 JP182 上。
10. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

实验十 红外通信芯片设计实验

一、实验目的

1. 了解红外遥控器的收发原理。
2. 学会用 VHDL 语言实现红外遥控器信号解码芯片的设计。

二、实验原理

红外线遥控是目前使用最广泛的一种通信和遥控手段。由于红外线遥控装置具有体积

小、功耗低、功能强、成本低等特点,因而继彩电、录像机之后,在录音机、音响设备、空调设备以及玩具等其他小型电器装置上也纷纷采用红外线遥控。工业设备中,在高压、辐射、有毒气体、粉尘等环境下,采用红外线遥控不仅可靠而且能有效地隔离电气干扰。

通用红外遥控系统由发射和接收两大部分组成,应用编/解码专用集成电路芯片来进行控制操作,发射部分包括键盘矩阵、编码调制、LED 红外发送器;接收部分包括光/电转换放大器,解调、解码电路,如图 6.48 所示。

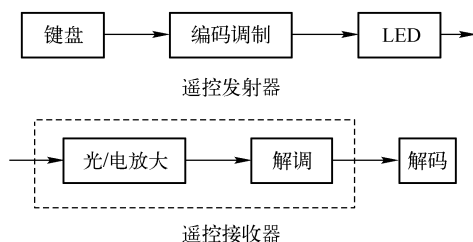


图 6.48 通用红外遥控系统

现以日本 NEC 的 $\mu\text{PD6121G}$ 组成发射电路为例说明编码原理。当发射器按键按下后,即有遥控码发出,所按的键不同对应的遥控编码也不同,这种遥控码具有以下特征:

采用脉宽调制的串行码,以脉宽为 0.565ms 、间隔为 0.56ms 、周期为 1.125ms 的组合表示二进制的“0”;以脉宽为 0.565ms 、间隔为 1.685ms 、周期为 2.25ms 的组合表示二进制的“1”,其波形如图 6.49 所示。

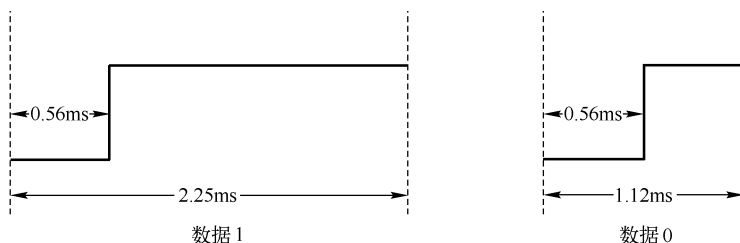


图 6.49 遥控码的“0”和“1”

由上述“0”和“1”组成的 32 位二进制码经 38kHz 的载频进行二次调制以提高发射效率,达到降低电源功耗的目的。然后再通过红外发射二极管产生红外线向空间发射。

$\mu\text{PD6121G}$ 产生的遥控编码是连续的 32 位二进制码组,其中前 16 位为用户识别码,能区别不同的电器设备,防止不同机种遥控码互相干扰。该芯片的用户识别码固定为十六进制 01H ;后 16 位为 8 位操作码(功能码)及其反码。遥控器在按键按下后,周期性地发出同一种 32 位二进制码,周期约为 108ms 。一组码本身的持续时间随它所包含的二进制“0”和“1”的个数不同而不同,大约在 45ms 至 63ms 之间,图 6.50 所示为发射波形图。

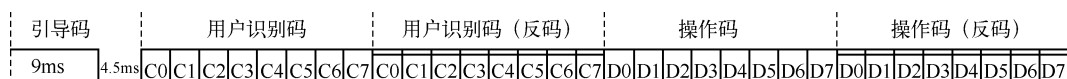


图 6.50 发射波形图

当一个按键按下超过 36ms ,振荡器使芯片激活,将发射一组 108ms 的编码脉冲,这 108ms 发射代码由一个起始码 (9ms)、一个结束码 (4.5ms)、低 8 位地址码 ($9\sim 18\text{ms}$)、高 8 位地址码 ($9\sim 18\text{ms}$)、8 位数据码 ($9\sim 18\text{ms}$) 和这 8 位数据的反码 ($9\sim 18\text{ms}$) 组成。如果按键按下超过 108ms 仍未松开,接下来发射的代码(连发代码)将仅由起始码 (9ms) 和结束码 (2.25ms) 组成。

DE2-115 开发板配备有一个红外接收（IR）模组（型号：IRM-V538N7/TR1），相关的电路图可以在 DE2-115 系统光盘里面的 DE2_115_datasheets\IR_Receiver 目录下找到。需要注意的是，这个一体化接收模组仅兼容 38kHz 载波脉宽调制模式。使用附带的 uPD6121G 芯片编码的遥控器可以产生与接收器匹配的调制信号。图 6.51 给出了 IR 相关的电路图，相关引脚配置可以在表 1 中找到。

表 6.15 IR 引脚配置

Signal Name	FPGA Pin No.	Description	I/O Standard
IRDA_RXD	PIN_Y15	IR_Receiver	3.3V

DE2-115 附带的遥控器如图 6.52 所示，按键编码信息如表 6.16 所示。

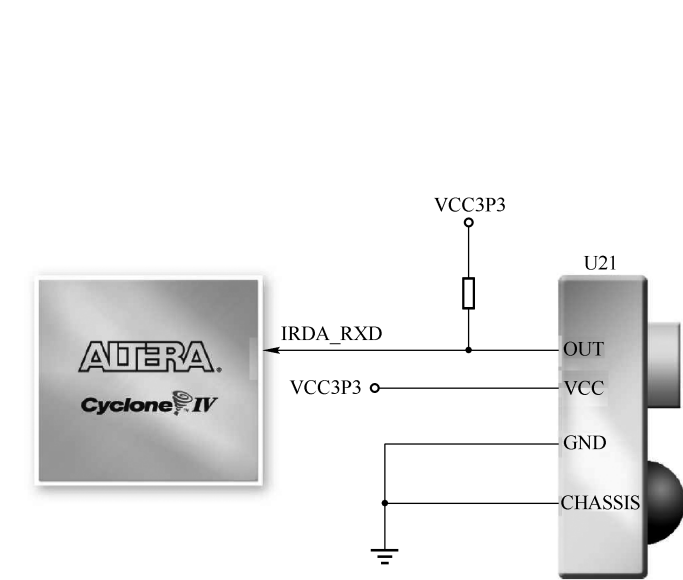


图 6.51 FPGA 与 IR 间的连接示意图



图 6.52 遥控器外形

表 6.16 遥控器按键编码信息

按键	按键编码	按键	按键编码	按键	按键编码	按键	按键编码
	0x0F		0x13		0x10		0x12
	0x01		0x02		0x03		0x1A
	0x04		0x05		0x06		0x1E
	0X07		0X08		0x09		0x1B
	0x11		0x00		0x17		0x1F
	0x16		0x14		0x18		0x0C

IR 接收器解调接收到的信号的过程如下：

当红外遥控器有按键按下时，会发送一个数据帧，波形如图 6.53 所示。帧数据的起始是一串引导码，然后是按键相关编码信息，最后是一个结束比特。

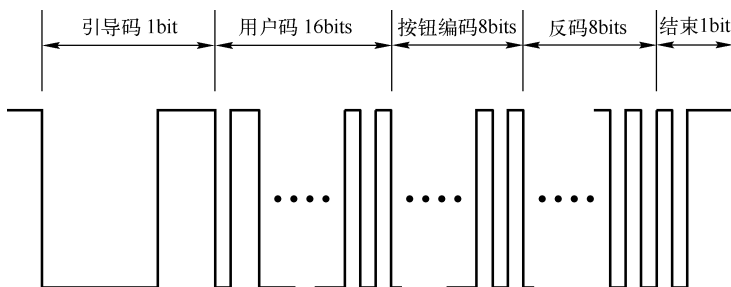


图 6.53 IR 遥控器发射波形（基带信号）

当 DE2-115 接收到数据帧后，一体化红外接收头的数据输出会送给 FPGA，FPGA 内部实现了帧数据解码所需电路，电路主要包括三个部分：编码检测器、状态机及移位寄存器，如图 6.54 所示。编码检测器检测引导码并将信息回馈给状态机。

一旦检测到引导码，状态机即转移到 GUIDANCE 状态。当编码检测器完成用户码的检测后，状态机转入 DATAREAD 状态。在这个状态，用户码、反码及设备码信息会移入移位寄存器，并译码输出给 7 段数码管。图 6.55 给出了状态机的状态转移过程。此设计需使用 50MHz 时钟输入。

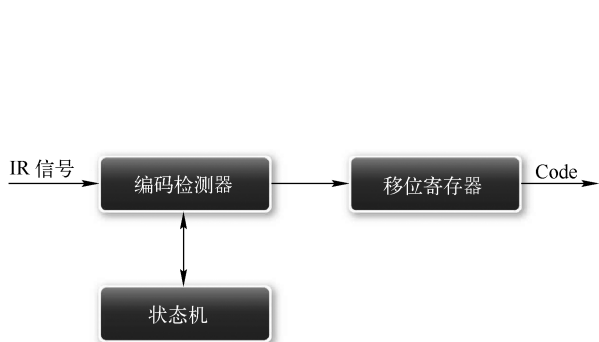


图 6.54 IR 接收控制器框图

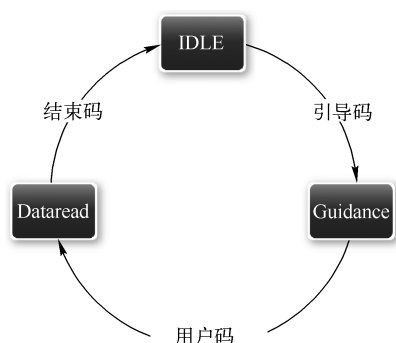


图 6.55 状态机状态转移图

三、实验内容

编写 VHDL 程序，用有限状态机方式实现用 FPGA 进行红外解码并控制 12 个 LED 灯的亮灭，同时用数码管显示红外解码的结果。

可参考的顶部实体原理图如图 6.56 所示。其中包含了分频模块、红外解码及控制模块以及数码管显示模块。

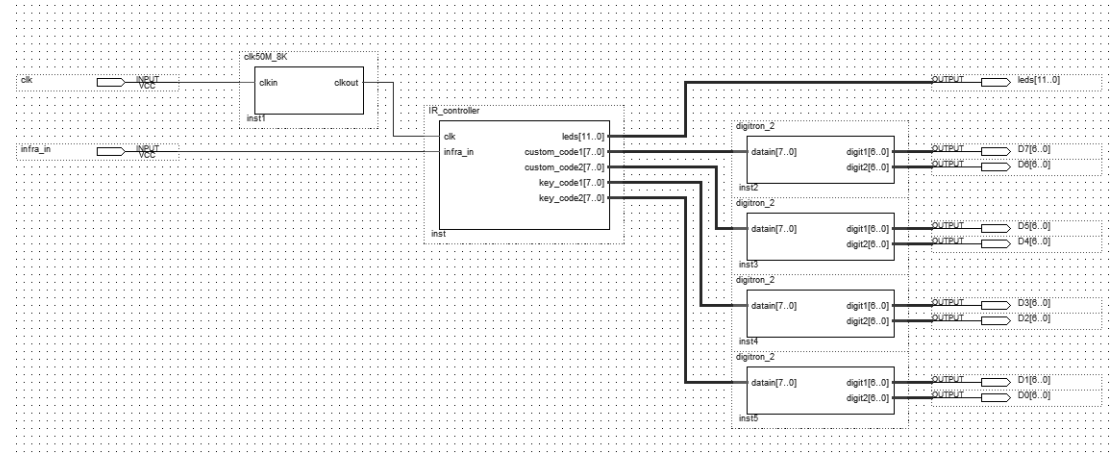


图 6.56 顶部实体原理图

四、实验步骤

1. 编写 VHDL 程序实现实验内容所要求的芯片设计，进行语法检查，直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直至得到正确的波形仿真结果。
3. 对 VHDL 语言实现的芯片设计进行逻辑综合，并进行综合后仿真。
4. 对 VHDL 语言实现的芯片设计进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
5. 布局布线后进行时序仿真。
6. 参考附录进行 FPGA 引脚分配。
7. 计算机通过写入电缆与硬件实验平台连接，硬件实验平台与电源连接，检查无误后，开启电源，把仿真正确的 VHDL 编译代码写入 DE2-115 硬件实验平台。
8. 进行硬件在线调试，检查结果是否符合设计要求。若不符合，需要修改 VHDL 程序，重复以上各个步骤，直到硬件在线调试正确。

实验十一 DDS 频率合成器芯片设计实验

一、实验目的

1. 理解 DDS 的基本原理。
2. 掌握用 VHDL 实现 DDS 芯片设计。
3. 学会用嵌入式逻辑分析仪 SignalTap II 观察片内信号。

二、实验原理

随着现代电子技术的不断发展，在通信系统中往往需要在一定频率范围内提供一系列稳

$\sim \frac{\pi}{2}$ 中 $3FF-x$ 地址的数据取反再加 1。

根据以上的分析, DDS 的合成频率为:

$$f_0 = \frac{f_{\text{clk}}}{4} \times \frac{\text{fword}}{2^{\text{ADDR} + \text{FSIZE}}}$$

2. ROM 查找表的设计

针对不同的可编程器件, ROM 查找表的设计所采用的方法也不同。主要是基于 lpm_rom 和 VHDL 选择语句这两种方法。

使用 lpm_rom 的波形存储表只需要产生数据文件 *.mif, 然后直接在定制 lpm_rom 时, 添加数据文件即可。不过这种方法只有在 FPGA 支持内部嵌入式阵列块 (EAB) 时才可以使用; 使用 VHDL 选择语句比较直观, 如图 6.58 所示, 但当输入数据量较大时, 这种方法是比较烦琐的。此次设计采用第一种方法。

```
subtype slv8 is std_logic_vector(7 downto 0);
type rom128x8 is array (0 to 127) of slv8;
constant sinus_rom : rom128x8 := (
  x"00", x"03", x"06", x"09", x"0c", x"0f", x"12", x"15",
  x"18", x"1b", x"1e", x"21", x"24", x"27", x"2a", x"2d",
  x"30", x"33", x"36", x"39", x"3b", x"3e", x"41", x"43",
  x"46", x"49", x"4b", x"4e", x"50", x"52", x"55", x"57",
  x"59", x"5b", x"5e", x"60", x"62", x"64", x"66", x"67",
  x"69", x"6b", x"6c", x"6e", x"70", x"71", x"72", x"74",
  x"75", x"76", x"77", x"78", x"79", x"7a", x"7b", x"7b",
  x"7c", x"7d", x"7d", x"7e", x"7e", x"7e", x"7e", x"7e",
  x"7f", x"7e", x"7e", x"7e", x"7e", x"7e", x"7d", x"7d",
  x"7c", x"7b", x"7b", x"7a", x"79", x"78", x"77", x"76",
  x"75", x"74", x"72", x"71", x"70", x"6e", x"6c", x"6b",
  x"69", x"67", x"66", x"64", x"62", x"60", x"5e", x"5b",
  x"59", x"57", x"55", x"52", x"50", x"4e", x"4b", x"49",
  x"46", x"43", x"41", x"3e", x"3b", x"39", x"36", x"33",
  x"30", x"2d", x"2a", x"27", x"24", x"21", x"1e", x"1b",
  x"18", x"15", x"12", x"0f", x"0c", x"09", x"06", x"03" );
```

图 6.58 使用 VHDL 选择语句定义 ROM

采用 ROM 压缩技术, 只存储 $0 \sim 2\pi$ 的正弦信号幅度值中的 $0 \sim \frac{\pi}{2}$ 的部分。

mif 文件是在编译和仿真过程中作为存储器 (ROM 或 RAM) 初始化的输入文件, 即 memory initialization file。创建 mif 文件的方式有很多种, 在本设计中, 利用 Matlab 来生成 mif 文件。mif 文件编写格式如下:

```
width=10;
depth=1024;
x=[0:1023]/1023*pi/2;
y=floor(1023*sin(x)); % 产生正弦信号 0~pi 之间幅度的向量
fid=fopen('C:\Users\Administrator\Desktop\sin_data.mif','w');
fprintf(fid,'--Generated by MATLAB,Used by QuartusII\n\n');
fprintf(fid,'WIDTH=%d;\n',width); % 写入存储位宽
fprintf(fid,'DEPTH=%d;\n\n',depth); % 写入存储深度
```

```

fprintf(fid, ADDRESS_RADIX=UNS; \n);           % 写入地址类型为无符号整型
fprintf(fid, DATA_RADIX=UNS; \n \n);         % 写入数据类型为无符号整型
fprintf(fid, CONTENT BEGIN \n);               % 起始内容
for i=0:1023
fprintf(fid, \t% ld:% ld; \n', i, y(i+1));
end
fprintf(fid, END; \n);
fclose(fid);

```

生成的 mif 文件的部分数据如图 6.59 所示。

3. SignalTap II 的原理及使用方法

SignalTap II 全称 SignalTap II Logic Analyzer，是第二代系统级调试工具，可以捕获和显示实时信号，观察在系统设计中的硬件和软件之间的互相作用。Quartus II 软件可以选择要捕获的信号、开始捕获的时间，以及要捕获多少数据样本。还可以选择时间数据从器件的存储器块通过 JTAG 端口传送至 SignalTap II Logic Analyzer，以及设置 I/O 引脚以供外部逻辑分析仪或示波器使用。将实时数据提供给工程师帮助调试。

SignalTap II 获取实时数据的原理是在工程中引入 Megafunction 中的 ELA (Embedded Logic Analyzer)，以预先设定的时钟采样实时数据，并存储于 FPGA 片上 RAM 资源中，然后通过 JTAG 传送回 Quartus II 进行分析。可见其实 SignalTap II 也是在工程额外加入了模块来采集信号，所以使用 SignalTap II 需要一定的代价，首先是逻辑单元 (ELA)，其次是 RAM，如果工程中剩余的 RAM 资源充足，则 SignalTap II 一次可以采集较多的数据，而如果 FPGA 资源已被工程耗尽，则无法使用 SignalTap II 调试。

基本操作：

(1) 首先确保工程能够成功综合，然后在 Quartus 界面下选择 Tools->SignalTap II Logic Analyzer，在此界面下确保 JTAG 和 FPGA 开发板能用，如图 6.60 所示。

Addr	+0	+1	+2	+3	+4	+5	+6	+7
0	0	1	3	4	6	7	9	10
8	12	14	15	17	18	20	21	23
16	25	26	28	29	31	32	34	36
24	37	39	40	42	43	45	47	48
32	50	51	53	54	56	58	59	61
40	62	64	65	67	69	70	72	73
48	75	76	78	80	81	83	84	86
56	87	89	90	92	94	95	97	98

图 6.59 mif 文件的部分数据

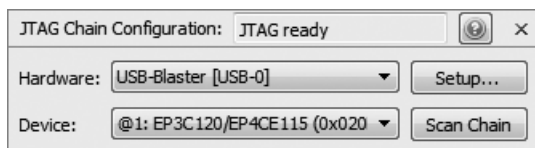


图 6.60 确保 Jtag 和 FPGA 开发板可用

(2) 在 Signal Configuration 中设置主时钟，单击“Clock”按钮后面的“...”，在 Filter 下选择 Post-Compilation，单击“List”按钮可以看到节点名，选择原理图上的全局时钟作为 SignalTap 的采样时钟，如图 6.61 所示。

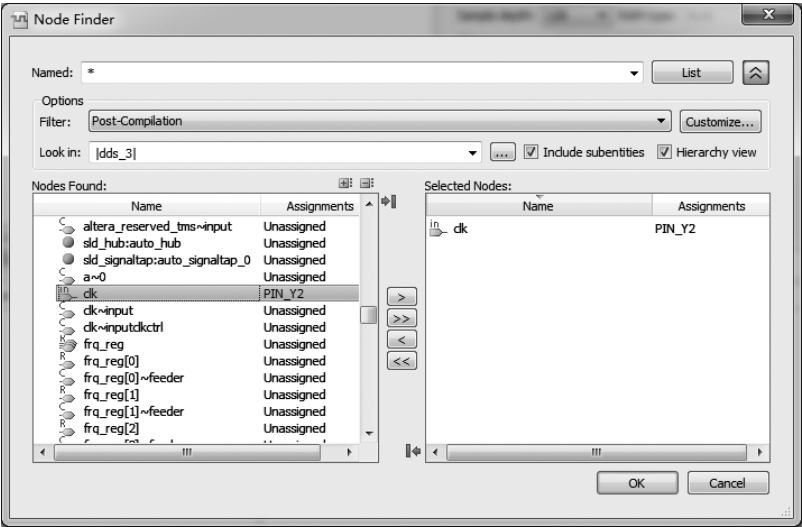


图 6.61 设置主时钟

(3) 选择采样深度如图 6.62 所示，采样深度是指采集到的数据能够存入的存储器大小，也表示每次能够观察到的信号深度，根据具体情况而定，深度越大，需要的资源越多，这里选择 128。

(4) 设置想要观察的节点，如图 6.63 所示。确保选中 Setup 标签，在空白区域双击以添加节点。在弹出界面的 Filter 栏中选择 Post-Compilation，单击“List”按钮，选择要观察的信号。添加成功后会在 Setup 和 Data 标签下显示，如图 6.64 所示。

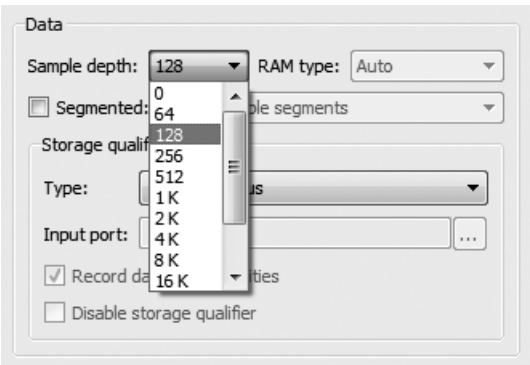


图 6.62 设置采样深度

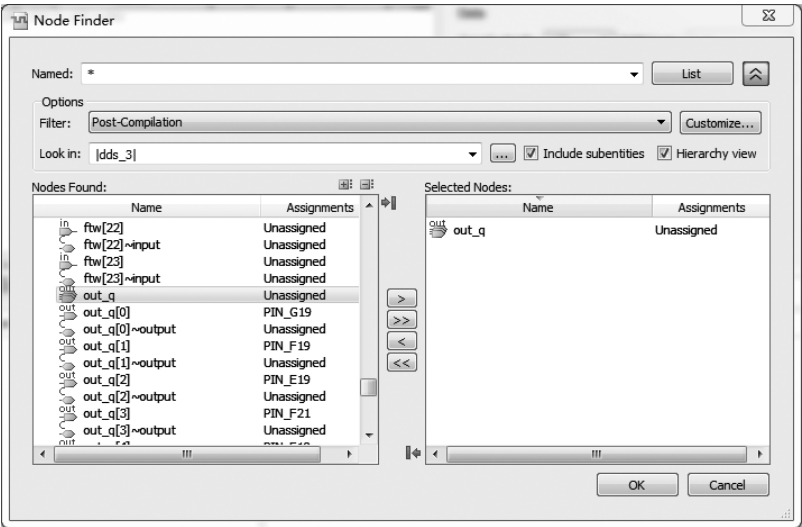


图 6.63 添加要观察的节点

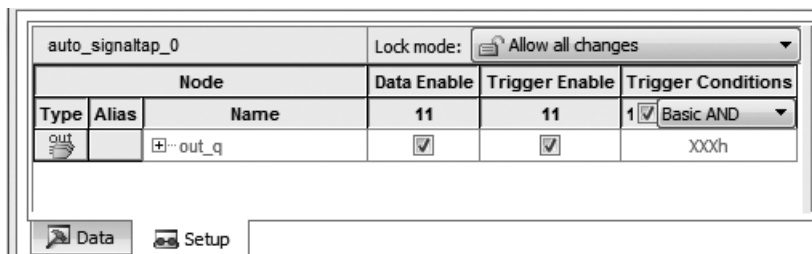


图 6.64 成功添加的节点

(5) 在 Quartus 界面下单击“Save all”按钮以保存所有文件，然后综合，再到 SignalTap 界面下选择 sof 文件下载，如图 6.65 所示。



图 6.65 下载文件

(6) 成功下载 sof 文件之后可以看到“Ready to acquire”。Instance Manager 右边有四个图标分别代表单次运行、连续运行、停止和读数据，如图 6.66 所示。单击连续运行图标，就能看到 SignalTap 在采集数据，观察到的数据如图 6.67 所示。

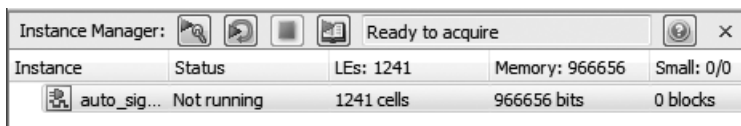


图 6.66

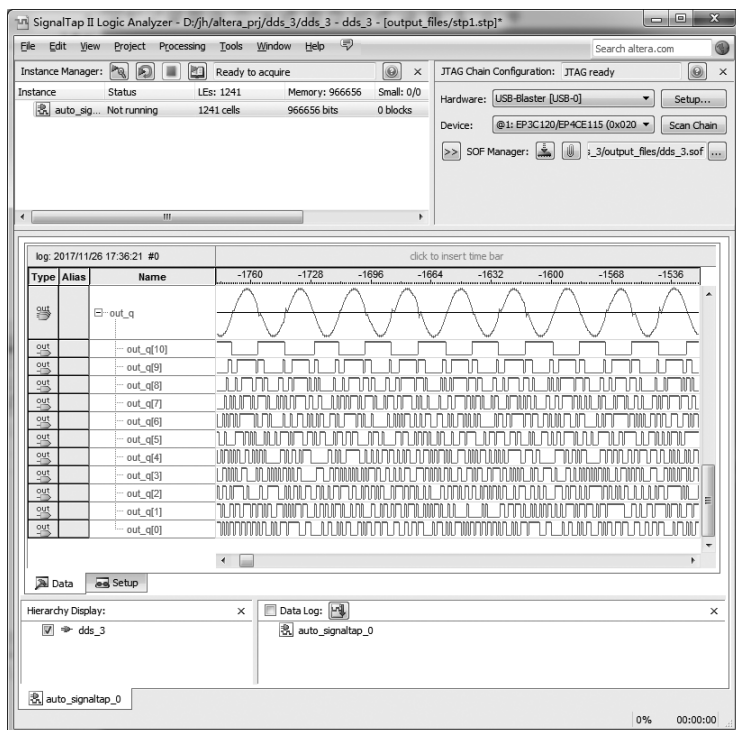


图 6.67 观察信号

(7) 因为使用 SignalTap，所以会额外消耗更多资源，如果不想使用 SignalTap，可以在 Assignment->Setting 中 SignalTap II Logic Analyzer 标签下进行设置，将 Enable SignalTap II Logic Analyzer 前面的勾选去掉即可，如图 6.68 所示，这样综合出来的电路就不带 SignalTap，资源占用就会减少。

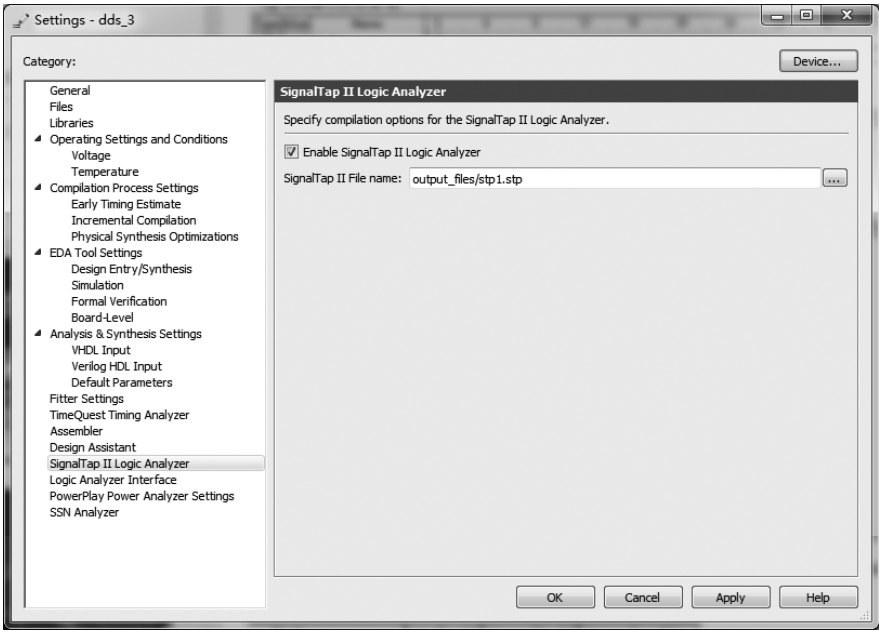


图 6.68 禁用 SignalTap

三、实验内容

根据上述 DDS 的基本原理，用 VHDL 语言描述直接数字频率合成模块，并用 ModelSim 进行仿真验证。本实验的部分 VHDL 代码如下：

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity DDS is
Port ( fword : in std_logic_vector(7 downto 0);      --频率控制字
      clk : in std_logic;                            --系统时钟
      rst : in std_logic;                            --复位信号
      dout : out std_logic_vector(10 downto 0)       --幅度值输出
);
end DDS;

architecture behavioral of DDS is
component DDS_ROM
```

--元件例化

```
Port ( address : in std_logic_vector(9 downto 0);  
      clock : in std_logic_vector;  
      q : out std_logic_vector(9 downto 0);  
end component;  
.....
```

四、实验步骤

1. 编写 DDS 模块的 VHDL 程序，进行语法检查，直到语法检查正确。
2. 用 VHDL 语言编写 TestBench 文件，对上述 VHDL 程序进行 ModelSim 功能仿真，如仿真波形有误，需重新修改 VHDL 程序，重复以上各个步骤，直至得到正确的波形仿真结果。
3. 进行逻辑综合，并进行综合后仿真。
4. 进行实现过程（包括翻译、映射、布局布线 3 个小步骤）。
5. 参考附录进行 FPGA 引脚分配。
6. 进行硬件在线调试，并用嵌入式逻辑分析仪观察片内信号。

实验十二 一个简单的 CPU 芯片设计

一、实验目的

1. 理解 CPU 芯片的内部架构。
2. 掌握用 VHDL 语言实现 CPU 芯片的设计。

二、实验原理

传统的计算机由三个主要单元组成：中央处理器（CPU）单元、存储器、输入/输出硬件。存储器是用来存储程序指令和数据的，输入/输出硬件是用来与其他设备通信的。这些单元是由总线相连接的，如图 6.69 所示。通常在线上传输的信号包括存储地址、存储数据和总线的状态。总线状态信号指示当前总线的操作是内存的读、内存的写或输入/输出操作。CPU 内部包括小数量的寄存器，这些寄存器是用来存储处理器内部数据的。诸如 PC、IR、AC、MAR、MDR 等处理器内部用来存储数据的寄存器是由 D 触发器实现的。在 CPU 内部还包括一个或多个算术逻辑单元（ALU），算术逻辑单元是用来做算术和逻辑运算的。通常，算术逻辑单元的操作包括加、减、逻辑与、逻辑或。寄存器和总线之间的连接是简单的点对点连接，当有几个寄存器驱动总线时，这个连接用多路选择器打开连接输出或三态输出。控制单元是一个复杂的有限状态机，它控制处理器内部的操作。处理器执行的主要操作是存储在内存中的指令序列。处理器从内存读或取指令，然后进行指令译码，决定进行哪些操作，接着执行指令。控制单元控制处理器中的这些操作序列。

计算机的程序就是一些指令序列。指令存储在存储器中，在这个实验中我们实现一个简单的计算机设计，每条指令由 16bits 组成，如图 6.70 所示。

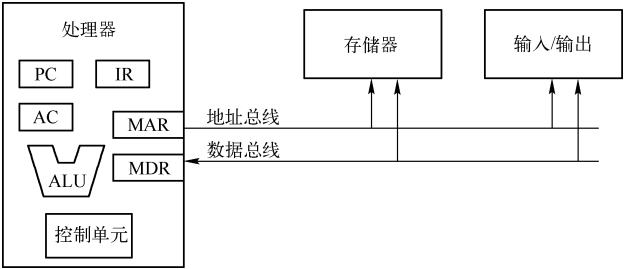


图 6.69 简单的计算机系统体系结构



图 6.70 简单 CPU 指令格式

每条指令由两部分组成：操作码（Opcode）和地址（Address）。通常，一条指令发送一系列的数据值，然后通过 ALU 执行这个操作。操作码说明这个操作，例如加、减，这是将被执行的操作。地址可能指向数据存储的位置，也可能指向另一条指令。本实验中的基本指令如表 6.17 所示。

表 6.17 基本指令

指 令	操 作	操作码的值
ADD address	AC <= AC+address 地址中的值	00
STORE address	address 地址中的值<= AC	01
LOAD address	AC<= address 地址中的值	02
JUMP address	PC<= address	03
JNEG address	If AC<0 Then PC <=address	04

下面给出一个程序的例子，计算 $A = B + C$ ，过程如表 6.18 所示。

表 6.18 汇编语言与机器码

汇 编 语 言	机 器 码
LOAD B	0211
ADD C	0012
STORE A	0110

处理器从存储器中读或取指令，然后进行指令译码，决定进行哪些操作，接着执行指令的过程如图 6.71 所示。一个简单的状态机——控制单元控制处理器中的这个操作序列。执行取指令、指令译码、执行指令需要几个寄存器传输这些操作和几个时钟周期。

为了保证程序能够连续执行，在 CPU 中必须设置程序计数器，用以存放下一条指令在主存中的地址，因此又称为指令计数器或指令指针 IP（Instruction Pointer）。在开始执行程序前，需将程序的起始地址即程序的第一条指令所在主存单元地址送入 PC，以便从程序的第一条指令开始执行。当现行指令执行完毕，通常由程序计数器提供后继指令地址，并送往主存的地址寄存器。当指令按顺序执行时，每读取一条指令后，PC 应加上一个增量（通常为刚读取指令所占主存的单元数），以指向下一条指令地址。当遇到转移指令时，需改变程序的执行顺

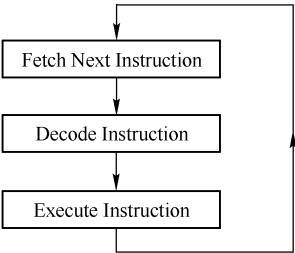


图 6.71 处理器取指、译码、执行循环

序, 则由转移指令形成转移地址送入 PC 作为后继指令地址。PC 计数功能可由 ALU 配合实现, 此时 PC 为一单纯寄存器; 也可让 PC 本身具有计数逻辑。

指令寄存器 IR (Instruction Register) 用来存放当前正在执行的一条指令。当执行一条指令时, 通常是先将其从存储器读出到主存数据寄存器 MDR (Memory Data Register) 中, 然后再送往 IR 中。

累加器 (AC) 主要是用来进行数据计算和存储临时的程序数据的寄存器。一个详细的取指令、指令译码和执行指令的过程如图 4 所示。

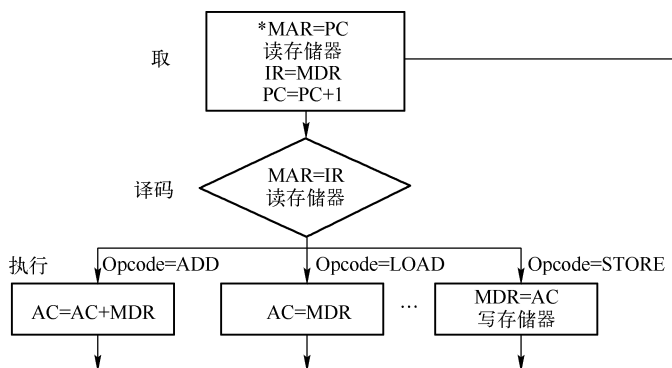


图 6.72 详细的取指令、指令译码和执行指令的过程

三、实验内容

根据上面介绍的实验原理, 设计一个简单的 CPU, 然后下载到硬件实验平台中进行测试。

四、实验步骤

1. 将系统分成几个模块, 然后编写各个模块的 VHDL 程序, 进行语法检查, 直到语法检查正确。

2. 用 VHDL 语言编写 TestBench 文件, 对上述 VHDL 程序进行 ModelSim 功能仿真, 如仿真波形有误, 需重新修改 VHDL 程序, 重复以上各个步骤, 直至得到正确的波形仿真结果。

3. 进行逻辑综合, 并进行综合后仿真。

4. 进行实现过程 (包括翻译、映射、布局布线 3 个小步骤)。

5. 布局布线后进行时序仿真。

6. 计算机通过写入电缆与硬件实验平台连接, 硬件实验平台与电源连接, 检查无误后, 开启电源, 把仿真正确的 VHDL 编译代码写入硬件实验平台中的 CPLD 或 FPGA 中。

7. 进行硬件在线调试, 检查结果是否符合设计要求。若不符合, 需要修改 VHDL 程序, 重复以上各个步骤, 直到硬件在线调试正确。

实验十三 LCD 显示 SOC 芯片设计实验

一、实验目的

1. 了解 SOC 和 SOPC 技术。
2. 掌握 SOPC Builder 与 Nios II IDE 的使用。
3. 掌握软硬件协同设计方法。

二、实验原理

片上系统 (System On a Chip, SOC) 是指在单芯片上集成微电子应用产品所需的全部功能系统, 以超深亚微米 (Very Deep Submicron, VDSM) 工艺和知识产权 (Intellectual Property, IP) 核复用 (Reuse) 技术为支撑。SOC 技术是当前大规模集成电路 (VLSI) 的发展趋势, 也是 21 世纪集成电路技术的主流, 其为集成电路产业和集成电路应用技术提供了前所未有的广阔市场和难得的发展机遇。SOC 为微电子应用产品研究、开发和生产提供了新型的优秀的技术方法和工具, 也是解决电子产品开发中及时上市 (Time to Market, TTM) 的主要技术与方法。SOPC (System On a Programmable Chip), 即基于可编程片上系统, 它是芯片级电子系统设计。SOPC 主要有两大支撑点: 可编程片上系统器件所能提供的片上资源和可复用 IP 核库所能提供的 IP 核资源, 可以用来实现软硬件一体的片上系统 SOC。NiosII 是 2004 Altera 正式推出的一款 32 位 RISC 嵌入式处理器。Nios II 系列软核处理器是 Altera 的第二代 FPGA 嵌入式处理器, 其性能超过 200DMIPS, 在 Altera FPGA 中实现仅需 35 美分。Altera 的 Stratix、Stratix GX、Stratix II 和 Cyclone 系列 FPGA 全面支持 Nios II 处理器。NiosII 处理器软核就像 51 单片机一样, 是一个 CPU, 只不过不是像 51 单片机那样的硬件芯片实物, 而是由硬件描述语言构成的一个软核 IP, 配置到 FPGA 里面就能当 CPU 用了, 有了 CPU 就可以支持软件编程了。Nios II 系列包括 3 种产品: Nios II/f (快速) ——最高的系统性能, 中等 FPGA 使用量; Nios II/s (标准) ——高性能, 低 FPGA 使用量; Nios II/e (经济) ——低性能, 最低的 FPGA 使用量。这 3 种产品具有 32 位处理器的基本结构单元——32 位指令大小, 32 位数据和地址路径, 32 位通用寄存器和 32 个外部中断源; 使用同样的指令集架构 (ISA), 二进制代码百分之百兼容, 设计者可以根据系统需求的变化更改 CPU, 选择满足性能和成本的最佳方案, 而不会影响已有的软件投入。特别是, Nios II 系列支持使用专用指令。专用指令是用户增加的硬件模块, 它增加了算术逻辑单元 (ALU)。用户能为系统中使用的每个 Nios II 处理器创建多达 256 个专用指令, 这使得设计者能够细致地调整系统硬件以满足性能目标。

SOPC Builder 集成在 Quartus II 软件中, 是传统的 Nios II 硬件设计工具, 为设计者提供了一个强大的可以快速开发设计及验证的 SOPC 系统设计平台, 如图 6.73 所示。SOPC Builder 可以灵活定制 Nios II CPU 的许多特性甚至指令, 搭建基于总线的系统。它包含了一系列的模块, 例如 Nios II 处理器、存储器、总线、DSP 等 IP 核, 为了将微处理器核、外围设备、存储器和其他 IP 核相连接起来, SOPC Builder 能够自动生成片上 Avalon 总线和总线了仲裁器等所需的逻辑。完成 Nios II 的硬件开发后, SOPC Builder 可自动生成与自定义的

Nios II CPU 和外设系统、存储器、外设地址映射等相应的软件开发包 SDK。在生成的 SDK 基础上,启动 Nios II IDE (Nios Integrated Develop Environment) 进入 Nios II 软件开发流程。Nios II IDE 是 Altera 公司提供的专门开发 SOPC 软件的集成开发环境,它提供了一系列的管理工具,能完成对软件项目的管理,并能够完成对源代码进行编辑、编译、连接、调试、下载等功能。同时,Nios IDE 还集成了许多软件组件,如 RTOS、LWIP、FS 等软件组件,并提供了图形化的参数编辑方式。基于这个平台可以方便地进行使用 SOPC 应用软件的设计。用户可使用汇编语言或 C,甚至 C++ 语言来进行嵌入式程序设计,使用 GNU 工具或其他第三方工具进行程序的编译连接以及调试。

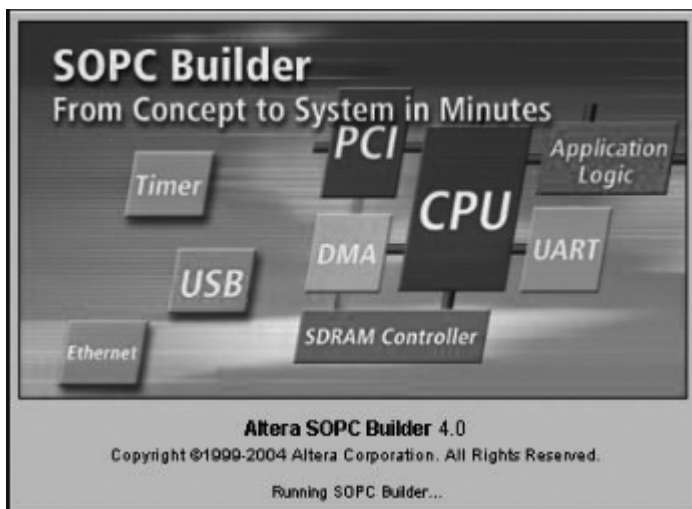


图 6.73 SOPC Builder 开发工具

SOPC 系统的开发流程一般分为硬件和软件两大部分,如图 6.74 所示,左侧流程图是硬件开发,右侧流程图是软件开发。硬件(按照习惯说法,将一个 SOPC 系统中的 Nios II CPU、外设、自定义组件等统称为硬件,它是由 IP 核和硬件描述语言来实现的;而在 Nios II CPU 上运行的程序称为软件)开发主要是创建 Nios II 系统,作为应用程序运行的平台;软件开发主要是根据系统应用的要求,利用 C/C++ 语言和系统所带的 API 函数编写实现特定功能的程序。这其中用到的主要工具是 Altera 公司的 Quartus II 和 Nios II IDE 以及集成在 Quartus II 中的 SOPC Builder。设计者首先根据任务要求决定系统需求,进行软、硬件划分,完成这项工作后,硬件工程师与软件工程师可以开始协同工作。硬件工程师用 Quartus II 建立工程,用 VHDL 硬件描述语言设计自定义组件,然后用 SOPC Builder 建立自己的 SOPC 系统,该系统主要包括 Nios II 软核处理器、片上存储器、定时器等 IP 核以及自己用 VHDL 硬件描述语言定义的组件,将生成的 SOPC 系统例化,并设置引脚分配、时序要求及其他设计约束,然后编译硬件设计并将 FPGA 设计下载到目标板中。在硬件工程师工作的同时,软件工程师可以用 Nios II IDE 开发应用软件,并在 Nios II IDE 中使用 Nios II 指令仿真器运行并调试软件,等硬件工程师将硬件设计下载到目标板中之后,软件工程师可将软件下载到目标板上的 Nios II 系统中,并在目标板上运行调试软件,如果发现设计不满足需求,则再改进硬件与软件的设计。

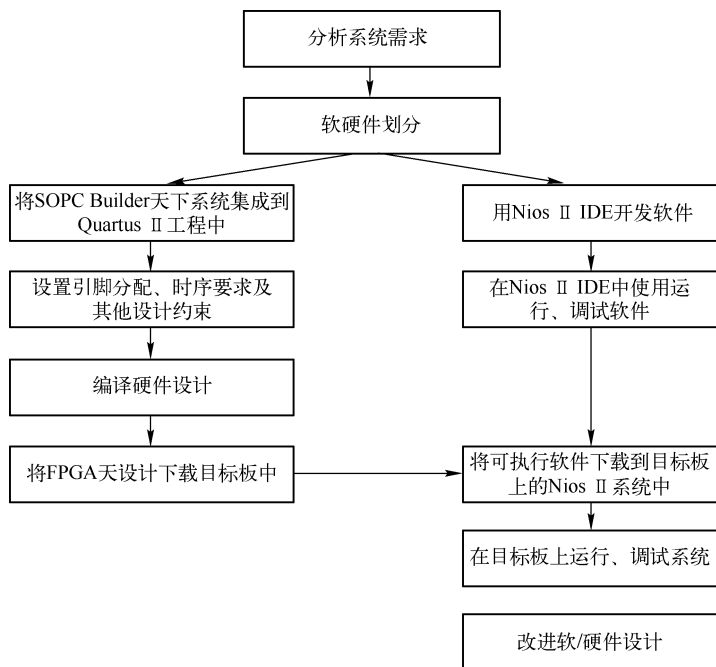


图 6.74 SOPC 系统设计流程

三、实验内容

1. 用 SOPC 技术实现一个字符液晶（Character LCD）SOC 系统，在字符液晶上显示“Hello LCD from NiosII!”。采用 SOPC Builder 工具在 DE2-115 平台上建立 SOPC 系统的硬件，这个系统包括一个 Nios II/s 嵌入式处理器、片上存储器（程序运行时将其导入 on-chip memory）、一个 JTAG UART 及 LCD 输出显示。用 C 语言实现 SOPC 系统的软件，在字符液晶上显示“Hello LCD from NiosII!”。

2. 用 SOPC 系统在 DE2-115 硬件实验平台上实现一个计数器。先在 DE2-115 平台上建立 SOPC 系统的硬件，这个系统包括一个 Nios II/s 嵌入式处理器、一个 JTAG UART 及一个定时器。另外，用 VHDL 语言实现一个自定义 7 段数码管组件，对自定义 7 段数码管组件进行了 8 次例化，分别对应于 7 段数码管的 HEX0 ~ HEX7。完成原理图设计：用 CLOCK_50 作为系统的时钟输入，SW [0] 作为复位信号输入，Nios II 的输出分别连接到 HEX0 ~ HEX7。启动 Nios II IDE，用 C 语言实现在 7 段数码管上显示计数器的当前值。

四、实验步骤

1. SOPC 硬件系统的建立

(1) 建立工程

启动 Quartus II 软件，用 New>New Project Wizard…菜单在 Quartus II 中新建一个工程，本实验中将工程的工作目录设定在 D:\de2_115\niosII_DE2_115，工程的名称为 HelloWorld。

(2) 用 SOPC Builder 建立一个新的 SOPC 硬件系统

有两种方式可以打开 SOPC Builder，可以直接单击页面右上角的 SOPC Builder 图标 ，

或者通过路径 Tool→SOPC Builder 打开。启动 SOPC Builder, 输入新系统的名称为 HelloWorld_System, 当然也可以输入其他名称, 但应注意, 所有的名称中都不能出现空格。Target HDL 选项可以选择 Verilog, 也可以选择 VHDL, 这里选择 VHDL。单击“OK”按钮, 进入 SOPC Builder 的主界面。此时系统会自动生成一个默认值为 50MHz 的驱动时钟 clk_0, 将其手动重命名为 clk, 并将其修改为 100MHz。

(3) 向系统中添加 Nios II 处理器

在 SOPC Builder 主界面左侧的组件列表中, 选中 Nios II Processor, 单击鼠标右键, 在弹出的菜单中选择 Add New Nios II Processor, 显示 Nios II 处理器配置界面。具体有以下三种: 经济型 CPU 内核 (Nios II/e), 最小的逻辑占用的优化, 占用逻辑元素 (LE) 最少, 功能最少; 标准型 CPU 内核 (Nios II/s), 平衡于性能和尺寸, 具有 Nios II CPU 核的一般功能; 全功能 CPU 内核 (Nios II/f), 最高性能优化, 具有 Nios II CPU 核的所有功能, 包括一些高级功能, 而需要使用的 LE 也是最多的。一般来说, 选择 Nios II/s 标准型 CPU 核即可满足一般系统需要。这里我们选择 Nios II/s 标准型内核, 其他参数采用默认值即可。从界面中可以看到, Nios II/s 占用约 1200 ~ 1400 个逻辑单元, 2 个 M4K RAM 块。由于可以添加指令缓存, 缓存需要占用额外的 M4K RAM 块, 因此 Nios II/s 与 Nios II/e 相比, 增加了指令缓存、分支预测、硬件乘法器及硬件除法器。Nios II/s 的最好性能可达 25 MIPS。单击“Next”按钮, 设置处理器的指令缓存和紧密耦合指令存储器, 选择指令缓存为 2K 字节, 不使用紧密耦合指令存储器。单击“Next”按钮, 设置 JTAG 调试模块, JTAG 调试模块分为四个级别, 每个级别的功能不同, 占用的逻辑资源也不同, 本设计选择占用逻辑资源最少的级别 Level 1。

(4) 添加片上存储器

片上存储器主要用于系统程序和数据的存储。在 SOPC Builder 主界面左侧的组件列表的 Memory 组中, 选中 On-Chip Memory (RAM 或 ROM), 单击鼠标右键, 在弹出的菜单中选择 Add New On-Chip Memory (RAM 或 ROM), 显示片上存储器的配置界面。配置中选择类型为 RAM, 数据宽度 (Data width) 默认为 32 位, 将存储大小改为 64kb, 单击“Finish”按钮完成添加。此时下方信息栏会出现地址冲突的错误提示, 可以忽略。

(5) 添加 JTAG UART

在 SOPC Builder 主界面左侧的组件列表的 Interface Protocols→Serial 组中, 选中 JTAG UART, 单击鼠标右键, 在弹出的菜单中选择 Add New JTAG UART, 显示 JTAG UART 配置界面。JTAG UART 用于与计算机通信, 使用默认配置即可, 单击“Finish”按钮完成 JTAG UART 的设置。

(6) 添加 LCD 模块和 System ID

在 SOPC Builder 组件列表的 Other 组中选中 System ID Peripheral, 单击鼠标右键, 在弹出的菜单中选择 Add New System ID Peripheral, 即可添加 System ID, 每个 Nios II 系统只能添加唯一的 ID。SOPC Builder 会使用 System ID 为每个系统提供识别符号, Nios II EDS 可以此防止使用者烧录了与 *.ptf 不符合的 *.sof。接受默认值即可, 单击“Finish”按钮完成。将 sysid_0 改成 sysid。注意, 一定要将名称改成 sysid, 因为 Nios II EDS 将会使用 sysid 做判断, 否则会产生错误。

(7) 设置复位地址和异常地址

当所有器件模块都添加以后, 下面的信息栏会发现很多的错误报警信息。此时右键单击

CPU 模块，选择 Edit 对 CPU 中的 Reset Vector（复位地址）和 Exception Vector（异常地址）进行设置，这两项地址选择 onchip_memory，即为之前添加的内部 RAM 存储模块。

(8) 调整基地址

SOPC Builder 为用户系统中每个 IP 模块制定默认的基地址，地址为一个 32 位的值，如果地址位出现重叠，则会出现错误警告。自动调整基地址的方法是选择 system→auto-assignment base address。分配完基地址之后，之前的错误提示就消失了。并可以通过 Auto-assign IRQs 自动分配中断优先级。

(9) 生成硬件系统

整个 SOPC 系统如图 6.75 所示。系统配置成功后，选择 System Generation，单击“Generate”按钮生成系统。当系统生成成功，单击“EXIT”按钮退出 SOPC Builder。

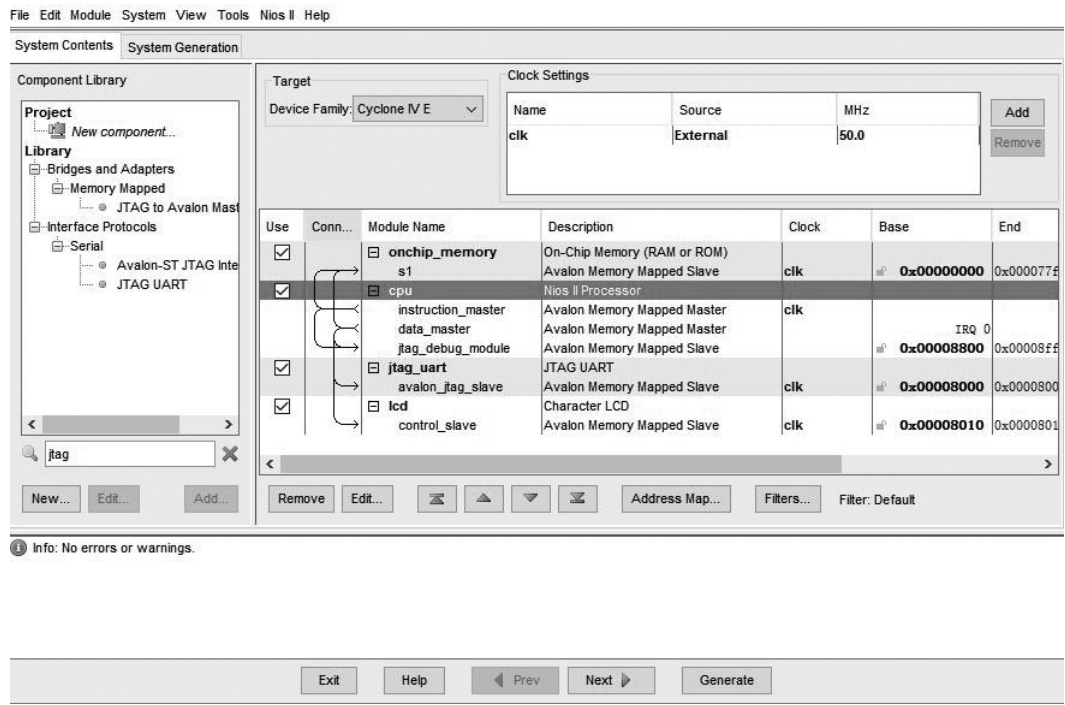


图 6.75 SOPC 硬件系统

(10) 用 VHDL 语言完成顶层实体

生成硬件系统之后，需要为工程建立一个顶层设计文件，这个设计文件的名称应该与工程的名称完全一致。设计文件可以是 Quartus II 允许的各种设计输入格式的文件，如 Verilog HDL、VHDL、AHDL 及原理图设计文件等。本实验中使用 VHDL。用 File>New 菜单建立一个新文件，选择 VHDL 文件，文件名称与工程名称保持一致，为 HelloWorld.vhd。将如下列出的 VHDL 代码保存至 HelloWorld.vhd。

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC.1164.ALL;

entity HelloWorld is
    Port ( iCLK_50 : in std_logic;
```

```

        iKEY      : in std_logic;
        oLCD_ON   : out std_logic;
        oLCD_BLON : out std_logic;
        oLCD_RW   : out std_logic;
        oLCD_EN   : out std_logic;
        oLCD_RS   : out std_logic;
        LCD_D     : inout std_logic_vector(7 downto 0)
    );
end HelloWorld;

architecture behavioral of HelloWorld is

component HelloWorld_System is
    Port ( clk : in std_logic;
          reset_n : in std_logic;
          LCD_E_from_the_lcd : out std_logic;
          LCD_RS_from_the_lcd : out std_logic;
          LCD_RW_from_the_lcd : out std_logic;
          LCD_data_to_and_from_the_lcd : inout std_logic_vector(7 downto 0)
    );
end component;

signal reset_n : std_logic;
begin
    oLCD_ON<= 1';
    oLCD_BLON<= 1';
    reset_n<= 1';
    u1: HelloWorld_System port map (
        iCLK_50,reset_n,oLCD_EN,oLCD_RS,oLCD_RW,LCD_D);
end behavioral;

```

(11) 引脚分配

接下来需要为设计分配引脚，可以在引脚规划器（Pin Planner）中手动分配引脚，也可以导入 DE2_115 系统光盘中提供的引脚分配表。在 DE2_115 光盘中有个名为 DE2_115_pin_assignments.csv 的文件，该文件中包含了 DE2_115 平台上 FPGA 的引脚分配信息。为方便起见，我们从 DE2_115_pin_assignments.csv 中导入引脚分配信息。

使用 Assignments>Import Assignments 菜单，选择 DE2_115_pin_assignments.csv 文件，按照提示导入引脚分配。

(12) 编译下载

当完成引脚配置之后，检查电路，进行工程编译：路径为 Processing→Start Compilation；或是单击右上方的编译快捷图标。工程编译完成，生成整个工程的整体报告，显示芯片利用率等信息。编译完成后，通过 Programmer 下载到板子上。路径为 Tools→Programmer；或者单击任务栏中的 programmer 图标打开。将硬件程序烧录入芯片中。

2. 软件设计

(1) 打开 Nios II IDE 软件，建立新工程：路径为 File->new->project；或者单击如图 6.76 所示的新建图标，选择 Nios II C/C++ Application，单“Next”按钮，进入工程设置页面。

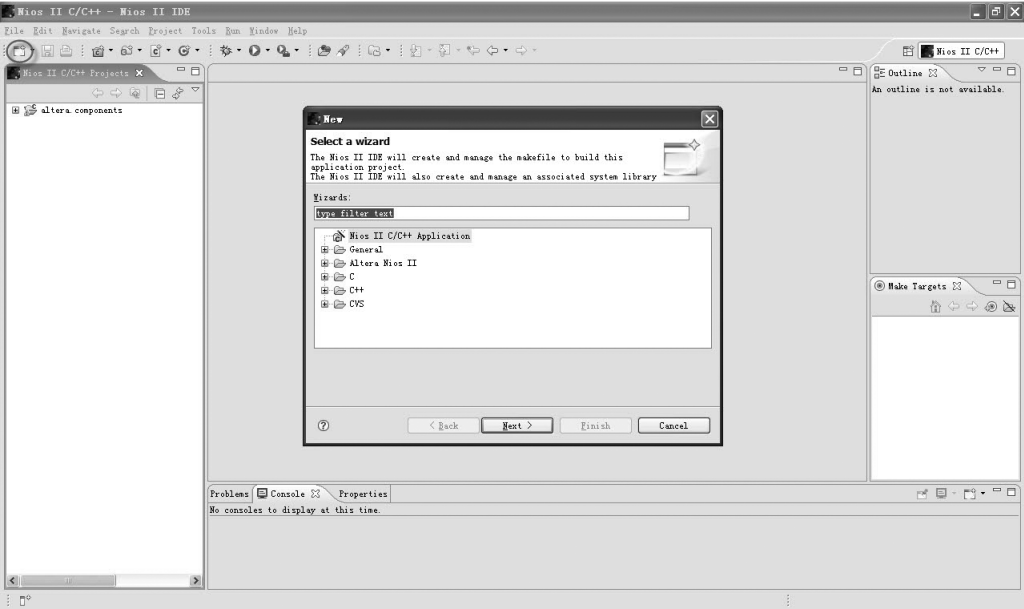


图 6.76 新工程建立

(2) 在工程设置页面中，如图 6.77 所示，Nios II 会给出许多模板，在这里，我们选择模

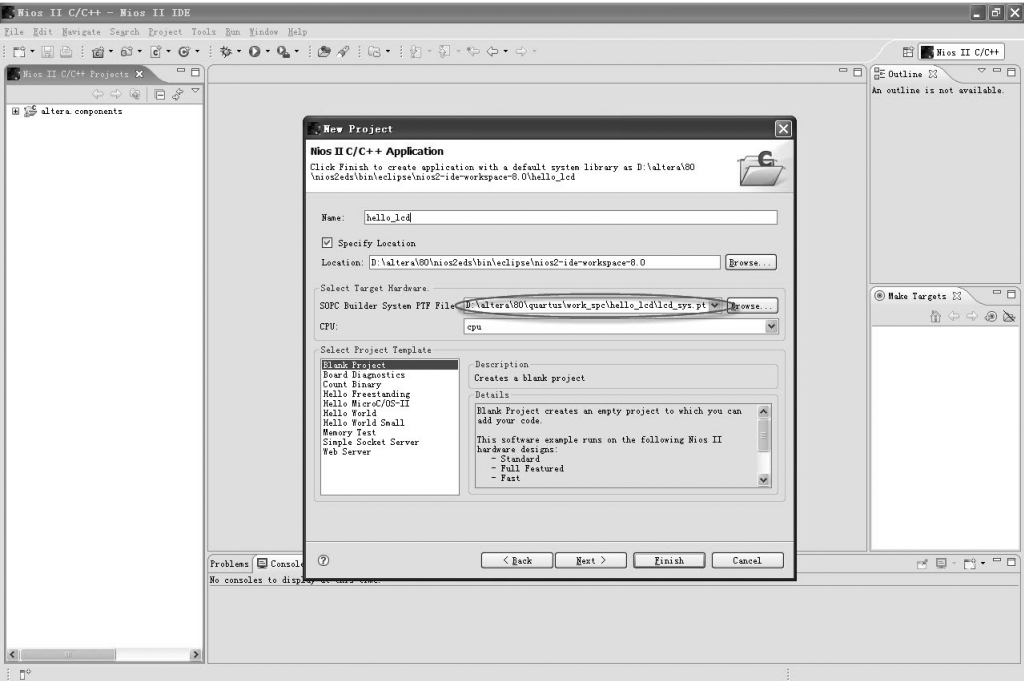


图 6.77 工程设置页面

板为空 (Blank Project)，选择 Specify Location 可以指定生成工程文件的路径。在 Select Target Hardware 中，选择刚刚生成的硬件系统 lcd_sys.ptf。这里需要注意的是，对比 Quartus II 中目标器件的选择，在工程建立之后，可以通过路径 Assignment→Device 来修改目标器件；而 Nios II 工程是建立在选定的硬件系统的基础之上，根据选定的硬件系统，来生成系统头文件“system.h”，系统文件会随着硬件系统的更新而更新，但是不能在工程建立之后改变目标系统，因而在此处必须选择正确的对应系统。完成系统配置之后，单击“Next”按钮进入下一页，选择“Create a new system library named”，单击“Finish”按钮完成工程建立。工程建立之后，Nios II IDE 左边列表将出现 3 个工程，并且程序会在后台自动索引，如图 6.78 所示。

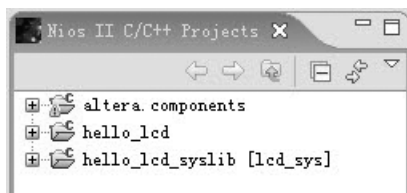


图 6.78 系统生成

(3) 选择菜单栏 Window → Preferences …，在弹出的对话框中选择 Run/Debug → Launching，出现如图 6.79 所示的页面，勾选掉 Build (if required) before launching，以减少程序下载和调试前的准备时间。

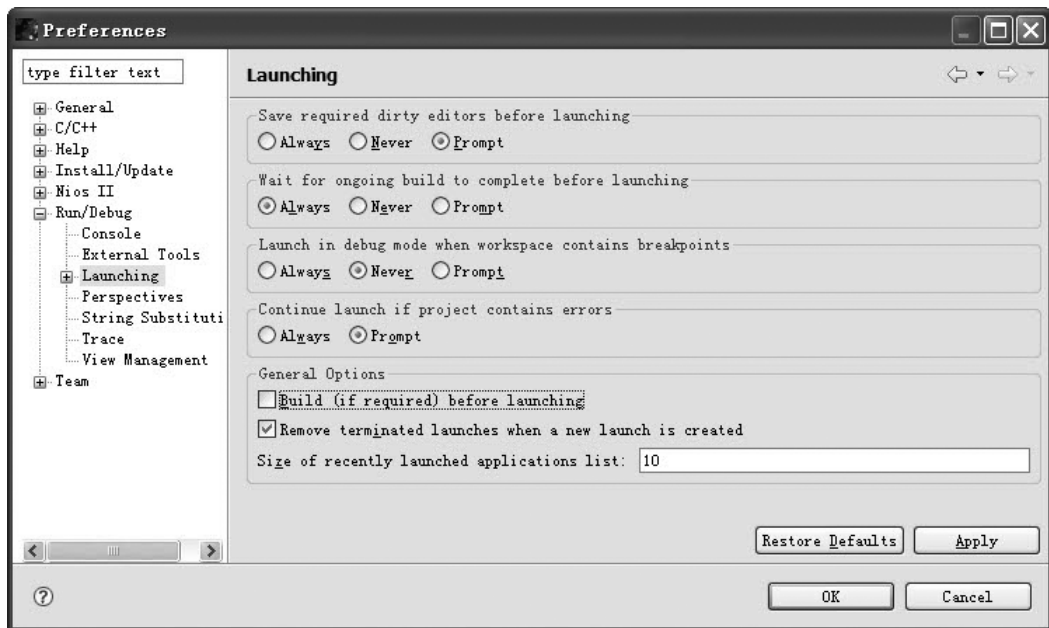


图 6.79 References 界面

(4) 选择工程 hello_lcd，右键单击选择 Properties，在 C/C++ Make Project 选项卡内选中 Build (Incremental Build)，在 Project References 选项卡内取消所有的勾选，确认。类似地，右键单击工程 hello_lcd_syslib → Properties，选中 C/C++ Make Project 选项卡中的 Build (Incremental Build)，如图 6.80 所示的页面。

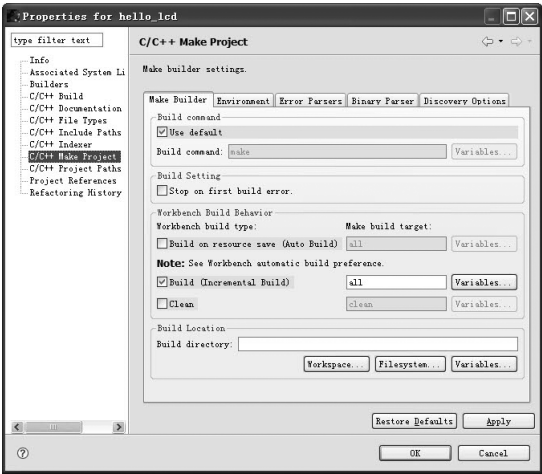


图 6.80 Properties for hello_lcd

(5) 在 properties for hello_lcd_syslib 中选中 system library 选项卡，在如图 6.81 所示界面中，勾选掉 Clean Exit (Flush Buffer) 和 Support C++，因为我们的程序不会退出，也不包含 C++ 的函数和库。选中 Program never exits、Reduce device drivers 和 Small C library 以减小程序体积。其他项保持默认设置即可。System Library 配置是 Nios II 中一个非常重要的配置界面。如图 6.81 中所示的 stdout、stderr、stdin 等选项的目标接口，作为标准输出、错误输出、标准输入的接口，这里我们将 stdout 接口选为“lcd”作为现实输出。

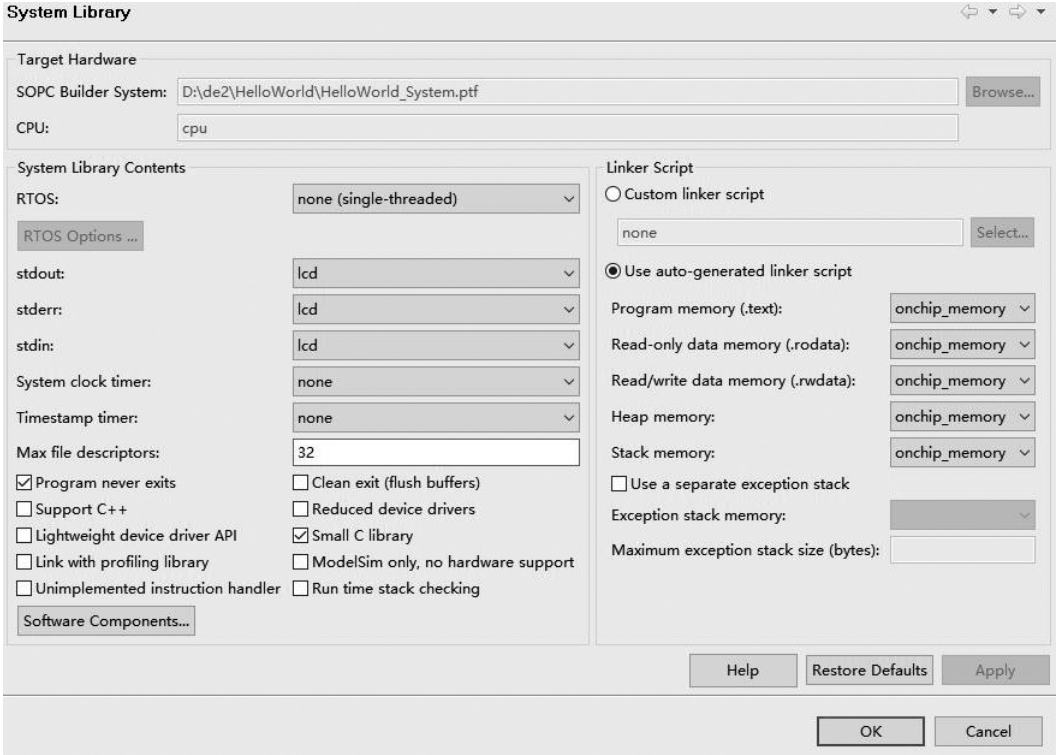


图 6.81 System Library 配置

(6) 右键单击 hello_lcd_syslib, 选择 Build Project。生成 hello_led 的系统工程文件, 如图 6.82 所示。

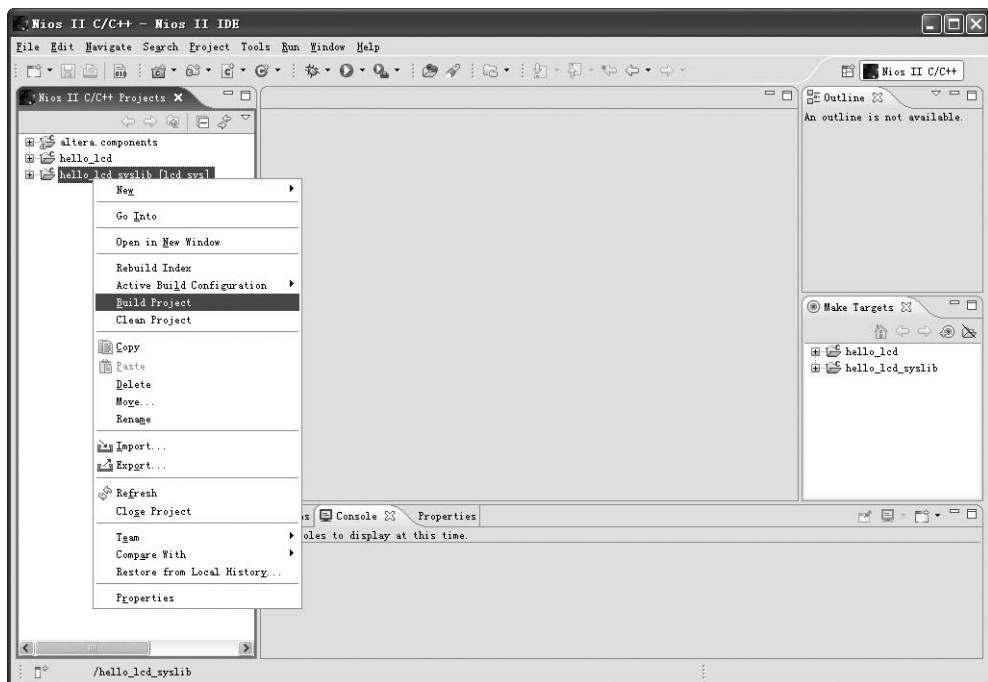


图 6.82 Build Project

(7) 键单右击工程 hello_lcd, 选择 New->Source file, 在弹出的对话框中指定文件名为 main.c。将以下 lcd 控制代码复制到其中:

```
#include "system.h"
#include <stdio.h>
int main()
{
    printf("Hello from Nios II!\n");
    return 0;
}
```

(8) 右键单击工程 hello_lcd, 选择 Build Project。编译完成后, 软件部分也就完成了, 如图 6.83 所示。

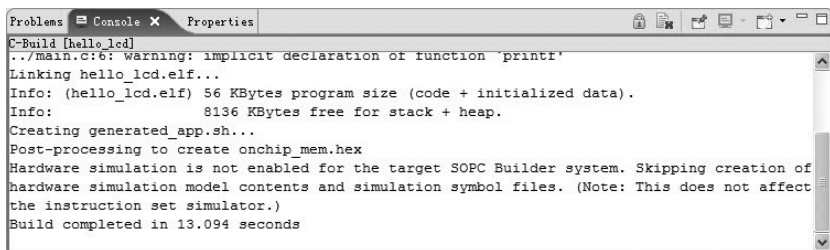


图 6.83 工程编译完成

(9) Nios II 中的调试器 (Debug): 在工程编译成功后, 可通过路径 Window→Open Perspective→Debug, 或如图 6.84 所示的路径进入 Debug 进行调试。Debug 包含以下基本调试功能: 运行控制、调用堆栈查看、软件断点、反汇编代码查看、调试信息查看、指令仿真器, 此外, 还支持硬件断点调试 ROM 或内存中的代码、数据触发、指令集仿真器等高级功能。

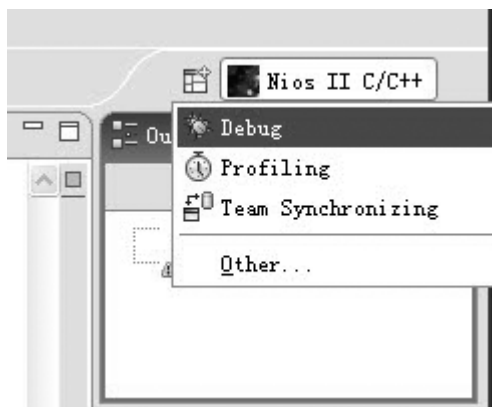


图 12 进入 Debug 调试界面

可通过路径 Windows→Show view 来打开信息查看窗口, 包括函数变量 Variable、寄存器 Register、断点 Breakpoint、储存器 Memory 以及表达式赋值函数的查看。还可进行程序的运行控制。本程序涉及较少, 因而这里不作调试演示。

附录

DE2-115 实验板引脚配置信息

AUD	To	Direction	Location	I/O Bank	VREF Group
AUD	AUD_ADCCDAT	Input	PIN_D2	1	B1_N0
AUD	AUD_ADCLRCK	Bidir	PIN_C2	1	B1_N0
AUD	AUD_BCLK	Bidir	PIN_F2	1	B1_N1
AUD	AUD_DACDAT	Output	PIN_D1	1	B1_N0
AUD	AUD_DACLCK	Bidir	PIN_E3	1	B1_N0
AUD	AUD_XCK	Output	PIN_E1	1	B1_N0
CLOCK	CLOCK2_50	Input	PIN_AG14	3	B3_N0
CLOCK	CLOCK3_50	Input	PIN_AG15	4	B4_N2
CLOCK	CLOCK_50	Input	PIN_Y2	2	B2_N0
DRAM	DRAM_ADDR[12]	Output	PIN_Y7	2	B2_N2
DRAM	DRAM_ADDR[11]	Output	PIN_AA5	2	B2_N2
DRAM	DRAM_ADDR[10]	Output	PIN_R5	2	B2_N0
DRAM	DRAM_ADDR[9]	Output	PIN_Y6	2	B2_N2
DRAM	DRAM_ADDR[8]	Output	PIN_Y5	2	B2_N2
DRAM	DRAM_ADDR[7]	Output	PIN_AA7	2	B2_N2
DRAM	DRAM_ADDR[6]	Output	PIN_W7	2	B2_N2
DRAM	DRAM_ADDR[5]	Output	PIN_W8	2	B2_N2
DRAM	DRAM_ADDR[4]	Output	PIN_V5	2	B2_N1
DRAM	DRAM_ADDR[3]	Output	PIN_P1	1	B1_N2
DRAM	DRAM_ADDR[2]	Output	PIN_U8	2	B2_N1
DRAM	DRAM_ADDR[1]	Output	PIN_V8	2	B2_N1
DRAM	DRAM_ADDR[0]	Output	PIN_R6	2	B2_N0
DRAM	DRAM_BA[1]	Output	PIN_R4	2	B2_N0
DRAM	DRAM_BA[0]	Output	PIN_U7	2	B2_N1
DRAM	DRAM_CAS_N	Output	PIN_V7	2	B2_N1
DRAM	DRAM_CKE	Output	PIN_AA6	2	B2_N2
DRAM	DRAM_CLK	Output	PIN_AE5	3	B3_N2

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
DRAM	DRAM_CS_N	Output	PIN_T4	2	B2_N0
DRAM	DRAM_DQ[31]	Bidir	PIN_U1	2	B2_N0
DRAM	DRAM_DQ[30]	Bidir	PIN_U4	2	B2_N0
DRAM	DRAM_DQ[29]	Bidir	PIN_T3	2	B2_N0
DRAM	DRAM_DQ[28]	Bidir	PIN_R3	2	B2_N0
DRAM	DRAM_DQ[27]	Bidir	PIN_R2	2	B2_N0
DRAM	DRAM_DQ[26]	Bidir	PIN_R1	2	B2_N0
DRAM	DRAM_DQ[25]	Bidir	PIN_R7	2	B2_N0
DRAM	DRAM_DQ[24]	Bidir	PIN_U5	2	B2_N1
DRAM	DRAM_DQ[23]	Bidir	PIN_L7	1	B1_N2
DRAM	DRAM_DQ[22]	Bidir	PIN_M7	1	B1_N2
DRAM	DRAM_DQ[21]	Bidir	PIN_M4	1	B1_N1
DRAM	DRAM_DQ[20]	Bidir	PIN_N4	1	B1_N2
DRAM	DRAM_DQ[19]	Bidir	PIN_N3	1	B1_N2
DRAM	DRAM_DQ[18]	Bidir	PIN_P2	1	B1_N2
DRAM	DRAM_DQ[17]	Bidir	PIN_L8	1	B1_N2
DRAM	DRAM_DQ[16]	Bidir	PIN_M8	1	B1_N2
DRAM	DRAM_DQ[15]	Bidir	PIN_AC2	2	B2_N1
DRAM	DRAM_DQ[14]	Bidir	PIN_AB3	2	B2_N1
DRAM	DRAM_DQ[13]	Bidir	PIN_AC1	2	B2_N1
DRAM	DRAM_DQ[12]	Bidir	PIN_AB2	2	B2_N0
DRAM	DRAM_DQ[11]	Bidir	PIN_AA3	2	B2_N1
DRAM	DRAM_DQ[10]	Bidir	PIN_AB1	2	B2_N0
DRAM	DRAM_DQ[9]	Bidir	PIN_Y4	2	B2_N1
DRAM	DRAM_DQ[8]	Bidir	PIN_Y3	2	B2_N1
DRAM	DRAM_DQ[7]	Bidir	PIN_U3	2	B2_N0
DRAM	DRAM_DQ[6]	Bidir	PIN_V1	2	B2_N0
DRAM	DRAM_DQ[5]	Bidir	PIN_V2	2	B2_N0
DRAM	DRAM_DQ[4]	Bidir	PIN_V3	2	B2_N0
DRAM	DRAM_DQ[3]	Bidir	PIN_W1	2	B2_N1
DRAM	DRAM_DQ[2]	Bidir	PIN_V4	2	B2_N0
DRAM	DRAM_DQ[1]	Bidir	PIN_W2	2	B2_N0
DRAM	DRAM_DQ[0]	Bidir	PIN_W3	2	B2_N2
DRAM	DRAM_DQM[3]	Output	PIN_N8	1	B1_N2
DRAM	DRAM_DQM[2]	Output	PIN_K8	1	B1_N2
DRAM	DRAM_DQM[1]	Output	PIN_W4	2	B2_N2

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
DRAM	DRAM_DQM[0]	Output	PIN_U2	2	B2_N0
DRAM	DRAM_RAS_N	Output	PIN_U6	2	B2_N1
DRAM	DRAM_WE_N	Output	PIN_V6	2	B2_N1
EEP	EEP_I2C_SCLK	Output	PIN_D14	8	B8_N0
EEP	EEP_I2C_SDAT	Bidir	PIN_E14	8	B8_N0
ENET	ENET0_GTX_CLK	Output	PIN_A17	7	B7_N2
ENET	ENET0_INT_N	Input	PIN_A21	7	B7_N1
ENET	ENET0_LINK100	Input	PIN_C14	8	B8_N0
ENET	ENET0_MDC	Output	PIN_C20	7	B7_N1
ENET	ENET0_MDIO	Bidir	PIN_B21	7	B7_N1
ENET	ENET0_RST_N	Output	PIN_C19	7	B7_N1
ENET	ENET0_RX_CLK	Input	PIN_A15	7	B7_N2
ENET	ENET0_RX_COL	Input	PIN_E15	7	B7_N2
ENET	ENET0_RX_CRS	Input	PIN_D15	7	B7_N2
ENET	ENET0_RX_DATA[3]	Input	PIN_C15	7	B7_N2
ENET	ENET0_RX_DATA[2]	Input	PIN_D17	7	B7_N1
ENET	ENET0_RX_DATA[1]	Input	PIN_D16	7	B7_N2
ENET	ENET0_RX_DATA[0]	Input	PIN_C16	7	B7_N2
ENET	ENET0_RX_DV	Input	PIN_C17	7	B7_N1
ENET	ENET0_RX_ER	Input	PIN_D18	7	B7_N1
ENET	ENET0_TX_CLK	Input	PIN_B17	7	B7_N2
ENET	ENET0_TX_DATA[3]	Output	PIN_B19	7	B7_N1
ENET	ENET0_TX_DATA[2]	Output	PIN_A19	7	B7_N1
ENET	ENET0_TX_DATA[1]	Output	PIN_D19	7	B7_N1
ENET	ENET0_TX_DATA[0]	Output	PIN_C18	7	B7_N1
ENET	ENET0_TX_EN	Output	PIN_A18	7	B7_N1
ENET	ENET0_TX_ER	Output	PIN_B18	7	B7_N1
ENET	ENET1_GTX_CLK	Output	PIN_C23	7	B7_N0
ENET	ENET1_INT_N	Input	PIN_D24	7	B7_N0
ENET	ENET1_LINK100	Input	PIN_D13	8	B8_N0
ENET	ENET1_MDC	Output	PIN_D23	7	B7_N0
ENET	ENET1_MDIO	Bidir	PIN_D25	7	B7_N0
ENET	ENET1_RST_N	Output	PIN_D22	7	B7_N0
ENET	ENET1_RX_CLK	Input	PIN_B15	7	B7_N2
ENET	ENET1_RX_COL	Input	PIN_B22	7	B7_N1
ENET	ENET1_RX_CRS	Input	PIN_D20	7	B7_N1

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
ENET	ENET1_RX_DATA[3]	Input	PIN_D21	7	B7_N0
ENET	ENET1_RX_DATA[2]	Input	PIN_A23	7	B7_N0
ENET	ENET1_RX_DATA[1]	Input	PIN_C21	7	B7_N0
ENET	ENET1_RX_DATA[0]	Input	PIN_B23	7	B7_N0
ENET	ENET1_RX_DV	Input	PIN_A22	7	B7_N1
ENET	ENET1_RX_ER	Input	PIN_C24	7	B7_N0
ENET	ENET1_TX_CLK	Input	PIN_C22	7	B7_N0
ENET	ENET1_TX_DATA[3]	Output	PIN_C26	7	B7_N0
ENET	ENET1_TX_DATA[2]	Output	PIN_B26	7	B7_N0
ENET	ENET1_TX_DATA[1]	Output	PIN_A26	7	B7_N0
ENET	ENET1_TX_DATA[0]	Output	PIN_C25	7	B7_N0
ENET	ENET1_TX_EN	Output	PIN_B25	7	B7_N0
ENET	ENET1_TX_ER	Output	PIN_A25	7	B7_N0
ENET	ENETCLK_25	Input	PIN_A14	8	B8_N0
EX	EX_IO[6]	Bidir	PIN_D9	8	B8_N1
EX	EX_IO[5]	Bidir	PIN_E10	8	B8_N1
EX	EX_IO[4]	Bidir	PIN_F14	8	B8_N0
EX	EX_IO[3]	Bidir	PIN_H14	8	B8_N0
EX	EX_IO[2]	Bidir	PIN_H13	8	B8_N0
EX	EX_IO[1]	Bidir	PIN_J14	8	B8_N0
EX	EX_IO[0]	Bidir	PIN_J10	8	B8_N1
FL	FL_ADDR[22]	Output	PIN_AD11	3	B3_N0
FL	FL_ADDR[21]	Output	PIN_AD10	3	B3_N2
FL	FL_ADDR[20]	Output	PIN_AE10	3	B3_N1
FL	FL_ADDR[19]	Output	PIN_AD12	3	B3_N0
FL	FL_ADDR[18]	Output	PIN_AC12	3	B3_N0
FL	FL_ADDR[17]	Output	PIN_AH12	3	B3_N0
FL	FL_ADDR[16]	Output	PIN_AA8	3	B3_N1
FL	FL_ADDR[15]	Output	PIN_Y10	3	B3_N2
FL	FL_ADDR[14]	Output	PIN_AC8	3	B3_N1
FL	FL_ADDR[13]	Output	PIN_AD8	3	B3_N2
FL	FL_ADDR[12]	Output	PIN_AA10	3	B3_N1
FL	FL_ADDR[11]	Output	PIN_AF9	3	B3_N1
FL	FL_ADDR[10]	Output	PIN_AE9	3	B3_N1
FL	FL_ADDR[9]	Output	PIN_AB10	3	B3_N1
FL	FL_ADDR[8]	Output	PIN_AB12	3	B3_N0

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
FL	FL_ADDR[7]	Output	PIN_AB13	3	B3_N0
FL	FL_ADDR[6]	Output	PIN_AA12	3	B3_N0
FL	FL_ADDR[5]	Output	PIN_AA13	3	B3_N0
FL	FL_ADDR[4]	Output	PIN_Y12	3	B3_N0
FL	FL_ADDR[3]	Output	PIN_Y14	3	B3_N0
FL	FL_ADDR[2]	Output	PIN_Y13	3	B3_N0
FL	FL_ADDR[1]	Output	PIN_AH7	3	B3_N1
FL	FL_ADDR[0]	Output	PIN_AG12	3	B3_N0
FL	FL_CE_N	Output	PIN_AG7	3	B3_N2
FL	FL_DQ[7]	Bidir	PIN_AF12	3	B3_N1
FL	FL_DQ[6]	Bidir	PIN_AH11	3	B3_N0
FL	FL_DQ[5]	Bidir	PIN_AG11	3	B3_N0
FL	FL_DQ[4]	Bidir	PIN_AF11	3	B3_N1
FL	FL_DQ[3]	Bidir	PIN_AH10	3	B3_N1
FL	FL_DQ[2]	Bidir	PIN_AG10	3	B3_N1
FL	FL_DQ[1]	Bidir	PIN_AF10	3	B3_N1
FL	FL_DQ[0]	Bidir	PIN_AH8	3	B3_N1
FL	FL_OE_N	Output	PIN_AG8	3	B3_N1
FL	FL_RST_N	Output	PIN_AE11	3	B3_N1
FL	FL_RY	Input	PIN_Y1	2	B2_N0
FL	FL_WE_N	Output	PIN_AC10	3	B3_N0
FL	FL_WP_N	Output	PIN_AE12	3	B3_N1
GPIO	GPIO[35]	Bidir	PIN_AG26	4	B4_N0
GPIO	GPIO[34]	Bidir	PIN_AH23	4	B4_N1
GPIO	GPIO[33]	Bidir	PIN_AH26	4	B4_N0
GPIO	GPIO[32]	Bidir	PIN_AF20	4	B4_N1
GPIO	GPIO[31]	Bidir	PIN_AG23	4	B4_N1
GPIO	GPIO[30]	Bidir	PIN_AE20	4	B4_N1
GPIO	GPIO[29]	Bidir	PIN_AF26	4	B4_N1
GPIO	GPIO[28]	Bidir	PIN_AH22	4	B4_N1
GPIO	GPIO[27]	Bidir	PIN_AE24	4	B4_N0
GPIO	GPIO[26]	Bidir	PIN_AG22	4	B4_N1
GPIO	GPIO[25]	Bidir	PIN_AE25	4	B4_N1
GPIO	GPIO[24]	Bidir	PIN_AH25	4	B4_N1
GPIO	GPIO[23]	Bidir	PIN_AD25	4	B4_N0
GPIO	GPIO[22]	Bidir	PIN_AG25	4	B4_N1

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
GPIO	GPIO[21]	Bidir	PIN_AD22	4	B4_N0
GPIO	GPIO[20]	Bidir	PIN_AF22	4	B4_N0
GPIO	GPIO[19]	Bidir	PIN_AF21	4	B4_N1
GPIO	GPIO[18]	Bidir	PIN_AE22	4	B4_N0
GPIO	GPIO[17]	Bidir	PIN_AC22	4	B4_N0
GPIO	GPIO[16]	Bidir	PIN_AF25	4	B4_N1
GPIO	GPIO[15]	Bidir	PIN_AE21	4	B4_N1
GPIO	GPIO[14]	Bidir	PIN_AF24	4	B4_N1
GPIO	GPIO[13]	Bidir	PIN_AF15	4	B4_N2
GPIO	GPIO[12]	Bidir	PIN_AD19	4	B4_N0
GPIO	GPIO[11]	Bidir	PIN_AF16	4	B4_N2
GPIO	GPIO[10]	Bidir	PIN_AC19	4	B4_N0
GPIO	GPIO[9]	Bidir	PIN_AE15	4	B4_N2
GPIO	GPIO[8]	Bidir	PIN_AD15	4	B4_N2
GPIO	GPIO[7]	Bidir	PIN_AE16	4	B4_N2
GPIO	GPIO[6]	Bidir	PIN_AD21	4	B4_N0
GPIO	GPIO[5]	Bidir	PIN_Y16	4	B4_N0
GPIO	GPIO[4]	Bidir	PIN_AC21	4	B4_N0
GPIO	GPIO[3]	Bidir	PIN_Y17	4	B4_N0
GPIO	GPIO[2]	Bidir	PIN_AB21	4	B4_N0
GPIO	GPIO[1]	Bidir	PIN_AC15	4	B4_N2
GPIO	GPIO[0]	Bidir	PIN_AB22	4	B4_N0
HEX	HEX0[6]	Output	PIN_H22	6	B6_N0
HEX	HEX0[5]	Output	PIN_J22	6	B6_N0
HEX	HEX0[4]	Output	PIN_L25	6	B6_N1
HEX	HEX0[3]	Output	PIN_L26	6	B6_N1
HEX	HEX0[2]	Output	PIN_E17	7	B7_N2
HEX	HEX0[1]	Output	PIN_F22	7	B7_N0
HEX	HEX0[0]	Output	PIN_G18	7	B7_N2
HEX	HEX1[6]	Output	PIN_U24	5	B5_N0
HEX	HEX1[5]	Output	PIN_U23	5	B5_N1
HEX	HEX1[4]	Output	PIN_W25	5	B5_N1
HEX	HEX1[3]	Output	PIN_W22	5	B5_N0
HEX	HEX1[2]	Output	PIN_W21	5	B5_N1
HEX	HEX1[1]	Output	PIN_Y22	5	B5_N0
HEX	HEX1[0]	Output	PIN_M24	6	B6_N2

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
HEX	HEX2[6]	Output	PIN_W28	5	B5_N1
HEX	HEX2[5]	Output	PIN_W27	5	B5_N1
HEX	HEX2[4]	Output	PIN_Y26	5	B5_N1
HEX	HEX2[3]	Output	PIN_W26	5	B5_N1
HEX	HEX2[2]	Output	PIN_Y25	5	B5_N1
HEX	HEX2[1]	Output	PIN_AA26	5	B5_N1
HEX	HEX2[0]	Output	PIN_AA25	5	B5_N1
HEX	HEX3[6]	Output	PIN_Y19	4	B4_N0
HEX	HEX3[5]	Output	PIN_AF23	4	B4_N0
HEX	HEX3[4]	Output	PIN_AD24	4	B4_N0
HEX	HEX3[3]	Output	PIN_AA21	4	B4_N0
HEX	HEX3[2]	Output	PIN_AB20	4	B4_N0
HEX	HEX3[1]	Output	PIN_U21	5	B5_N0
HEX	HEX3[0]	Output	PIN_V21	5	B5_N1
HEX	HEX4[6]	Output	PIN_AE18	4	B4_N2
HEX	HEX4[5]	Output	PIN_AF19	4	B4_N1
HEX	HEX4[4]	Output	PIN_AE19	4	B4_N1
HEX	HEX4[3]	Output	PIN_AH21	4	B4_N2
HEX	HEX4[2]	Output	PIN_AG21	4	B4_N2
HEX	HEX4[1]	Output	PIN_AA19	4	B4_N0
HEX	HEX4[0]	Output	PIN_AB19	4	B4_N0
HEX	HEX5[6]	Output	PIN_AH18	4	B4_N2
HEX	HEX5[5]	Output	PIN_AF18	4	B4_N1
HEX	HEX5[4]	Output	PIN_AG19	4	B4_N2
HEX	HEX5[3]	Output	PIN_AH19	4	B4_N2
HEX	HEX5[2]	Output	PIN_AB18	4	B4_N0
HEX	HEX5[1]	Output	PIN_AC18	4	B4_N1
HEX	HEX5[0]	Output	PIN_AD18	4	B4_N1
HEX	HEX6[6]	Output	PIN_AC17	4	B4_N2
HEX	HEX6[5]	Output	PIN_AA15	4	B4_N2
HEX	HEX6[4]	Output	PIN_AB15	4	B4_N2
HEX	HEX6[3]	Output	PIN_AB17	4	B4_N1
HEX	HEX6[2]	Output	PIN_AA16	4	B4_N2
HEX	HEX6[1]	Output	PIN_AB16	4	B4_N2
HEX	HEX6[0]	Output	PIN_AA17	4	B4_N1
HEX	HEX7[6]	Output	PIN_AA14	3	B3_N0

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
HEX	HEX7[5]	Output	PIN_AG18	4	B4_N2
HEX	HEX7[4]	Output	PIN_AF17	4	B4_N2
HEX	HEX7[3]	Output	PIN_AH17	4	B4_N2
HEX	HEX7[2]	Output	PIN_AG17	4	B4_N2
HEX	HEX7[1]	Output	PIN_AE17	4	B4_N2
HEX	HEX7[0]	Output	PIN_AD17	4	B4_N2
HSMC	HSMC_CLKIN0	Input	PIN_AH15	4	B4_N2
HSMC	HSMC_CLKIN_P1	Input	PIN_J27	6	B6_N2
HSMC	HSMC_CLKIN_P2	Input	PIN_Y27	5	B5_N0
HSMC	HSMC_CLKOUT0	Output	PIN_AD28	5	B5_N2
HSMC	HSMC_CLKOUT_P1	Output	PIN_G23	6	B6_N0
HSMC	HSMC_CLKOUT_P2	Output	PIN_V23	5	B5_N1
HSMC	HSMC_D[3]	Bidir	PIN_AF27	5	B5_N2
HSMC	HSMC_D[2]	Bidir	PIN_AE27	5	B5_N2
HSMC	HSMC_D[1]	Bidir	PIN_AE28	5	B5_N2
HSMC	HSMC_D[0]	Bidir	PIN_AE26	5	B5_N2
HSMC	HSMC_RX_D_P[16]	Input	PIN_T21	5	B5_N0
HSMC	HSMC_RX_D_P[15]	Input	PIN_R22	5	B5_N0
HSMC	HSMC_RX_D_P[14]	Input	PIN_P21	5	B5_N0
HSMC	HSMC_RX_D_P[13]	Input	PIN_P25	6	B6_N2
HSMC	HSMC_RX_D_P[12]	Input	PIN_N25	6	B6_N2
HSMC	HSMC_RX_D_P[11]	Input	PIN_L21	6	B6_N0
HSMC	HSMC_RX_D_P[10]	Input	PIN_U25	5	B5_N0
HSMC	HSMC_RX_D_P[9]	Input	PIN_T25	5	B5_N0
HSMC	HSMC_RX_D_P[8]	Input	PIN_R25	5	B5_N0
HSMC	HSMC_RX_D_P[7]	Input	PIN_M25	6	B6_N2
HSMC	HSMC_RX_D_P[6]	Input	PIN_L23	6	B6_N1
HSMC	HSMC_RX_D_P[5]	Input	PIN_K25	6	B6_N1
HSMC	HSMC_RX_D_P[4]	Input	PIN_H25	6	B6_N1
HSMC	HSMC_RX_D_P[3]	Input	PIN_G25	6	B6_N0
HSMC	HSMC_RX_D_P[2]	Input	PIN_F26	6	B6_N1
HSMC	HSMC_RX_D_P[1]	Input	PIN_D26	6	B6_N0
HSMC	HSMC_RX_D_P[0]	Input	PIN_F24	6	B6_N0
HSMC	HSMC_TX_D_P[16]	Output	PIN_U22	5	B5_N0
HSMC	HSMC_TX_D_P[15]	Output	PIN_V27	5	B5_N1
HSMC	HSMC_TX_D_P[14]	Output	PIN_U27	5	B5_N0

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
HSMC	HSMC_TX_D_P[13]	Output	PIN_R27	5	B5_N0
HSMC	HSMC_TX_D_P[12]	Output	PIN_V25	5	B5_N1
HSMC	HSMC_TX_D_P[11]	Output	PIN_L27	6	B6_N2
HSMC	HSMC_TX_D_P[10]	Output	PIN_J25	6	B6_N1
HSMC	HSMC_TX_D_P[9]	Output	PIN_P27	6	B6_N2
HSMC	HSMC_TX_D_P[8]	Output	PIN_J23	6	B6_N0
HSMC	HSMC_TX_D_P[7]	Output	PIN_H23	6	B6_N0
HSMC	HSMC_TX_D_P[6]	Output	PIN_K21	6	B6_N0
HSMC	HSMC_TX_D_P[5]	Output	PIN_M27	6	B6_N2
HSMC	HSMC_TX_D_P[4]	Output	PIN_K27	6	B6_N1
HSMC	HSMC_TX_D_P[3]	Output	PIN_G27	6	B6_N1
HSMC	HSMC_TX_D_P[2]	Output	PIN_F27	6	B6_N1
HSMC	HSMC_TX_D_P[1]	Output	PIN_E27	6	B6_N1
HSMC	HSMC_TX_D_P[0]	Output	PIN_D27	6	B6_N0
I2C	I2C_SCLK	Output	PIN_B7	8	B8_N1
I2C	I2C_SDAT	Bidir	PIN_A8	8	B8_N1
I2C	IRDA_RXD	Input	PIN_Y15	3	B3_N0
KEY	KEY[3]	Input	PIN_R24	5	B5_N0
KEY	KEY[2]	Input	PIN_N21	6	B6_N2
KEY	KEY[1]	Input	PIN_M21	6	B6_N1
KEY	KEY[0]	Input	PIN_M23	6	B6_N2
LCD	LCD_BLON	Output	PIN_L6	1	B1_N2
LCD	LCD_DATA[7]	Bidir	PIN_M5	1	B1_N2
LCD	LCD_DATA[6]	Bidir	PIN_M3	1	B1_N1
LCD	LCD_DATA[5]	Bidir	PIN_K2	1	B1_N1
LCD	LCD_DATA[4]	Bidir	PIN_K1	1	B1_N1
LCD	LCD_DATA[3]	Bidir	PIN_K7	1	B1_N1
LCD	LCD_DATA[2]	Bidir	PIN_L2	1	B1_N2
LCD	LCD_DATA[1]	Bidir	PIN_L1	1	B1_N2
LCD	LCD_DATA[0]	Bidir	PIN_L3	1	B1_N1
LCD	LCD_EN	Output	PIN_L4	1	B1_N1
LCD	LCD_ON	Output	PIN_L5	1	B1_N1
LCD	LCD_RS	Output	PIN_M2	1	B1_N2
LCD	LCD_RW	Output	PIN_M1	1	B1_N2
LED	LEDG[8]	Output	PIN_F17	7	B7_N2
LED	LEDG[7]	Output	PIN_G21	7	B7_N1

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
LED	LEDG[6]	Output	PIN_G22	7	B7_N2
LED	LEDG[5]	Output	PIN_G20	7	B7_N1
LED	LEDG[4]	Output	PIN_H21	7	B7_N2
LED	LEDG[3]	Output	PIN_E24	7	B7_N1
LED	LEDG[2]	Output	PIN_E25	7	B7_N1
LED	LEDG[1]	Output	PIN_E22	7	B7_N0
LED	LEDG[0]	Output	PIN_E21	7	B7_N0
LED	LEDR[17]	Output	PIN_H15	7	B7_N2
LED	LEDR[16]	Output	PIN_G16	7	B7_N2
LED	LEDR[15]	Output	PIN_G15	7	B7_N2
LED	LEDR[14]	Output	PIN_F15	7	B7_N2
LED	LEDR[13]	Output	PIN_H17	7	B7_N2
LED	LEDR[12]	Output	PIN_J16	7	B7_N2
LED	LEDR[11]	Output	PIN_H16	7	B7_N2
LED	LEDR[10]	Output	PIN_J15	7	B7_N2
LED	LEDR[9]	Output	PIN_G17	7	B7_N1
LED	LEDR[8]	Output	PIN_J17	7	B7_N2
LED	LEDR[7]	Output	PIN_H19	7	B7_N2
LED	LEDR[6]	Output	PIN_J19	7	B7_N2
LED	LEDR[5]	Output	PIN_E18	7	B7_N1
LED	LEDR[4]	Output	PIN_F18	7	B7_N1
LED	LEDR[3]	Output	PIN_F21	7	B7_N0
LED	LEDR[2]	Output	PIN_E19	7	B7_N0
LED	LEDR[1]	Output	PIN_F19	7	B7_N0
LED	LEDR[0]	Output	PIN_G19	7	B7_N2
OTG	OTG_ADDR[1]	Output	PIN_C3	8	B8_N2
OTG	OTG_ADDR[0]	Output	PIN_H7	1	B1_N0
OTG	OTG_CS_N	Output	PIN_A3	8	B8_N2
OTG	OTG_DACK_N[1]	Output	PIN_D4	8	B8_N2
OTG	OTG_DACK_N[0]	Output	PIN_C4	8	B8_N2
OTG	OTG_DATA[15]	Bidir	PIN_G4	1	B1_N0
OTG	OTG_DATA[14]	Bidir	PIN_F3	1	B1_N0
OTG	OTG_DATA[13]	Bidir	PIN_F1	1	B1_N1
OTG	OTG_DATA[12]	Bidir	PIN_G3	1	B1_N0
OTG	OTG_DATA[11]	Bidir	PIN_G2	1	B1_N1
OTG	OTG_DATA[10]	Bidir	PIN_G1	1	B1_N1

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
OTG	OTG_DATA[9]	Bidir	PIN_H4	1	B1_N0
OTG	OTG_DATA[8]	Bidir	PIN_H3	1	B1_N0
OTG	OTG_DATA[7]	Bidir	PIN_H6	1	B1_N0
OTG	OTG_DATA[6]	Bidir	PIN_J7	1	B1_N1
OTG	OTG_DATA[5]	Bidir	PIN_J3	1	B1_N1
OTG	OTG_DATA[4]	Bidir	PIN_J4	1	B1_N1
OTG	OTG_DATA[3]	Bidir	PIN_K3	1	B1_N1
OTG	OTG_DATA[2]	Bidir	PIN_J5	1	B1_N1
OTG	OTG_DATA[1]	Bidir	PIN_K4	1	B1_N1
OTG	OTG_DATA[0]	Bidir	PIN_J6	1	B1_N1
OTG	OTG_DREQ[1]	Input	PIN_B4	8	B8_N2
OTG	OTG_DREQ[0]	Input	PIN_J1	1	B1_N2
OTG	OTG_FSPEED	Bidir	PIN_C6	8	B8_N2
OTG	OTG_INT[1]	Input	PIN_D5	8	B8_N2
OTG	OTG_INT[0]	Input	PIN_A6	8	B8_N1
OTG	OTG_LSPEED	Bidir	PIN_B6	8	B8_N1
OTG	OTG_RD_N	Output	PIN_B3	8	B8_N2
OTG	OTG_RST_N	Output	PIN_C5	8	B8_N2
OTG	OTG_WR_N	Output	PIN_A4	8	B8_N2
PS2	PS2_CLK	Bidir	PIN_G6	1	B1_N0
PS2	PS2_CLK2	Bidir	PIN_G5	1	B1_N0
PS2	PS2_DAT	Bidir	PIN_H5	1	B1_N1
PS2	PS2_DAT2	Bidir	PIN_F5	1	B1_N0
SD	SD_CLK	Output	PIN_AE13	3	B3_N0
SD	SD_CMD	Bidir	PIN_AD14	3	B3_N0
SD	SD_DAT[3]	Bidir	PIN_AC14	3	B3_N0
SD	SD_DAT[2]	Bidir	PIN_AB14	3	B3_N0
SD	SD_DAT[1]	Bidir	PIN_AF13	3	B3_N0
SD	SD_DAT[0]	Bidir	PIN_AE14	3	B3_N0
SD	SD_WP_N	Input	PIN_AF14	3	B3_N0
SMA	SMA_CLKIN	Input	PIN_AH14	3	B3_N0
SMA	SMA_CLKOUT	Output	PIN_AE23	4	B4_N0
SRAM	SRAM_ADDR[19]	Output	PIN_T8	2	B2_N1
SRAM	SRAM_ADDR[18]	Output	PIN_AB8	3	B3_N2
SRAM	SRAM_ADDR[17]	Output	PIN_AB9	3	B3_N2
SRAM	SRAM_ADDR[16]	Output	PIN_AC11	3	B3_N0

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
SRAM	SRAM_ADDR[15]	Output	PIN_AB11	3	B3_N1
SRAM	SRAM_ADDR[14]	Output	PIN_AA4	2	B2_N1
SRAM	SRAM_ADDR[13]	Output	PIN_AC3	2	B2_N1
SRAM	SRAM_ADDR[12]	Output	PIN_AB4	2	B2_N2
SRAM	SRAM_ADDR[11]	Output	PIN_AD3	2	B2_N1
SRAM	SRAM_ADDR[10]	Output	PIN_AF2	2	B2_N2
SRAM	SRAM_ADDR[9]	Output	PIN_T7	2	B2_N0
SRAM	SRAM_ADDR[8]	Output	PIN_AF5	3	B3_N2
SRAM	SRAM_ADDR[7]	Output	PIN_AC5	2	B2_N2
SRAM	SRAM_ADDR[6]	Output	PIN_AB5	2	B2_N2
SRAM	SRAM_ADDR[5]	Output	PIN_AE6	3	B3_N2
SRAM	SRAM_ADDR[4]	Output	PIN_AB6	2	B2_N2
SRAM	SRAM_ADDR[3]	Output	PIN_AC7	3	B3_N2
SRAM	SRAM_ADDR[2]	Output	PIN_AE7	3	B3_N1
SRAM	SRAM_ADDR[1]	Output	PIN_AD7	3	B3_N2
SRAM	SRAM_ADDR[0]	Output	PIN_AB7	3	B3_N1
SRAM	SRAM_CE_N	Output	PIN_AF8	3	B3_N1
SRAM	SRAM_DQ[15]	Bidir	PIN_AG3	3	B3_N2
SRAM	SRAM_DQ[14]	Bidir	PIN_AF3	3	B3_N2
SRAM	SRAM_DQ[13]	Bidir	PIN_AE4	3	B3_N2
SRAM	SRAM_DQ[12]	Bidir	PIN_AE3	2	B2_N2
SRAM	SRAM_DQ[11]	Bidir	PIN_AE1	2	B2_N1
SRAM	SRAM_DQ[10]	Bidir	PIN_AE2	2	B2_N1
SRAM	SRAM_DQ[9]	Bidir	PIN_AD2	2	B2_N1
SRAM	SRAM_DQ[8]	Bidir	PIN_AD1	2	B2_N1
SRAM	SRAM_DQ[7]	Bidir	PIN_AF7	3	B3_N1
SRAM	SRAM_DQ[6]	Bidir	PIN_AH6	3	B3_N2
SRAM	SRAM_DQ[5]	Bidir	PIN_AG6	3	B3_N2
SRAM	SRAM_DQ[4]	Bidir	PIN_AF6	3	B3_N2
SRAM	SRAM_DQ[3]	Bidir	PIN_AH4	3	B3_N2
SRAM	SRAM_DQ[2]	Bidir	PIN_AG4	3	B3_N2
SRAM	SRAM_DQ[1]	Bidir	PIN_AF4	3	B3_N2
SRAM	SRAM_DQ[0]	Bidir	PIN_AH3	3	B3_N2
SRAM	SRAM_LB_N	Output	PIN_AD4	3	B3_N2
SRAM	SRAM_OE_N	Output	PIN_AD5	3	B3_N2
SRAM	SRAM_UB_N	Output	PIN_AC4	2	B2_N2

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
SRAM	SRAM_WE_N	Output	PIN_AE8	3	B3_N1
SW	SW[17]	Input	PIN_Y23	5	B5_N2
SW	SW[16]	Input	PIN_Y24	5	B5_N2
SW	SW[15]	Input	PIN_AA22	5	B5_N2
SW	SW[14]	Input	PIN_AA23	5	B5_N2
SW	SW[13]	Input	PIN_AA24	5	B5_N2
SW	SW[12]	Input	PIN_AB23	5	B5_N2
SW	SW[11]	Input	PIN_AB24	5	B5_N2
SW	SW[10]	Input	PIN_AC24	5	B5_N2
SW	SW[9]	Input	PIN_AB25	5	B5_N1
SW	SW[8]	Input	PIN_AC25	5	B5_N2
SW	SW[7]	Input	PIN_AB26	5	B5_N1
SW	SW[6]	Input	PIN_AD26	5	B5_N2
SW	SW[5]	Input	PIN_AC26	5	B5_N2
SW	SW[4]	Input	PIN_AB27	5	B5_N1
SW	SW[3]	Input	PIN_AD27	5	B5_N2
SW	SW[2]	Input	PIN_AC27	5	B5_N2
SW	SW[1]	Input	PIN_AC28	5	B5_N2
SW	SW[0]	Input	PIN_AB28	5	B5_N1
TD	TD_CLK27	Input	PIN_B14	8	B8_N0
TD	TD_DATA[7]	Input	PIN_F7	8	B8_N2
TD	TD_DATA[6]	Input	PIN_E7	8	B8_N2
TD	TD_DATA[5]	Input	PIN_D6	8	B8_N2
TD	TD_DATA[4]	Input	PIN_D7	8	B8_N2
TD	TD_DATA[3]	Input	PIN_C7	8	B8_N2
TD	TD_DATA[2]	Input	PIN_D8	8	B8_N2
TD	TD_DATA[1]	Input	PIN_A7	8	B8_N1
TD	TD_DATA[0]	Input	PIN_E8	8	B8_N2
TD	TD_HS	Input	PIN_E5	8	B8_N2
TD	TD_RESET_N	Output	PIN_G7	8	B8_N2
TD	TD_VS	Input	PIN_E4	8	B8_N2
UART	UART_CTS	Output	PIN_G14	8	B8_N0
UART	UART_RTS	Input	PIN_J13	8	B8_N0
UART	UART_RXD	Input	PIN_G12	8	B8_N1
UART	UART_TXD	Output	PIN_G9	8	B8_N2
VGA	VGA_B[7]	Output	PIN_D12	8	B8_N0

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
VGA	VGA_B[6]	Output	PIN_D11	8	B8_N1
VGA	VGA_B[5]	Output	PIN_C12	8	B8_N0
VGA	VGA_B[4]	Output	PIN_A11	8	B8_N0
VGA	VGA_B[3]	Output	PIN_B11	8	B8_N0
VGA	VGA_B[2]	Output	PIN_C11	8	B8_N1
VGA	VGA_B[1]	Output	PIN_A10	8	B8_N0
VGA	VGA_B[0]	Output	PIN_B10	8	B8_N0
VGA	VGA_BLANK_N	Output	PIN_F11	8	B8_N1
VGA	VGA_CLK	Output	PIN_A12	8	B8_N0
VGA	VGA_G[7]	Output	PIN_C9	8	B8_N1
VGA	VGA_G[6]	Output	PIN_F10	8	B8_N1
VGA	VGA_G[5]	Output	PIN_B8	8	B8_N1
VGA	VGA_G[4]	Output	PIN_C8	8	B8_N1
VGA	VGA_G[3]	Output	PIN_H12	8	B8_N1
VGA	VGA_G[2]	Output	PIN_F8	8	B8_N2
VGA	VGA_G[1]	Output	PIN_G11	8	B8_N1
VGA	VGA_G[0]	Output	PIN_G8	8	B8_N2
VGA	VGA_HS	Output	PIN_G13	8	B8_N0
VGA	VGA_R[7]	Output	PIN_H10	8	B8_N1
VGA	VGA_R[6]	Output	PIN_H8	8	B8_N2
VGA	VGA_R[5]	Output	PIN_J12	8	B8_N0
VGA	VGA_R[4]	Output	PIN_G10	8	B8_N1
VGA	VGA_R[3]	Output	PIN_F12	8	B8_N1
VGA	VGA_R[2]	Output	PIN_D10	8	B8_N1
VGA	VGA_R[1]	Output	PIN_E11	8	B8_N1
VGA	VGA_R[0]	Output	PIN_E12	8	B8_N1
VGA	VGA_SYNC_N	Output	PIN_C10	8	B8_N0
VGA	VGA_VS	Output	PIN_C13	8	B8_N0
HSMC	HSMC_CLKIN_N1	Unknown	PIN_J28	6	B6_N2
HSMC	HSMC_CLKIN_N2	Unknown	PIN_Y28	5	B5_N0
HSMC	HSMC_TX_D_N[0]	Unknown	PIN_D28	6	B6_N0
HSMC	HSMC_RX_D_N[0]	Unknown	PIN_F25	6	B6_N0
HSMC	HSMC_RX_D_N[1]	Unknown	PIN_C27	6	B6_N0
HSMC	HSMC_TX_D_N[1]	Unknown	PIN_E28	6	B6_N1
HSMC	HSMC_TX_D_N[2]	Unknown	PIN_F28	6	B6_N1
HSMC	HSMC_RX_D_N[2]	Unknown	PIN_E26	6	B6_N1

(续)

AUD	To	Direction	Location	I/O Bank	VREF Group
HSMC	HSMC_TX_D_N[3]	Unknown	PIN_G28	6	B6_N1
HSMC	HSMC_RX_D_N[3]	Unknown	PIN_G26	6	B6_N0
HSMC	HSMC_TX_D_N[4]	Unknown	PIN_K28	6	B6_N1
HSMC	HSMC_RX_D_N[4]	Unknown	PIN_H26	6	B6_N1
HSMC	HSMC_TX_D_N[5]	Unknown	PIN_M28	6	B6_N2
HSMC	HSMC_RX_D_N[5]	Unknown	PIN_K26	6	B6_N1
HSMC	HSMC_TX_D_N[6]	Unknown	PIN_K22	6	B6_N0
HSMC	HSMC_RX_D_N[6]	Unknown	PIN_L24	6	B6_N2
HSMC	HSMC_TX_D_N[7]	Unknown	PIN_H24	6	B6_N0
HSMC	HSMC_RX_D_N[7]	Unknown	PIN_M26	6	B6_N2
HSMC	HSMC_TX_D_N[8]	Unknown	PIN_J24	6	B6_N0
HSMC	HSMC_RX_D_N[8]	Unknown	PIN_R26	5	B5_N0
HSMC	HSMC_TX_D_N[9]	Unknown	PIN_P28	6	B6_N2
HSMC	HSMC_RX_D_N[9]	Unknown	PIN_T26	5	B5_N0
HSMC	HSMC_TX_D_N[10]	Unknown	PIN_J26	6	B6_N1
HSMC	HSMC_RX_D_N[10]	Unknown	PIN_U26	5	B5_N0
HSMC	HSMC_TX_D_N[11]	Unknown	PIN_L28	6	B6_N2
HSMC	HSMC_RX_D_N[11]	Unknown	PIN_L22	6	B6_N0
HSMC	HSMC_TX_D_N[12]	Unknown	PIN_V26	5	B5_N1
HSMC	HSMC_RX_D_N[12]	Unknown	PIN_N26	6	B6_N2
HSMC	HSMC_TX_D_N[13]	Unknown	PIN_R28	5	B5_N0
HSMC	HSMC_RX_D_N[13]	Unknown	PIN_P26	6	B6_N2
HSMC	HSMC_TX_D_N[14]	Unknown	PIN_U28	5	B5_N0
HSMC	HSMC_RX_D_N[14]	Unknown	PIN_R21	5	B5_N0
HSMC	HSMC_TX_D_N[15]	Unknown	PIN_V28	5	B5_N1
HSMC	HSMC_RX_D_N[15]	Unknown	PIN_R23	5	B5_N0
HSMC	HSMC_TX_D_N[16]	Unknown	PIN_V22	5	B5_N1
HSMC	HSMC_RX_D_N[16]	Unknown	PIN_T22	5	B5_N0
HSMC	HSMC_CLKOUT_N2	Unknown	PIN_V24	5	B5_N1
HSMC	HSMC_CLKOUT_N1	Unknown	PIN_G24	6	B6_N0

参 考 文 献

1. 杨之廉 申明编著,“超大规模集成电路设计方法学导论”第2版,清华大学出版社,1999。
2. San Jose. MAX+PLUS II GETTING STARTED. ALTERA CORPORATION
3. San Jose. Introduction to Quartus II. ALTERA CORPORATION
4. 陈耀和 编著.VHDL 语言设计技术,北京,电子工业出版社,2004。
5. 林明权 等编著.VHDL 数字控制系统设计范例,北京,电子工业出版社,2003。
6. 潘松,黄继业,编著.EDA 技术实用教程,北京,科学出版社,2002。
7. James O. Hamblen and Michael D. Furman, PAPID PROTOTYPING OF DIGITAL SYSTEMS. Kluwer Academic Publisher, 2001.
8. [美] Rochit Rajsuman 著,于敦山 盛世敏 田泽译,SoC 设计与测试,北京,北京航空航天大学出版社,2003。
9. 廖裕评等编著。逻辑电路设计 DE2-115 实战宝典。友晶科技 出版。
10. FPGA 与 SOPC 设计教程——DE2 实践,张志刚 编著,西安电子科技大学出版社,2007。
11. 王金明主编,数字系统设计与 VHDL,北京,电子工业出版社,2010
12. 书 1) VHDL 硬件描述语言与数字逻辑电路设计(修订版),西电出版社,侯伯亨
13. 书 4) VHDL 入门/解惑/经典实例/经验总结,北航出版社,黄任
《VHDL 硬件描述语言与数字逻辑电路设计》侯伯亨 顾新 西安电子科技大学出版社 1999;
《VHDL 编程与仿真》王毅平 张振荣 人民邮电出版社 2000 年 7 月